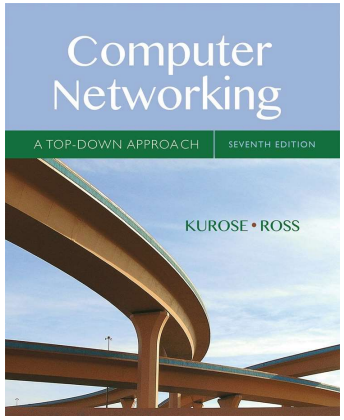
目录

计算机网络-第一章（中科大-郑老师） 2
计算机网络-第二章（中科大-郑老师） 30
计算机网络-第三章（中科大-郑老师） 69
计算机网络-第四章（中科大-郑老师） 105
计算机网络-第五章（中科大-郑老师） 131
计算机网络-第六章（中科大-郑老师） 161

第 1 章
计算机网络概述

中国科学技术大学
自动化系　郑烇
改编自Jim kurose, Keith Ross



Computer Networking: A Top Down Approach
7th edition
Jim Kurose, Keith Ross
Addison-Wesley
April 2016

第1章：概论

目标:
- 了解基本术语和概念
- 掌握网络的基本原理
- 为后续章节更深入的学习打下基础
- 方法：
  - 以Internet为例子

提纲:
- 什么是Internet？
- 什么是协议?
- 网络边缘
- 接入网、物理媒体
- 网络核心：分组交换、线路交换
- Internet/ISP 结构
- 性能：丢包、延时、吞吐量
- 协议层次、服务模型
- 历史

中科大信院2021秋

第1章：提纲

1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

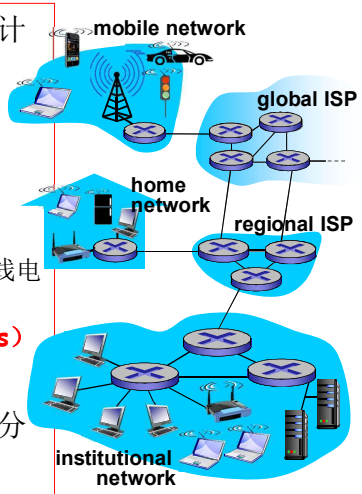
中科大信院2021秋

什么是Internet：从具体构成角度

- 节点
  - 主机及其上运行的应用程序
  - 路由器、交换机等网络交换设备
- 边：通信链路
  - 接入网链路：主机连接到互联网的链路
  - 主干链路：路由器间的链路
- 协议

什么是Internet：从具体构成角度（续）

- 数以亿计的、互联的计算设备：
  - 主机 = 端系统
  - 运行网络应用程序
- 通信链路
  - 光纤、同轴电缆、无线电、卫星
  - 传输速率 = 带宽（bps）
- 分组交换设备：转发分组 (packets)
  - 路由器和交换机



中科大信院2021秋

“Fun” 互联网设备

IP picture frame
http://www.ceiva.com/

Web-enabled toaster + weather forecaster

Tweet-a-watt: monitor energy use

Internet refrigerator

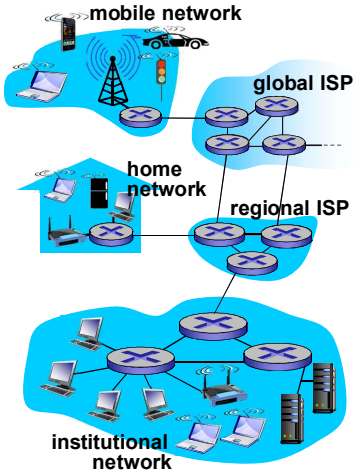
Slingbox: watch, control cable TV remotely

Internet phones

中科大信院2021秋

什么是Internet：从具体构成角度（续）

- 协议控制发送、接收消息
  - 如TCP、IP、HTTP、FTP、PPP
- Internet：“网络的网络”
  - 松散的层次结构，互连的ISP
  - 公共Internet vs. 专用intranet
- Internet标准
  - RFC: Request for comments
  - IETF: Internet Engineering Task Force



中科大信院2021秋

什么是协议？

人类协议：

- “几点了？”
- “我有个问题”
- 你好

… 发送特定的消息
… 收到消息时采取的特定行动或其他事件

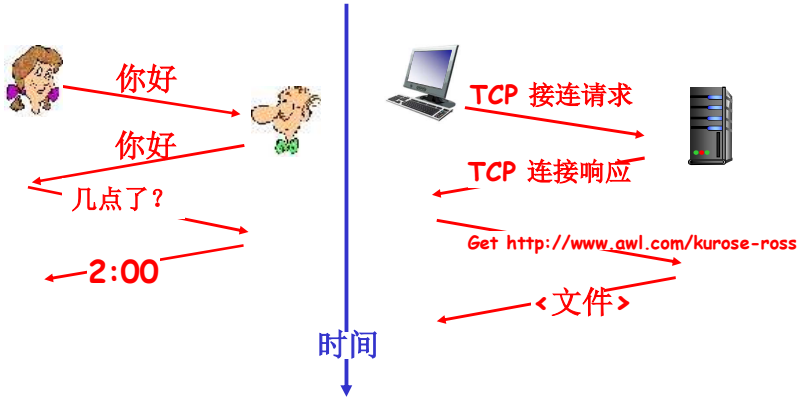
网络协议：

- 类似人类协议
- 机器之间的协议而非人与人之间的协议
- Internet 中所有的通信行为都受协议制约

协议定义了在两个或多个通信实体之间交换的报文格式和次序，以及在报文传输和/或接收或其他事件方面所采取的动作

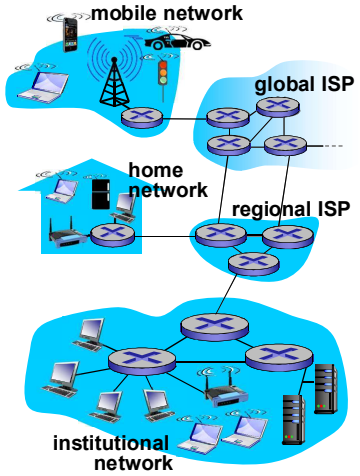
什么是协议？

人类协议和计算机网络协议示例



什么是Internet：从服务角度

- 使用通信设施进行通信的分布式应用
  - Web、VoIP、email、分布式游戏、电子商务、社交网络……
- 通信基础设施为apps提供编程接口（通信服务）
  - 将发送和接收数据的apps与互联网连接起来
  - 为app应用提供服务选择，类似于邮政服务：
    - 无连接不可靠服务
    - 面向连接的可靠服务

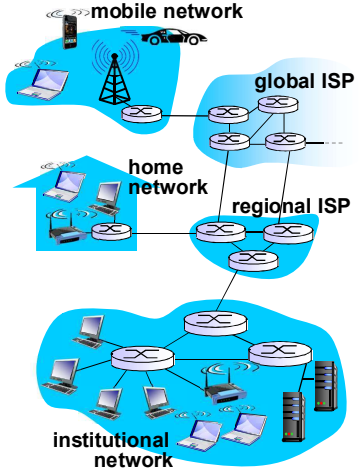


第1章：提纲

1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

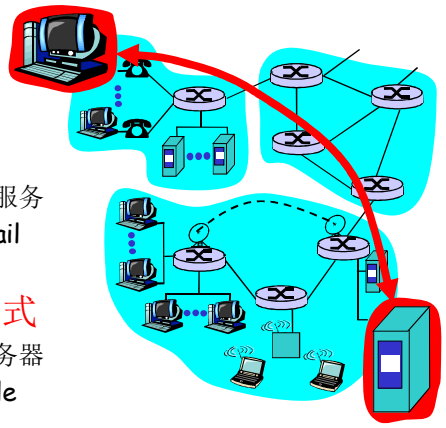
网络结构

- 网络边缘：
  - 主机
  - 应用程序（客户端和服务器）
- 网络核心：
  - 互连着的路由器
  - 网络的网络
- 接入网、物理媒体：
  - 有线或者无线通信链路



网络边缘

- 端系统（主机）：
  - 运行应用程序
  - 如Web、email
  - 在 “网络的边缘”
- 客户/服务器模式
  - 客户端向服务器请求、接收服务
  - 如Web浏览器/服务器；email客户端/服务器
- 对等（peer-peer ）模式
  - 很少（甚至没有）专门的服务器
  - 如 Gnutella、KaZaA、Emule



网络边缘：采用网络设施的面向连接服务

目标：在端系统之间传输数据

- 握手：在数据传输之前做好准备
  - 人类协议中：你好、你好
  - 两个通信主机之间为连接建立状态
- TCP - 传输控制协议（ Transmission Control Protocol ）
  - Internet上面向连接的服务

TCP 服务 [RFC 793]

- 可靠地、按顺序地传送数据
  - 确认和重传
- 流量控制
  - 发送方不会淹没接收方
- 拥塞控制
  - 当网络拥塞时，发送方降低发送速率

网络边缘：采用基础设施的无连接服务

目标：在端系统之间传输数据

- 无连接服务
- UDP - 用户数据报协议 (User Datagram Protocol) [RFC 768]:
  - 无连接
  - 不可靠数据传输
  - 无流量控制
  - 无拥塞控制

使用TCP的应用：

- HTTP (Web), FTP (文件传送), Telnet (远程登录), SMTP (email)

使用 UDP的应用：

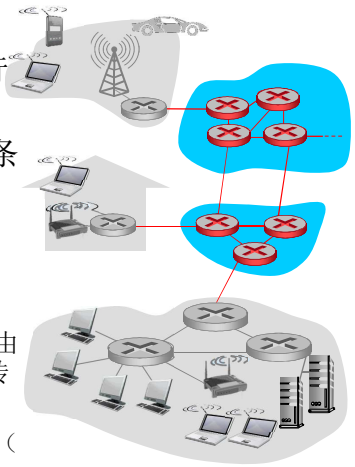
- 流媒体、远程会议、 DNS、Internet电话

第1章：提纲

1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

网络核心

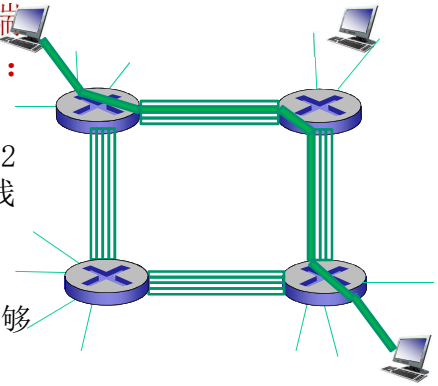
- 网络核心：路由器的网状网络
- 基本问题：数据怎样通过网络进行传输？
  - 电路交换：为每个呼叫预留一条专有电路：如电话网
  - 分组交换：
    - 将要传送的数据分成一个个单位：分组
    - 将分组从一个路由器传到相邻路由器（hop），一段段最终从源端传到目标端
    - 每段：采用链路的最大传输能力（带宽）



网络核心：电路交换

端到端的资源被分配给从源端到目标端的呼叫 “call”：

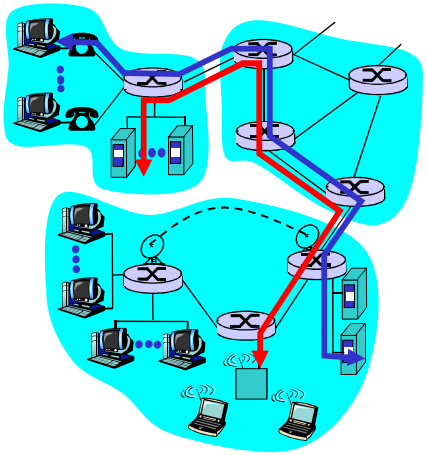
- 图中，每段链路有4条线路：
  - 该呼叫采用了上面链路的第2个线路，右边链路的第1个线路（piece）
- 独享资源：不同享
  - 每个呼叫一旦建立起来就能够保证性能
- 如果呼叫没有数据发送，被分配的资源就会被浪费 *(no sharing)*
- 通常被传统电话网络采用



网络核心：电路交换

为呼叫预留端-端资源

- 链路带宽、交换能力
- 专用资源：不共享
- 保证性能
- 要求建立呼叫连接



网络核心：电路交换

网络资源（如带宽）被分成片

- 为呼叫分配片
- 如果某个呼叫没有数据，则其资源片处于空闲状态（不共享）

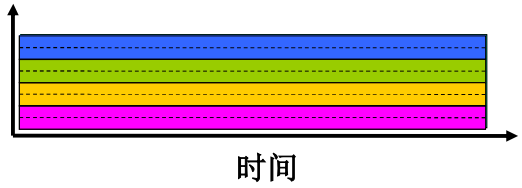
- 将带宽分成片
  - 频分(Frequency-division multiplexing)
  - 时分(Time-division multiplexing)
  - 波分(Wave-division multiplexing)

电路交换：FDM 与 TDM

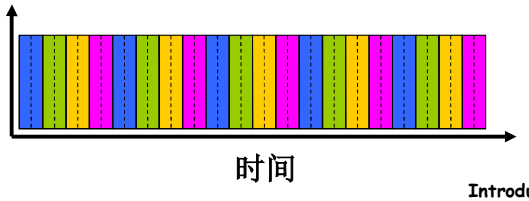
FDM

Example:

4 users



TDM



计算举例

- 在一个电路交换网络上，从主机A到主机B发送一个640,000比特的文件需要多长时间？
  - 所有的链路速率为1.536 Mbps
  - 每条链路使用时隙数为24的TDM
  - 建立端-端的电路需500 ms

每条链路的速率（一个时间片）：1.536Mbps/24 = 64kbps

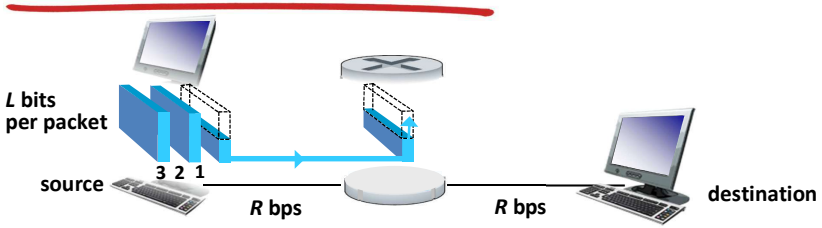
传输时间：640kb/64kps = 10s

共用时间：传输时间+建立链路时间=10s + 500ms = 10.5s

电路交换不适合计算机之间的通信

- 连接建立时间长
- 计算机之间的通信有突发性，如果使用线路交换，则浪费的片较多
  - 即使这个呼叫没有数据传递，其所占据的片也不能够被别的呼叫使用
- 可靠性不高？

网络核心：分组交换



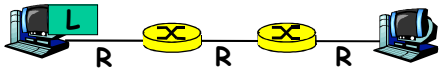
以分组为单位存储-转发方式

- 网络带宽资源不再分分为一个个片，传输时使用全部带宽
- 主机之间传输的数据被分为一个个分组

资源共享，按需使用：

- 存储-转发：分组每次移动一跳（ hop ）
  - 在转发之前，节点必须收到整个分组
  - 延迟比线路交换要大
  - 排队时间

分组交换：存储-转发



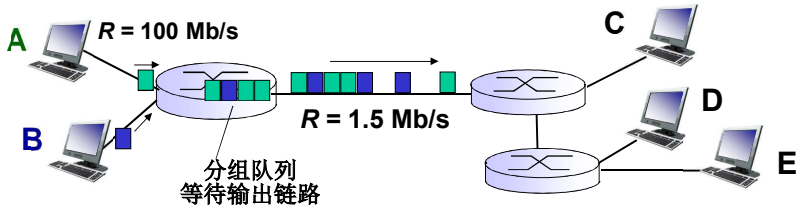
- 被传输到下一个链路之前，整个分组必须到达路由器：存储-转发
- 在一个速率为R bps的链路，一个长度为L bits 的分组的存储转发延时： L/R s

Example:

- L = 7.5 Mbits
- R = 1.5 Mbps
- 3次存储转发的延时 = 15 s

中科大信院2021秋

分组交换：排队延迟和丢失



排队和延迟：

- 如果到达速率>链路的输出速率：
  - 分组将会排队，等待传输
  - 如果路由器的缓存用完了，分组将会被抛弃

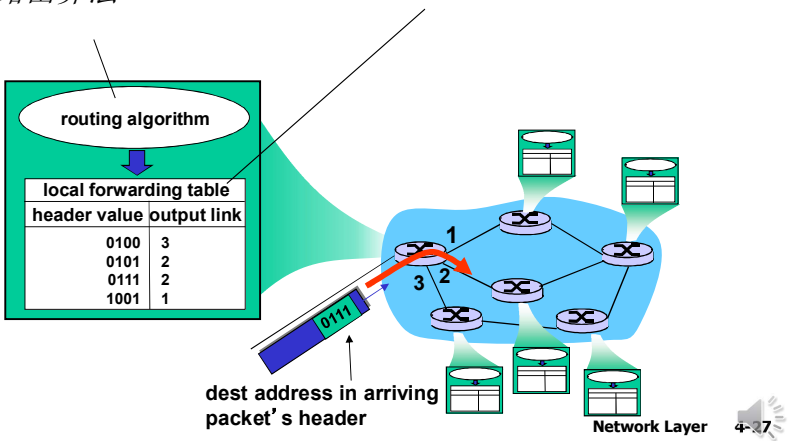
中科大信院2021秋

网络核心的关键功能

路由： 决定分组采用的源到目标的路径

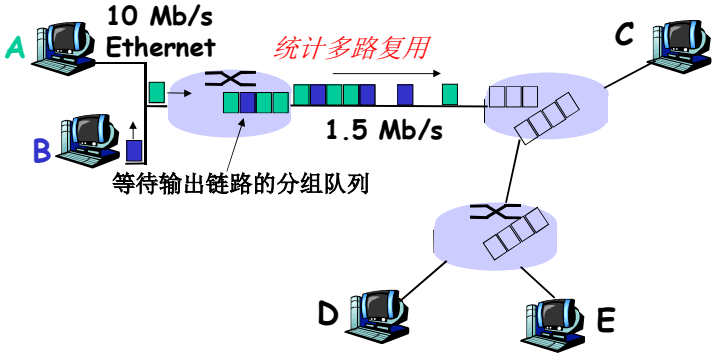
- *路由算法*

转发： 将分组从路由器的输入链路转移到输出链路



中科大信院2021秋

分组交换：统计多路复用



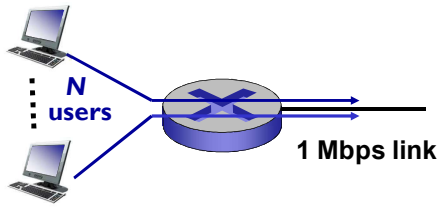
A&B 时分复用 链路资源

A & B 分组没有固定的模式➔ 统计多路复用

分组交换 vs. 电路交换

同样的网络资源，分组交换允许更多用户使用网络！

- 1 Mb/s 链路
- 每个用户：
  - 活动时100 kb/s
  - 10% 的时间是活动的



- 电路交换：
  - 10 用户
- 分组交换：
  - 35 用户时
  - >=10个用户活动的概率为0.0004

$$1-\sum_{n=0}^{9}\binom{35}{n}p^n(1-p)^{35-n}$$

中科大信院2021秋

分组交换 vs. 电路交换

分组交换是“突发数据的胜利者？”

- 适合于对突发式数据传输
  - 资源共享
  - 简单，不必建立呼叫
- 过度使用会造成网络拥塞：分组延时和丢失
  - 对可靠地数据传输需要协议来约束：拥塞控制
- Q: 怎样提供类似电路交换的服务？
  - 保证音频/视频应用需要的带宽
  - 一个仍未解决的问题(chapter 7)

Q: **预约服务（线路交换）对比按需服务（分组交换）的例子?**

中科大信院2021秋

分组交换网络：存储-转发

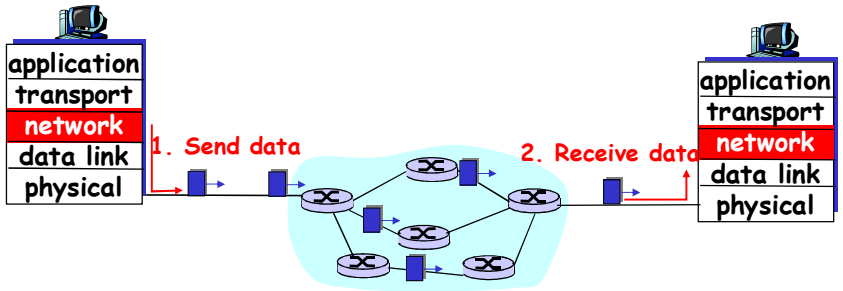
- 分组交换: 分组的存储转发一段一段从源端传到目标端，按照有无网络层的连接，分成：

1. 数据报网络：
   - 分组的目标地址决定下一跳
   - 在不同的阶段，路由可以改变
   - 类似：问路
   - Internent
2. 虚电路网络：
   - 每个分组都带标签（虚电路标识 VC ID），标签决定下一跳
   - 在呼叫建立时决定路径，在整个呼叫中路径保持不变
   - 路由器维持每个呼叫的状态信息
   - X.25 和ATM

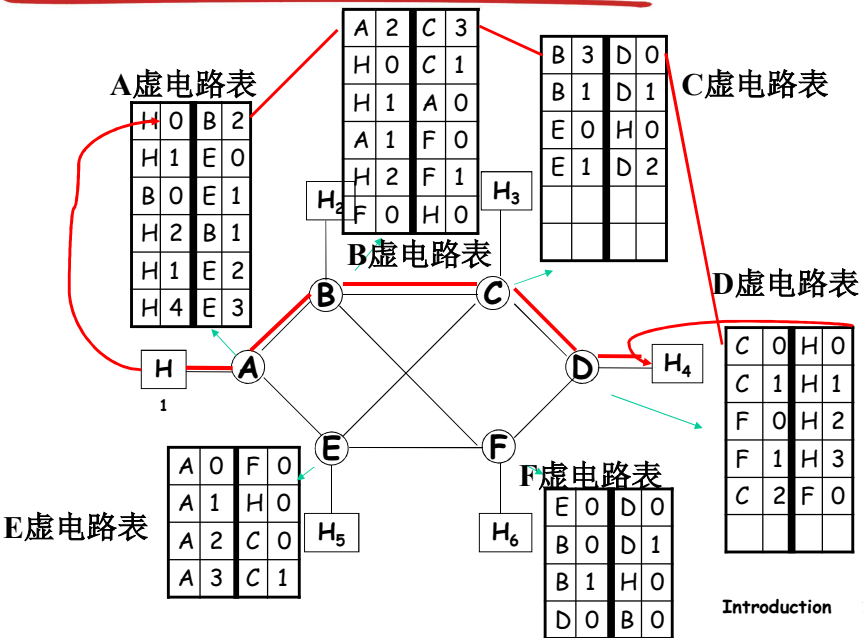
中科大信院2021秋

数据报(datagram) 的工作原理

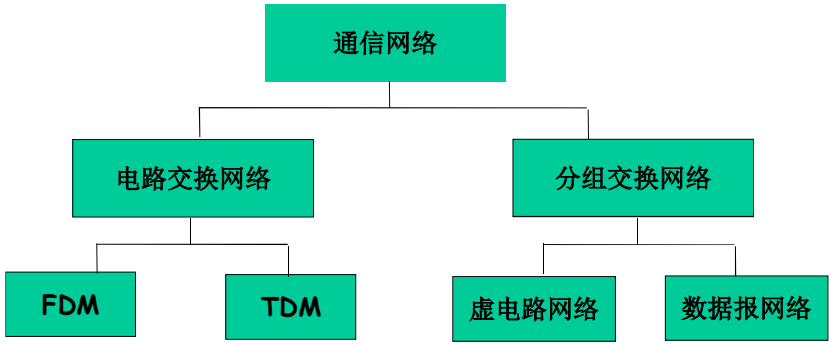
- 在通信之前,无须建立起一个连接,有数据就传输
- 每一个分组都独立路由(路径不一样,可能会失序)
- 路由器根据分组的目标地址进行路由



虚电路(virtual circuit)的工作原理



网络分类



第1章：提纲

1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

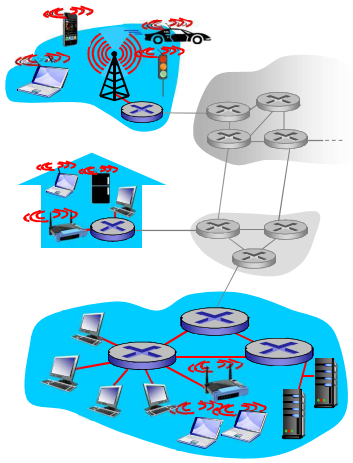
接入网络和物理媒体

Q: 怎样将端系统和边缘路由器连接？

- 住宅接入网络
- 单位接入网络 （学校、公司）
- 无线接入网络

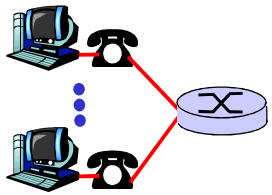
注意：

- 接入网络的带宽 (bits per second) ？
- 共享/专用？



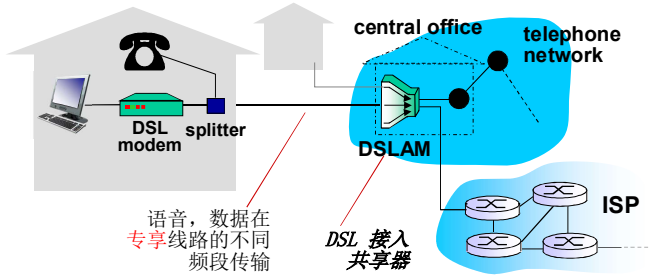
住宅接入：modem

- 将上网数据调制加载音频信号上，在电话线上传输，在局端将其中的数据解调出来；反之亦然
  - 调频
  - 调幅
  - 调相位
  - 综合调制
- 拨号调制解调器
  - 56Kbps 的速率直接接入路由器(通常更低)
  - 不能同时上网和打电话：不能总是在线



中科大信院2021秋

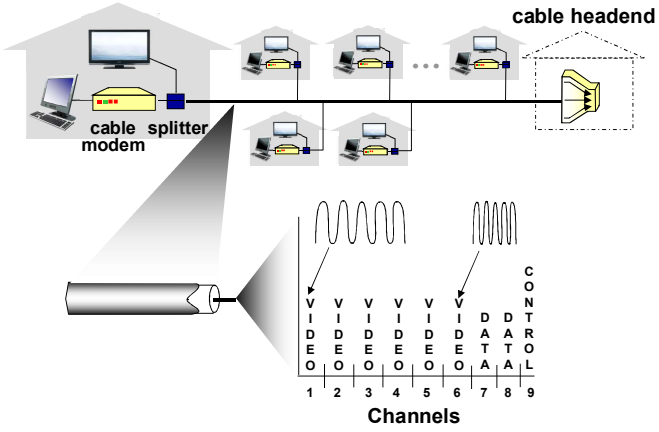
接入网：digital subscriber line (DSL)



- 采用现存的到交换局DSLAM的电话线
  - DSL线路上的数据被传到互联网
  - DSL线路上的语音被传到电话网
- < 2.5 Mbps上行传输速率(typically < 1 Mbps)
- < 24 Mbps下行传输速率(typically < 10 Mbps)

中科大信院2021秋

接入网：线缆网络

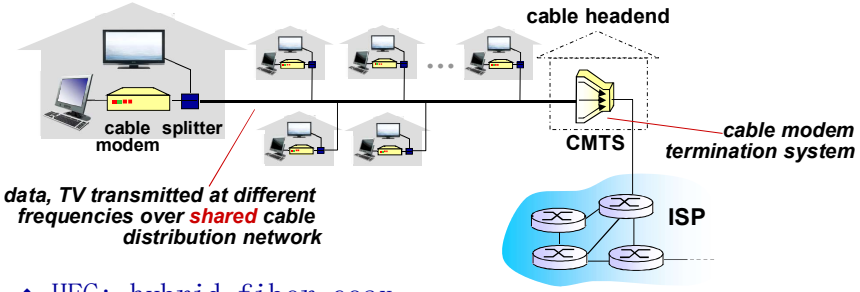


有线电视信号线缆双向改造

FDM: 在不同频段传输不同信道的数据，数字电视和上网数据（上下行）

中科大信院2021秋

接入网：线缆网络



- HFC: hybrid fiber coax
  - 非对称：最高30Mbps的下行传输速率，2 Mbps 上行传输速率
- 线缆和光纤网络将个家庭用户接入到 ISP 路由器
- 各用户共享到线缆头端的接入网络
  - 与DSL不同， DSL每个用户一个专用线路到CO（central office）

住宅接入：电缆模式

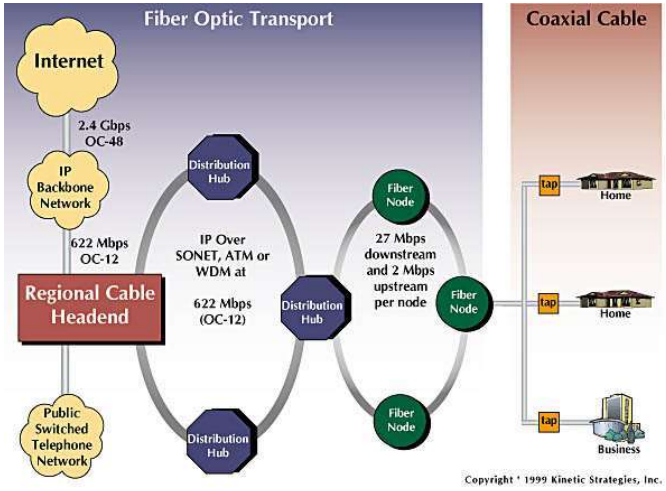
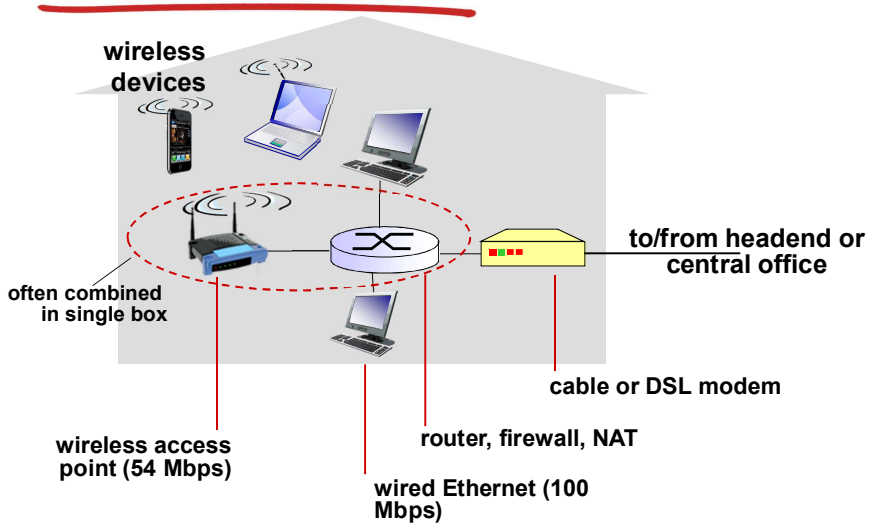
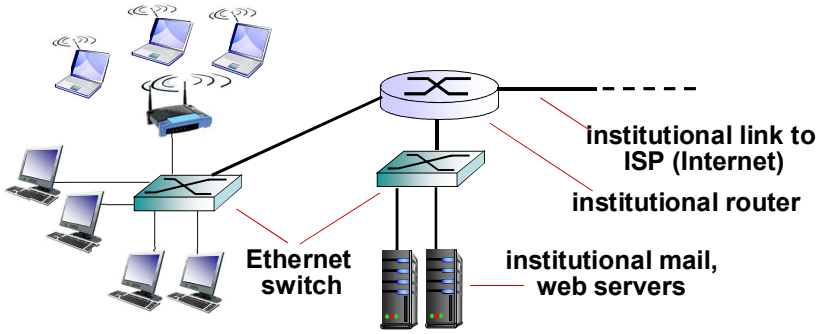


Diagram: http://www.cabledatacomnews.com/cmic/diagram.html

接入网：家庭网络



企业接入网络(Ethernet)



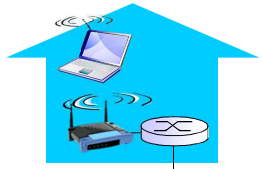
- 经常被企业或者大学等机构采用
  - 10 Mbps，100Mbps，1Gbps，10Gbps传输率
  - 现在，端系统经常直接接到以太网络交换机上

无线接入网络

- 各无线端系统共享无线接入网络（端系统到无线路由器）
  - 通过基站或者叫接入点

无线LANs：

- 建筑物内部（100 ft）
- 802.11b/g (WiFi)：11，54 Mbps 传输速率



广域无线接入

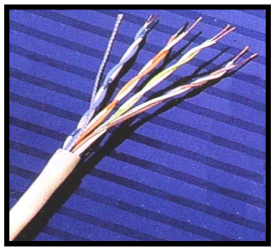
- 由电信运营商提供（cellular），10’s km
- 1 到 10 Mbps
- 3G，4G：LTE

物理媒体

- Bit: 在发送-接收对间传播
- 物理链路：连接每个发送-接收对之间的物理媒体
- 导引型媒体:
  - 信号沿着固体媒介被导引：同轴电缆、光纤、 双绞线
- 非导引型媒体：
  - 开放的空间传输电磁波或者光信号，在电磁或者光信号中承载数字数据

双绞线 (TP)

- 两根绝缘铜导线拧合
  - 5类：100Mbps 以太网，Gbps 千兆位以太网
  - 6类：10Gbps万兆以太网



物理媒体：同轴电缆、光纤

同轴电缆：

- 两根同轴的铜导线
- 双向
- 基带电缆：
  - 电缆上一个单个信道
  - Ethernet
- 宽带电缆：
  - 电缆上有多个信道
  - HFC

光纤和光缆：

- 光脉冲，每个脉冲表示一个bit，在玻璃纤维中传输
- 高速：
  - 点到点的高速传输（如10 Gps-100Gbps传输速率 ）
- 低误码率：在两个中继器之间可以有很长的距离，不受电磁噪声的干扰
- 安全

物理媒介：无线链路

- 开放空间传输电磁波，携带要传输的数据
- 无需物理“线缆”
- 双向
- 传播环境效应：
  - 反射
  - 吸收
  - 干扰

无线链路类型:

- 地面微波
  - e.g. up to 45 Mbps channels
- LAN (e.g., WiFi)
  - 11Mbps, 54 Mbps, 540Mbps…
- wide-area (e.g., 蜂窝)
  - 3G cellular: ~ 几Mbps
  - 4G 10Mbps
  - 5G 数Gbps
- 卫星
  - 每个信道Kbps 到45Mbps (或者多个聚集信道)
  - 270 msec端到端延迟
  - 同步静止卫星和低轨卫星

第1章：提纲

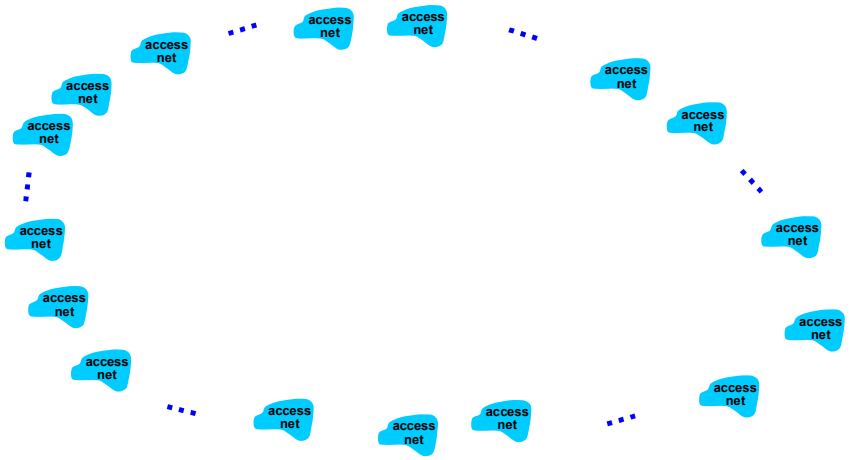
1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

互联网络结构：网络的网络

- 端系统通过接入ISPs (Internet Service Providers)连接到互联网
  - 住宅，公司和大学的ISPs
- 接入ISPs相应的必须是互联的
  - 因此任何2个端系统可相互发送分组到对方
- 导致的“网络的网络”非常复杂
  - 发展和演化是通过经济的和国家的政策来驱动的
- 让我们采用渐进方法来描述当前互联网的结构

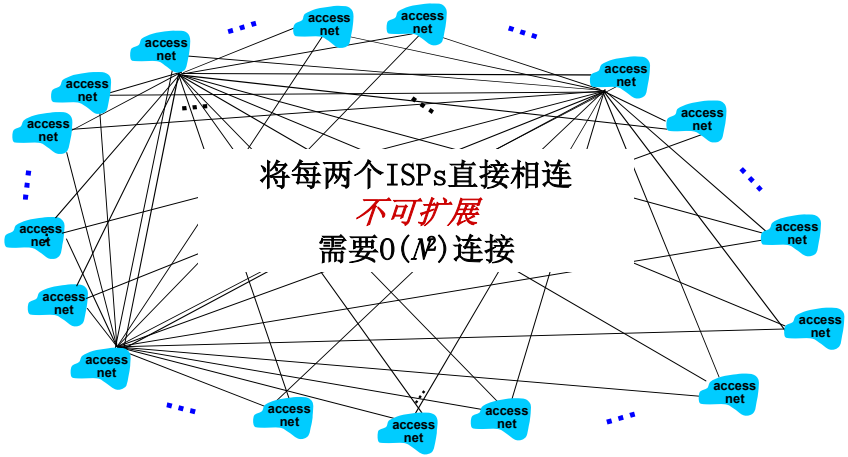
互联网络结构：网络的网络

问题： 给定数百万接入ISPs，如何将它们互联到一起



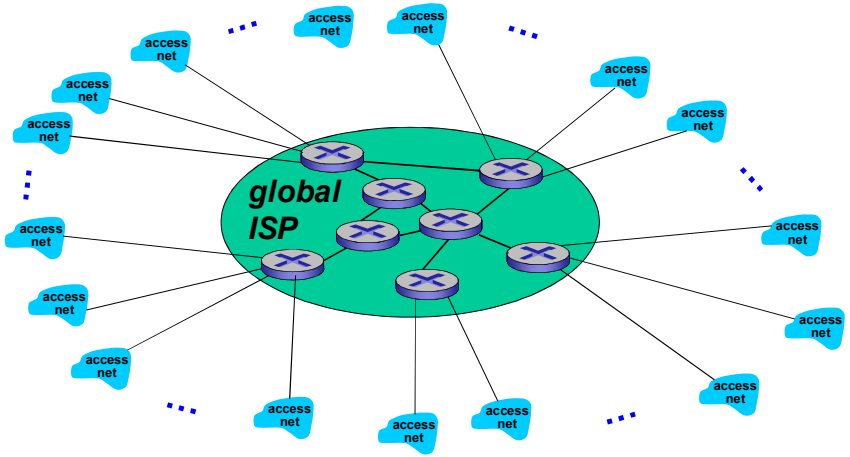
互联网络结构：网络的网络

问题： 给定数百万接入ISPs，如何将它们互联到一起



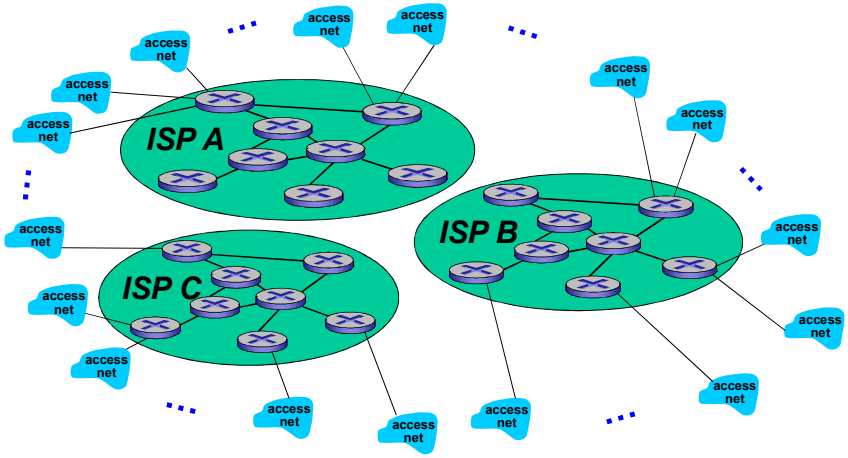
互联网络结构：网络的网络

选项：将每个接入ISP都连接到全局ISP（全局范围内覆盖）？
客户ISPs和提供者ISPs有经济合约



互联网络结构：网络的网络

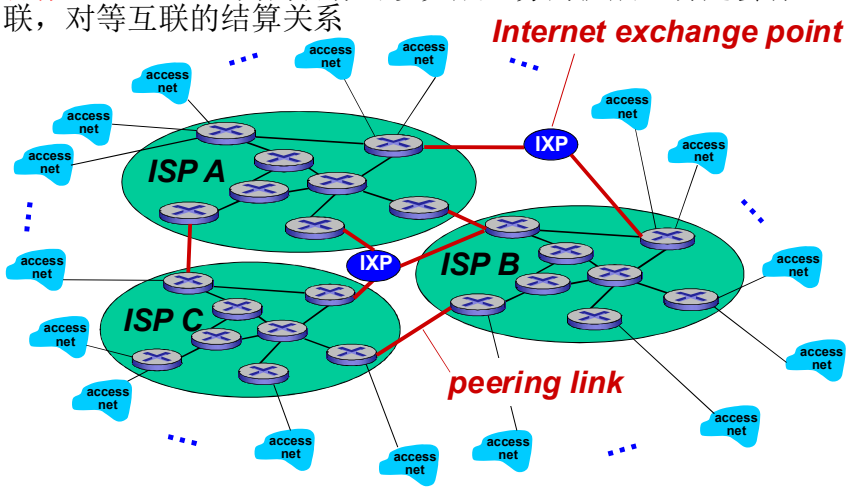
但是，如果全局ISP是可行的业务，那会有竞争者
有利可图，一定会有竞争



中科大信院2021秋

互联网络结构：网络的网络

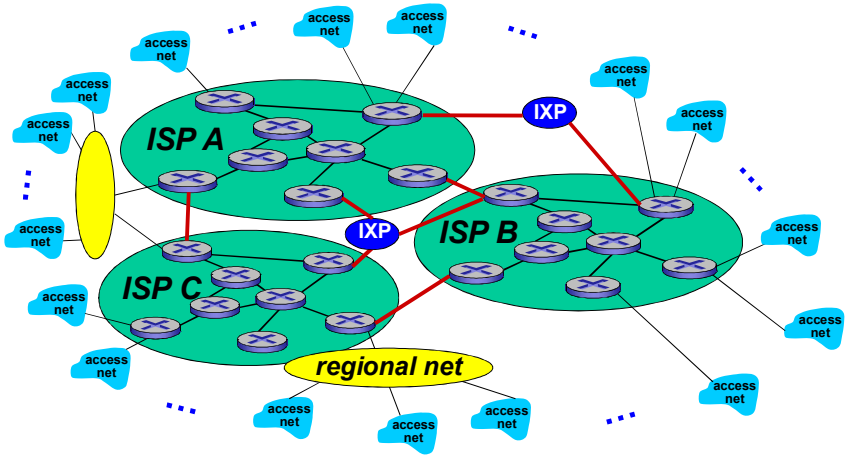
竞争：但如果全局ISP是有利可为的业务，那会有竞争者
合作：通过ISP之间的合作可以完成业务的扩展，肯定会有互联，对等互联的结算关系



中科大信院2021秋

互联网络结构：网络的网络

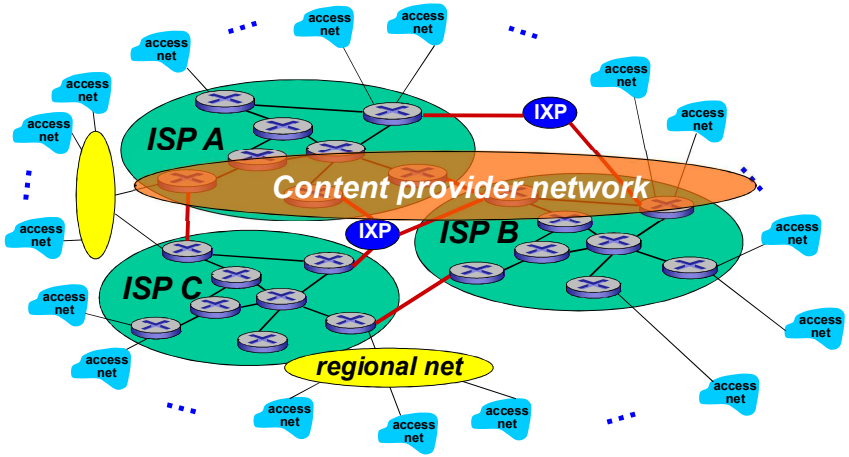
…然后业务会细分（全球接入和区域接入），区域网络将出现，用与将接入ISPs连接到全局ISPs



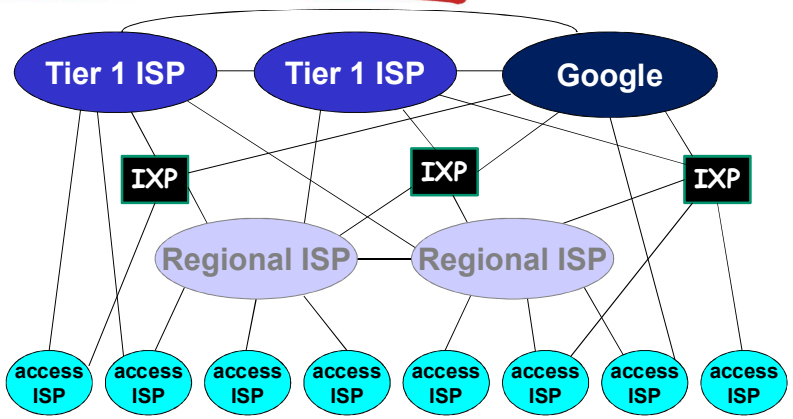
中科大信院2021秋

互联网络结构：网络的网络

然后内容提供商网络（Internet Content Providers, e.g., Google, Microsoft, Akamai）可能会构建它们自己的网络，将它们的服务、内容更加靠近端用户，向用户提供更好的服务，减少自己的运营支出



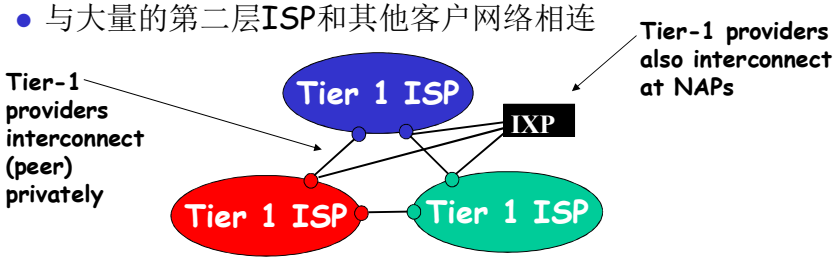
互联网络结构：网络的网络



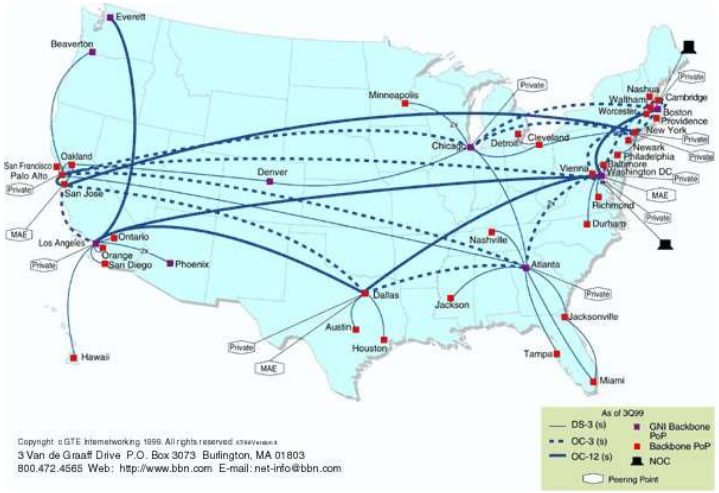
- 在网络的最中心，一些为数不多的充分连接的大范围网络（分布广、节点有限、但是之间有着多重连接）
  - “tier-1” commercial ISPs (e.g., Level 3, Sprint, AT&T, NTT)，国家或者国际范围的覆盖
  - content provider network (e.g., Google)：将它们的数据中心接入ISP，方便周边用户的访问；通常私有网络之间用专网绕过第一层ISP和区域ISPs

Internet结构：network of networks

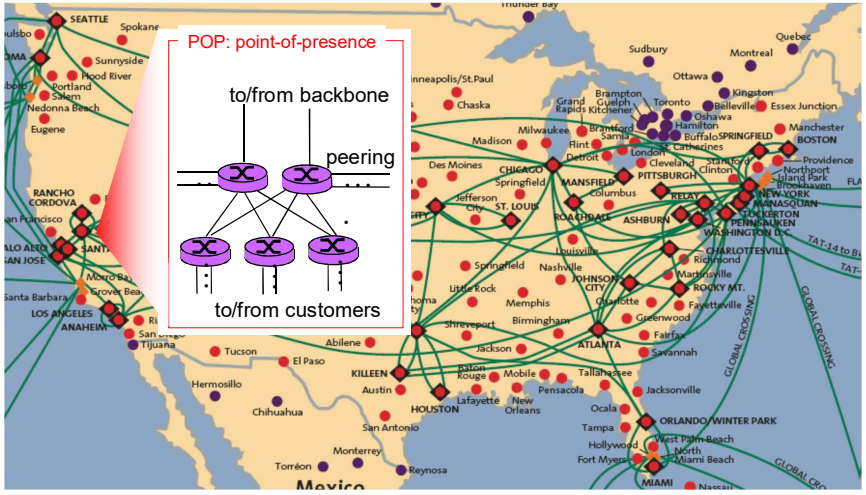
- 松散的层次模型
- 中心：第一层ISP（如UUNet, BBN/Genuity, Sprint, AT&T）国家/国际覆盖，速率极高
  - 直接与其他第一层ISP相连
  - 与大量的第二层ISP和其他客户网络相连



e.g. BBN/GTE US backbone network

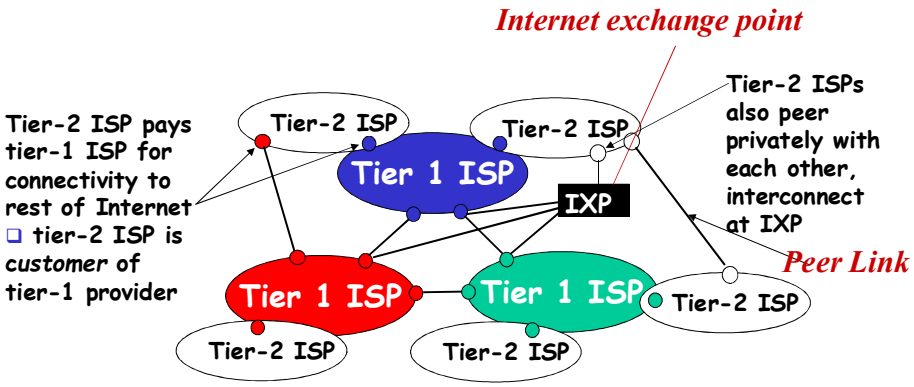


Tier-1 ISP: e.g., Sprint



Internet 结构：network of networks

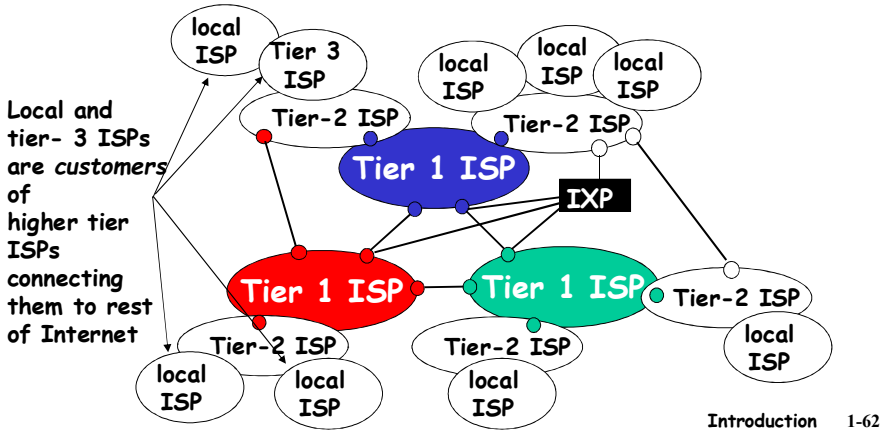
- 第二层ISP: 更小些的 (通常是区域性的) ISP
  - 与一个或多个第一层ISPs，也可能与其他第二层ISP



中科大信院2021秋

Internet结构：network of networks

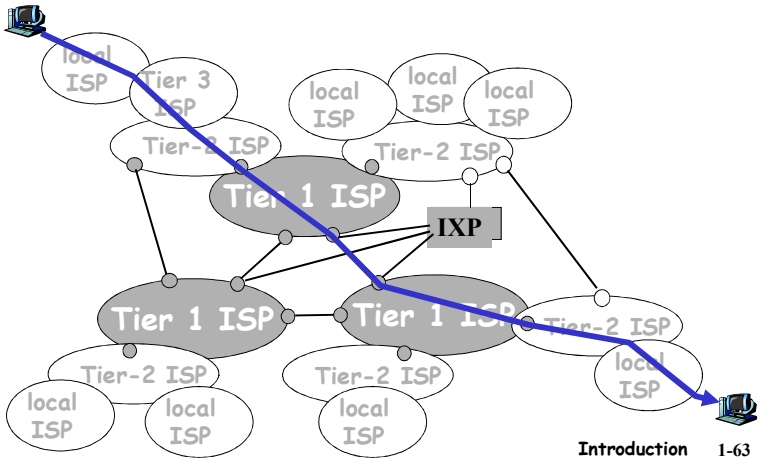
- 第三层ISP与其他本地ISP
  - 接入网 (与端系统最近)



中科大信院2021秋

Internet结构：network of networks

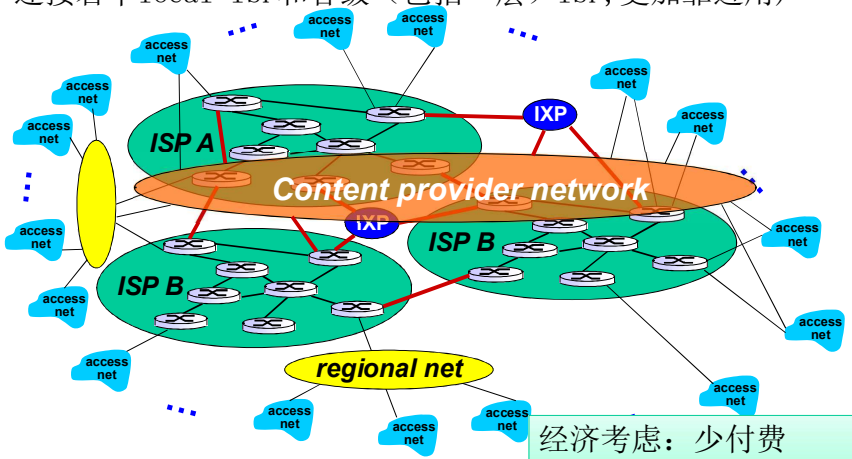
- 一个分组要经过许多网络！



中科大信院2021秋

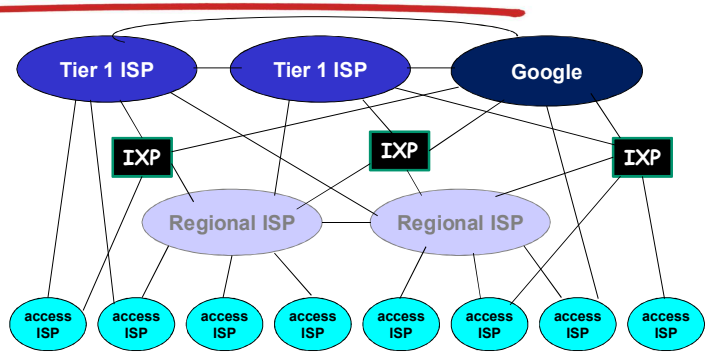
互联网结构：网络的网络

- 很多内容提供商(如：Google, Akamai)可能会部署自己的网络,连接自己的在各地的DC（数据中心），走自己的数据
- 连接若干local ISP和各级（包括一层）ISP,更加靠近用户



经济考虑：少付费
用户体验考虑：更快

ISP之间的连接



- POP: 高层ISP面向客户网络的接入点，涉及费用结算
  - 如一个低层ISP接入多个高层ISP，多宿（multi home）
- 对等接入：2个ISP对等互接，不涉及费用结算
- IXP：多个对等ISP互联互通之处，通常不涉及费用结算
  - 对等接入
- ICP自己部署专用网络，同时和各级ISP连接

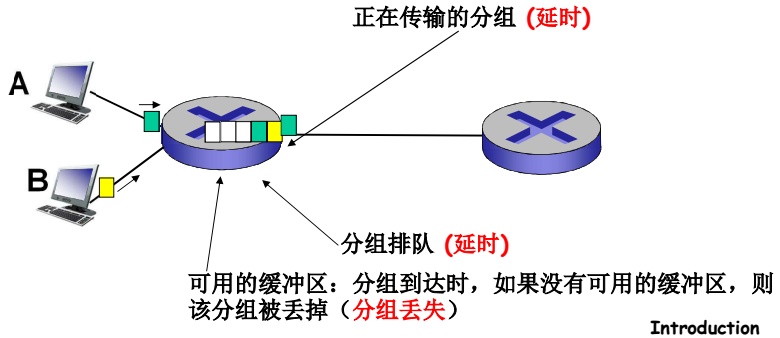
第1章：提纲

1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

分组丢失和延时是怎样发生的？

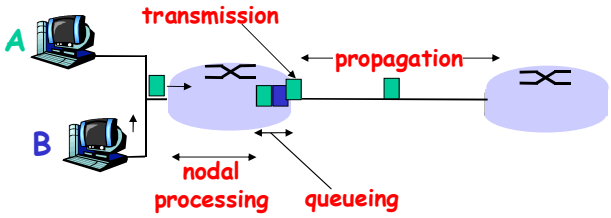
在路由器缓冲区的分组队列

- 分组到达链路的速率超过了链路输出的能力
- 分组等待排到队头、被传输



四种分组延时

- 1. 节点处理延时：
  - 检查 bit级差错
  - 检查分组首部和决定将分组导向何处
- 2. 排队延时
  - 在输出链路上等待传输的时间
  - 依赖于路由器的拥塞程度

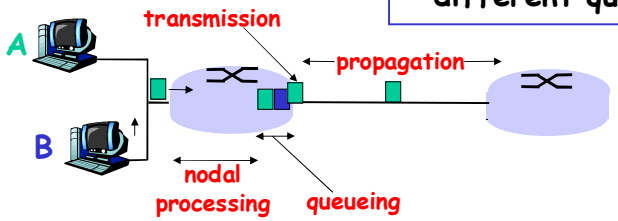


四种分组延时

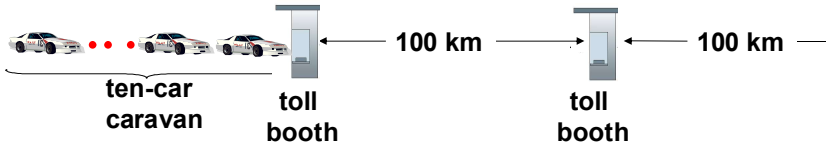
3. 传输延时:
- R=链路带宽(bps)
- L=分组长度(bits)
- 将分组发送到链路上的时间= L/R
- 存储转发延时

4. 传播延时:
- d = 物理链路的长度
- s = 在媒体上的传播速度 (~2x10^8 m/sec)
- 传播延时 = d/s

Note: s and R are very different quantities!

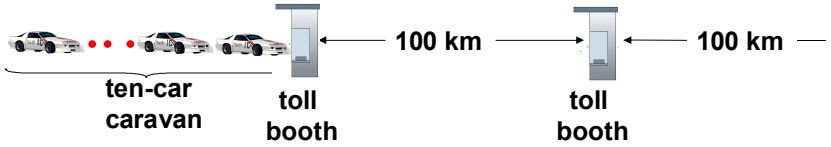


车队类比



- 汽车以100 km/hr 的速度传播
- 收费站服务每辆车需 12s(传输时间)
- 汽车~bit; 车队 ~ 分组
- Q: 在车队在第二个收费站排列好之前需要多长时间?
  - 即：从车队的第一辆车到达第一个收费站开始计时，到这个车队的最后一辆车离开第二个收费站，共需要多少时间
- 将车队从收费站输送到公路上的时间 = 12*10 = 120s
- 最后一辆车从第一个收费站到第二个收费站的传播时间：100km/(100km/hr)= 1 hr
- A: 62 minutes

车队类比（续）



- 汽车以1000 km/hr 的速度传播汽车
- 收费站服务每辆车需 1分钟
- Q:在所有的汽车被第一个收费站服务之前，汽车会到达第二个收费站吗?

- Yes！ 7分钟后，第一辆汽车到达了第二个收费站，而第一个收费站仍有3辆汽车
- 在整个分组被第一个路由器传输之前，第一个比特已经到达了第二个路由器！

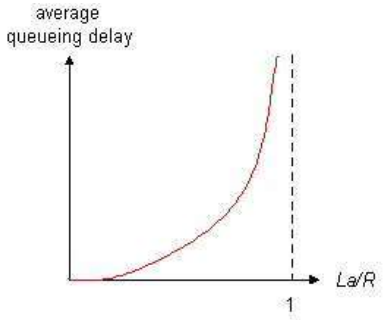
节点延时

$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

- d_{proc} = 处理延时
  - 通常是微秒数量级或更少
- d_{queue} = 排队延时
  - 取决于拥塞程度
- d_{trans} = 传输延时
  - = L/R, 对低速率的链路而言很大（如拨号），通常为微秒级到毫秒级
- d_{prop} = 传播延时
  - 几微秒到几百毫秒

排队延时

- R=链路带宽 (bps)
- L=分组长度 (bits)
- a=分组到达队列的平均速率



流量强度 = La/R

- La/R ~ 0: 平均排队延时很小
- La/R -> 1: 延时变得很大
- La/R > 1: 比特到达队列的速率超过了从该队列输出的速率，平均排队延时将趋向无穷大！

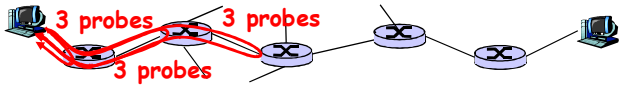
La/R ~ 0

La/R -> 1

设计系统时流量强度不能大于1！

Internet的延时和路由

- Internet 的延时和路由是什么样的呢?
- Traceroute 诊断程序: 提供从源端，经过路由器，到目的的延时测量
  - For all *i*:
    - 沿着目的的路径，向每个路由器发送3个探测分组
    - 路由器 *i* 将向发送方返回一个分组
    - 发送方对发送和回复之间间隔计时



Internet的延时和路由

traceroute: gaia.cs.umass.edu to www.eurecom.fr

Three delay measements from gaia.cs.umass.edu to cs-gw.cs.umass.edu

```
1  cs-gw (128.119.240.254)  1 ms  1 ms  2 ms
2  border1-rt-fa5-1-0.gw.umass.edu (128.119.3.145)  1 ms  1 ms  2 ms
3  cht-vbns.gw.umass.edu (128.119.3.130)  6 ms 5 ms 5 ms
4  jn1-at1-0-0-19.wor.vbns.net (204.147.132.129)  16 ms 11 ms 13 ms
5  jn1-so7-0-0-0.wae.vbns.net (204.147.136.136)  21 ms 18 ms 18 ms
6  abilene-vbns.abilene.ucaid.edu (198.32.11.9)  22 ms  18 ms  22 ms
7  nycm-wash.abilene.ucaid.edu (198.32.8.46)  22 ms  22 ms  22 ms
8  62.40.103.253 (62.40.103.253)  104 ms 109 ms 106 ms
9  de2-1.de1.de.geant.net (62.40.96.129)  109 ms 102 ms 104 ms
10  de.fr1.fr.geant.net (62.40.96.50)  113 ms 121 ms 114 ms
11  renater-gw.fr1.fr.geant.net (62.40.103.54)  112 ms  114 ms  112 ms
12  nio-n2.cssi.renater.fr (193.51.206.13)  111 ms  114 ms  116 ms
13  nice.cssi.renater.fr (195.220.98.102)  123 ms  125 ms  124 ms
14  r3t2-nice.cssi.renater.fr (195.220.98.110)  126 ms  126 ms  124 ms
15  eurecom-valbonne.r3t2.ft.net (193.48.50.54)  135 ms  128 ms  133 ms
16  194.214.211.25 (194.214.211.25)  126 ms  128 ms  126 ms
17  * * *
18  * * *
19  fantasia.eurecom.fr (193.55.113.142)  132 ms  128 ms  136 ms
```

trans-oceanic link

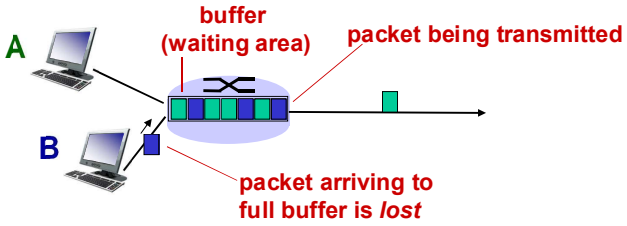
* means no reponse (probe lost, router not replying)

关于traceroute

- 在Windows系统下
  - **Tracerert** hostname
    - 如 **Tracerert** www.gucas.ac.cn
  - 更完整的例子
    - tracert [-d] [-h maximum_hops] [-j computer-list] [-w timeout] target_name
    - 请见帮助: http://www.linkwan.com/gb/broadmeter/article/trace-help.htm
- 测试网址:
  - www.traceroute.org
  - www.linkwan.com

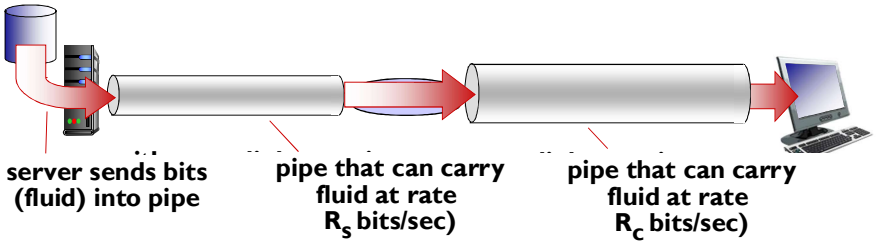
分组丢失

- 链路的队列缓冲区容量有限
- 当分组到达一个满的队列时，该分组将会丢失
- 丢失的分组可能会被前一个节点或源端系统重传，或根本不重传



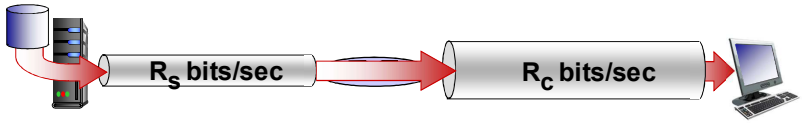
吞吐量

- *吞吐量：* 在源端和目标端之间传输的速率（数据量/单位时间）
  - *瞬间吞吐量:* 在一个时间点的速率
  - *平均吞吐量:* 在一个长时间内平均值

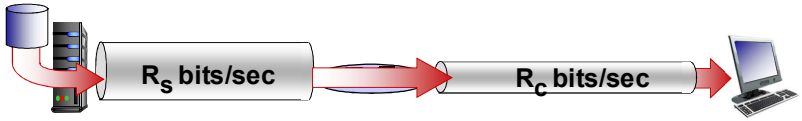


吞吐量

- $R_s < R_c$ 端到端平均吞吐是多少?



- $R_s > R_c$ 端到端平均吞吐是多少?
- ?



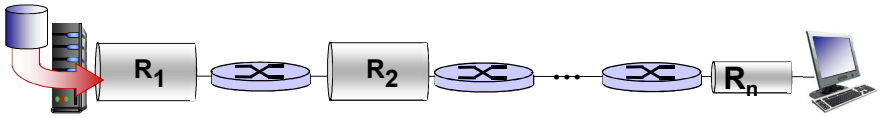
瓶颈链路

端到端路径上，限制端到端吞吐的链路

其他节点都不传输，吞吐量min{Rs,Rc}

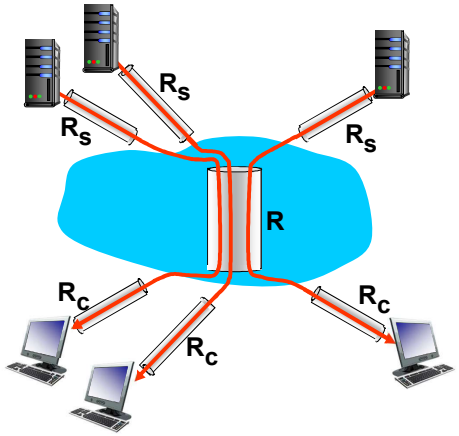
吞吐量

- 端到端平均吞吐=min{R_1，R_2, …, R_n}



吞吐量：互联网场景

- 链路上的每一段实际可用带宽R_i’ =?
- 端到端吞吐量：$\min\{R_i'\}$
- 每个连接上的端到端吞吐：$\min(R_c, R_s, R/10)$
- 实际上：R_c 或者 R_s 经常是瓶颈



10 个连接(公平地) 共享Rbps的瓶颈链路

第1章：提纲

1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

协议层次

网络是一个复杂的系统!

- 网络功能繁杂：数字信号的物理信号承载、点到点、路由、rdt、进程区分、应用等
- 现实来看，网络的许多构成元素和设备:
  - 主机
  - 路由器
  - 各种媒体的链路
  - 应用
  - 协议
  - 硬件, 软件

问题是：

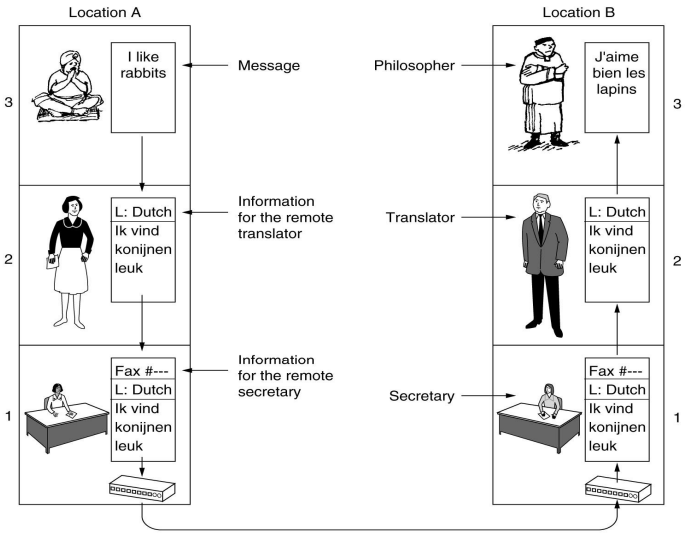
如何组织和实现这个复杂的网络功能？

现实生活中的例子：航线系统

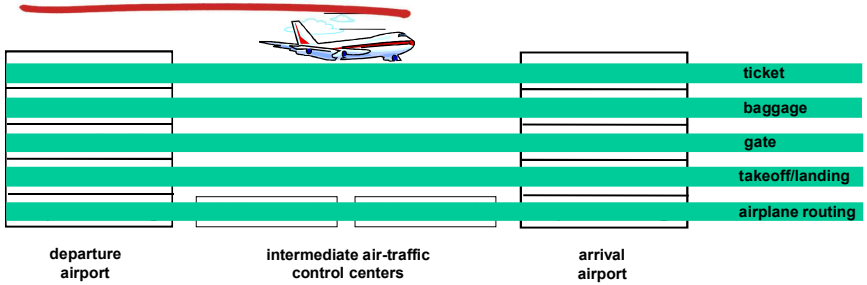
票务 (购买)	票务 (投诉)
行李 (托运)	行李 (认领)
登机口 (登机)	登机口 (离机)
从跑道起飞	从跑道着陆
按航线飞行	按航线飞行

按航线飞行

例子2:两位异地哲学家的交流



航线的功能层次



层次化方式实现复杂网络功能:

- 将网络复杂的功能分层功能明确的层次，每一层实现了其中一个或一组功能，功能中有其上层可以使用的功能：服务
- 本层协议实体相互交互执行本层的协议动作，目的是实现本层功能，通过接口为上层提供更好的服务
- 在实现本层协议的时候，直接利用了下层所提供的服务
- 本层的服务：借助下层服务实现的本层协议实体之间交互带来的新功能（上层可以利用的）+更下层所提供的服务

服务和服务访问点

- 服务(Service)：低层实体向上层实体提供它们之间的通信的能力
  - 服务用户(service user)
  - 服务提供者(service provider)
- 原语(primitive)：上层使用下层服务的形式，高层使用低层提供的服务，以及低层向高层提供服务都是通过服务访问原语来进行交互的---形式
- 服务访问点 SAP (Services Access Point)：上层使用下层提供的服务通过层间的接口—地点；
  - 例子:邮箱
  - 地址(address)：下层的一个实体支撑着上层的多个实体，SAP有标志不同上层实体的作用
  - 可以有不同的实现，队列
  - 例子:传输层的SAP: 端口(port)

服务的类型

- 面向连接的服务和无连接的服务-方式
  - 面向连接的服务(Connection-oriented Service)
    - 连接(Connection)：两个通信实体为进行通信而建立的一种结合
    - 面向连接的服务通信的过程：建立连接，通信，拆除连接
    - 面向连接的服务的例子：网络层的连接被成为虚电路
    - 适用范围：对于大的数据块要传输; 不适合小的零星报文
    - 特点：保序
    - 服务类型:
      - 可靠的信息流 传送页面(可靠的获得,通过接收方的确认)
      - 可靠的字节流 远程登录
      - 不可靠的连接 数字化声音

服务的类型

- 面向连接的服务和无连接的服务
  - 无连接的服务(Connectionless Service)
    - 无连接服务：两个对等层实体在通信前不需要建立一个连接，不预留资源；不需要通信双方都是活跃；(例：寄信)
    - 特点：不可靠、可能重复、可能失序
    - IP分组，数据包；
    - 适用范围：适合传送零星数据；
    - 服务类型：
      - 不可靠的数据报 电子方式的函件
      - 有确认的数据报 挂号信
      - 请求回答 信息查询

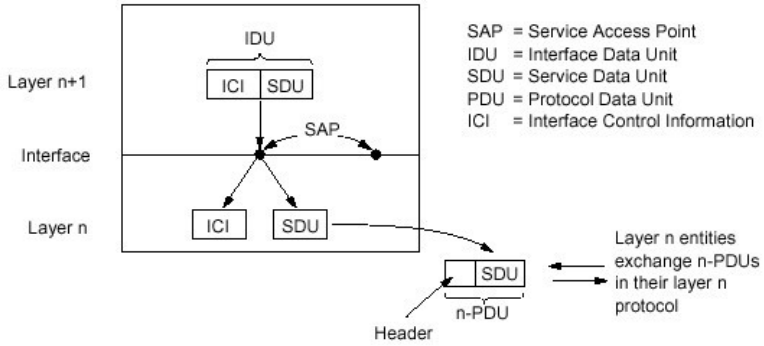
中科大信院2021秋

服务和协议

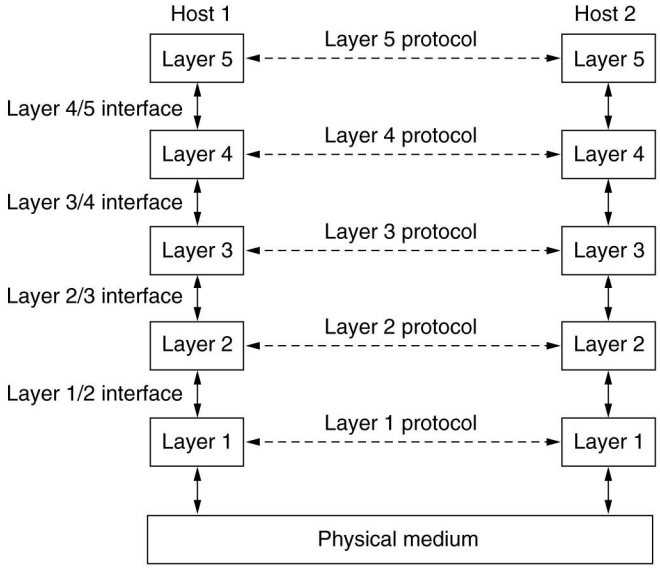
- 服务与协议的区别
  - 服务(Service)：低层实体向上层实体提供它们之间的通信的能力，是通过原语(primitive)来操作的，垂直
  - 协议(protocol)：对等层实体(peer entity)之间在相互通信的过程中，需要遵循的规则的集合，水平
- 服务与协议的联系
  - 本层协议的实现要靠下层提供的服务来实现
  - 本层实体通过协议为上层提供更高级的服务

中科大信院2021秋

数据单元(DU)



中科大信院2021秋



分层处理和实现复杂系统的好处？

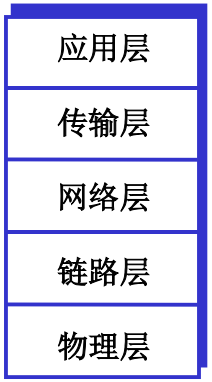
对付复杂的系统

- 概念化：结构清晰，便于标示网络组件，以及描述其相互关系
  - 分层参考模型
- 结构化：模块化更易于维护和系统升级
  - 改变某一层服务的实现不影响系统中的其他层次
    - 对于其他层次而言是透明的
  - 如改变登机程序并不影响系统的其它部分
    - 改变2个秘书使用的通信方式不影响2个翻译的工作
    - 改变2个翻译使用的语言也不影响上下2个层次的工作
- 分层思想被认为有害的地方？

中科大信院2021秋

Internet 协议栈

- 应用层：网络应用
  - 为人类用户或者其他应用进程提供网络应用服务
  - FTP, SMTP, HTTP,DNS
- 传输层：主机之间的数据传输
  - 在网络层提供的端到端通信基础上，细分为进程到进程，将不可靠的通信变成可靠地通信
  - TCP, UDP
- 网络层：为数据报从源到目的选择路由
  - 主机主机之间的通信，端到端通信，不可靠
  - IP, 路由协议
- 链路层：相邻网络节点间的数据传输
  - 2个相邻2点的通信，点到点通信，可靠或不可靠
  - 点对对协议PPP, 802.11(wifi), Ethernet
- 物理层：在线路上传送bit



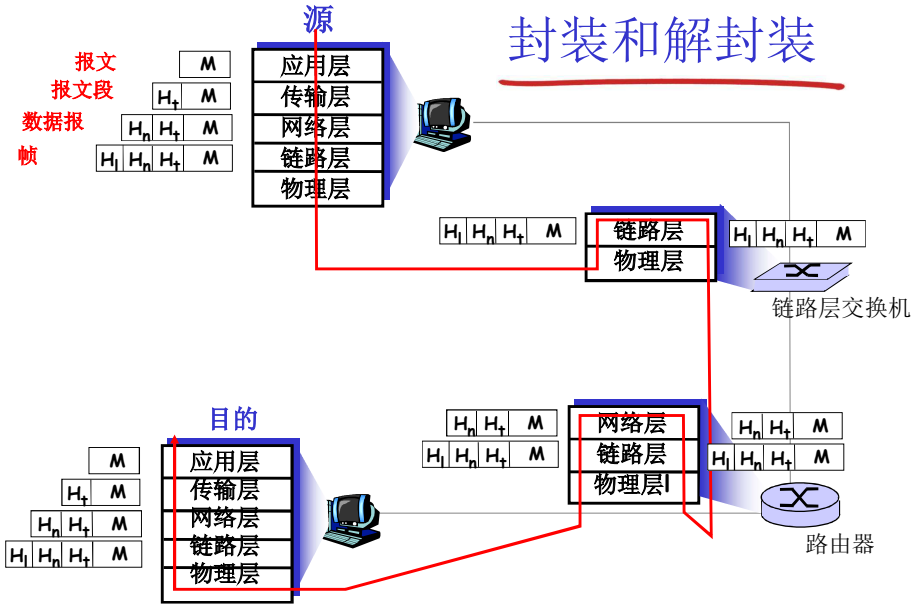
中科大信院2021秋

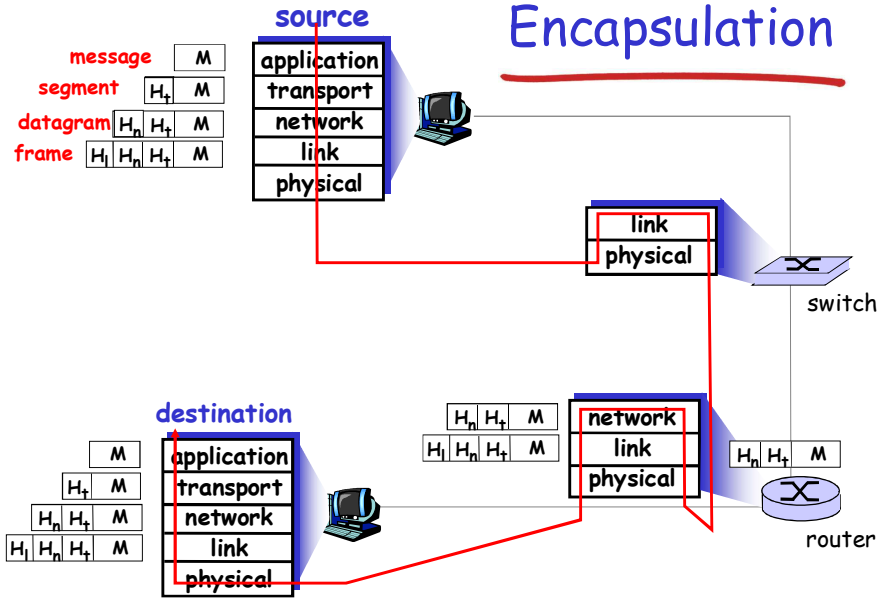
ISO/OSI 参考模型

- *表示层：* 允许应用解释传输的数据，e.g.，加密，压缩，机器相关的表示转换
- *会话层：* 数据交换的同步，检查点，恢复
- 互联网协议栈没有这两层！
  - 这些服务，如果需要的话，必须被应用实现
  - 需要吗?

中科大信院2021秋

封装和解封装





中科大信院2021秋

各层次的协议数据单元

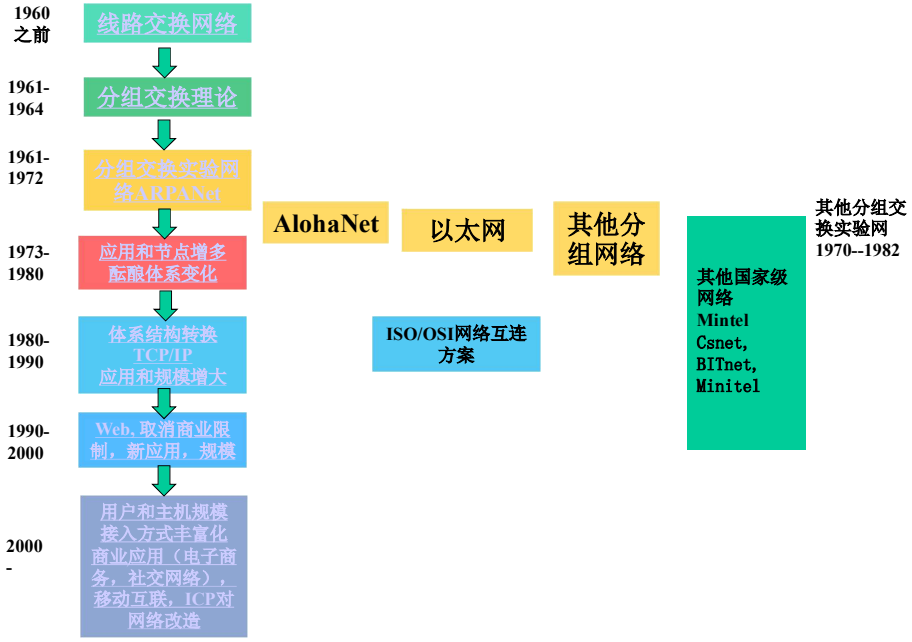
- 应用层：报文(message)
- 传输层：报文段(segment)：TCP段，UDP数据报
- 网络层：分组packet（如果无连接方式：数据报datagram）
- 数据链路层：帧(frame)
- 物理层：位(bit)

中科大信院2021秋

第1章：提纲

1.1 什么是Internet?
1.2 网络边缘
1.3 网络核心
1.4 接入网和物理媒体
1.5 Internet结构和ISP
1.6 分组延时、丢失和吞吐量
1.7 协议层次及服务模型
1.8 历史

中科大信院2021秋



早期（1960以前）计算机网络

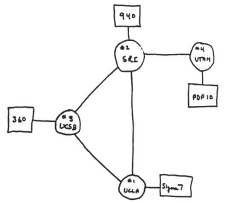
- 线路交换网络
- 线路交换的特性使得其不适合计算机之间的通信
  - 线路建立时间过长
  - 独享方式占用通信资源，不适合突发性很强的计算机之间的通信
  - 可靠性不高：非常不适合军事通信
- 三个小组独立地开展分组交换的研究
  - 1961: Kleinrock(MIT)，排队论，展现了分组交换的有效性
  - 1964: Baran(美国兰德公司) - 军用网络上的分组交换
  - 1964：Donald（英国）等，NPL

返回

Internet 历史

1961-1972: 早期的分组交换概念

- 1967: 美国高级研究计划研究局考虑ARPAnet
  - Kleinrock在MIT的同事
- 1969: 第一个 ARPAnet 节点开始工作,UCLA
  - IMP:接口报文处理机
- 1969年底: 4个节点



THE ARPA NETWORK

- 1972:
  - ARPAnet 公众演示
  - 网络控制协议是第一个端系统直接的主机-主机协议
    - NCP协议：相当于传输层和网络层在一起，支持应用开发
  - 第一个e-mail 程序（BBN）
  - ARPAnet有15个节点

返回

Internet历史

1972-1980: 专用网络和网络互联

- 出现了很多对以后来说重要的网络形式，雨后春笋
  - 1970: ALOHAnet,夏威夷上的微波网络
  - 1973: Metcalfe在博士论文中提出了Ethernet
  - ATM网络
  - ALOHAnet，Telenet，Cyclades法国等
- 1970后期，网络体系结构的必要性
  - 专用的体系结构： DECnet, SNA, XNA
  - 标准化的体系结构
- 1974: 网际互联的Cerf and Kahn 体系结构
- 1979: ARPAnet的规模在持续增加，体系结构也在酝酿着变化，以支持网络互联和其他目的（性能）需求
  - 节点数目增加，有200个节点

Cerf and Kahn 网络互联原则:
- 极简、自治
- 尽力而为（best effort）服务模型
- 无状态的路由器
- 分布控制

定义了今天的Internet体系结构

返回

Internet 历史

1980-1990: 体系结构变化，网络数量激增，应用丰富

- 1983: TCP/IP部署，标记日
  - NCP分化成2个层次，TCP/IP，从而出现UDP
  - 覆盖式IP解决网络互联问题
  - 主机设备和网络交换设备分开
- 1982: smtp e-mail协议定义
- 1983: DNS 定义，完成域名到IP地址的转换
- 1985: ftp 协议定义
- 1988: TCP拥塞控制
- 其他网络形式的发展
  - 新的国家级网络: Csnet, BITnet, NSFnet, Minitel
  - 1985年：ISO/OSI提出，时机不对且太繁琐，
- 100,000主机连接到网络联邦

返回

Internet历史

1990, 2000's: 商业化, Web, 新的应用

- 1990年代初: NSF对ARPAnet 的访问网，双主干，ARPAnet退役
- 1991: NSF放宽了对NSFnet用于商业目的的限制 (1995退役)，ASFNET非盈利性机构维护，后面叫Internet
- UNIX 中TCP/IP的**免费捆绑**
- 1990年代初: Web
  - hypertext [Bush 1945, Nelson 1960's]
  - HTML, HTTP: Berners-Lee
  - 1994: Mosaic (Netscape, andreesen)
  - 1990年代后期: Web的商业化

1990后期 – 21世纪:

- TCP/IP体系结构的**包容性**，在其上部署应用便捷，出现非常多的应用
- 新一代**杀手级应用**（即时讯息，P2P 文件共享，社交网络等）更进一步促进互联网的发展
- 安全问题不断出现和修订（互联网的补丁对策）
- 2001网络泡沫，使得一些好公司沉淀下来（谷歌，微软，苹果，Yahoo，思科）
- 主干网的速率达到Gbps

返回

互联网历史

2005–现在

- ~50+亿主机：包括智能手机和平板
- 宽带接入的快速部署
- 高速无线接入无处不在：移动互联时代
  - 4G部署,5G蓄势待发
  - 带宽大，终端性能高，价格便宜，应用不断增多
- 在线社交网络等新型应用的出现:
  - Facebook：10亿用户
  - 微信，qq：数十亿用户
- 内容提供商 (Google, Microsoft)创建他们自己的网络
  - 通过自己的专用网络提供对搜索、视频内容和电子邮件的即刻访问
- 电子商务，大学，企业在云中运行他们的服务 (eg, Amazon EC2)
- 体系结构酝酿着大的变化，未来网络蠢蠢欲动

返回

小结

覆盖大量的材料!

- Internet
- 什么是协议
- 网络边缘，核心，接入网络
  - 分组交换 vs. 电路交换
- Internet/ISP 结构
- 性能: 丢失，延时，吞吐量
- 层次模型和服务模型
- 历史

小结1

- 组成角度看 什么是互联网
  - 边缘：端系统（包括应用）+接入网
  - 核心：网络交换设备+通信链路
  - 协议：对等层实体通信过程中遵守的规则的集合
    - 语法，语义，时序
- 为了实现复杂的网络功能，采用分层方式设计、实现和调试
  - 应用层，传输层，网络层，数据链路层，物理层
  - 协议数据单位：
    - 报文，报文段，分组，帧，位
- 从 服务角度看互联网
  - 通信服务基础设施
    - 提供的通信服务：面向连接 无连接
  - 应用
- 应用之间的交互
  - C/S模式
  - P2P模式

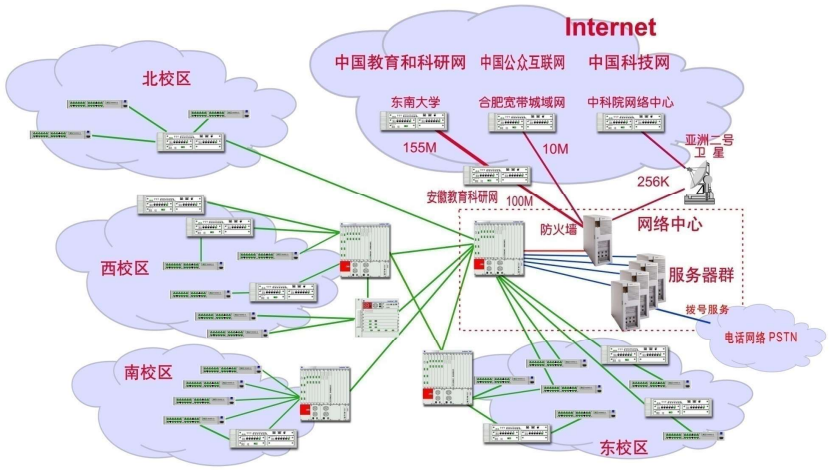
小结2

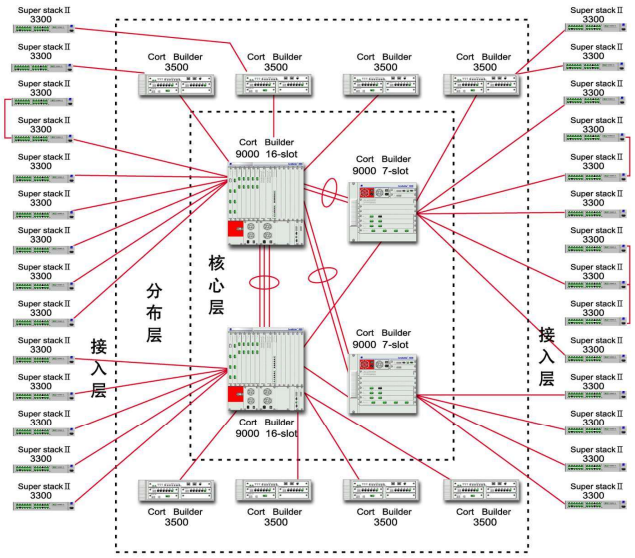
- 数据交换
  - 分组数据交换
  - 线路交换
- 比较 线路交换和分组交换
- 分组交换的2种方式
  - 虚电路
  - 数据报
- 接入网和物理媒介
  - 接入网技术：
    - 住宅：ADSL，拨号，cable modem
    - 单位：以太网
    - 无线接入方式
  - 物理媒介
    - 光纤，同轴电缆，以太网，双绞线
- ISP层次结构

小结3

- 分组交换网络中延迟和丢失是如何发生的
  - 延迟的组成：处理、传输、传播、排队
- 网络的分层体系结构
  - 分层体系结构
  - 服务
  - 协议数据单元
  - 封装与解封装
- 历史

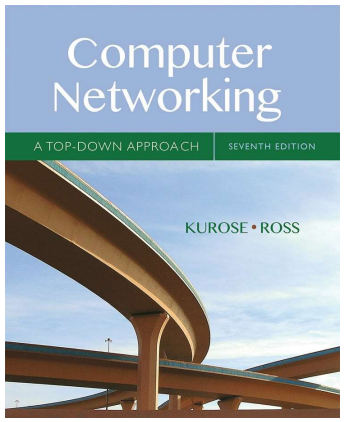
中国科大校园网络拓扑结构示意图





中国科大校园网主干改造方案物理结构图

第2章 应用层



Computer Networking: A Top Down Approach
7th edition
Jim Kurose, Keith Ross
Addison-Wesley
April 2016

中国科学技术大学
自动化系 郑烇
改编自Jim kurose, Keith Ross

第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 Email
SMTP
POP3
IMAP
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字（Socket）编程
2.9 UDP 套接字编程

第2章：应用层

目标：

- 网络应用的**原理**：网络应用协议的概念和实现方面
  - 传输层的服务模型
  - 客户-服务器模式
  - 对等模式(peer-to-peer)
  - 内容分发网络
- 网络应用的**实例**：互联网流行的应用层协议
  - HTTP
  - FTP
  - SMTP / POP3 / IMAP
  - DNS
- **编程**：网络应用程序
  - Socket API

一些网络应用的例子

- E-mail
- Web
- 文本消息
- 远程登录
- P2P文件共享
- 即时通信
- 多用户网络游戏
- 流媒体(YouTube, Hulu, Netflix)
- Internet 电话
- 实时电视会议
- 社交网络
- 搜索
- ……

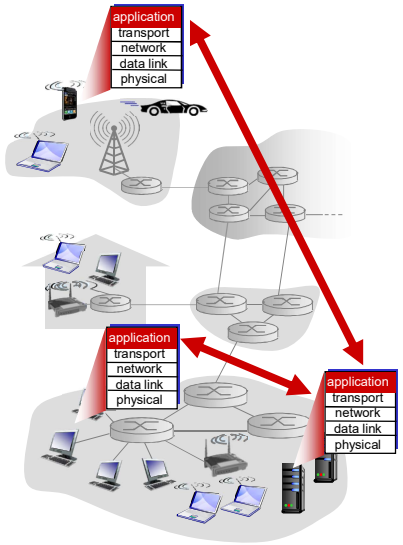
创建一个新的网络应用

编程

- 在不同的端系统上运行
- 通过网络基础设施提供的服务，应用进程彼此通信
- 如Web:
  - Web 服务器软件与浏览器软件通信

网络核心中没有应用层软件

- 网络核心没有应用层功能
- 网络应用只在端系统上存在，快速网络应用开发和部署



第2章：提纲

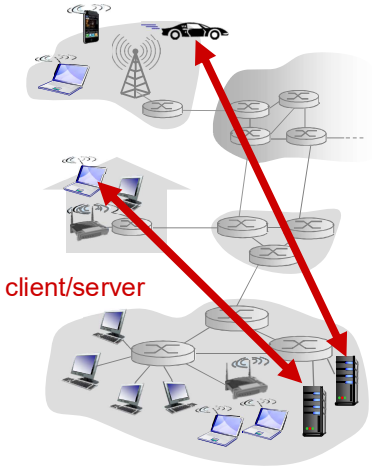
2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

网络应用的体系结构

可能的应用架构：

- 客户-服务器模式（C/S:client/server）
- 对等模式(P2P:Peer To Peer)
- 混合体：客户-服务器和对等体系结构

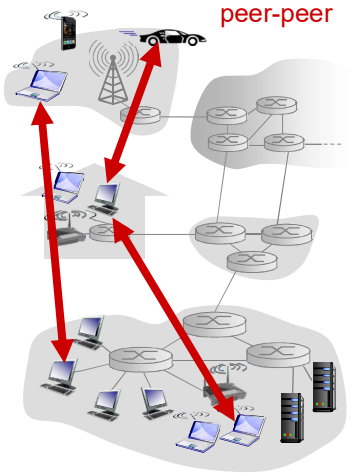
客户-服务器（C/S）体系结构



- 服务器:
  - 一直运行
  - 固定的IP地址和周知的端口号（约定）
  - 扩展性：服务器场
    - 数据中心进行扩展
    - 扩展性差
- 客户端:
  - 主动与服务器通信
  - 与互联网有间歇性的连接
  - 可能是动态IP 地址
  - 不直接与其它客户端通信

对等体（P2P）体系结构

- （几乎）没有一直运行的服务器
- 任意端系统之间可以进行通信
- 每一个节点既是客户端又是服务器
  - 自扩展性-新peer节点带来新的服务能力，当然也带来新的服务请求
- 参与的主机间歇性连接且可以改变IP 地址
  - 难以管理
- 例子: Gnutella，迅雷



C/S和P2P体系结构的混合体

Napster

- 文件搜索：集中
  - 主机在中心服务器上注册其资源
  - 主机向中心服务器查询资源位置
- 文件传输：P2P
  - 任意Peer节点之间

即时通信

- 在线检测：集中
  - 当用户上线时，向中心服务器注册其IP地址
  - 用户与中心服务器联系，以找到其在线好友的位置
- 两个用户之间聊天：P2P

进程通信

进程：在主机上运行的应用程序

- 在同一个主机内，使用进程间通信机制通信（操作系统定义）
- 不同主机，通过交换报文（Message）来通信
  - 使用OS提供的通信服务
  - 按照应用协议交换报文
    - 借助传输层提供的服务

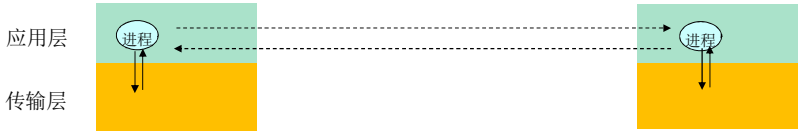
clients, servers

客户端进程：发起通信的进程

服务器进程：等待连接的进程

- 注意：P2P架构的应用也有客户端进程和服务器进程之分

分布式进程通信需要解决的问题

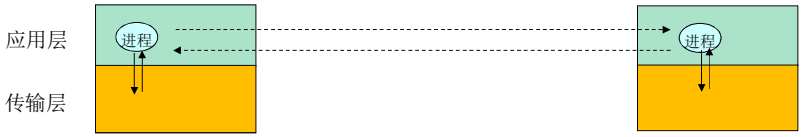


- 问题1：进程标示和寻址问题（服务用户）
- 问题2：传输层-应用层提供服务是如何（服务）
  - 位置：层间界面的SAP （TCP/IP：socket）
  - 形式：应用程序接口API （TCP/IP：socket API）
- 问题3：如何使用传输层提供的服务，实现应用进程之间的报文交换，实现应用（用户使用服务）
  - 定义应用层协议：报文格式，解释，时序等
  - 编制程序，使用OS提供的API，调用网络基础设施提供通信服务传报文，实现应用时序等；

问题1：对进程进行编址（addressing）

- 进程为了接收报文，必须有一个标识
  - 即：SAP（发送也需要标示）
  - 主机：唯一的 32位IP地址
    - 仅仅有IP地址不能够唯一标示一个进程；在一台端系统上有很多应用进程在运行
  - 所采用的传输层协议：TCP or UDP
  - 端口号（Port Numbers）
- 一些知名端口号的例子：
  - HTTP: TCP 80 Mail: TCP25 ftp:TCP 2
- 一个进程：用IP+port标示 端节点
- 本质上，一对主机进程之间的通信由2个端节点构成

问题2：传输层提供的服务-需要穿过层间的信息



- 层间接口必须要携带的信息
  - 要传输的报文（对于本层来说：SDU）
  - 谁传的：自己的应用进程标示：IP+TCP(UDP) 端口
  - 传给谁：对方的应用进程标示：对方的IP+TCP(UDP)端口号
- 传输层实体（tcp或者udp实体）根据这些信息进行TCP报文段（UDP数据报）的封装
  - 源端口号，目标端口号，数据等
  - 将IP地址往下交IP实体，用于封装IP数据报：源IP,目标IP

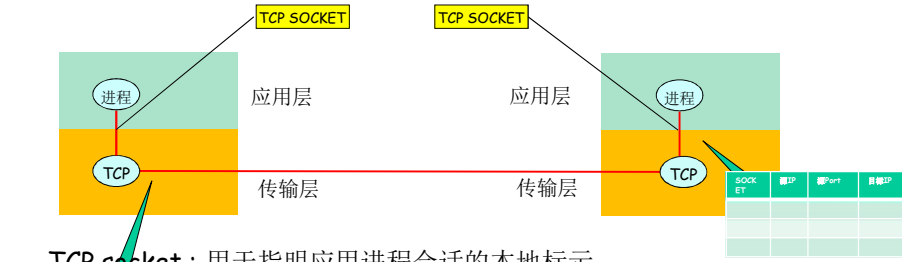
问题2：传输层提供的服务-层间信息的代表

- 如果Socket API 每次传输报文，都携带如此多的信息，太繁琐易错，不便于管理
- 用个代号标示通信的双方或者单方：socket
- 就像OS打开文件返回的句柄一样
  - 对句柄的操作，就是对文件的操作
- TCP socket：
  - TCP服务，两个进程之间的通信需要之前要建立连接
    - 两个进程通信会持续一段时间，通信关系稳定
  - 可以用一个整数表示两个应用实体之间的通信关系，本地标示
  - 穿过层间接口的信息量最小
  - TCP socket：源IP,源端口，目标IP，目标IP,目标端口

TCP之上的套接字（socket）

- 对于使用面向连接服务（TCP）的应用而言，套接字是4元组的一个具有本地意义的标示
  - 4元组：(源IP，源port，目标IP，目标port)
  - 唯一的指定了一个会话（2个进程之间的会话关系）
  - 应用使用这个标示，与远程的应用进程通信
  - 不必在每一个报文的发送都要指定这4元组
  - 就像使用操作系统打开一个文件，OS返回一个文件句柄一样，以后使用这个文件句柄，而不是使用这个文件的目录名、文件名
  - 简单，便于管理

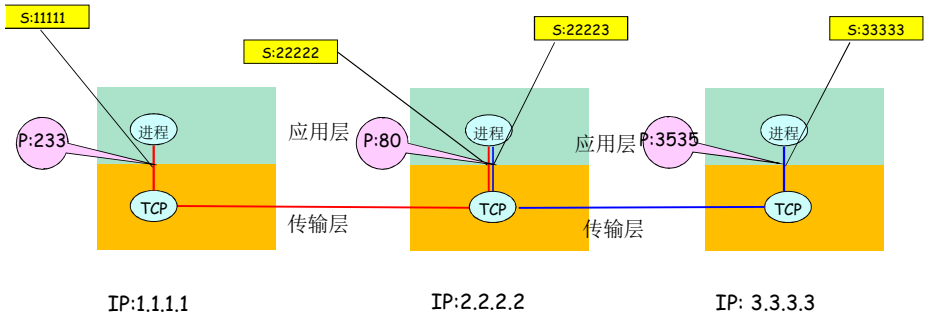
TCP socket



TCP socket：用于指明应用进程会话的本地标示

SOCKET	源IP	源Port	目标IP	目标Port	状态。。

TCP socket



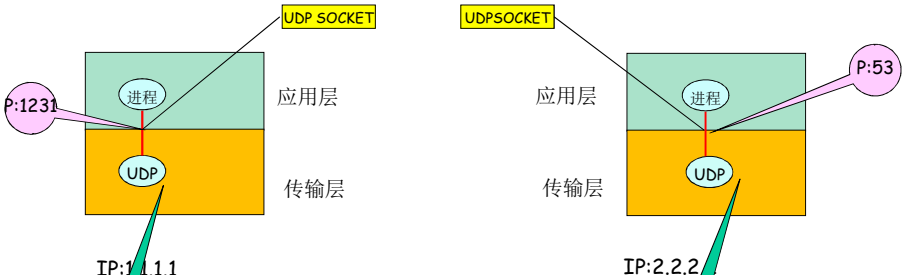
问题2：传输层提供的服务-层间信息代码

- UDP socket：
  - UDP服务，两个进程之间的通信需要之前无需建立连接
    - 每个报文都是独立传输的
    - 前后报文可能给不同的分布式进程
  - 因此，只能用一个整数表示本应用实体的标示
    - 因为这个报文可能传给另外一个分布式进程 -1
  - 穿过层间接口的信息大小最小
  - UDP socket：本IP,本端口
  - 但是传输 报文时：必须要提供对方IP，port
    - 接收报文时： 传输层需要上传对方的IP，port

UDP之上的套接字（socket）

- 对于使用无连接服务（UDP）的应用而言，套接字是2元组的一个具有本地意义的标示
  - 2元组：IP，port （源端指定）
  - UDP套接字指定了应用所在的一个端节点（end point）
  - 在发送数据报时，采用创建好的本地套接字（标示ID），就不必在发送每个报文中指明自己所采用的ip和port
  - 但是在发送报文时，必须要指定对方的ip和udp port(另外一个段节点)

UDP socket



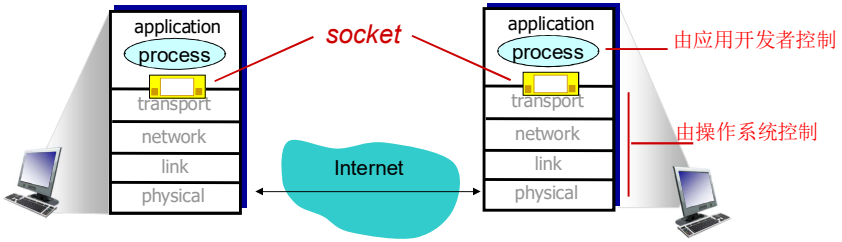
UDP socket：用于指明应用进程所在端节点的本地标示

SOCKET	IP	Port	

SOCKET	IP	Port	

套接字（Socket）

- 进程向套接字发送报文或从套接字接收报文
- 套接字 <-> 门户
  - 发送进程将报文**推出**门户，发送进程依赖于传输层设施在另外一侧的门将报文交付给接受进程
  - 接收进程从另外一端的门户**收到**报文（依赖于传输层设施）



问题3：如何使用传输层提供的服务实现应用

- 定义应用层协议：报文格式，解释，时序等
- 编制程序，通过API调用网络基础设施提供通信服务传报文，解析报文，实现应用时序等

应用层协议

- 定义了：运行在不同端系统上的应用**进程**如何相互交换报文
  - 交换的报文类型：请求和应答报文
  - 各种报文类型的语法：报文中的各个字段及其描述
  - 字段的语义：即字段取值的含义
  - 进程何时、如何发送报文及对报文进行响应的规则
- 应用协议仅仅是应用的一个组成部分
  - Web应用：HTTP协议，web客户端，web服务器，HTML

公开协议：
- 由RFC文档定义
- 允许互操作
- 如HTTP, SMTP

专用（私有）协议：
- 协议不公开
- 如：Skype

应用需要传输层提供什么样的服务？
如何描述传输层的服务？

数据丢失率
- 有些应用则要求100%的可靠数据传输（如文件）
- 有些应用（如音频）能容忍一定比例以下的数据丢失

延迟
- 一些应用 出于有效性考虑，对数据传输有严格的时间限制
  - Internet 电话、交互式游戏
  - 延迟、延迟差

吞吐
- 一些应用（如多媒体）必须需要最小限度的吞吐，从而使得应用能够有效运转
- 一些应用能充分利用可供使用的吞吐(弹性应用)

安全性
- 机密性
- 完整性
- 可认证性（鉴别）

常见应用对传输服务的要求

应用	数据丢失率	吞吐	时间敏感性
文件传输	不能丢失	弹性	不
e-mail	不能丢失	弹性	不
Web 文档	不能丢失	弹性	不
实时音视频	容忍丢失	音频： 5kbps-1Mbps 视频：0kbps-5Mbps	是，100ms
存储音视频	容忍丢失	同上	是，几秒
交互式游戏	容忍丢失	几kbps ~10kbps	是，100ms
即时讯息	不能丢失	弹性	是和不是

Internet 传输层提供的服务

TCP 服务：
- 可靠的传输服务
- 流量控制：发送方不会淹没接受方
- 拥塞控制：当网络出现拥塞时，能抑制发送方
- 不能提供的服务：时间保证、最小吞吐保证和安全
- 面向连接：要求在客户端进程和服务器进程之间建立连接

UDP 服务：
- 不可靠数据传输
- 不提供的服务：可靠，流量控制、拥塞控制、时间、带宽保证、建立连接

Q: 为什么要有 UDP?

UDP存在的必要性

- 能够区分不同的进程，而IP服务不能
  - 在IP提供的主机到主机端到端功能的基础上，区分了主机的应用进程
- 无需建立连接，省去了建立连接时间，适合事务性的应用
- 不做可靠性的工作，例如检错重发，适合那些对实时性要求比较高而对正确性要求不高的应用
  - 因为为了实现可靠性（准确性、保序等），必须付出时间代价（检错重发）
- 没有拥塞控制和流量控制，应用能够按照设定的速度发送数据
  - 而在TCP上面的应用，应用发送数据的速度和主机向网络发送的实际速度是不一致的，因为有流量控制和拥塞控制

Internet应用及其应用层协议和传输协议

应用	应用层协议	下层的传输协议
e-mail	SMTP [RFC 2821]	TCP
远程终端访问	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
文件传输	FTP [RFC 959]	TCP
流媒体	专用协议 (如RealNetworks)	TCP 或 UDP
Internet电话	专用协议 (如Net2Phone)	TCP或UDP

安全TCP

TCP & UDP

- 都没有加密
- 明文通过互联网传输，甚至密码

SSL

- 在TCP上面实现，提供加密的TCP连接
- 私密性
- 数据完整性
- 端到端的鉴别

SSL在应用层

- 应用采用SSL库，SSL库使用TCP通信

SSL socket API

- 应用通过API将明文交给socket，SSL将其加密在互联网上传输
- 详见第8章

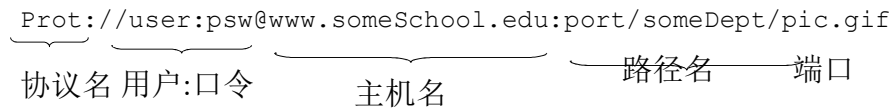
第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

Web 与 HTTP

一些术语

- Web页：由一些对象组成
- 对象可以是HTML文件、JPEG图像、Java小程序、声音剪辑文件等
- Web页含有一个基本的HTML文件，该基本HTML文件又包含若干对象的引用（链接）
- 通过URL对每个对象进行引用
  - 访问协议，用户名，口令字，端口，目录文件等；
- URL格式:

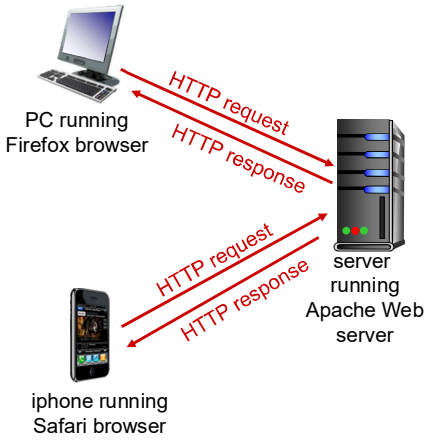


HTTP概况

HTTP: 超文本传输协议

- Web的应用层协议
- 客户/服务器模式
  - 客户: 请求、接收和显示Web对象的浏览器
  - 服务器: 对请求进行响应，发送对象的Web服务器

- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068



HTTP概况(续)

使用TCP:

- 客户发起一个与服务器的TCP连接 (建立套接字)，端口号为 80
- 服务器接受客户的TCP连接
- 在浏览器(HTTP客户端)与 Web服务器(HTTP服务器 server)交换HTTP报文 (应用层协议报文)
- TCP连接关闭

HTTP是无状态的

- 服务器并不维护关于客户的任何信息

维护状态的协议很复杂！

- 必须维护历史信息(状态)
- 如果服务器/客户端死机，它们的状态信息可能不一致，二者的信息必须是一致
- 无状态的服务器能够支持更多的客户端

HTTP连接

非持久HTTP

- 最多只有一个对象在TCP连接上发送
- 下载多个对象需要多个TCP连接
- HTTP/1.0使用非持久连接

持久HTTP

- 多个对象可以在一个（在客户端和服务器之间的）TCP连接上传输
- HTTP/1.1 默认使用持久连接

非持久HTTP连接

假设用户输入URL

`www.someSchool.edu/someDept/home.index`

(包含文本和10个就jpeg图像的引用)

1a. HTTP客户端在端口号80发起一个到服务器www.someSchool.edu的连接

1b. 位于主机www.someSchool.edu的HTTP服务器在80号端口等待连接，接受连接并通知客户端

2. HTTP客户端向TCP连接的套接字发送HTTP请求报文，报文表示客户端需要对象someDepartment/home.index

3. HTTP服务器接收到请求报文，检索出被请求的对象，将对象封装在一个响应报文，并通过其套接字象客户端发送

时间

非持久HTTP (续)

4. HTTP关闭TCP连接。

5. HTTP客户端收到包含html文件的响应报文，并显示html。然后对html文件进行检查，找到10引用对象

时间

6. 对10jpeg对象，重复1-5步

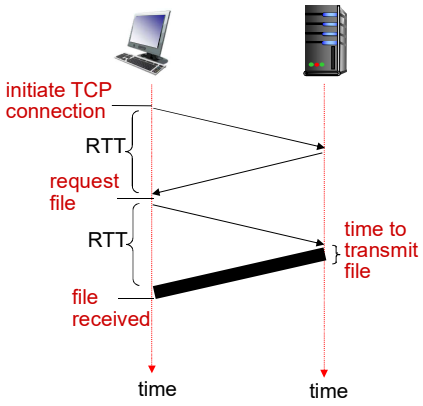
响应时间模型

往返时间RTT（round-trip time）：一个小的分组从客户端到服务器，在回到客户端的时间（传输时间忽略）

响应时间：

- 一个RTT用来发起TCP连接
- 一个 RTT用来HTTP请求并等待HTTP响应
- 文件传输时间

共：2RTT+传输时间



持久HTTP

非持久HTTP的缺点：

- 每个对象要2个 RTT
- 操作系统必须为每个TCP连接分配资源
- 但浏览器通常打开并行TCP连接，以获取引用对象

持久HTTP

- 服务器在发送响应后，仍保持TCP连接
- 在相同客户端和服务器之间的后续请求和响应报文通过相同的连接进行传送
- 客户端在遇到一个引用对象的时候，就可以尽快发送该对象的请求

非流水方式的持久HTTP：

- 客户端只能在收到前一个响应后才能发出新的请求
- 每个引用对象花费一个RTT

流水方式的持久HTTP：

- HTTP/1.1的默认模式
- 客户端遇到一个引用对象就立即产生一个请求
- 所有引用（小）对象只花费一个RTT是可能的

HTTP请求报文

- 两种类型的HTTP报文：请求、响应
- HTTP请求报文:
  - ASCII (人能阅读)

请求行 (GET, POST, HEAD命令)

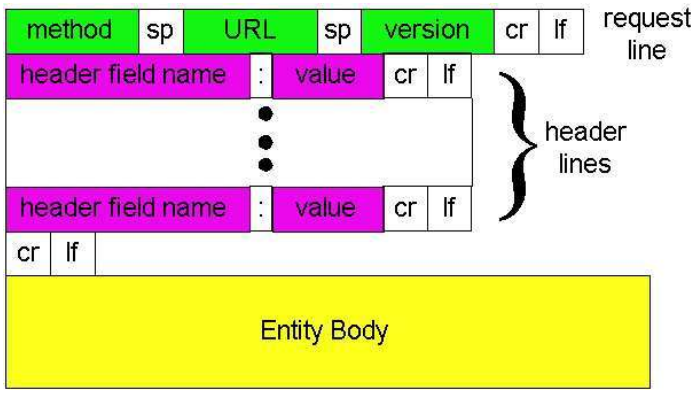
首部行

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language:fr
```

换行回车符，表示报文结束

(一个额外的换行回车符)

HTTP请求报文：通用格式



提交表单输入

Post方式：

- 网页通常包括表单输入
- 包含在实体主体(entity body)中的输入被提交到服务器

URL方式：

- 方法：GET
- 输入通过请求行的URL字段上载

www.somesite.com/animalsearch?monkeys&banana

http://www.baidu.com/s?wd=xx+yy+zzz&cl=3

参数：wd，cl

参数值：XX+YY+zzz，3

方法类型

HTTP/1.0

- GET
- POST
- HEAD
  - 要求服务器在响应报文中不包含请求对象 → 故障跟踪

HTTP/1.1

- GET, POST, HEAD
- PUT
  - 将实体主体中的文件上载到URL字段规定的路径
- DELETE
  - 删除URL字段规定的文件

HTTP响应报文

状态行 (协议版本、状态码和相应状态信息)

首部行

数据，如请求的HTML文件

```
HTTP/1.1 200 OK\r\n
Connection close\r\n
Date: Thu, 06 Aug 1998 12:00:15 GMT\r\r
Server: Apache/1.3.0 (Unix) \r\n
Last-Modified: Mon, 22 Jun 1998 …... \r
Content-Length: 6821\r\n
Content-Type: text/html\r\n
\r\n
\r\n
data data data data data ...
```

HTTP响应状态码

位于服务器→客户端的响应报文中的首行

一些状态码的例子：

200 OK
- 请求成功，请求对象包含在响应报文的后续部分

301 Moved Permanently
- 请求的对象已经被永久转移了：新的URL在响应报文的Location:首部行中指定
- 客户端软件自动用新的URL去获取对象

400 Bad Request
- 一个通用的差错代码，表示该请求不能被服务器解读

404 Not Found
- 请求的文档在该服务上没有找到

505 HTTP Version Not Supported

Trying out HTTP (client side) for yourself

1. Telnet to your favorite Web server:

```
telnet cis.poly.edu 80
```

opens TCP connection to port 80 (default HTTP server port) at cis.poly.edu. anything typed in sent to port 80 at cis.poly.edu

2. type in a GET HTTP request:

```
GET /~ross/ HTTP/1.1
Host: cis.poly.edu
```

by typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

3. look at response message sent by HTTP server!

(or use Wireshark to look at captured HTTP request/response)

用户-服务器状态：cookies

大多数主要的门户网站使用 cookies

4个组成部分：

1) 在HTTP响应报文中有一个cookie的首部行

2)在HTTP请求报文含有一个cookie的首部行

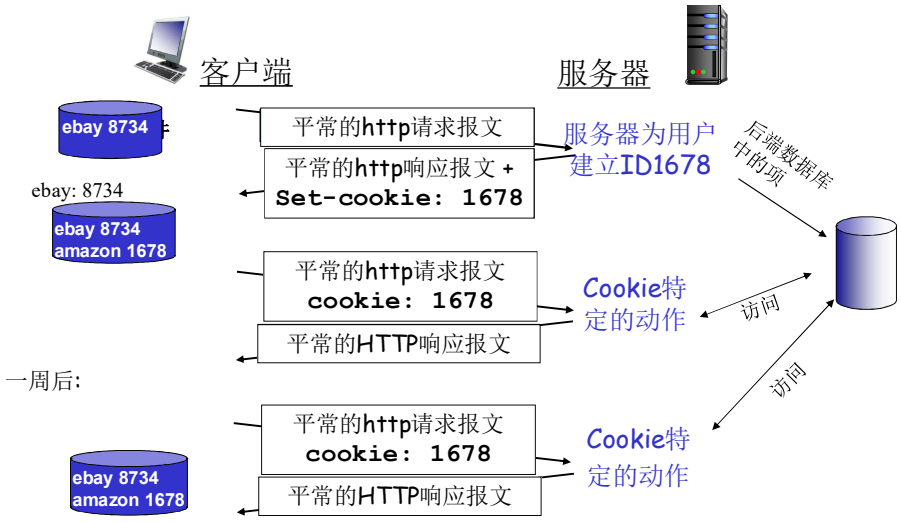
3) 在用户端系统中保留有一个cookie文件，由用户的浏览器管理

4) 在Web站点有一个后端数据库

例子：

- Susan总是用同一个PC使用Internet Explore上网
- 她第一次访问了一个使用了Cookie的电子商务网站
- 当最初的HTTP请求到达服务器时，该Web站点产生一个唯一的ID，并以此作为索引在它的后端数据库中产生一个项

Cookies: 维护状态



Cookies (续)

Cookies能带来什么：

- 用户验证
- 购物车
- 推荐
- 用户状态 (Web e-mail)

如何维持状态：

- 协议端节点：在多个事务上，发送端和接收端维持状态
- cookies：http报文携带状态信息

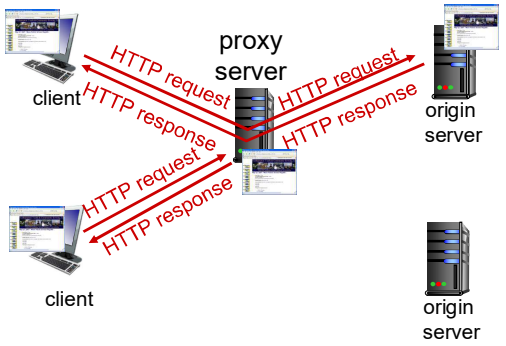
Cookies与隐私：

- Cookies允许站点知道许多关于用户的信息
- 可能将它知道的东西卖给第三方
- 使用重定向和cookie的搜索引擎还能知道用户更多的信息
  - 如通过某个用户在大量站点上的行为，了解其个人浏览方式的大致模式
- 广告公司从站点获得信息

Web缓存 (代理服务器)

目标：不访问**原始**服务器，就满足客户的请求

- 用户设置浏览器： 通过缓存访问Web
- 浏览器将所有的HTTP请求发给缓存
  - 在缓存中的对象：缓存直接返回对象
  - 如对象不在缓存，缓存请求原始服务器，然后再将对象返回给客户端



Web缓存

- 缓存既是客户端又是服务器
- 通常缓存是由ISP安装 (大学、公司、居民区ISP)

为什么要使用Web缓存？

- 降低客户端的请求响应时间
- 可以大大减少一个机构内部网络与Internent接入链路上的流量
- 互联网大量采用了缓存：可以使较弱的ICP也能够有效提供内容

缓存示例

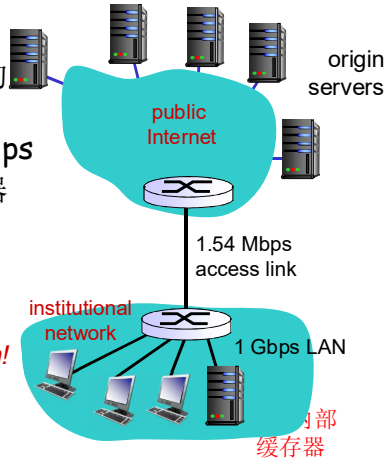
假设

- 平均对象大小 = 100kb
- 机构内浏览器对原始服务器的平均请求率为 = 15请求/s
- 平均到浏览器的速率：1.5Mbps
- 机构内部路由器到原始服务器再返回到路由器的的延时 （Internet 延时）= 2s
- 接入链路带宽：1.54Mbps

结果

- LAN的流量强度 = 15%
- 接入链路上的流量强度 = 99%
- 总延时 = LAN延时 + 接入延时 + Internet 延时

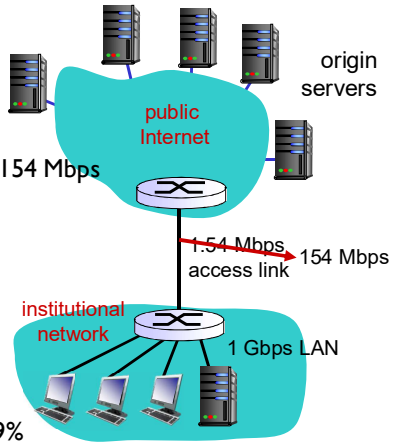
= ms + 分 + 2s



缓存示例：更快的接入链路

假设

- 平均对象大小 = 100kb
- 机构内浏览器对原始服务器的平均请求率为 = 15请求/s
- 平均到浏览器的速率：1.5Mbps
- 机构内部路由器到原始服务器再返回到路由器的的延时 （Internet 延时）= 2s
- 接入链路带宽：1.54Mbps



结果

- LAN的流量强度 = 15%
- 接入链路上的流量强度 = 99%
- 总延时 = LAN延时 + 接入延时 + Internet 延时 = ms + 分 + 2s → msecs

9.9%

代价： 增加了接入链路带宽（非常昂贵！）

缓存例子：安装本地缓存

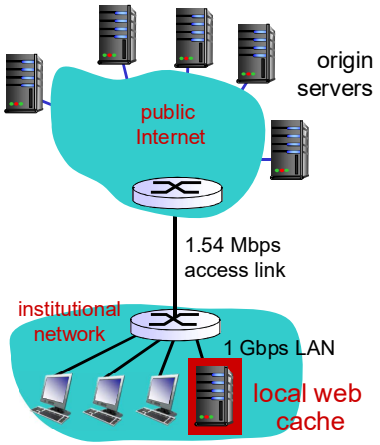
假设

- 平均对象大小 = 100kb
- 机构内浏览器对原始服务器的平均请求率为 = 15请求/s
- 平均到浏览器的速率：1.5Mbps
- 机构内部路由器到原始服务器再返回到路由器的的延时 （Internet 延时）= 2s
- 接入链路带宽：1.54Mbps

结果

- LAN 利用率：15%
- 接入网络利用率： ?
- 总体延迟= ?

How to compute link utilization, delay?

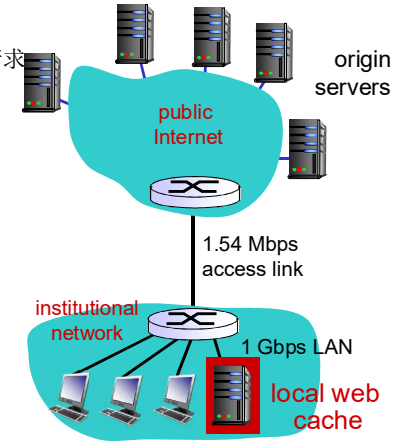


代价： web缓存(廉价!)

缓存例子：安装本地缓存

计算链路利用率，有缓存的延迟：

- 假设缓存命中率0.4
  - 40%请求在缓存中被满足，其他60%的请求需要被原始服务器满足



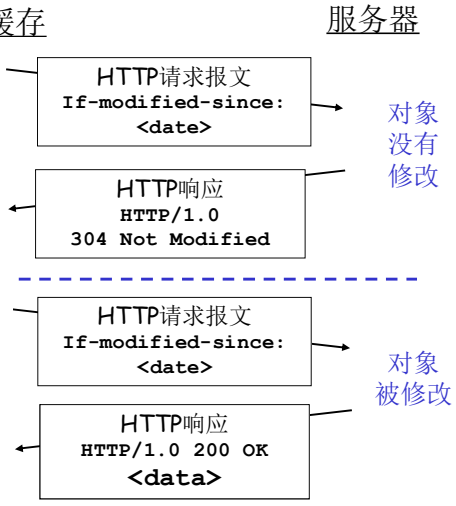
- 接入链路利用率：
  - 60%的请求采用接入链路
- 进过接入链路到达浏览器的数据速率 = 0.6*1.50 Mbps = .9 Mbps
  - 利用率= 0.9/1.54 = .58
- 总体延迟：
  - = 0.6 *（从原始服务器获取对象的延迟）+0.4 *（从缓存获取对象的延迟）
  - = 0.6 (2.01) + 0.4 (~msecs)
  - = ~ 1.2 secs
  - 比安装154Mbps链路还来得小（而且比较便宜!）

条件GET方法

- 目标：如果缓存器中的对象拷贝是最新的，就不要发送对象
- 缓存器：在HTTP请求中指定缓存拷贝的日期

  `If-modified-since: <date>`
- 服务器：如果缓存拷贝陈旧，则响应报文没包含对象：

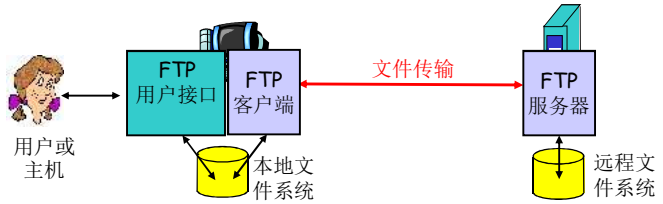
  `HTTP/1.0 304 Not Modified`



第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

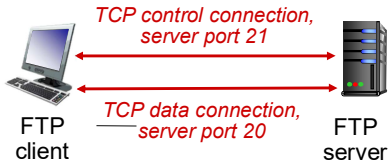
FTP: 文件传输协议



- 向远程主机上传输文件或从远程主机接收文件
- 客户/服务器模式
  - 客户端：发起传输的一方
  - 服务器：远程主机
- ftp: RFC 959
- ftp服务器：端口号为21

FTP: 控制连接与数据连接分开

- FTP客户端与FTP服务器通过端口21联系，并使用TCP为传输协议
- 客户端通过控制连接获得身份确认
- 客户端通过控制连接发送命令浏览远程目录
- 收到一个文件传输命令时，服务器打开一个到客户端的数据连接
- 一个文件传输完成后，服务器关闭连接



- 服务器打开第二个TCP数据连接用来传输另一个文件
- 控制连接： 带外（ “out of band” ）传送
- FTP服务器维护用户的状态信息：当前路径、用户帐户与控制连接对应

有状态

FTP命令、响应

命令样例：

- 在**控制连接**上以*ASCII*文本方式传送
- **`USER username`**
- **`PASS password`**
- **`LIST`**：请服务器返回远程主机当前目录的文件列表
- **`RETR filename`**：从远程主机的当前目录检索文件 (gets)
- **`STOR filename`**：向远程主机的当前目录存放文件 (puts)

返回码样例：

- 状态码和状态信息 (同HTTP)
- **`331 Username OK, password required`**
- **`125 data connection already open; transfer starting`**
- **`425 Can’ t open data connection`**
- **`452 Error writing file`**

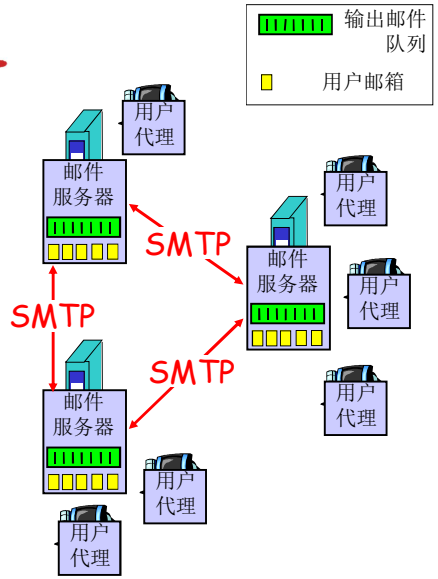
第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

电子邮件（EMail）

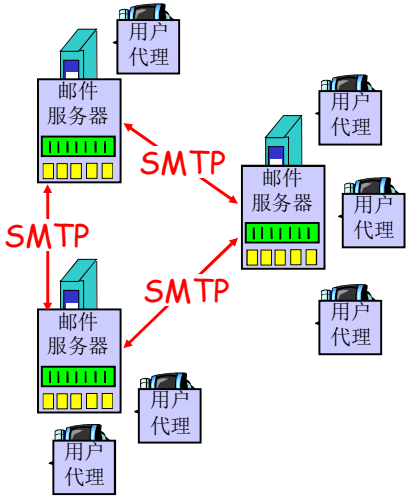
3个主要组成部分：
- 用户代理
- 邮件服务器
- 简单邮件传输协议：SMTP

用户代理
- 又名“邮件阅读器”
- 撰写、编辑和阅读邮件
- 如Outlook、Foxmail
- 输出和输入邮件保存在服务器上



EMail: 邮件服务器

邮件服务器
- 邮箱中管理和维护发送给用户的邮件
- 输出报文队列保持待发送邮件报文
- 邮件服务器之间的SMTP协议：发送email报文
  - 客户：发送方邮件服务器
  - 服务器：接收端邮件服务器

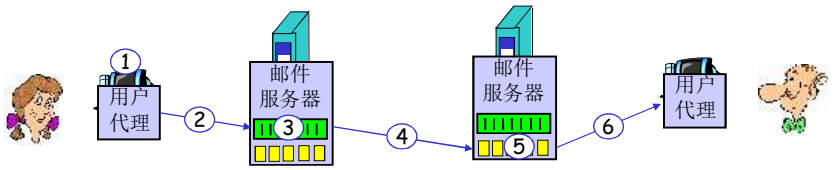


EMail: SMTP [RFC 2821]

- 使用TCP在客户端和服务器之间传送报文，端口号为25
- 直接传输：从发送方服务器到接收方服务器
- 传输的3个阶段
  - 握手
  - 传输报文
  - 关闭
- 命令/响应交互
  - 命令：ASCII文本
  - 响应：状态码和状态信息
- 报文必须为7位ASCII码

举例：Alice给Bob发送报文

1) Alice使用用户代理撰写邮件并发送给 bob@someschool.edu
2) Alice的用户代理将邮件发送到她的邮件服务器；邮件放在报文队列中
3) SMTP的客户端打开到Bob邮件服务器的TCP连接
4) SMTP客户端通过TCP连接发送Alice的邮件
5) Bob的邮件服务器将邮件放到Bob的邮箱
6) Bob调用他的用户代理阅读邮件



简单的SMTP交互

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```


Try SMTP interaction for yourself:

- telnet servername 25
- see 220 reply from server
- enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands

above lets you send email without using email client (reader)

SMTP：总结

- SMTP使用持久连接
- SMTP要求报文（首部和主体）为7位ASCII编码
- SMTP服务器使用CRLF.CRLF决定报文的尾部

HTTP比较：

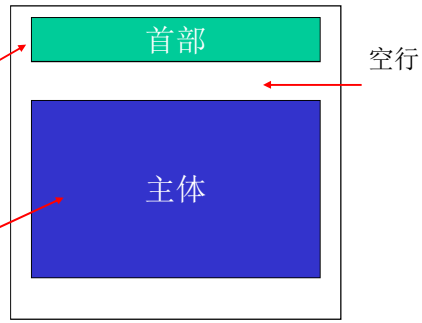
- HTTP：拉（pull）
- SMTP：推（push）
- 二者都是ASCII形式的命令/响应交互、状态码
- HTTP：每个对象封装在各自的响应报文中
- SMTP：多个对象包含在一个报文中

邮件报文格式

SMTP：交换email报文的协议
RFC 822: 文本报文的标准：

- 首部行：如，
  - To:
  - From:
  - Subject:

  与SMTP命令不同！
- 主体
  - 报文，只能是ASCII码字符



报文格式：多媒体扩展

- MIME：多媒体邮件扩展（multimedia mail extension），RFC 2045, 2056
- 在报文首部用额外的行申明MIME内容类型

MIME版本

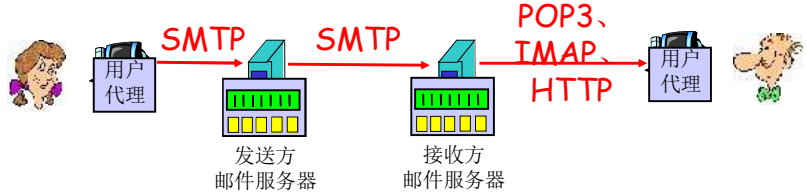
数据的编码方式

多媒体数据类型、子类型和参数申明

编码好的数据

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.........................
......base64 encoded data
```


邮件访问协议



- SMTP: 传送到接收方的邮件服务器
- 邮件访问协议：从服务器访问邮件
  - POP：邮局访问协议（Post Office Protocol）[RFC 1939]
    - 用户身份确认 (代理<-->服务器) 并下载
  - IMAP：Internet邮件访问协议（Internet Mail Access Protocol）[RFC 1730]
    - 更多特性 (更复杂)
    - 在服务器上处理存储的报文
  - HTTP：Hotmail , Yahoo! Mail等
    - 方便

POP3协议

用户确认阶段

- 客户端命令：
  - **user:** 申明用户名
  - **pass:** 口令
- 服务器响应
  - **+OK**
  - **-ERR**

事物处理阶段，客户端：

- **list:** 报文号列表
- **retr:** 根据报文号检索报文
- **dele:** 删除
- **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (续) 与 IMAP

POP3（续）

- 先前的例子使用 “下载并删除”模式。
  - 如果改变客户机，Bob不能阅读邮件
- “下载并保留”：不同客户机上为报文的拷贝
- POP3在会话中是无状态的

本地管理文件夹

IMAP

- IMAP服务器将每个报文与一个文件夹联系起来
- 允许用户用目录来组织报文
- 允许用户读取报文组件
- IMAP在会话过程中保留用户状态：
  - 目录名、报文ID与目录名之间映射

远程管理文件夹

第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

2.5 DNS(Domain Name System)

- DNS的必要性
  - IP地址标识主机、路由器
  - 但IP地址不好记忆，不便人类使用(没有意义)
  - 人类一般倾向于使用一些有意义的字符串来标识Internet上的设备

    例如：gzheng@ustc.edu.cn 所在的邮件服务器

    www.ustc.edu.cn 所在的web服务器
  - 存在着“字符串”—IP地址的转换的必要性
  - 人类用户提供要访问机器的“字符串”名称
  - 由DNS负责转换成为二进制的网络地址

DNS系统需要解决的问题

- 问题1：如何命名设备
  - 用有意义的字符串：好记，便于人类用使用
  - 解决一个平面命名的重名问题：层次化命名
- 问题2：如何完成名字到IP地址的转换
  - 分布式的数据库维护和响应名字查询
- 问题3：如何维护：增加或者删除一个域，需要在域名系统中做哪些工作

DNS(Domain Name System)的历史

- ARPANET的名字解析解决方案
  - 主机名：没有层次的一个字符串（一个平面）
  - 存在着一个（集中）维护站：维护着一张 主机名-IP地址的映射文件：Hosts.txt
  - 每台主机定时从维护站取文件
- ARPANET解决方案的问题
  - 当网络中主机数量很大时
    - 没有层次的主机名称很难分配
    - 文件的管理、发布、查找都很麻烦

DNS(Domain Name System)总体思路和目标

- DNS的主要思路
  - 分层的、基于域的命名机制
  - 若干分布式的数据库完成名字到IP地址的转换
  - 运行在UDP之上端口号为53的应用服务
  - 核心的Internet功能，但以应用层协议实现
    - 在网络边缘处理复杂性
- DNS主要目的：
  - 实现主机名-IP地址的转换(name/IP translate)
  - 其它目的
    - 主机别名到规范名字的转换：Host aliasing
    - 邮件服务器别名到邮件服务器的正规名字的转换：Mail server aliasing
    - 负载均衡：Load Distribution

问题1：DNS名字空间(The DNS Name Space)

- DNS域名结构
  - 一个层面命名设备会有很多重名
  - NDS采用层次树状结构的 命名方法
  - Internet 根被划为几百个顶级域(top lever domains)
    - 通用的(generic)

      .com; .edu ; .gov ; .int ; .mil ; .net ; .org

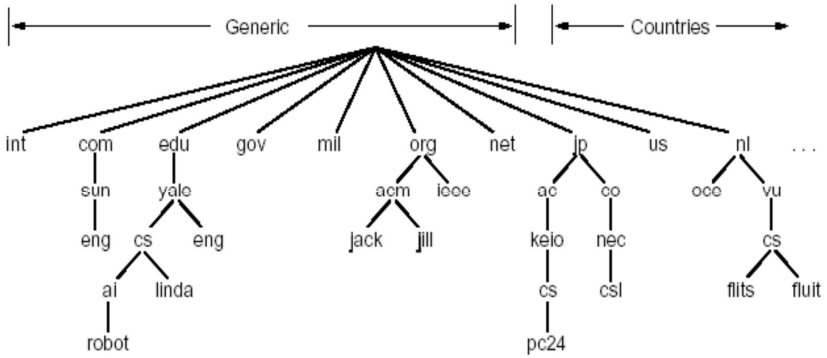
      .firm ; .hsop ; .web ; .arts ; .rec ;
    - 国家的(countries)

      .cn ; .us ; .nl ; .jp
  - 每个(子)域下面可划分为若干子域(subdomains)
  - 树叶是主机

DNS: 根名字服务器

共有13个根名字服务器

DNS名字空间(The DNS Name Space)



DNS名字空间(The DNS Name Space)

- 域名(Domain Name)
  - 从本域往上，直到树根
  - 中间使用“.”间隔不同的级别
  - 例如：ustc.edu.cn
  - auto.ustc.edu.cn
  - www.auto. ustc.edu.cn
  - 域的域名：可以用于表示一个域
  - 主机的域名：一个域上的一个主机

DNS名字空间(The DNS Name Space)

- 域名的管理
  - 一个域管理其下的子域

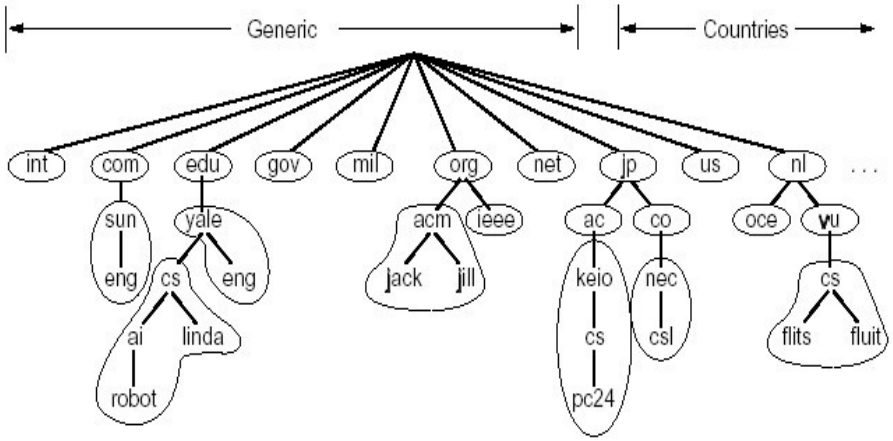
    .jp 被划分为 ac.jp co.jp

    .cn 被划分为 edu.cn com.cn
  - 创建一个新的域，必须征得它所属域的同意
- 域与物理网络无关
  - 域遵从组织界限，而不是物理网络
    - 一个域的主机可以不在一个网络
    - 一个网络的主机不一定在一个域
  - 域的划分是逻辑的，而不是物理的

问题2：解析问题-名字服务器(Name Server)

- 一个名字服务器的问题
  - 可靠性问题：单点故障
  - 扩展性问题：通信容量
  - 维护问题：远距离的集中式数据库
- 区域(zone)
  - 区域的划分有区域管理者自己决定
  - 将DNS名字空间划分为互不相交的区域，每个区域都是树的一部分
  - 名字服务器：
    - 每个区域都有一个名字服务器：维护着它所管辖区域的权威信息(authoritative record)
    - 名字服务器允许被放置在区域之外，以保障可靠性

名字空间划分为若干区域：Zone



权威DNS服务器：组织机构的DNS服务器， 提供组织机构服务器（如Web和mail）可访问的主机和IP之间的映射
组织机构可以选择实现自己维护或由某个服务提供商来维护

TLD服务器

- 顶级域(TLD)服务器：负责顶级域名（如com, org, net, edu和gov）和所有国家级的顶级域名（如cn, uk, fr, ca, jp ）
  - Network solutions 公司维护com TLD服务器
  - Educause公司维护edu TLD服务器

区域名字服务器维护资源记录

- 资源记录(resource records)
  - 作用：维护 域名-IP地址(其它)的映射关系
  - 位置：Name Server的分布式数据库中
- RR格式：(domain_name, ttl, type,class,Value)
  - Domain_name: 域名
  - Ttl: time to live：生存时间(权威，缓冲记录)
  - Class 类别 ：对于Internet，值为IN
  - Value 值：可以是数字，域名或ASCII串
  - Type 类别：资源记录的类型—见下页

DNS记录

DNS ：保存资源记录(RR)的分布式数据库

RR 格式：(name, value, type, ttl)

- Type=A
  - **Name为主机**
  - **Value为IP地址**
- Type=NS
  - **Name域名**(如foo.com)
  - **Value为该域名的权威服务器的域名**
- Type=CNAME
  - **Name为规范名字的别名**
    www.ibm.com 的规范名字为 servereast.backup2.ibm.com
  - **value** 为规范名字
- Type=MX
  - **Value为name对应的邮件服务器的名字**

TTL：生存时间，决定了资源记录应当从缓存中删除的时间

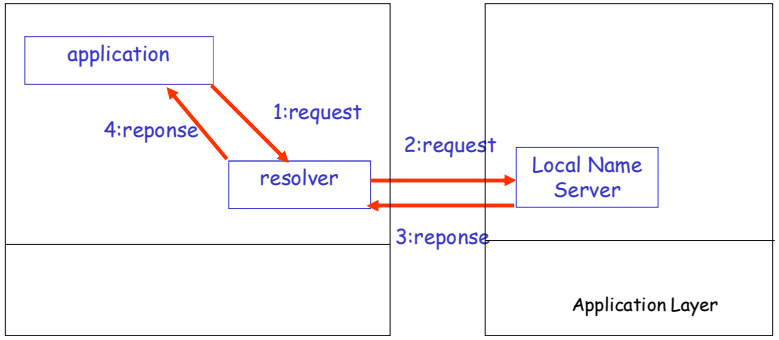
资源记录(resource records)

- 一个例子

```
; Authoritative data for cs.vu.nl
cs.vu.nl.       86400   IN SOA    star boss (952771,7200,7200,2419200,86400)
cs.vu.nl.       86400   IN TXT    "Faculteit Wiskunde en Informatica."
cs.vu.nl.       86400   IN TXT    "Vrije Universiteit Amsterdam."
cs.vu.nl.       86400   IN MX     1 zephyr.cs.vu.nl.
cs.vu.nl.       86400   IN MX     2 top.cs.vu.nl.

flits.cs.vu.nl. 86400   IN HINFO  Sun Unix
flits.cs.vu.nl. 86400   IN A      130.37.16.112
flits.cs.vu.nl. 86400   IN A      192.31.231.165
flits.cs.vu.nl. 86400   IN MX     1 flits.cs.vu.nl.
flits.cs.vu.nl. 86400   IN MX     2 zephyr.cs.vu.nl.
flits.cs.vu.nl. 86400   IN MX     3 top.cs.vu.nl.
www.cs.vu.nl.   86400   IN CNAME  star.cs.vu.nl
ftp.cs.vu.nl.   86400   IN CNAME  zephyr.cs.vu.nl

rowboat                 IN A      130.37.56.201
                        IN MX     1 rowboat
                        IN MX     2 zephyr
                        IN HINFO  Sun Unix

little-sister           IN A      130.37.62.23
                        IN HINFO  Mac MacOS

laserjet                IN A      192.31.231.216
                        IN HINFO  "HP Laserjet IIISi" Proprietary
```

DNS(Domain Name System)

- DNS大致工作过程
  - 应用调用 解析器(resolver)
  - 解析器作为客户 向Name Server发出查询报文（封装在UDP段中）
  - Name Server返回响应报文(name/ip)



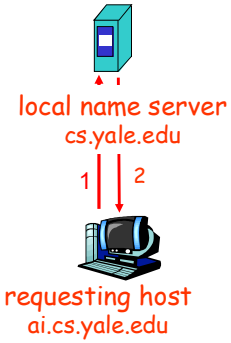
本地名字服务器（Local Name Server）

- 并不严格属于层次结构
- 每个ISP (居民区的ISP、公司、大学）都有一个本地DNS服务器
  - 也称为“默认名字服务器”
- 当一个主机发起一个DNS查询时，查询被送到其本地DNS服务器
  - 起着代理的作用，将查询转发到层次结构中

名字服务器(Name Server)

- 名字解析过程
  - 目标名字在Local Name Server中
    - 情况1：查询的名字在该区域内部
    - 情况2：缓存(cashing)

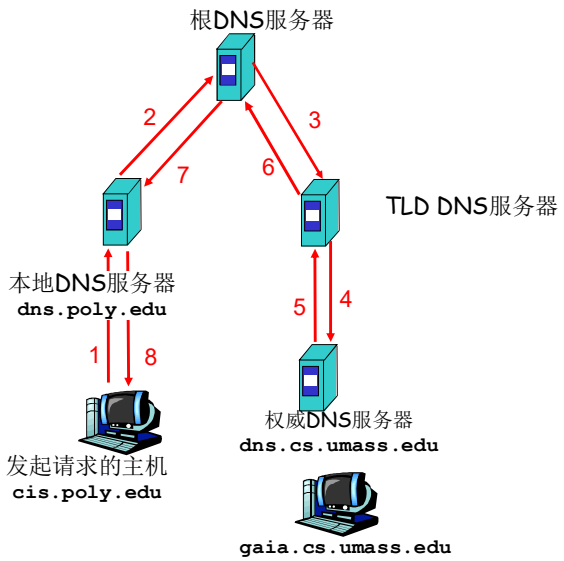
当与本地名字服务器不能解析名字时，联系根名字服务器 顺着根-TLD 一直找到 权威名字服务器



递归查询

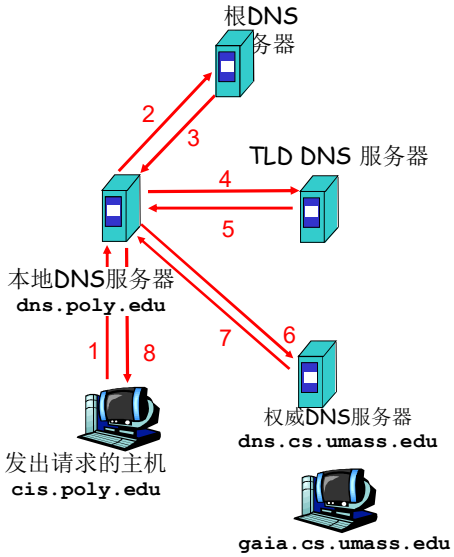
递归查询：

- 名字解析负担都放在当前联络的名字服务器上
- 问题：根服务器的负担太重
- 解决： 迭代查询 (iterated queries)



迭代查询

- 主机cis.poly.edu 想知道主机 gaia.cs.umass.edu 的IP地址



- 根（及各级域名）服务器返回的不是查询结果，而是下一个NS的地址
- 最后由权威名字服务器给出解析结果

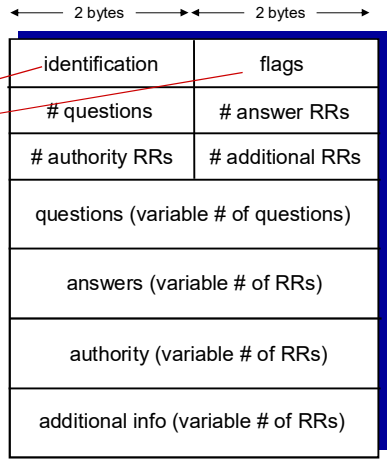
- 当前联络的服务器给出可以联系的服务器的名字
- “我不知道这个名字，但可以向这个服务器请求”

DNS协议、报文

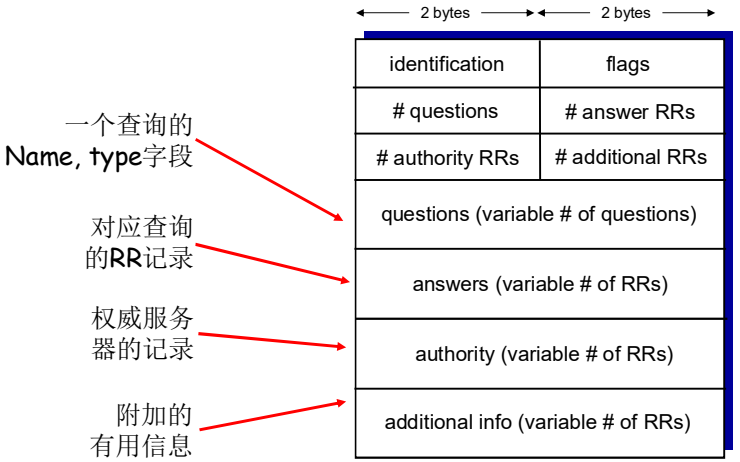
DNS协议：查询和响应报文的报文格式相同

报文首部

- 标识符（ID）：16位
- flags:
  - 查询/应答
  - 希望递归
  - 递归可用
  - 应答为权威



DNS协议、报文



提高性能：缓存

- 一旦名字服务器学到了一个映射，就将该映射缓存起来
- 根服务器通常都在本地服务器中缓存着
  - 使得根服务器不用经常被访问
- 目的：提高效率
- 可能存在的问题：如果情况变化，缓存结果和权威资源记录不一致
- 解决方案：TTL（默认2天）

问题3：维护问题：新增一个域

- 在上级域的名字服务器中增加两条记录，指向这个新增的子域的域名 和 域名服务器的地址
- 在新增子域 的名字服务器上运行名字服务器，负责本域的名字解析： 名字->IP地址

例子：在com域中建立一个“Network Utopia”

- 到注册登记机构注册域名networkutopia.com
  - 需要向该机构提供权威DNS服务器（基本的、和辅助的）的名字和IP地址
  - 登记机构在com TLD服务器中插入两条RR记录:

```
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
```

- 在networkutopia.com的权威服务器中确保有
  - 用于Web服务器的www.networkuptopia.com的类型为A的记录
  - 用于邮件服务器mail.networkutopia.com的类型为MX的记录

攻击DNS

总的说来，DNS比较健壮

DDoS 攻击

- 对根服务器进行流量轰炸攻击：发送大量ping
  - 没有成功
  - 原因 1：根目录服务器配置了流量过滤器，防火墙
  - 原因 2：Local DNS 服务器缓存了TLD服务器的IP地址，因此无需查询根服务器
- 向TLD服务器流量轰炸攻击：发送大量查询
  - 可能更危险
  - 效果一般，大部分DNS缓存了TLD

重定向攻击

- 中间人攻击
  - 截获查询，伪造回答，从而攻击某个（DNS回答指定的IP）站点
- DNS中毒
  - 发送伪造的应答给DNS服务器，希望它能够缓存这个虚假的结果
- 技术上较困难：分布式截获和伪造

利用DNS基础设施进行DDoS

- 伪造某个IP进行查询， 攻击这个目标IP
- 查询放大，响应报文比查询报文大
- 效果有限

第2章：提纲

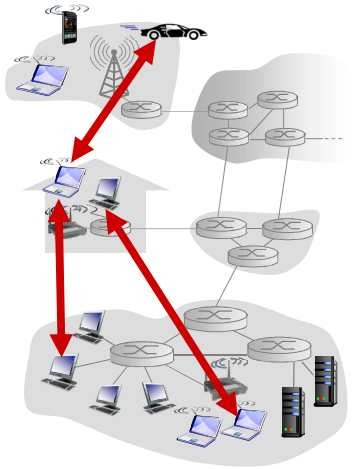
2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

纯P2P架构

- 没有（或极少）一直运行的服务器
- 任意端系统都可以直接通信
- 利用peer的服务能力
- Peer节点间歇上网，每次IP地址都有可能变化

例子：

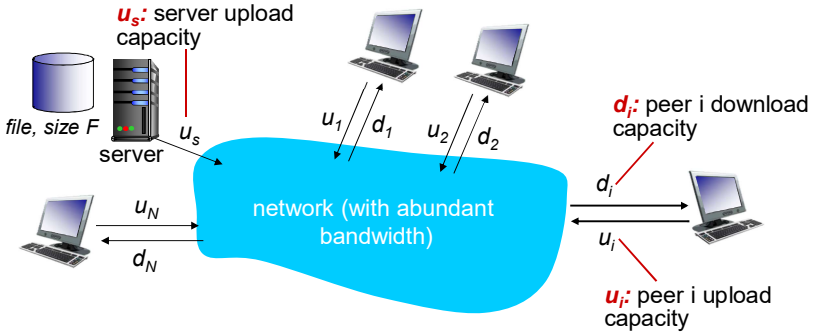
- 文件分发（BitTorrent）
- 流媒体(KanKan)
- VoIP（Skype）



文件分发：C/S vs P2P

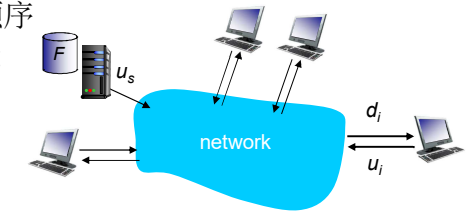
*问题：*从一台服务器分发文件（大小F）到N个peer需要多少时间？

- Peer节点上下载能力是有限的资源



文件分发时间：C/S模式

- *服务器传输：* 都是由服务器发送给peer，服务器必须顺序传输（上载）N个文件拷贝：
  - 发送一个copy：F/u_s
  - 发送N个copy：NF/u_s
- *客户端：* 每个客户端必须下载一个文件拷贝
  - d_{min} = 客户端最小的下载速率
  - 下载带宽最小的客户端下载的时间：F/d_{min}

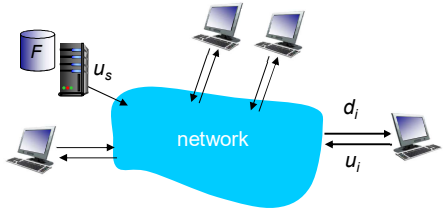


采用C-S方法将一个F大小的文件分发给N个客户端耗时 $D_{c\text{-}s} \geq max\{NF/u_s, F/d_{min}\}$

随着N线性增长

文件分发时间：P2P模式

- *服务器传输：*最少需要上载一份拷贝
  - 发送一个拷贝的时间：F/u_s
- *客户端：* 每个客户端必须下载一个拷贝
  - 最小下载带宽客户单耗时：：F/d_{min}
- *客户端：* 所有客户端总体下载量NF
  - 最大上载带宽是：$u_s + \Sigma u_i$
  - 除了服务器可以上载，其他所有的peer节点都可以上载



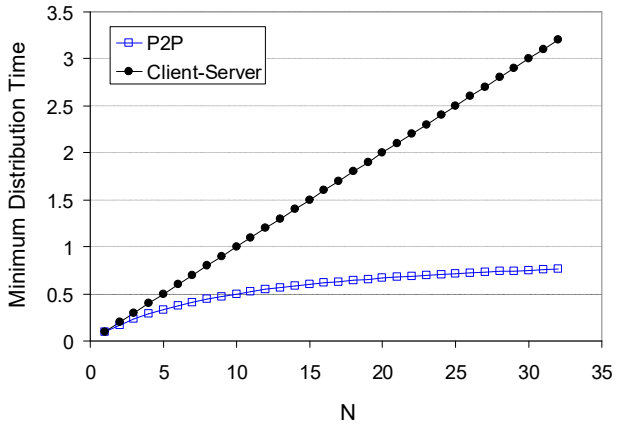
采用P2P方法将一个F大小的文件分发给N个客户端耗时 $D_{P2P} \geq max\{F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i)\}$

分子随着N线性变化，每个节点需要下载，整体下载量随着N增大 …

… 分母也是如此，随着peer节点的增多每个peer也带了服务能力

Client-server vs. P2P：例子

客户端上载速率 = u， F/u = 1 小时， $u_s = 10u$， $d_{min} \geq u_s$

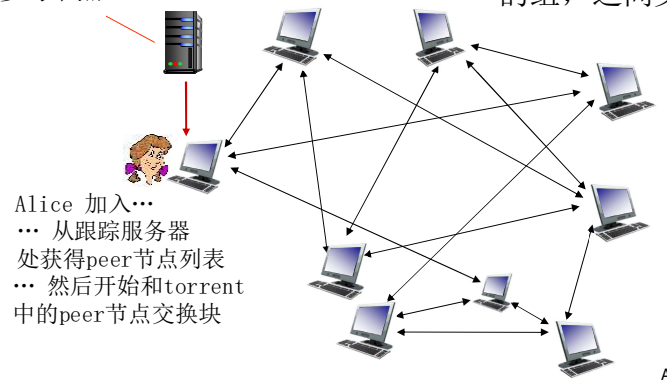


P2P文件分发： BitTorrent

- 文件被分为一个个块256KB
- 网络中的这些peers发送接收文件块，相互服务

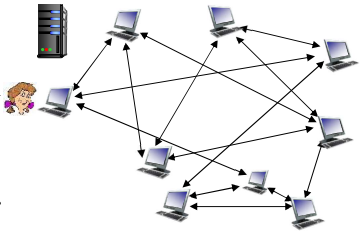
*tracker:*跟踪torrent中参与节点

Torrent（洪流）： 节点的组，之间交换文件块



P2P文件分发： BitTorrent

- Peer加入torrent：
  - 一开始没有块，但是将会通过其他节点处累积文件块
  - 向跟踪服务器注册，获得peer节点列表，和部分peer节点构成邻居关系（“连接”）



- 当peer下载时，该peer可以同时向其他节点提供上载服务
- Peer可能会变换用于交换块的peer节点
- *扰动churn:* peer节点可能会上线或者下线
- 一旦一个peer拥有整个文件，它会（自私的）离开或者保留（利他主义）在torrent中

BitTorrent：请求，发送文件块

请求块：

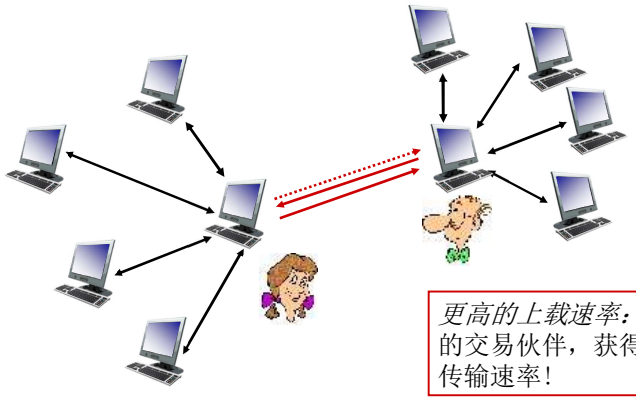
- 在任何给定时间，不同peer节点拥有一个文件块的子集
- 周期性的，Alice节点向邻居询问他们拥有哪些块的信息
- Alice向peer节点请求它希望的块，稀缺的块

发送块：一报还一报tit-for-tat

- Alice向4个peer发送块，这些块向它自己提供最大带宽的服务
  - 其他peer被Alice阻塞（将不会从Alice处获得服务）
  - 每10秒重新评估一次：前4位
- 每个30秒：随机选择其他peer节点，向这个节点发送块
  - “优化疏通” 这个节点
  - 新选择的节点可以加入这个top 4

BitTorrent: tit-for-tat

(1) Alice “优化疏通” Bob
(2) Alice 变成了Bob的前4位提供者；Bob答谢Alice
(3) Bob 变成了Alice的前4提供者



更高的上载速率： 发现更好的交易伙伴，获得更快的文件传输速率！

P2P文件共享

例子

- Alice在其笔记本电脑上运行P2P客户端程序
- 间歇性地连接到Internet，每次从其ISP得到新的IP地址
- 请求“双截棍.MP3”
- 应用程序显示其他有“双截棍.MP3” 拷贝的对等方
- Alice选择其中一个对等方，如Bob.
- 文件从Bob’s PC传送到Alice的笔记本上：HTTP
- 当Alice下载时，其他用户也可以从Alice处下载
- Alice的对等方既是一个Web客户端，也是一个瞬时Web服务器

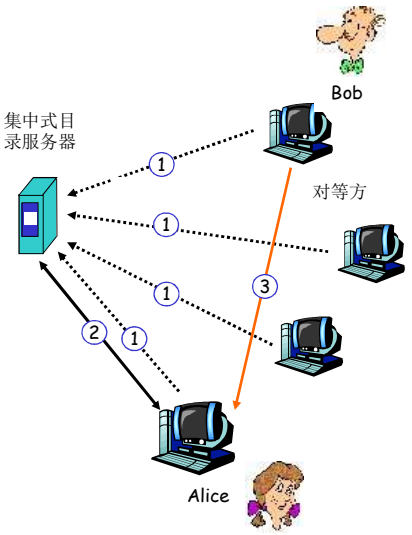
所有的对等方都是服务器
= 可扩展性好！

P2P文件共享

- 两大问题：
  - 如何定位所需资源
  - 如何处理对等方的加入与离开
- 可能的方案
  - 集中
  - 分散
  - 半分散

P2P：集中式目录

最初的“Napster”设计
1) 当对等方连接时，它告知中心服务器：
   - IP地址
   - 内容
2) Alice查询 “双截棍.MP3”
3) Alice从Bob处请求文件



P2P：集中式目录中存在的问题

- 单点故障
- 性能瓶颈
- 侵犯版权

文件传输是分散的，而定位内容则是高度集中的

查询洪泛：Gnutella

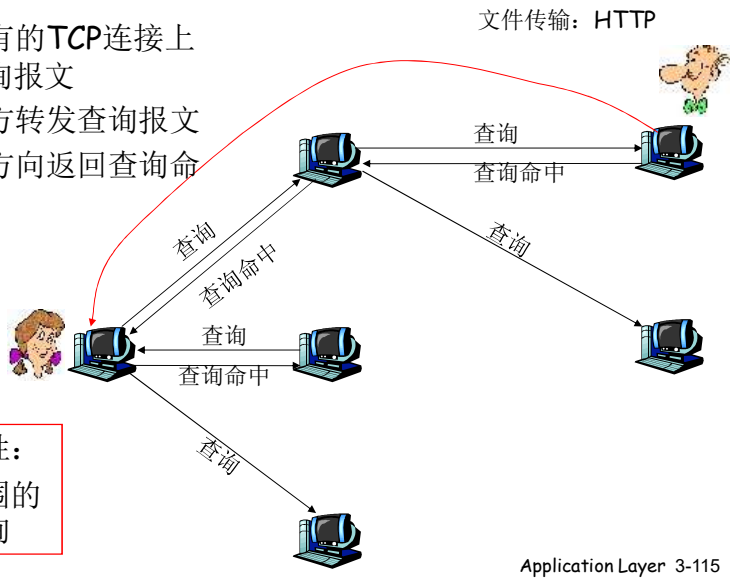
- 全分布式
  - 没有中心服务器
- 开放文件共享协议
- 许多Gnutella客户端实现了Gnutella协议
  - 类似HTTP有许多的浏览器

覆盖网络：图

- 如果X和Y之间有一个TCP连接，则二者之间存在一条边
- 所有活动的对等方和边就是覆盖网络
- 边并不是物理链路
- 给定一个对等方，通常所连接的节点少于10个

Gnutella：协议

- 在已有的TCP连接上发送查询报文
- 对等方转发查询报文
- 以反方向返回查询命中报文



可扩展性：
限制范围的洪泛查询

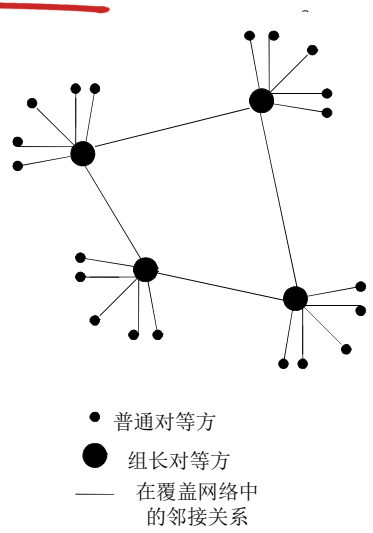
Gnutella：对等方加入

1. 对等方X必须首先发现某些已经在覆盖网络中的其他对等方：使用可用对等方列表
   自己维持一张对等方列表（经常开机的对等方的IP）
   联系维持列表的Gnutella站点
2. X接着试图与该列表上的对等方建立TCP连接，直到与某个对等方Y建立连接
3. X向Y发送一个Ping报文，Y转发该Ping报文
4. 所有收到Ping报文的对等方以Pong报文响应
   IP地址、共享文件的数量及总字节数
5. X收到许多Pong报文，然后它能建立其他TCP连接

对等方离开？

利用不匀称性：KaZaA

- 每个对等方要么是一个组长，要么隶属于一个组长
  - 对等方与其组长之间有TCP连接
  - 组长对之间有TCP连接
- 组长跟踪其所有的孩子的内容
- 组长与其他组长联系
  - 转发查询到其他组长
  - 获得其他组长的数据拷贝



KaZaA：查询

- 每个文件有一个散列标识码和一个描述符
- 客户端向其组长发送关键字查询
- 组长用匹配进行响应：
  - 对每个匹配：元数据、散列标识码和IP地址
- 如果组长将查询转发给其他组长，其他组长也以匹配进行响应
- 客户端选择要下载的文件
  - 向拥有文件的对等方发送一个带散列标识码的HTTP请求

Kazaa小技巧

- 请求排队
  - 限制并行上载的数量
  - 确保每个被传输的文件从上载节点接收一定量的带宽
- 激励优先权
  - 鼓励用户上载文件
  - 加强系统的扩展性
- 并行下载
  - 从多个对等方下载同一个文件的不同部分
    - HTTP的字节范围首部
    - 更快地检索一个文件

Distributed Hash Table (DHT)

- 哈希表
- DHT方案
- 环形DHT 以及覆盖网络
- Peer波动

第2章：提纲

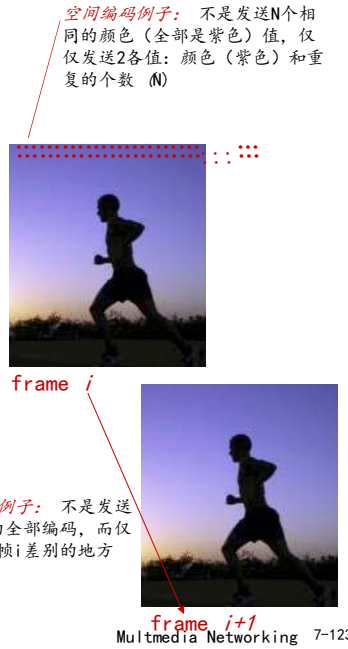
2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

视频流化服务和CDN：上下文

- 视频流量：占据着互联网大部分的带宽
  - Netflix，YouTube：占据37%，16% 的ISP下行流量
  - ~1B YouTube 用户，~75M Netflix用户
- 挑战：规模性–如何服务者 ~1B 用户？
  - 单个超级服务器无法提供服务（为什么）
- 挑战：异构性
  - 不同用户拥有不同的能力（例如：有线接入和移动用户；带宽丰富和受限用户）
- *解决方案：*分布式的，应用层面的基础设施

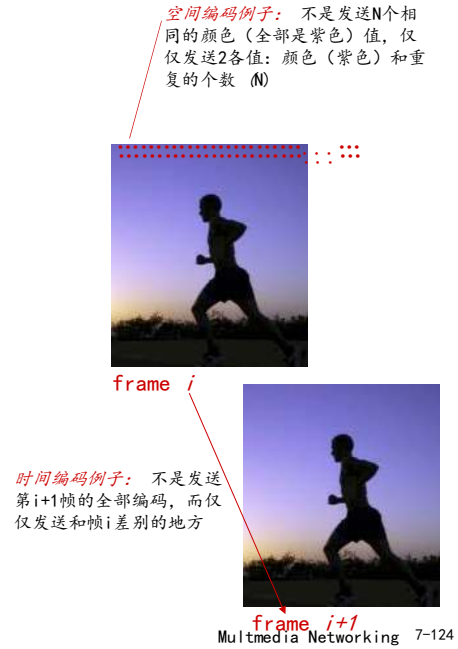
多媒体：视频

- 视频：固定速度显示的图像序列
  - e.g. 24 images/sec
- 网络视频特点：
  - 高码率：>10x于音频，高的网络带宽需求
  - 可以被压缩
  - 90%以上的网络流量是视频
- 数字化图像：像素的阵列
  - 每个像素被若干bits表示
- 编码：使用图像内和图像间的冗余来降低编码的比特数
  - 空间冗余（图像内）
  - 时间冗余（相邻的图像间）



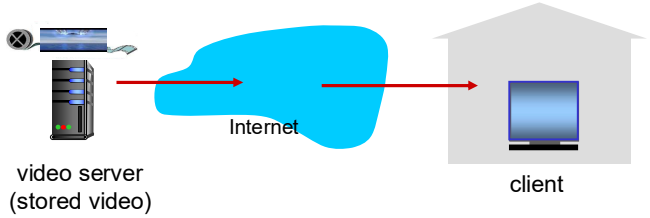
多媒体：视频

- CBR：(constant bit rate)：以固定速率编码
- VBR：(variable bit rate)：视频编码速率随时间的变化而变化
- 例子：
  - MPEG 1 (CD-ROM) 1.5 Mbps
  - MPEG2 (DVD) 3-6 Mbps
  - MPEG4 (often used in Internet, < 1 Mbps)



存储视频的流化服务：

简单场景：



多媒体流化服务：DASH

- DASH: Dynamic, Adaptive Streaming over HTTP
- *服务器：*
  - 将视频文件分割成多个块
  - 每个块独立存储，编码于不同码率（8-10种）
  - *告示文件（manifest file）：*提供不同块的URL
- *客户端：*
  - 先获取告示文件
  - 周期性地测量服务器到客户端的带宽
  - 查询告示文件，在一个时刻请求一个块，HTTP头部指定字节范围
    - 如果带宽足够，选择最大码率的视频块
    - 会话中的不同时刻，可以切换请求不同的编码块（取决于当时的可用带宽）

流式多媒体技术3：DASH

- “智能”客户端：客户端自适应决定
  - 什么时候去请求块（不至于缓存挨饿，或者溢出）
  - 请求什么编码速率的视频块（当带宽够用时，请求高质量的视频块）
  - 哪里去请求块（可以向离自己近的服务器发送URL，或者向高可用带宽的服务器请求）

Content Distribution Networks

- *挑战：*服务器如何通过网络向上百万用户同时流化视频内容（上百万视频内容)?
- *选择1：*单个的、大的超级服务中心“mega-server”
  - 服务器到客户端路径上跳数较多，瓶颈链路的带宽小导致停顿
  - “二八规律”决定了网络同时充斥着同一个视频的多个拷贝，效率低（付费高、带宽浪费、效果差）
  - 单点故障点，性能瓶颈
  - 周边网络的拥塞

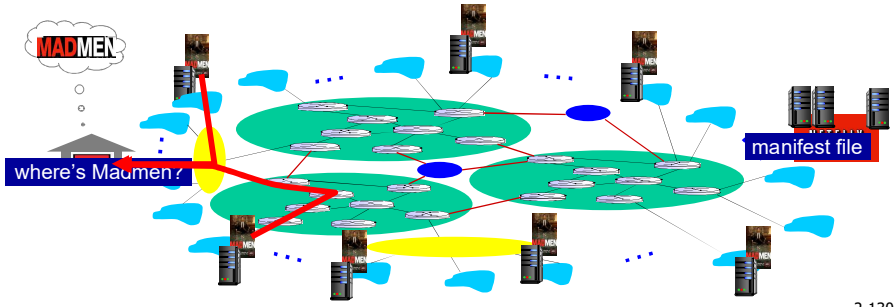
评述：相当简单，但是这个方法不可扩展

Content distribution networks

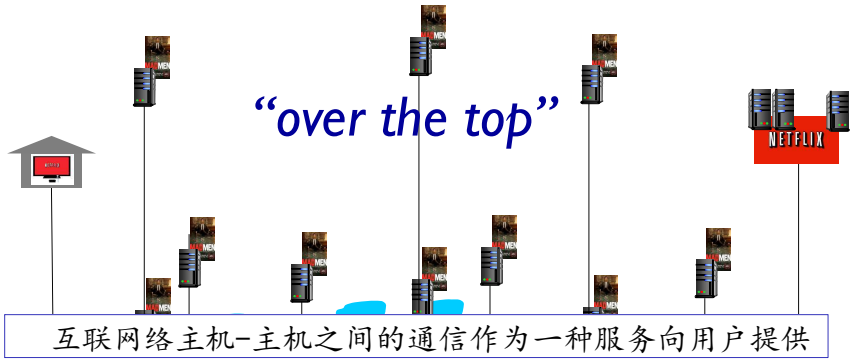
- *选项2:* 通过CDN，全网部署缓存节点，存储服务内容，就近为用户提供服务，提高用户体验
  - *enter deep:* 将CDN服务器深入到许多接入网
    - 更接近用户，数量多，离用户近，管理困难
    - Akamai，1700个位置
  - *bring home:* 部署在少数(10个左右)关键位置，如将服务器簇安装于POP附近（离若干1st ISP POP较近）
    - 采用租用线路将服务器簇连接起来
    - Limelight

Content Distribution Networks (CDNs)

- CDN：在CDN节点中存储内容的多个拷贝
  - e.g. Netflix stores copies of MadMen
- 用户从CDN中请求内容
  - 重定向到最近的拷贝，请求内容
  - 如果网络路径拥塞，可能选择不同的拷贝



Content Distribution Networks (CDNs)



OTT 挑战: 在拥塞的互联网上复制内容

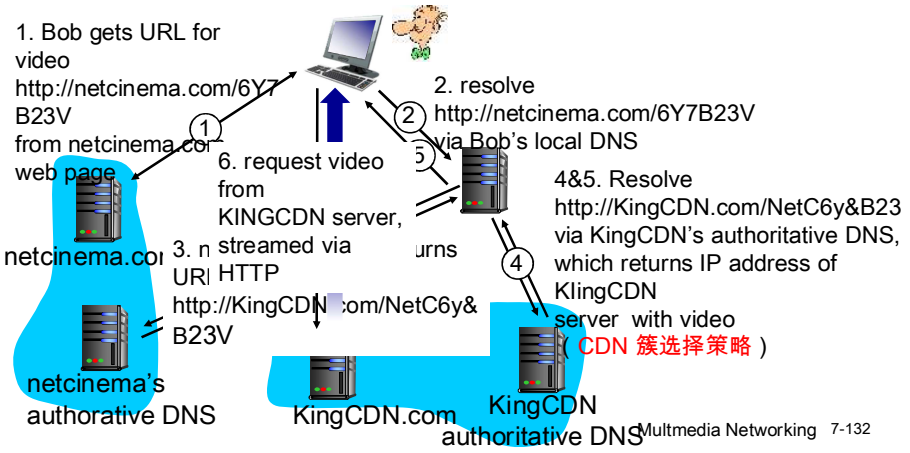
- 从哪个CDN节点中获取内容?
- 用户在网络拥塞时的行为?
- 在哪些CDN节点中存储什么内容?

more .. in chapter 7

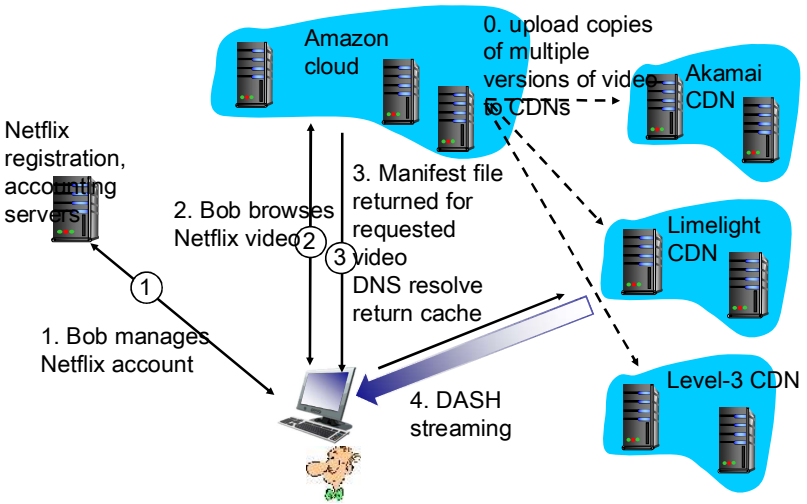
CDN: "简单"内容访问场景

Bob(客户端)请求视频http://netcinema.com/6Y7B23V

- 视频存储在CDN，位于http://KingCDN.com/NetC6y&B23V



案例学习：Netflix



第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

Socket编程

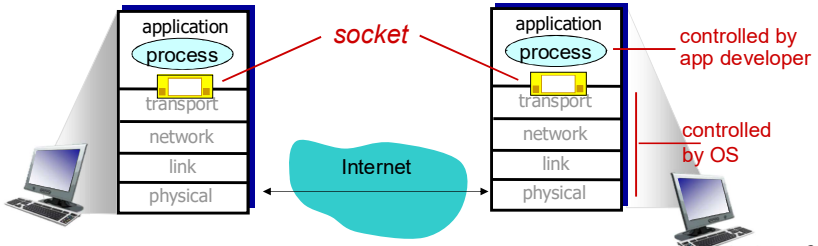
应用进程使用传输层提供的服务才能够交换报文，实现应用协议，实现应用

TCP/IP：应用进程使用Socket API访问传输服务

地点：界面上的SAP(Socket） 方式：Socket API

目标：学习如何构建能借助sockets进行通信的C/S应用程序

socket：分布式应用进程之间的门，传输层协议提供的端到端服务接口



Socket编程

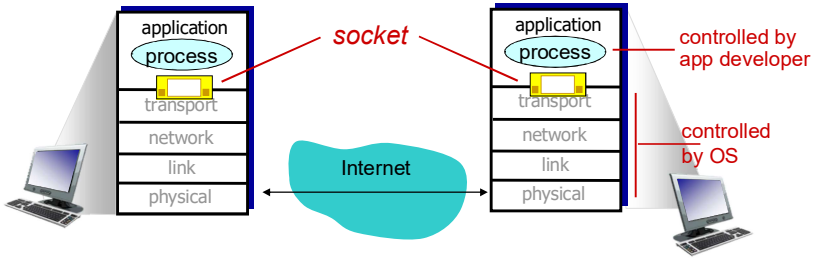
2种传输层服务的socket类型：

- *TCP:* 可靠的、字节流的服务
- *UDP:* 不可靠（数据UDP数据报）服务

TCP套接字编程

套接字：应用进程与端到端传输协议（TCP或UDP）之间的门户

TCP服务：从一个进程向另一个进程可靠地传输字节流



TCP套接字编程

服务器首先运行，等待连接建立

1：服务器进程必须先处于运行状态

- 创建欢迎socket
- 和本地端口捆绑
- 在欢迎socket上阻塞式等待接收用户的连接

客户端主动和服务器建立连接：

2：创建客户端本地套接字（隐式捆绑到本地port）

- 指定服务器进程的IP地址和端口号，与服务器进程连接

4：连接API调用有效时，客户端P与服务器建立了TCP连接

3：当与客户端连接请求到来时

- 服务器接受来自用户端的请求，解除阻塞式等待，返回一个新的socket（与欢迎socket不一样），与客户端通信
  - 允许服务器与多个客户端通信
  - 使用源IP和源端口来区分不同的客户端

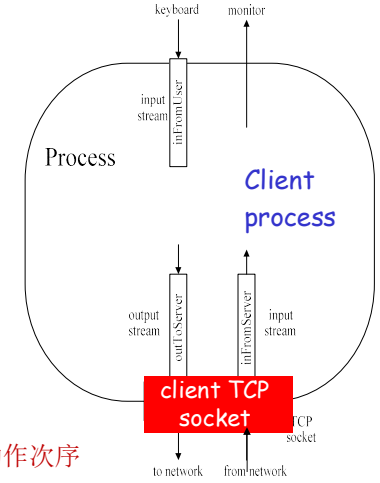
从应用程序的角度

TCP在客户端和服务器进程之间提供了可靠的、字节流（管道）服务

TCPsocket编程

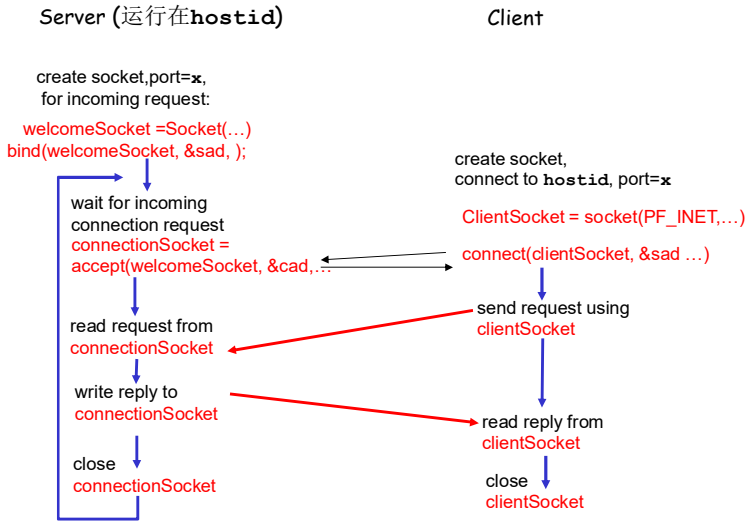
C/S模式的应用样例:

1) 客户端从标准输入装置读取一行字符，发送给服务器
2) 服务器从socket读取字符
3) 服务器将字符转换成大写，然后返回给客户端
4) 客户端从socket中读取一行字符，然后打印出来



实际上，这里描述了C-S之间交互的动作次序

C/S socket 交互: TCP



数据结构 sockaddr_in

IP地址和port捆绑关系的数据结构（标示进程的端节点）

```
struct sockaddr_in {
  short sin_family;    //AF_INET
  u_short sin_port;   // port
   struct in_addr sin_addr ;
          //  IP address, unsigned long
 char sin_zero[8]; // align
 };
```


数据结构 hostent

域名和IP地址的数据结构

```
struct hostent
    {      char *h_name;
           char **h_aliases;
           int h_addrtype;
           int h_length;        /*地址长度*/
           char **h_addr_list;
           #define h_addr h_addr_list[0];
     }
```

作为调用域名解析函数时的参数
返回后，将IP地址拷贝到 sockaddr_in的IP地址部分

例子: C客户端(TCP)

```
/* client.c  */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address of server */
int clientSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry  */

char Sentence[128];
char modifiedSentence[128];

host = argv[1]; port = atoi(argv[2]);

clientSocket = socket(PF_INET, SOCK_STREAM, 0);
          memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
          sad.sin_family = AF_INET; /* set family to Internet */
          sad.sin_port = htons((u_short)port);
          ptrh = gethostbyname(host);
                   /* Convert host name to IP address */
          memcpy(&sad.sin_addr, ptrh->h_addr, ptrh->h_length);
                   //将IP地址拷贝到sad.sin_addr
        connect(clientSocket, (struct sockaddr *)&sad, sizeof(sad));
```

Create client socket, connect to server

例子: C客户端(TCP) （续）

```
Get input stream from user  → gets(Sentence);

Send line to server  → n=write(clientSocket, Sentence, strlen(Sentence)+1);

Read line from server  → n=read(clientSocket, modifiedSentence, sizeof(modifiedSentence

                         printf("FROM SERVER: %s\n",modifiedSentence);

Close connection  → close(clientSocket);
                    }
```

例子: C服务器（TCP）

```
/* server.c */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address of server*/
struct sockaddr_in cad; /*client */
int welcomeSocket, connectionSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry */

char clientSentence[128];
char capitalizedSentence[128];

port = atoi(argv[1]);

welcomeSocket = socket(PF_INET, SOCK_STREAM, 0);
        memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
        sad.sin_family = AF_INET; /* set family to Internet */
        sad.sin_addr.s_addr = INADDR_ANY; /* set the local IP address */
        sad.sin_port = htons((u_short)port);/* set the port number */
bind(welcomeSocket, (struct sockaddr *)&sad, sizeof(sad));
```

Create welcoming socket at port
&
Bind a local address

例子: C服务器（TCP）（续）

```
/* Specify the maximum number of clients that can be queued */
listen(welcomeSocket, 10)

while(1) {

    connectionSocket=accept(welcomeSocket, (struct sockaddr *)&cad, &alen);

    n=read(connectionSocket, clientSentence, sizeof(clientSentence));

    /* capitalize Sentence and store the result in capitalizedSentence*/


    n=write(connectionSocket, capitalizedSentence, strlen(capitalizedSentence)+1);

    close(connectionSocket);
    }
}
```

Wait, on welcoming socket for contact by a client

Write out the result to socket

End of while loop, loop back and wait for another client connection

第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

UDP Socket编程

UDP: 在客户端和服务器之间没有连接

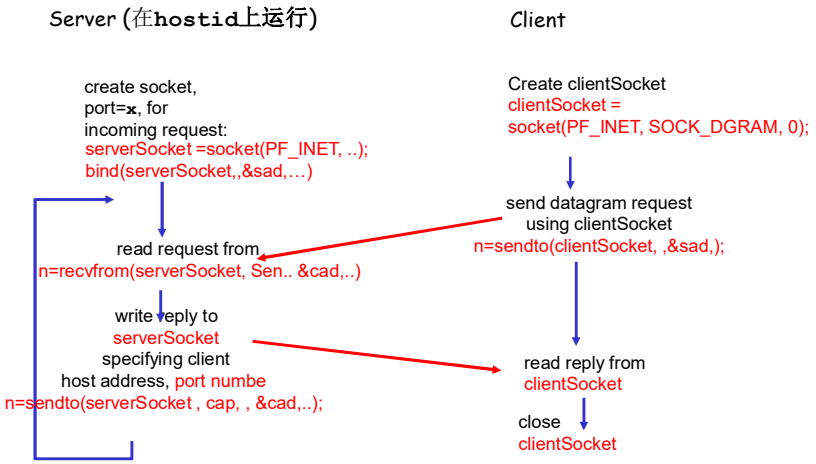
- 没有握手
- 发送端在每一个报文中明确地指定目标的IP地址和端口号
- 服务器必须从收到的分组中提取出发送端的IP地址和端口号

UDP: 传送的数据可能乱序，也可能丢失

进程视角看UDP服务

UDP 为客户端和服务器提供不可靠的字节组的传送服务

Client/server socket 交互: UDP



样例: C客户端 (UDP)

```
/* client.c  */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address */
int clientSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry */

char Sentence[128];
char modifiedSentence[128];

host = argv[1]; port = atoi(argv[2]);

clientSocket = socket(PF_INET, SOCK_DGRAM, 0);

/* determine the server's address */
        memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
        sad.sin_family = AF_INET; /* set family to Internet */
        sad.sin_port = htons((u_short)port);
        ptrh = gethostbyname(host);
            /* Convert host name to IP address */
        memcpy(&sad.sin_addr, ptrh->h_addr, ptrh->h_length);
```

创建客户端 socket,
没有连接到服务器

样例: C客户端 (UDP)（续）

```
Get input stream from user →  gets(Sentence);

Send line to server →  addr_len =sizeof(struct sockaddr);
                       n=sendto(clientSocket, Sentence, strlen(Sentence)+1,
                                (struct sockaddr *) &sad, addr_len);

Read line from server →  n=recvfrom(clientSocket, modifiedSentence, sizeof(modifiedSentence),
                                    (struct sockaddr *) &sad, &addr_len);

                         printf("FROM SERVER: %s\n",modifiedSentence);

Close connection →  close(clientSocket);
                    }
```


样例: C服务器(UDP)

```
/* server.c  */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address */
struct sockaddr_in cad;
int serverSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry */

char clientSentence[128];
char capitalizedSentence[128];

port = atoi(argv[1]);

serverSocket = socket(PF_INET, SOCK_DGRAM, 0);
        memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
        sad.sin_family = AF_INET; /* set family to Internet */
        sad.sin_addr.s_addr = INADDR_ANY; /* set the local IP address */
        sad.sin_port = htons((u_short)port);/* set the port number */
bind(serverSocket, (struct sockaddr *)&sad, sizeof(sad));
```

Create welcoming socket at port
&
Bind a local address

样例: C服务器(UDP)（续）

```
while(1) {

   n=recvfrom(serverSocket, clientSentence, sizeof(clientSentence), 0
              (struct sockaddr *) &cad, &addr_len );

   /* capitalize Sentence and store the result in capitalizedSentence*/

   n=sendto(serverSocket , capitalizedSentence, strlen(capitalizedSentence)+1,
            (struct sockaddr *) &cad, &addr_len);

   }
}
```

Receive messages from clients

Write out the result to socket

End of while loop, loop back and wait for another client connection

第2章：小结

- 应用程序体系结构
  - 客户-服务器
  - P2P
  - 混合
- 应用程序需要的服务品质描述:
  - 可靠性、带宽、延时、安全
- Internet传输层服务模式
  - 可靠的、面向连接的服务：TCP
  - 不可靠的数据报：UDP
- 流行的应用层协议:
  - HTTP
  - FTP
  - SMTP, POP, IMAP
  - DNS
- Socket编程

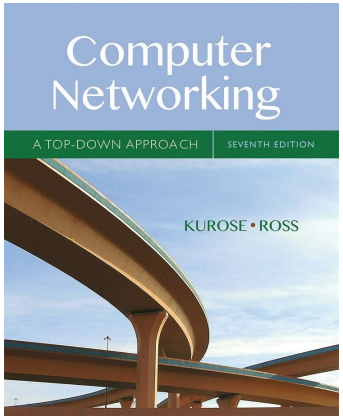
第2章：小结

更重要的：学习协议的知识

- 应用层协议报文类型：请求/响应报文：
  - 客户端请求信息或服务
  - 服务器以数据、状态码进行响应
- 报文格式：
  - 首部：关于数据信息的字段
  - 数据：被交换的信息
- 控制报文 vs. 数据报文
  - 带内、带外
- 集中式 vs. 分散式
- 无状态 vs. 维护状态
- 可靠的 vs. 不可靠的报文传输
- 在网络边缘处理复杂性

一个协议定义了在两个或多个通信实体之间交换报文的格式和次序、以及就一条报文传输和接收或其他事件采取的动作

第 3 章
传 输 层



中国科学技术大学
自动化系 郑烇
改编自Jim kurose, Keith Ross

Computer Networking: A Top Down Approach
7th edition
Jim Kurose, Keith Ross
Addison-Wesley
April 2016

第三章：传输层

目标：

- 理解传输层的工作原理
  - 多路复用/解复用
  - 可靠数据传输
  - 流量控制
  - 拥塞控制
- 学习Internet的传输层协议
  - UDP：无连接传输
  - TCP：面向连接的可靠传输
  - TCP的拥塞控制

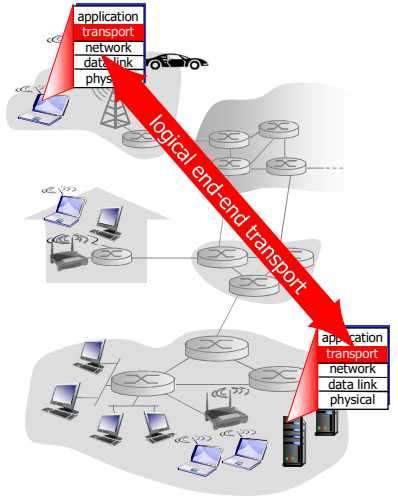
第三章：提纲

3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理
3.7 TCP 拥塞控制

传输服务和协议

- 为运行在不同主机上的应用进程提供逻辑通信
- 传输协议运行在端系统
  - 发送方：将应用层的报文分成报文段，然后传递给网络层
  - 接收方：将报文段重组成报文，然后传递给应用层
- 有多个传输层协议可供应用选择
  - Internet: TCP和UDP



传输层 vs. 网络层

- 网络层服务：主机之间的逻辑通信
- 传输层服务：进程间的逻辑通信
  - 依赖于网络层的服务
    - 延时、带宽
  - 并对网络层的服务进行增强
    - 数据丢失、顺序混乱、加密

类比：东西2个家庭的通信

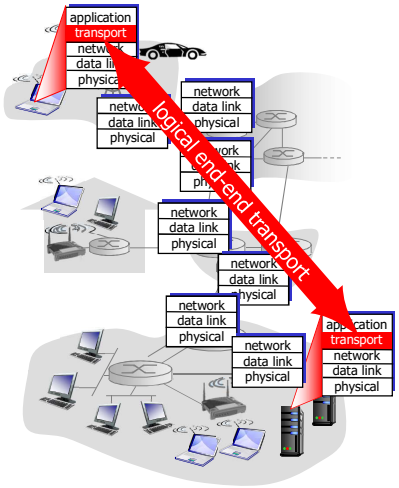
Ann家的12个小孩给另Bill家的12个小孩发信

- 主机 = 家庭
- 进程 = 小孩
- 应用层报文= 信封中的信件
- 传输协议= Ann 和 Bill
  - 为家庭小孩提供复用解复用服务
- 网络层协议 = 邮政服务
  - 家庭-家庭的邮包传输服务

有些服务是可以加强的：不可靠 -> 可靠；安全
但有些服务是不可以被加强的：带宽，延迟

Internet传输层协议

- 可靠的、保序的传输： TCP
  - 多路复用、解复用
  - 拥塞控制
  - 流量控制
  - 建立连接
- 不可靠、不保序的传输：UDP
  - 多路复用、解复用
  - 没有为尽力而为的IP服务添加更多的其它额外服务
- 都不提供的服务：
  - 延时保证
  - 带宽保证



第三章：提纲

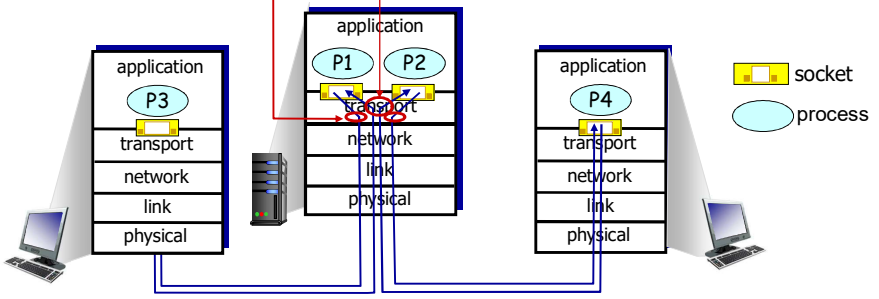
3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理
3.7 TCP 拥塞控制

多路复用/解复用

在发送方主机**多路复用**
从多个套接字接收来自多个进程的报文，根据套接字对应的**IP地址和端口号**等信息对报文段用头部加以封装 (该头部信息用于以后的解复用)

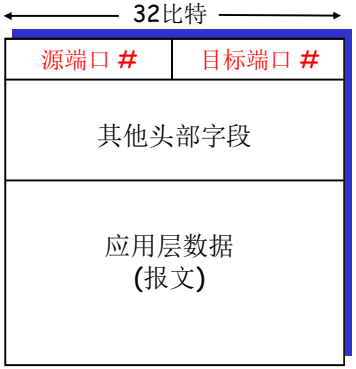
在接收方主机多路**解复用**
根据报文段的头部信息中的**IP地址和端口号**将接收到的报文段发给正确的套接字(和对应的应用进程)



多路解复用工作原理

UDP和TCP不同

- 解复用作用：TCP或UDP实体采用哪些信息，将段的数据部分交给正确的socket，从而交给正确的进程
- 主机收到IP数据报
  - 每IP数据报有源IP地址和目标IP地址
  - IP数据报承载一个传输层报文段，每个段有一个**源端口号和目标端口号**
- 主机联合使用**IP地址**和**端口号**将报文段发送给合适的套接字，交给相应的进程(线程)
  - TCP使用4元组
  - UDP使用2元组（目标IP，目标Port）



TCP/UDP报文段格式

无连接(UDP)多路解复用

- 创建套接字：

服务器端：

```
serverSocket=socket(PF_INET,
    SOCK_DGRAM,0);
bind(serverSocket, &sad, sizeof(sad));
```

serverSocket和Sad指定的端口号捆绑

客户端：

UDP和TCP不同

```
ClientSocket=socket(PF_INET,
    SOCK_DGRAM,0);
```

没有Bind，ClientSocket和OS为之分配的某个端口号捆绑（客户端使用什么端口号无所谓，客户端主动找服务器）

- 在接收端，UDP套接字用二元组标识 (目标IP地址、目标端口号)
- 当主机收到UDP报文段：
  - 检查报文段的目标端口号
  - 用该端口号将报文段定位给套接字
- 如果两个不同源IP地址/源端口号的数据报，但是有相同的目标IP地址和端口号，则被定位到相同的套接字

PID	Socket	Des IP	Des Port
12345	77888	202.38.64.1	80
12346	77999	202.38.64.1	11010
……			

无连接的多路解复用

- 回顾：创建拥有本地端口号的套接字

```
DatagramSocket mySocket1
= new DatagramSocket(12534);
```

- 回顾：当创建UDP段采用端口号，可以指定：
  - 目标IP地址
  - 目标端口号

- 当主机接收到UDP段时：
  - 检查UDP段中的目标端口号
  - 将UDP段交给具备那个端口号的套接字

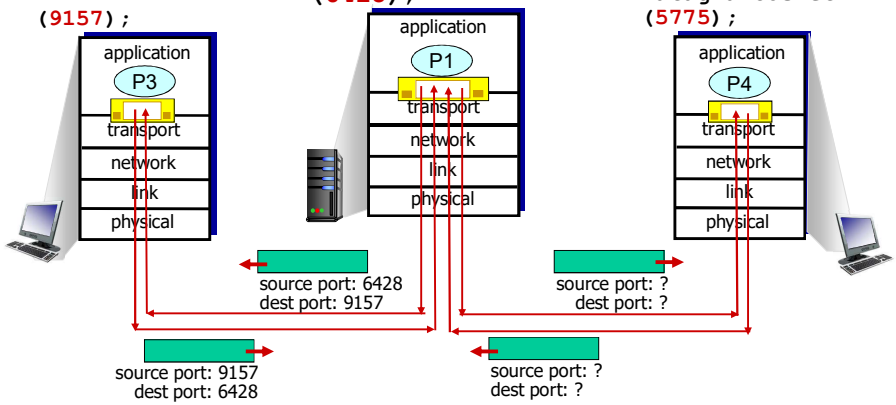
具备相同目标IP地址和目标端口号，即使是源IP地址 或/且源端口号的不同IP数据报，将会被传到相同的目标UDP套接字上

无连接多路复用：例子

```
DatagramSocket
mySocket2 = new
DatagramSocket
(9157);
```

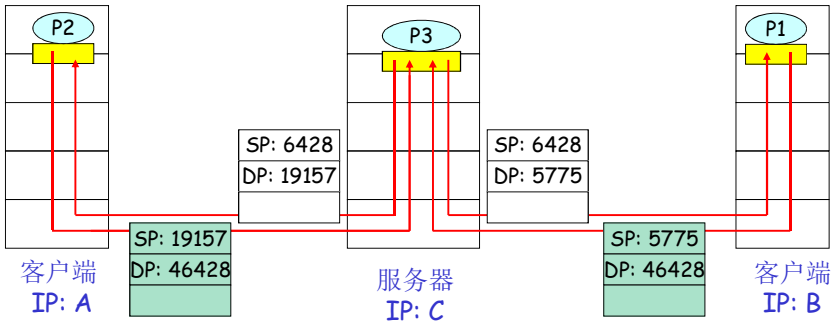
```
DatagramSocket
serverSocket = new
DatagramSocket
(6428);
```

```
DatagramSocket
mySocket1 = new
DatagramSocket
(5775);
```



无连接多路解复用(续)

```
serverSocket = serverSocket=socket(PF_INET,SOCK_DGRAM,0);
bind(serverSocket, sad, sizeof(sad));
```



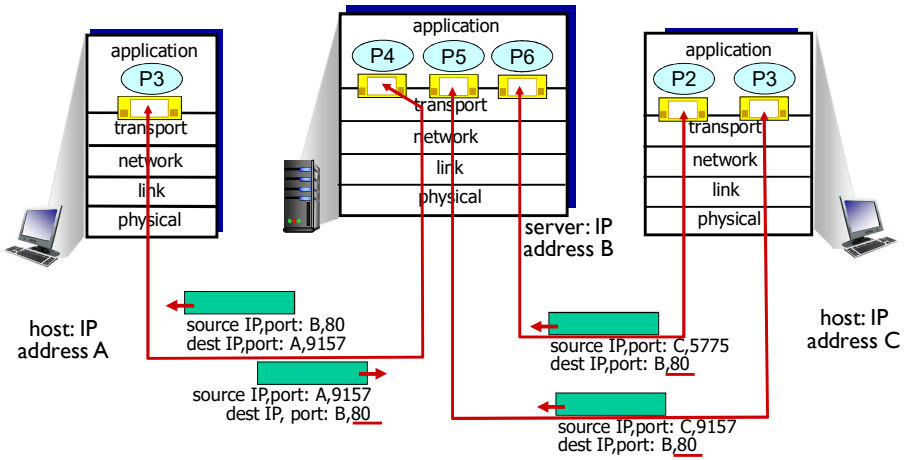
在每个接收到的报文段中都提供了"返回地址"

Socket	PID	Des IP	Des Port
77888	P3	C	6428
.....			

面向连接(TCP)的多路复用

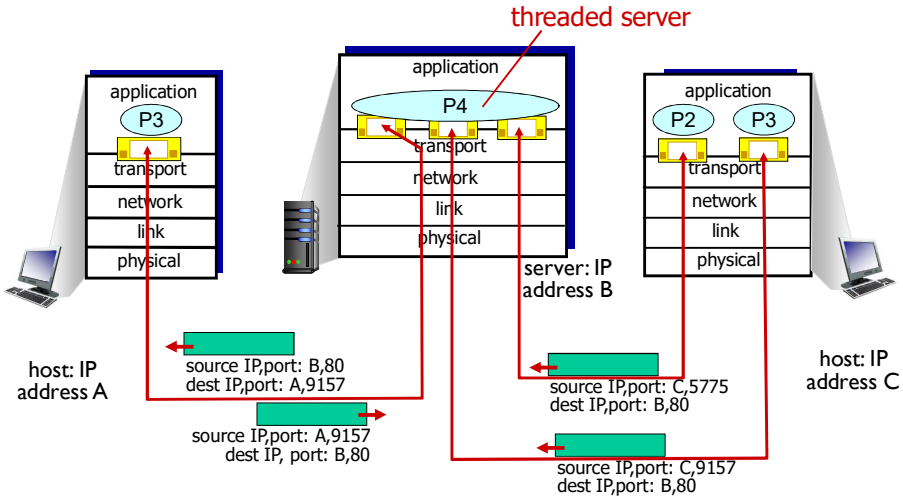
- TCP套接字:四元组本地标识:
  - 源IP地址
  - 源端口号
  - 目的IP地址
  - 目的端口号
- 解复用：接收主机用这四个值来将数据报定位到合适的套接字
- 服务器能够在一个TCP端口上同时支持多个TCP套接字：
  - 每个套接字由其四元组标识（有不同的源IP和源PORT）
- Web服务器对每个连接客户端有不同的套接字
  - 非持久对每个请求有不同的套接字

面向连接的解复用：例子



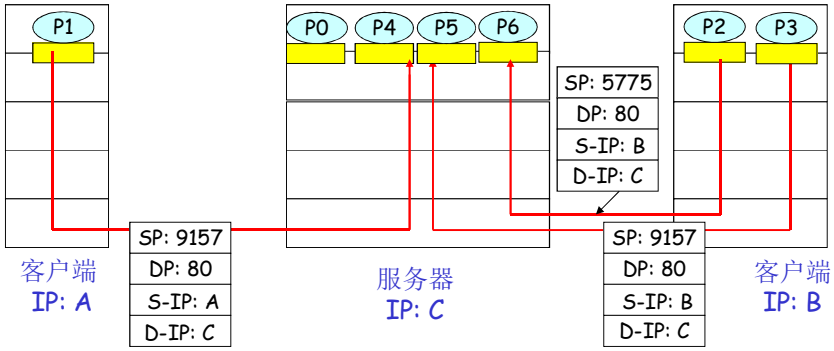
three segments, all destined to IP address: B,
dest port: 80 are demultiplexed to *different* sockets

面向连接的解复用：例子



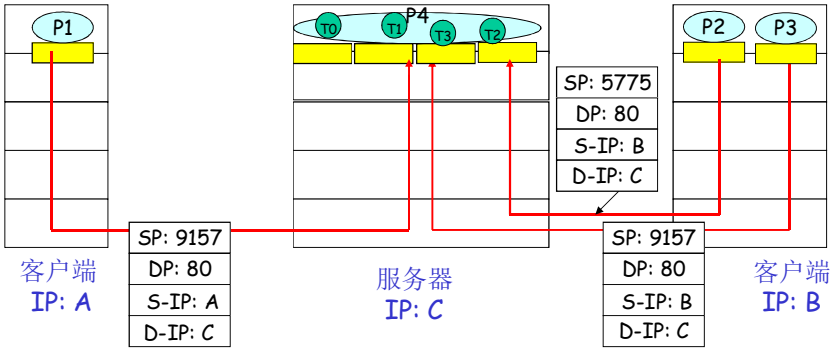
面向

Socket	PID	SIP	SPort	DIP	DPort
90000	P0			C	80
90004	P4	A	9157	C	80
90005	P5	B	9157	C	80
90006	P6	B	5775	C	80



Socke	PID	SIP	SPort	DIP	DPort
80011	P1	C	80	A	9157

面向连接的多路复用：多线程Web Server



- 一个进程下面可能有多个线程：由多个线程分别为客户提供服务
- 在这个场景下，还是根据4元组决定将报文段内容同一个进程下的不同线程
- 解复用到不同线程

第三章：提纲

3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

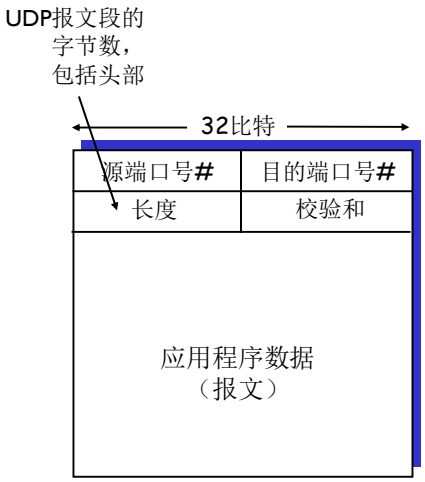
3.6 拥塞控制原理
3.7 TCP 拥塞控制

UDP: User Datagram Protocol [RFC 768]

- "no frills," "bare bones"Internet传输协议
- "尽力而为"的服务，报文段可能
  - 丢失
  - 送到应用进程的报文段乱序
- 无连接：
  - UDP发送端和接收端之间没有握手
  - 每个UDP报文段都被独立地处理

- UDP 被用于：
  - 流媒体（丢失不敏感，速率敏感、应用可控制传输速率）
  - DNS
  - SNMP
- 在UDP上可行可靠传输：
  - 在应用层增加可靠性
  - 应用特定的差错恢复

UDP：用户数据报协议



UDP报文段格式

为什么要有UDP?

- 不建立连接 （会增加延时）
- 简单：在发送端和接收端没有连接状态
- 报文段的头部很小(开销小)
- 无拥塞控制和流量控制：UDP可以尽可能快的发送报文段
  - 应用->传输的速率= 主机->网络的速率

UDP校验和

目标： 检测在被传输报文段中的差错 (如比特反转)

发送方：

- 将报文段的内容视为16比特的整数
- 校验和：报文段的加法和 （1的补运算）
- 发送方将校验和放在UDP的校验和字段

接收方：

- 计算接收到的报文段的校验和
- 检查计算出的校验和与校验和字段的内容是否相等：
  - 不相等---检测到差错
  - 相等---没有检测到差错，但也许还是有差错
    - 残存错误

Internet校验和的例子

- 注意：当数字相加时，在最高位的进位要回卷，再加到结果上

例子：两个16比特的整数相加

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
回卷	(1)	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
和	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
校验和	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

- 目标端：校验范围+校验和=1111111111111111 通过校验
  - 否则没有通过校验
- 注：求和时，必须将进位回卷到结果上

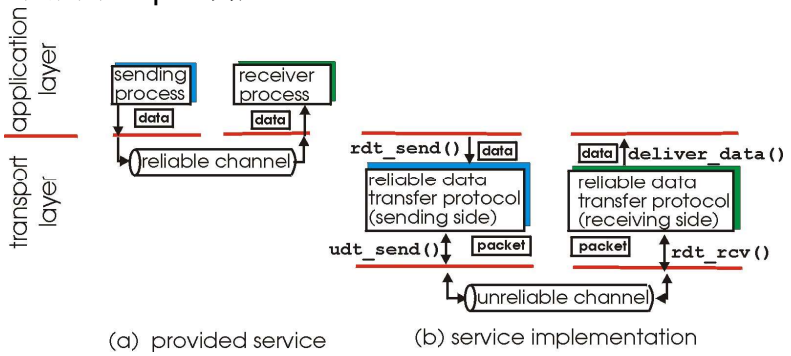
第三章：提纲

3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理
3.7 TCP 拥塞控制

可靠数据传输（rdt）的原理

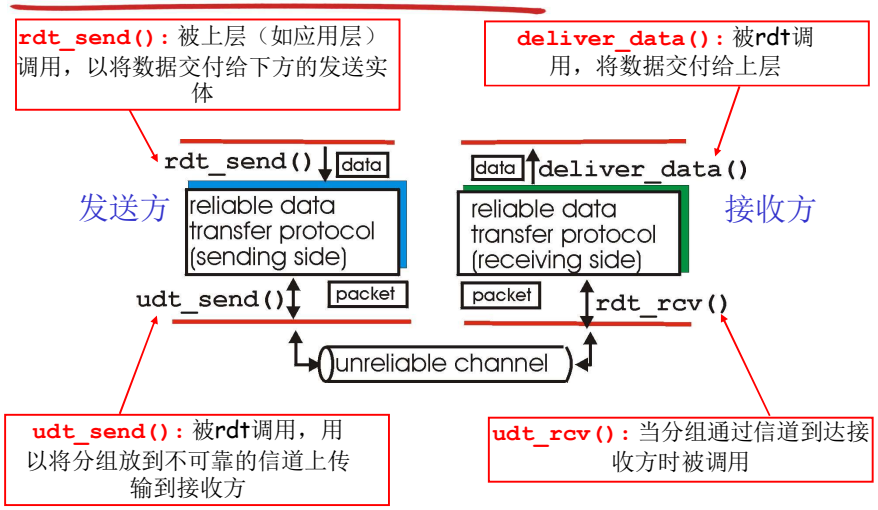
- rdt在应用层、传输层和数据链路层都很重要
- 是网络Top 10问题之一



(a) provided service (b) service implementation

- 信道的不可靠特点决定了可靠数据传输协议（ rdt ）的复杂性

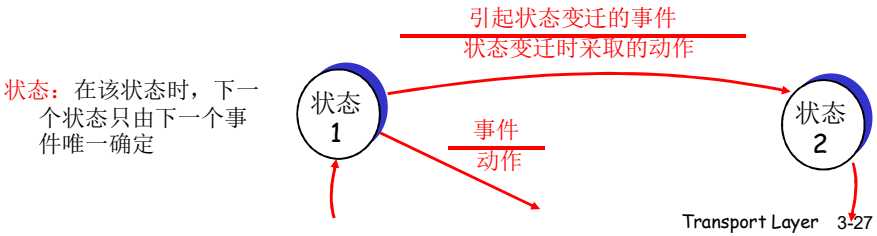
可靠数据传输：问题描述



可靠数据传输：问题描述（续）

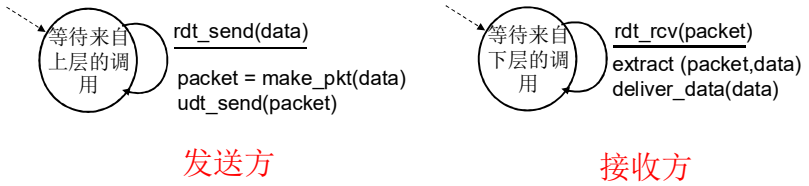
我们将：

- 渐增式地开发可靠数据传输协议（ rdt ）的发送方和接收方
- 只考虑单向数据传输
  - 但控制信息是双向流动的！
- 双向的数据传输问题实际上是2个单向数据传输问题的综合
- 使用有限状态机 (FSM) 来描述发送方和接收方



Rdt1.0：在可靠信道上的可靠数据传输

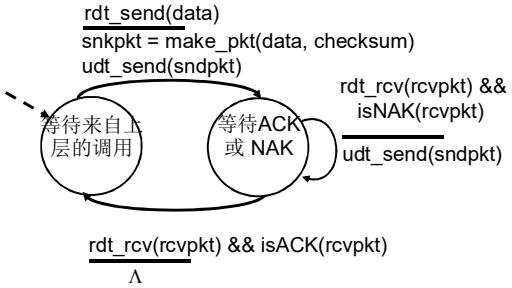
- 下层的信道是完全可靠的
  - 没有比特出错
  - 没有分组丢失
- 发送方和接收方的FSM
  - 发送方将数据发送到下层信道
  - 接收方从下层信道接收数据



Rdt2.0：具有比特差错的信道

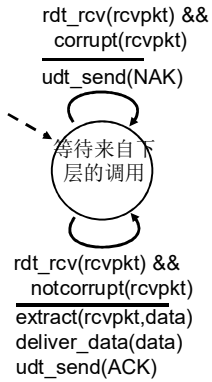
- 下层信道可能会出错：将分组中的比特翻转
  - 用校验和来检测比特差错
- 问题：怎样从差错中恢复：
  - 确认(ACK)：接收方显式地告诉发送方分组已被正确接收
  - 否定确认(NAK): 接收方显式地告诉发送方分组发生了差错
    - 发送方收到NAK后，发送方重传分组
- rdt2.0中的新机制：采用差错控制编码进行差错检测
  - 发送方差错控制编码、缓存
  - 接收方使用编码检错
  - 接收方的反馈：控制报文（ACK，NAK）：接收方→发送方
  - 发送方收到反馈相应的动作

rdt2.0：FSM描述

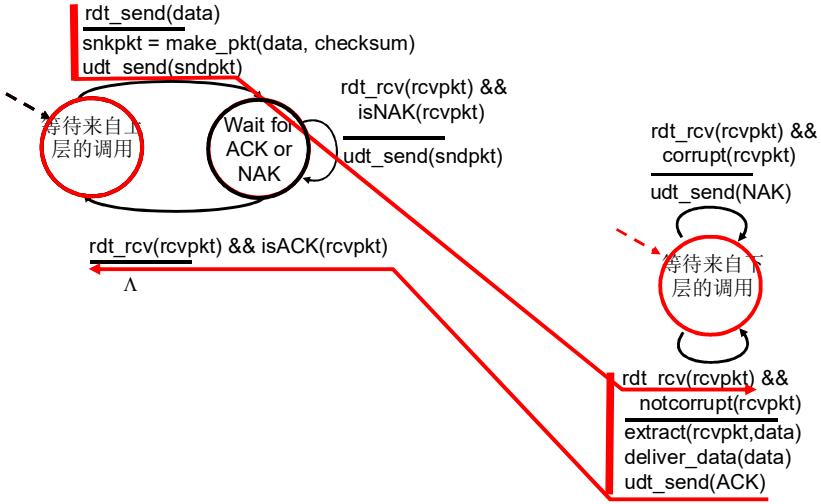


发送方

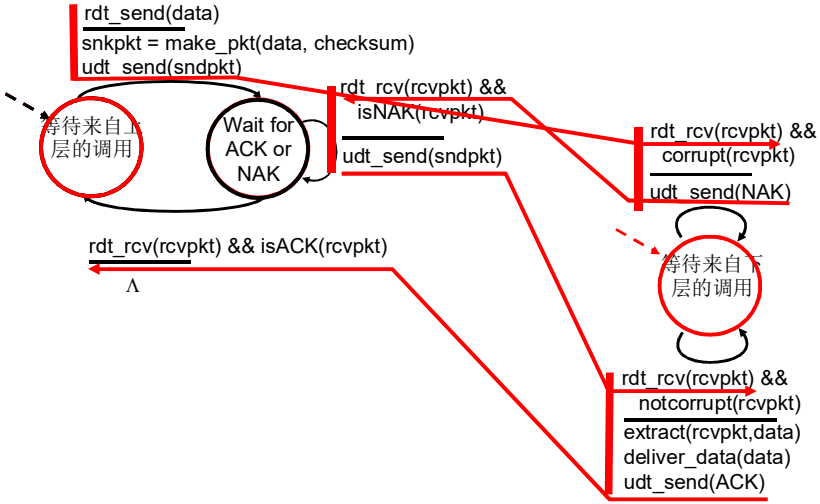
接收方



rdt2.0：没有差错时的操作



rdt2.0：有差错时



rdt2.0的致命缺陷！ -> rdt2.1

如果ACK/NAK出错？

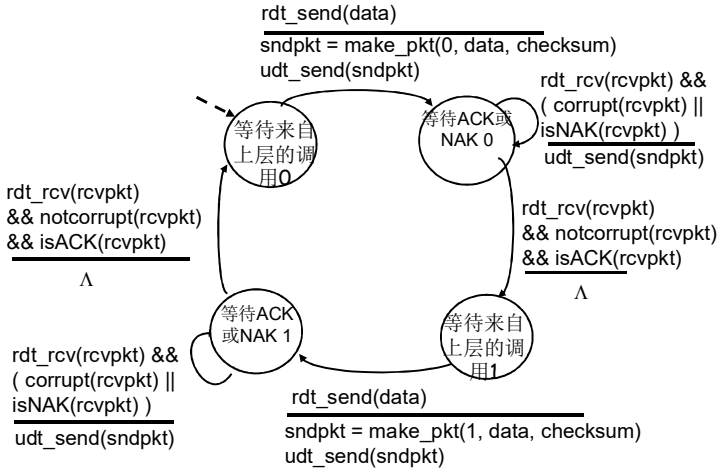
- 发送方不知道接收方发生了什么事情！
- 发送方如何做？
  - 重传？可能重复
  - 不重传？可能死锁(或出错)
- 需要引入新的机制
  - 序号

处理重复：

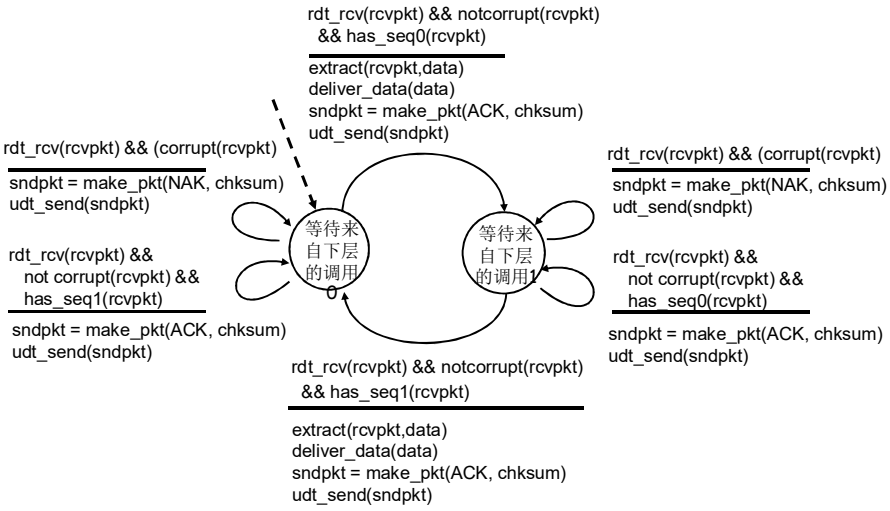
- 发送方在每个分组中加入序号
- 如果ACK/NAK出错，发送方重传当前分组
- 接收方丢弃（不发给上层）重复分组

停等协议
发送方发送一个分组，然后等待接收方的应答

rdt2.1：发送方处理出错的ACK/NAK



rdt2.1：接收方处理出错的ACK/NAK



rdt2.1：讨论

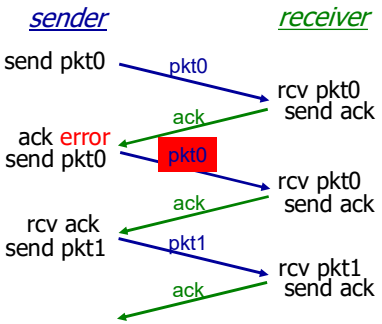
发送方：

- 在分组中加入序列号
- 两个序列号（0，1）就足够了
  - 一次只发送一个未经确认的分组
- 必须检测ACK/NAK是否出错（需要EDC ）
- 状态数变成了两倍
  - 必须记住当前分组的序列号为0还是1

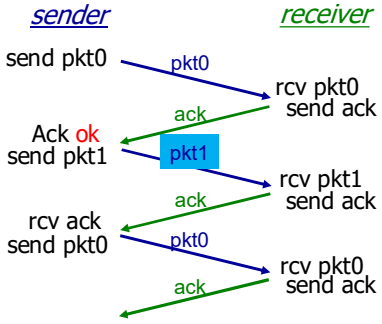
接收方：

- 必须检测接收到的分组是否是重复的
  - 状态会指示希望接收到的分组的序号为0还是1
- 注意：接收方并不知道发送方是否正确收到了其ACK/NAK
  - 没有安排确认的确认
  - 具体解释见下页

rdt2.1的运行



(a) ack error



(b) ack right

接收方不知道它最后发送的ACK/NAK是否被正确地收到

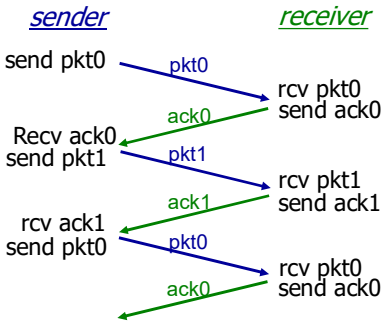
- 发送方不对收到的ack/nak给确认，没有所谓的确认的确认；
- 接收方发送ack，如果后面接收方收到的是：
  - 老分组p0？则之前的ack 错误
  - 下一个分组？P1，之前的ack正确

rdt2.2：无NAK的协议

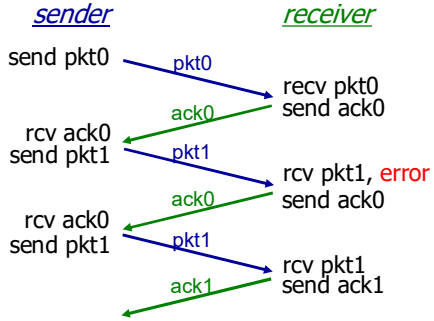
- 功能同rdt2.1，但只使用ACK(ack 要编号）
- 接收方对最后正确接收的分组发ACK，以替代NAK
  - 接收方必须显式地包含被正确接收分组的序号
- 发送方收到重复的ACK（如：再次收到ack0）时，与收到NAK采取相同的动作：重传当前分组
- 为后面的一次发送多个数据单位做一个准备
  - 一次能够发送多个
  - 每一个的应答都有：ACK，NACK；麻烦
  - 使用对前一个数据单位的ACK，代替本数据单位的nak
  - 确认信息减少一半，协议处理简单

NAK free

rdt2.2的运行

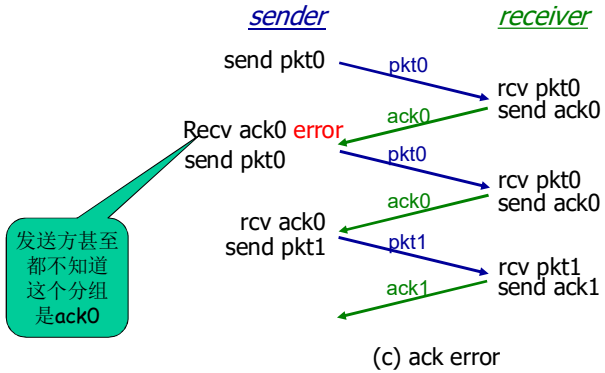


(a) No error(packet or ack error)



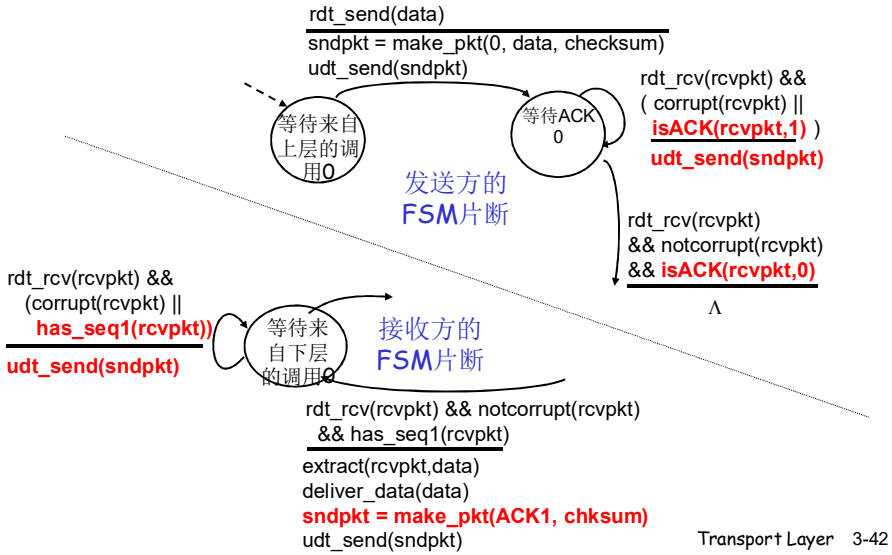
(b) packet error

rdt2.2的运行



(c) ack error

rdt2.2：发送方和接收方片断



rdt3.0：具有比特差错和分组丢失的信道

新的假设：下层信道可能会丢失分组（数据或ACK）

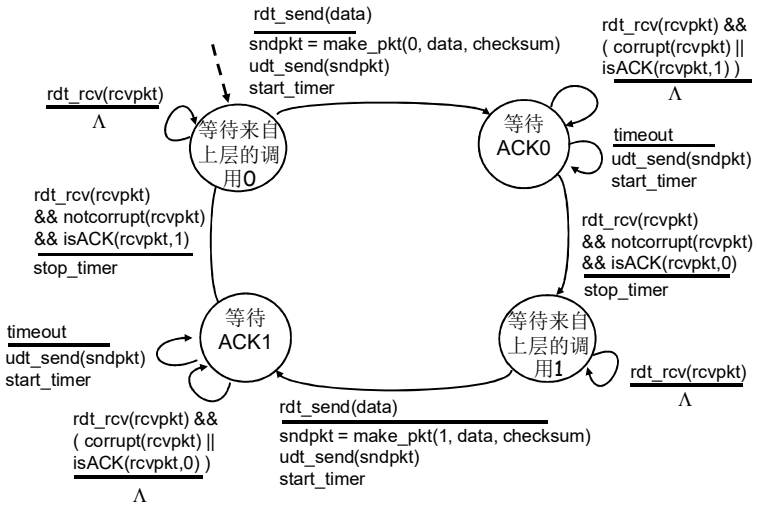
- 会死锁
- 机制还不够处理这种状况：
  - 检验和
  - 序列号
  - ACK
  - 重传

方法：发送方等待ACK一段合理的时间

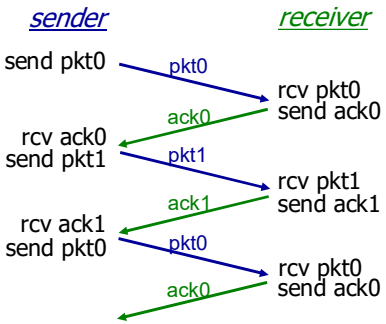
- 发送端超时重传：如果到时没有收到ACK->重传
- 问题：如果分组（或ACK）只是被延迟了：
  - 重传将会导致数据重复，但利用序列号已经可以处理这个问题
  - 接收方必须指明被正确接收的序列号
- 需要一个倒计数定时器

链路层的timeout时间确定的
传输层timeout时间是适应式的

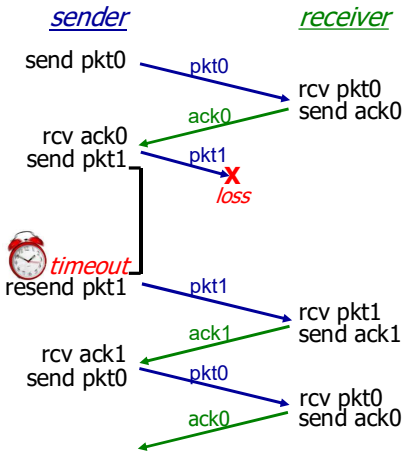
rdt3.0 发送方



rdt3.0的运行

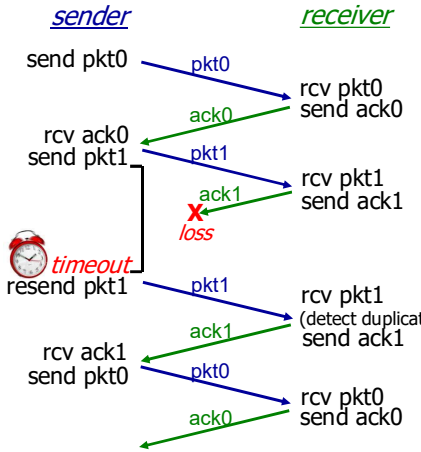


(a) no loss

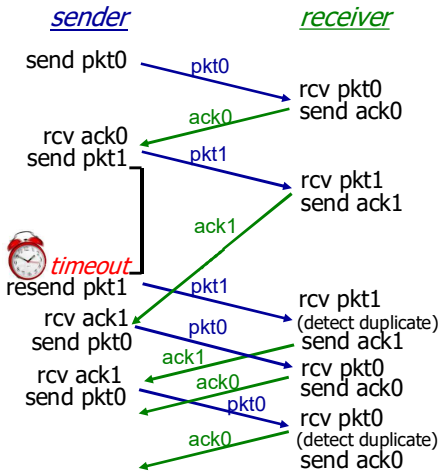


(b) packet loss

rdt3.0的运行



(c) ACK loss



(d) premature timeout/ delayed ACK

- 过早超时（延迟的ACK）也能够正常工作；但是效率较低，一半的分组和确认是重复的；
- 设置一个合理的超时时间也是比较重要的；

rdt3.0的性能

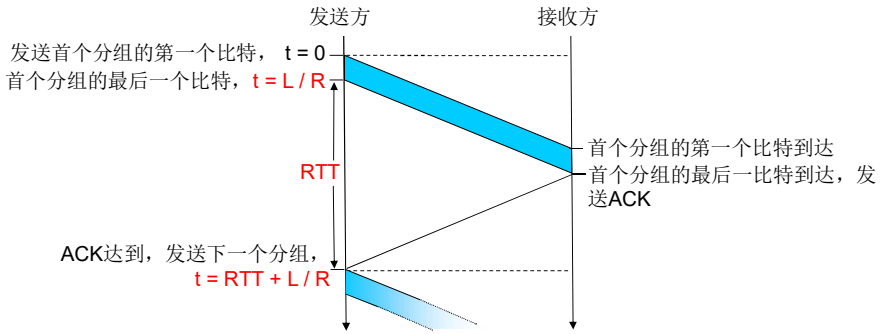
- rdt3.0可以工作，但链路容量比较大的情况下，性能很差
  - 链路容量比较大，一次发一个PDU 的不能够充分利用链路的传输能力
- 例：1 Gbps的链路，15 ms端-端传播延时，分组大小为1kB：

$$T_{transmit} = \frac{L\text{(分组长度，比特)}}{R\text{(传输速率，bps)}} = \frac{8kb/pkt}{10^9\ b/sec} = 8\mu s$$

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

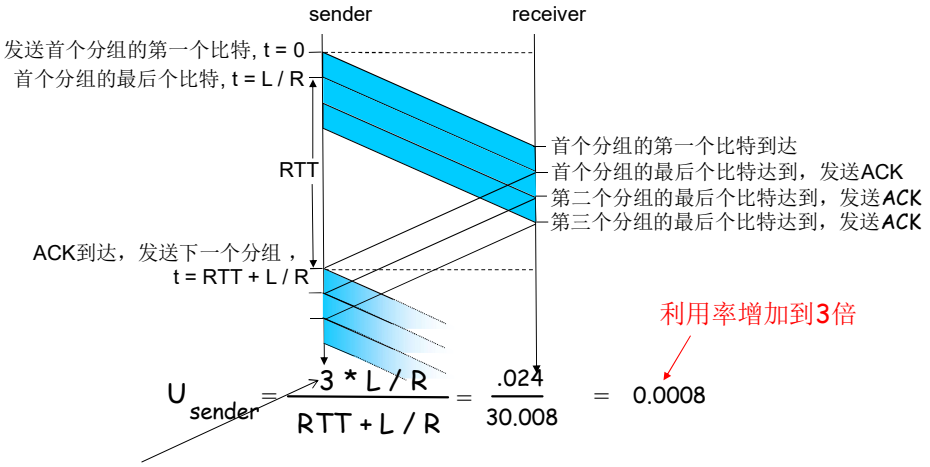
  - U_{sender}：利用率 - 忙于发送的时间比例
  - 每30ms发送1KB的分组→ 270kbps=33.75kB/s 的吞吐量（在1 Gbps 链路上）
  - 瓶颈在于：网络协议限制了物理资源的利用！

rdt3.0：停-等操作



$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

流水线：提高链路利用率

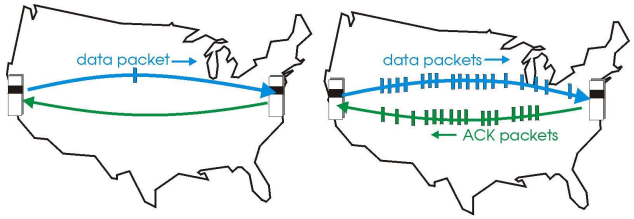


- 增加n,能提高链路利用率
- 但当达到某个n,其u=100%时,无法再通过增加n，提高利用率
- 瓶颈转移了->链路带宽

流水线协议

流水线：允许发送方在未得到对方确认的情况下一次发送多个分组

- 必须增加序号的范围:用多个bit表示分组的序号
- 在发送方/接收方要有缓冲区
  - 发送方缓冲：未得到确认，可能需要重传；
  - 接收方缓存：上层用户取用数据的速率≠接收到的数据速率；接收到的数据可能乱序，排序交付（可靠）



(a) a stop-and-wait protocol in operation　(b) a pipelined protocol in operation

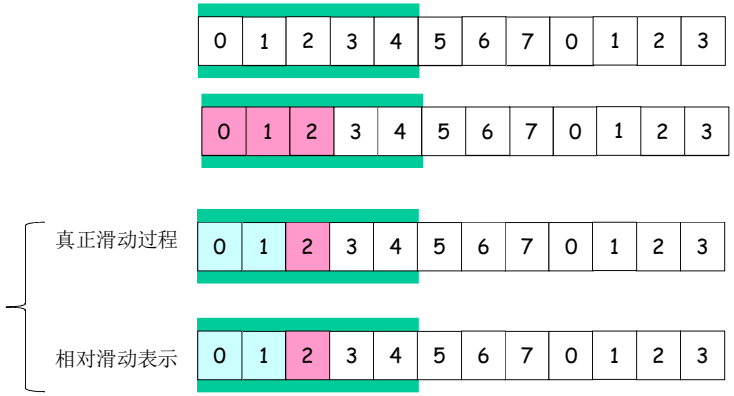
- 两种通用的流水线协议：回退N步(GBN)和选择重传(SR)

通用：滑动窗口(slide window)协议

- 发送缓冲区
  - 形式：内存中的一个区域，落入缓冲区的分组可以发送
  - 功能：用于存放已发送，但是没有得到确认的分组
  - 必要性：需要重发时可用
- 发送缓冲区的大小：一次最多可以发送多少个未经确认的分组
  - 停止等待协议=1
  - 流水线协议>1，合理的值，不能很大，链路利用率不能够超100%
- 发送缓冲区中的分组
  - 未发送的：落入发送缓冲区的分组，可以连续发送出去；
  - 已经发送出去的、等待对方确认的分组：发送缓冲区的分组只有得到确认才能删除

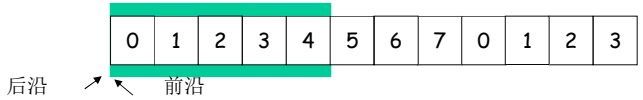
发送窗口滑动过程-相对表示方法

- 采用相对移动方式表示，分组不动
- 可缓冲范围移动，代表一段可以发送的权力



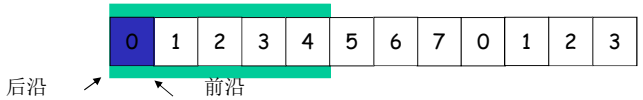
滑动窗口(slide window)协议

- 发送窗口：发送缓冲区内容的一个范围
  - 那些已发送但是未经确认分组的序号构成的空间
- 发送窗口的最大值<=发送缓冲区的值
- 一开始：没有发送任何一个分组
  - 后沿=前沿
  - 之间为发送窗口的尺寸=0
- 每发送一个分组，前沿前移一个单位



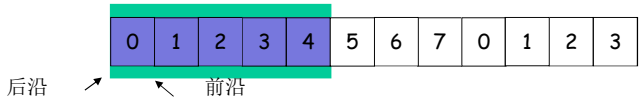
发送窗口的移动->前沿移动

- 发送窗口前沿移动的极限：不能够超过发送缓冲区



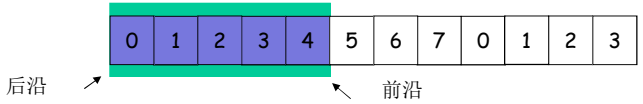
发送窗口的移动->前沿移动

- 发送窗口前沿移动的极限：不能够超过发送缓冲区



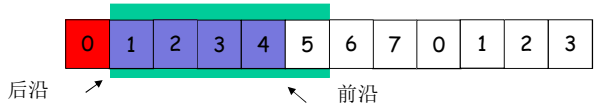
发送窗口的移动->后沿移动

- 发送窗口后沿移动
  - 条件：收到老分组的确认
  - 结果：发送缓冲区罩住新的分组，来了分组可以发送
  - 移动的极限：不能够超过前沿



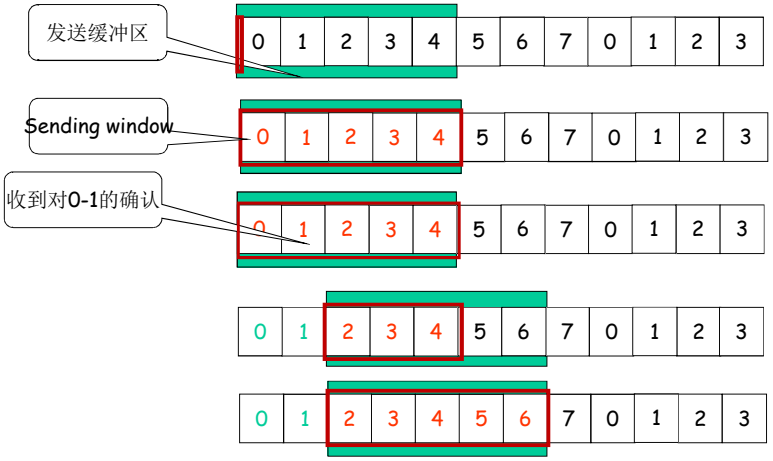
发送窗口的移动->后沿移动

- 发送窗口后沿移动
  - 条件：收到老分组(后沿)的确认
  - 结果：发送缓冲区罩住新的分组，来了分组可以发送
  - 移动的极限：不能够超过前沿



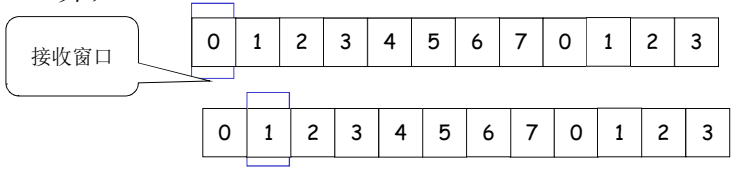
3.5 滑动窗口(slide window)协议

- 滑动窗口技术
  - 发送窗口 (sending window)



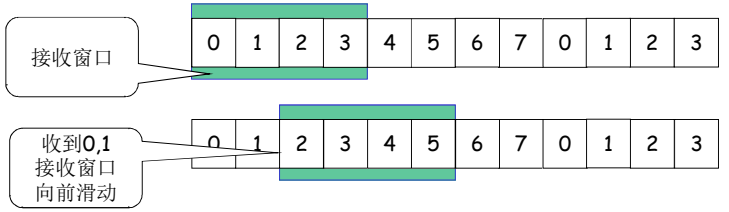
滑动窗口(slide window)协议-接收窗口

- 接收窗口 (receiving window)=接收缓冲区
  - 接收窗口用于控制哪些分组可以接收；
    - 只有收到的分组序号落入接收窗口内才允许接收
    - 若序号在接收窗口之外，则丢弃；
  - 接收窗口尺寸Wr=1，则只能顺序接收；
  - 接收窗口尺寸Wr>1 ，则可以乱序接收
    - 但提交给上层的分组，要按序
  - 例子：Wr = 1，在0的位置；只有0号分组可以接收；向前滑动一个，罩在1的位置，如果来了第2号分组，则丢弃；

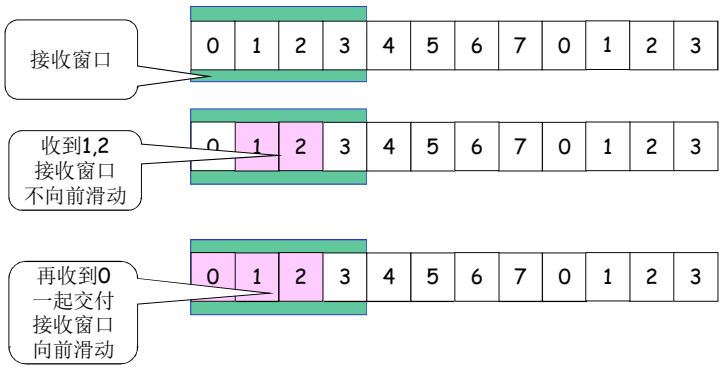


滑动窗口(slide window)协议-接收窗口

- 接收窗口的滑动和发送确认
  - 滑动：
    - 低序号的分组到来，接收窗口移动；
    - 高序号分组乱序到，缓存但不交付（因为要实现rdt，不允许失序），不滑动
  - 发送确认：
    - 接收窗口尺寸=1； 发送连续收到的最大的分组确认（累计确认）
    - 接收窗口尺寸>1； 收到分组，发送那个分组的确认（非累计确认）



滑动窗口(slide window)协议-接收窗口



正常情况下的2个窗口互动

- 发送窗口
  - 有新的分组落入发送缓冲区范围，发送->前沿滑动
  - 来了老的低序号分组的确认->后沿向前滑动->新的分组可以落入发送缓冲区的范围
- 接收窗口
  - 收到分组，落入到接收窗口范围内，接收
  - 是低序号，发送确认给对方
- 发送端上面来了分组->发送窗口滑动->接收窗口滑动->发确认

异常情况下GBN的2窗口互动

- 发送窗口
  - 新分组落入发送缓冲区范围，发送->前沿滑动
  - 超时重发机制让发送端将发送窗口中的所有分组发送出去
  - 来了老分组的重复确认->后沿不向前滑动->新的分组无法落入发送缓冲区的范围（此时如果发送缓冲区有新的分组可以发送）
- 接收窗口
  - 收到乱序分组，没有落入到接收窗口范围内，抛弃
  - （重复）发送老分组的确认，累计确认；

异常情况下SR的2窗口互动

- 发送窗口
  - 新分组落入发送缓冲区范围，发送->前沿滑动
  - 超时重发机制让发送端将超时的分组重新发送出去
  - 来了乱序分组的确认->后沿不向前滑动->新的分组无法落入发送缓冲区的范围（此时如果发送缓冲区有新的分组可以发送）
- 接收窗口
  - 收到乱序分组，落入到接收窗口范围内，接收
  - 发送该分组的确认，单独确认；

GBN协议和SR协议的异同

- 相同之处
  - 发送窗口>1
  - 一次能够可发送多个未经确认的分组
- 不同之处
  - GBN :接收窗口尺寸=1
    - 接收端：只能顺序接收
    - 发送端：从表现来看，一旦一个分组没有发成功，如：0,1,2,3,4；假如1未成功，234都发送出去了，要返回1再发送；GB1
  - SR: 接收窗口尺寸>1
    - 接收端：可以乱序接收
    - 发送端：发送0,1,2,3,4，一旦1未成功，2,3,4,已发送，无需重发，选择性发送1

流水线协议：总结

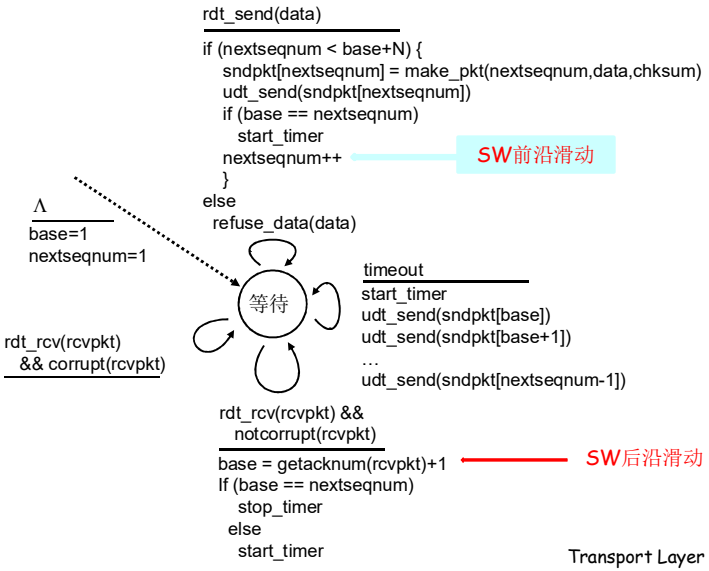
Go-back-N:

- 发送端最多在流水线中有N个未确认的分组
- 接收端只是发送累计型确认*cumulative ack*
  - 接收端如果发现gap，不确认新到来的分组
- 发送端拥有对最老的未确认分组的定时器
  - 只需设置一个定时器
  - 当定时器到时时，重传所有未确认分组

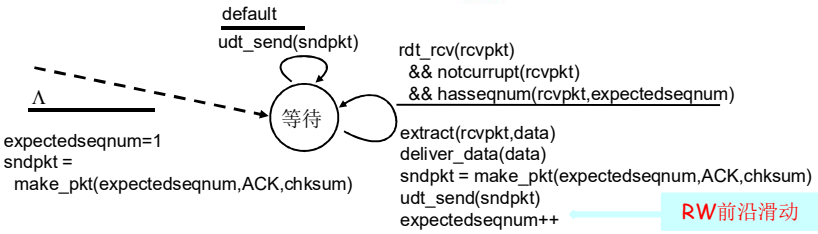
Selective Repeat:

- 发送端最多在流水线中有N个未确认的分组
- 接收方对每个到来的分组单独确认*individual ack*（非累计确认）
- 发送方为每个未确认的分组保持一个定时器
  - 当超时定时器到时，只是重发到时的未确认分组

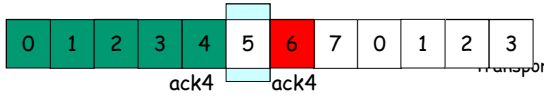
GBN：发送方扩展的FSM



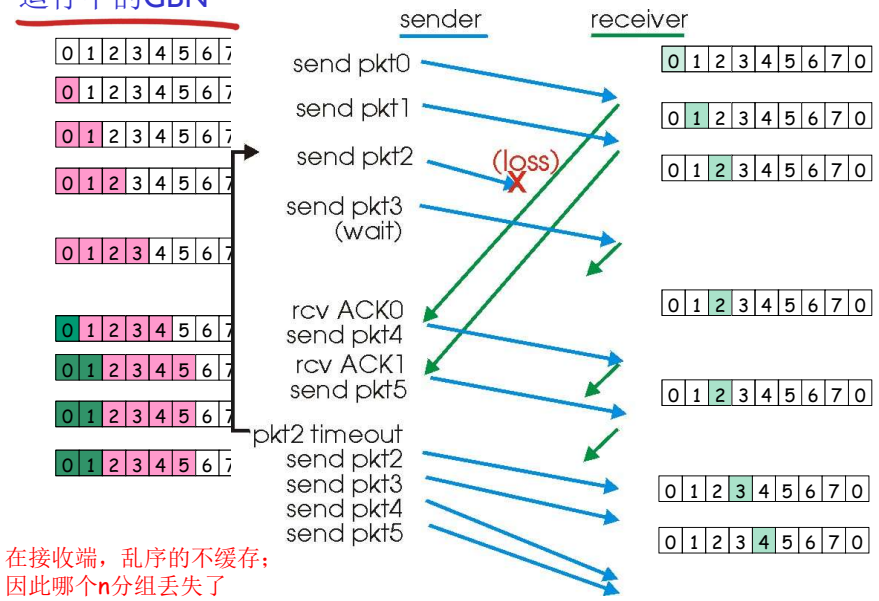
GBN：接收方扩展的FSM



- 只发送ACK：对顺序接收的最高序号的分组
  - 可能会产生重复的ACK
  - 只需记住**expectedseqnum**；接收窗口=1
    - **只一个变量就可表示接收窗口**
- 对乱序的分组：
  - 丢弃（不缓存）→ 在接收方不被缓存！
  - 对顺序接收的最高序号的分组进行确认-累计确认



运行中的GBN



在接收端，乱序的不缓存；
因此哪个n分组丢失了
GB到那个分组n；
即使n以后的分组传送都是正确的

选择重传SR

- 接收方对每个正确接收的分组，分别发送ACKn（非累积确认）
  - 接收窗口>1
    - 可以缓存乱序的分组
  - 最终将分组按顺序交付给上层
- 发送方只对那些没有收到ACK的分组进行重发-选择性重发
  - 发送方为每个未确认的分组设定一个定时器
- 发送窗口的最大值（发送缓冲区）限制发送未确认分组的个数

选择重传

发送方

从上层接收数据：

- 如果下一个可用于该分组的序号可在发送窗口中，则发送

timeout(n):

- 重新发送分组n，重新设定定时器

ACK(n) in [sendbase,sendbase+N]:

- 将分组n标记为已接收
- 如n为最小未确认的分组序号，将base移到下一个未确认序号

接收方

分组n [rcvbase, rcvbase+N-1]

- 发送ACK(n)
- 乱序：缓存
- 有序：该分组及以前缓存的序号连续的分组交付给上层，然后将窗口移到下一个仍未被接收的分组

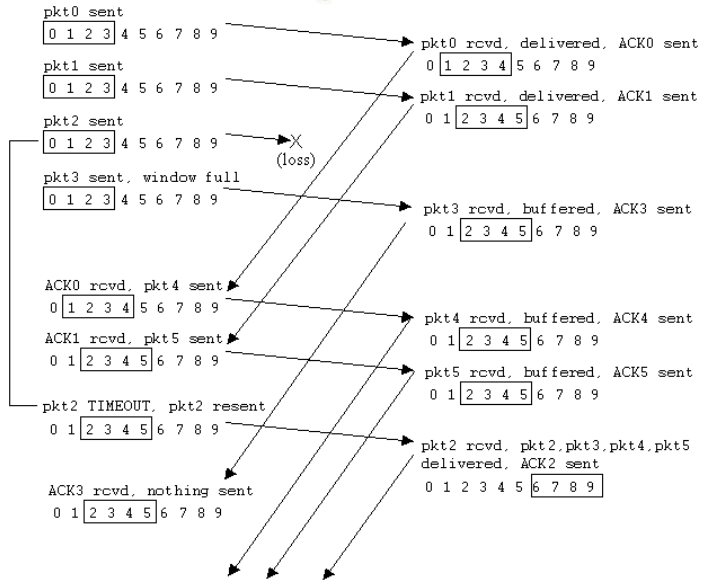
分组n [rcvbase-N, rcvbase-1]

- ACK(n)

其它：

- 忽略该分组

选择重传SR的运行



对比GBN和SR

	GBN	SR
优 点	简单，所需资源少（接收方一个缓存单元）	出错时，重传一个代价小
缺 点	一旦出错，回退N步代价大	复杂，所需要资源多（接收方多个缓存单元）

- 适用范围
  - 出错率低：比较适合GBN，出错非常罕见，没有必要用复杂的SR，为罕见的事件做日常的准备和复杂处理
  - 链路容量大（延迟大、带宽大）：比较适合SR而不是GBN，一点出错代价太大

窗口的最大尺寸

- GBN: 2^n-1
- SR:2^{n-1}

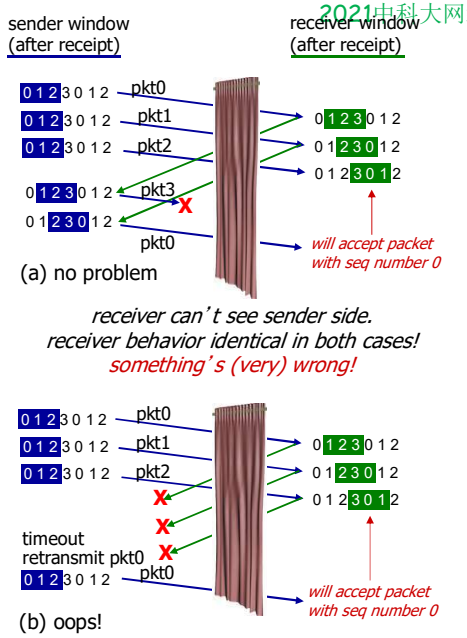
例如：n=2; 序列号：0, 1, 2, 3

- GBN =3
- SR=2

SR的例子：

- 接收方看不到二者的区别！
- 将重复数据误认为新数据 (a)

Q: 序号大小与窗口大小之间的关系？



第3章：提纲

3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

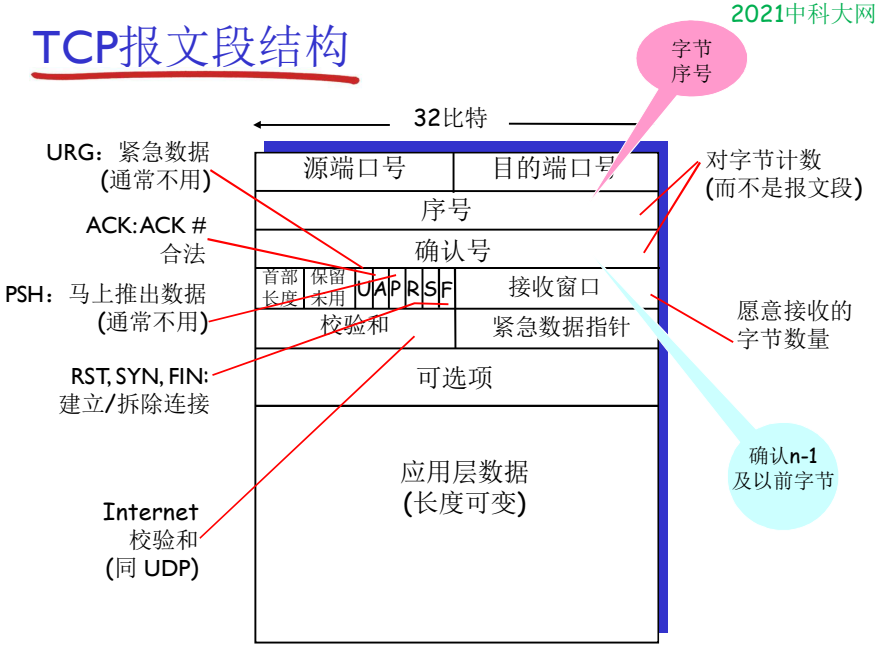
3.6 拥塞控制原理
3.7 TCP 拥塞控制

TCP：概述 RFCs: 793, 1122, 1323, 2018, 2581

- 点对点：
  - 一个发送方，一个接收方
- 可靠的、按顺序的字节流：
  - 没有报文边界
- 管道化（流水线）：
  - TCP拥塞控制和流量控制设置窗口大小
- 发送和接收缓存

- 全双工数据：
  - 在同一连接中数据流双向流动
  - MSS：最大报文段大小
- 面向连接：
  - 在数据交换之前，通过握手（交换控制报文） 初始化发送方、接收方的状态变量
- 有流量控制：
  - 发送方不会淹没接收方

TCP报文段结构



TCP 序号，确认号

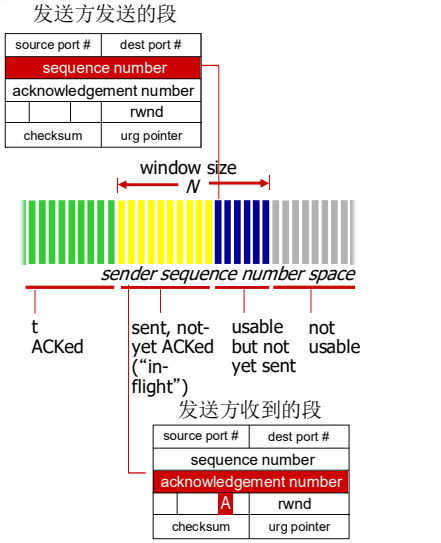
序号：

- 报文段首字节的在字节流的编号

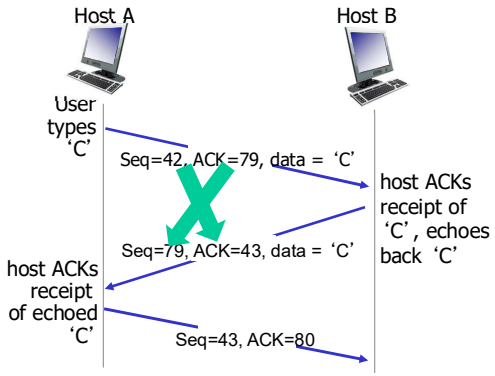
确认号:

- 期望从另一方收到的下一个字节的序号
- 累积确认

Q:接收方如何处理乱序的报文段-没有规定



TCP序号和确认号



简单telne场景

TCP往返延时（RTT）和超时

Q: 怎样设置TCP超时？

- 比RTT要长
  - 但RTT是变化的
- 太短：太早超时
  - 不必要的重传
- 太长：对报文段丢失反应太慢，消极

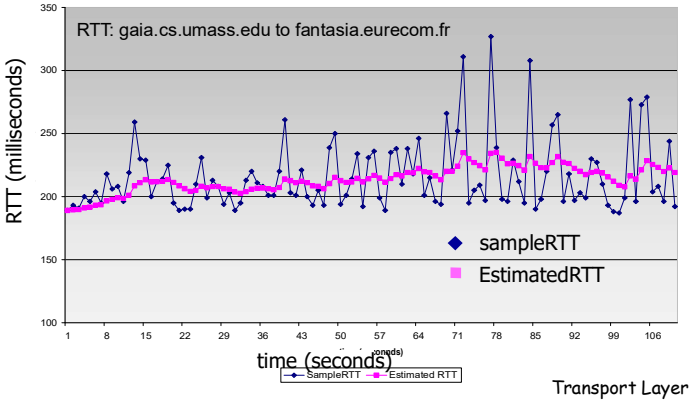
Q: 怎样估计RTT?

- **SampleRTT:** 测量从报文段发出到收到确认的时间
  - 如果有重传，忽略此次测量
- **SampleRTT**会变化，因此估计的RTT应该比较平滑
  - 对几个最近的测量值求平均，而不是仅用当前的**SampleRTT**

TCP往返延时（RTT）和超时

EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT

- 指数加权移动平均
- 过去样本的影响呈指数衰减
- 推荐值：α = 0.125



TCP往返延时（RTT）和超时

设置超时

- **EstimtedRTT** + 安全边界时间
  - **EstimatedRTT变化**大 (方差大)→ 较大的安全边界时间
- SampleRTT会偏离EstimatedRTT多远：

DevRTT = (1-β)*DevRTT +
β*|SampleRTT-EstimatedRTT|

(推荐值：β = 0.25)

超时时间间隔设置为：

TimeoutInterval = EstimatedRTT + 4*DevRTT

estimated RTT　　“safety margin”

第3章：提纲

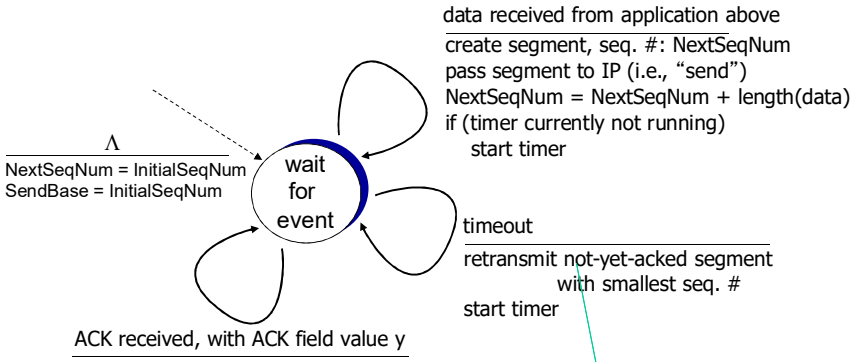
3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理
3.7 TCP 拥塞控制

TCP：可靠数据传输

- TCP在IP不可靠服务的基础上建立了rdt
  - 管道化的报文段
    - GBN or SR
  - 累积确认（像GBN）
  - 单个重传定时器（像GBN）
  - 是否可以接受乱序的，没有规范
- 通过以下事件触发重传
  - 超时（只重发那个最早的未确认段：SR）
  - 重复的确认
    - 例子：收到了ACK50,之后又收到3个ACK50
- 首先考虑简化的TCP发送方：
  - 忽略重复的确认
  - 忽略流量控制和拥塞控制

TCP 发送方(简化版)



只传送具有最小序号的未确段像SR，不是将所有未确认的都传一遍

TCP发送方事件：

从应用层接收数据：

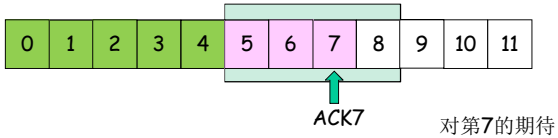
- 用nextseq创建报文段
- 序号nextseq为报文段首字节的字节流编号
- 如果还没有运行，启动定时器
  - 定时器与最早未确认的报文段关联
  - 过期间隔：TimeOutInterval

超时：

- 重传后沿最老的报文段
- 重新启动定时器

收到确认：

- 如果是对尚未确认的报文段确认
  - 更新已被确认的报文序号
  - 如果当前还有未被确认的报文段，重新启动定时器




```
NextSeqNum = InitialSeqNum
SendBase = InitialSeqNum

loop (forever) {
  switch(event)

  event: data received from application above
     create TCP segment with sequence number NextSeqNum
     if (timer currently not running)
        start timer
     pass segment to IP
     NextSeqNum = NextSeqNum + length(data)

  event: timer timeout
     retransmit not-yet-acknowledged segment with
        smallest sequence number
     start timer

  event: ACK received, with ACK field value of y
     if (y > SendBase) {
        SendBase = y
        if (there are currently not-yet-acknowledged segments)
           start timer
        }

} /* end of loop forever */
```


简化的TCP 发送方

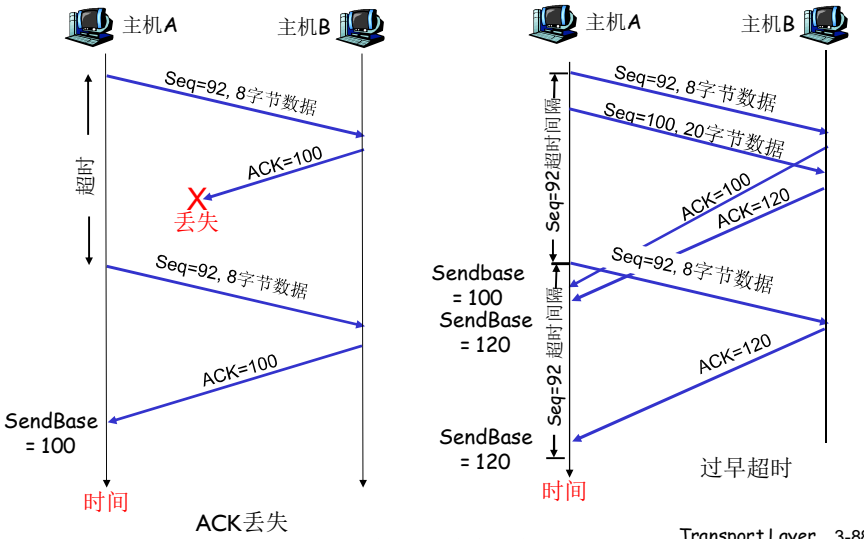
注释：

• SendBase-1: 最后一个累积确认的字节

例：

• SendBase-1 = 71; y= 73，因此接收方期望73+；
y > SendBase，因此新的数据被确认

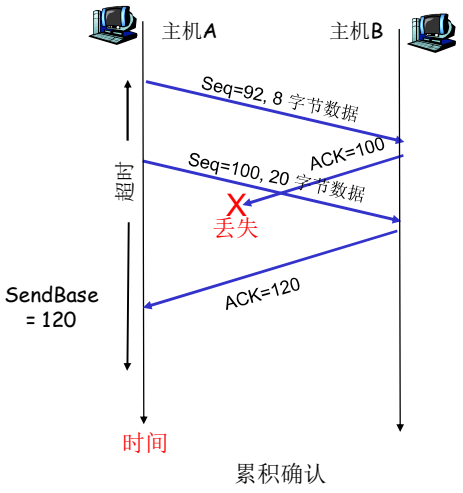
TCP：重传



ACK丢失

对顺序收到的最高字节确认：

TCP：重传



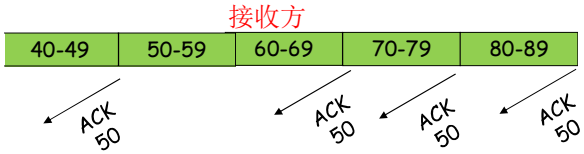
累积确认

产生TCP ACK的建议 [RFC 1122, RFC 2581]

接收方的事件	TCP接收方动作
所期望序号的报文段按序到达。所有在期望序号之前的数据都已经被确认	延迟的ACK。对另一个按序报文段的到达最多等待500ms。如果下一个报文段在这个时间间隔内没有到达，则发送一个ACK。
有期望序号的报文段到达。另一个按序报文段等待发送ACK	立即发送单个累积ACK，以确认两个按序报文段。
比期望序号大的报文段乱序到达。检测出数据流中的间隔	立即发送重复的ACK，指明下一个期待字节的序号
能部分或完全填充接收数据间隔的报文段到达。	若该报文段起始于间隔（gap）的低端，则立即发送ACK。

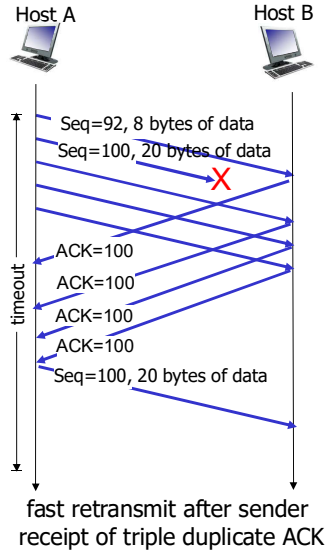
快速重传

- 超时周期往往太长：
  - 在重传丢失报文段之前的延时太长
- 通过重复的ACK来检测报文段丢失
  - 发送方通常连续发送大量报文段
  - 如果报文段丢失，通常会引起多个重复的ACK



- 如果发送方收到同一数据的3个冗余ACK，重传最小序号的段：
  - 快速重传：在定时器过时之前重发报文段
  - 它假设跟在被确认的数据后面的数据丢失了
    - 第一个ACK是正常的；
    - 收到第二个该段的ACK，表示接收方收到一个该段后的乱序段；
    - 收到第3，4个该段的ack，表示接收方收到该段之后的2个，3个乱序段，可能性非常大段丢失了

TCP快速重传



fast retransmit after sender receipt of triple duplicate ACK

快速重传算法：

```
event: ACK received, with ACK field value of y
            if (y > SendBase) {
                SendBase = y
                if (there are currently not-yet-acknowledged segments)
                    start timer
            }
            else {
                increment count of dup ACKs received for y
                if (count of dup ACKs received for y = 3) {
                    resend segment with sequence number y
                }
```

已确认报文段的一个重复确认

快速重传

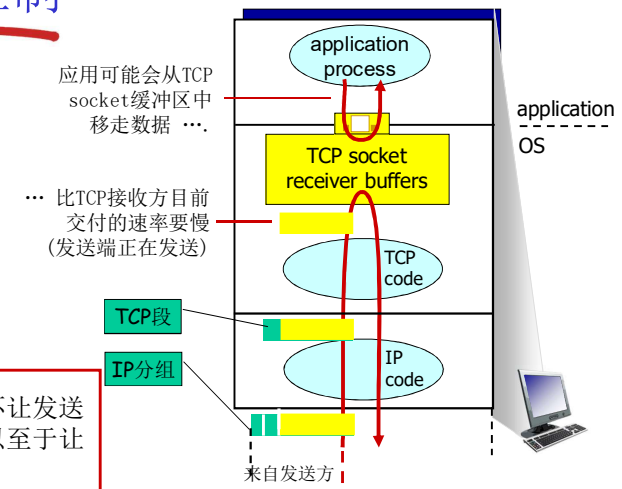
第3章：提纲

3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理
3.7 TCP 拥塞控制

TCP 流量控制

缓冲区TCP往里面写，app从当中读取



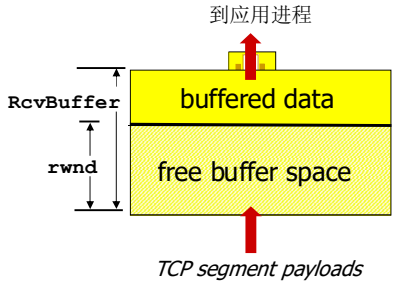
流量控制

接收方控制发送方，不让发送方发送的太多、太快以至于让接收方的缓冲区溢出

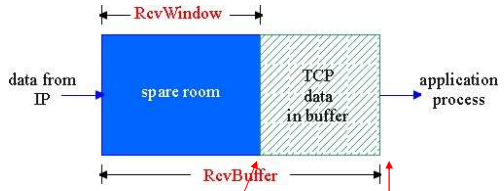
接收方协议栈

TCP流量控制

- 接收方在其向发送方的TCP段头部的rwnd字段“通告”其空闲buffer大小
  - RcvBuffer大小通过socket选项设置（典型默认大小为4096 字节）
  - 很多操作系统自动调整 RcvBuffer
- 发送方限制未确认（“in-flight”）字节的个数≤接收方发送过来的 rwnd 值
- 保证接收方不会被淹没



TCP流量控制



（假设TCP接收方丢弃乱序的报文段）

- 缓存中的可用的空间

= RcvWindow

= RcvBuffer-[LastByteRcvd - LastByteRead]

第3章：提纲

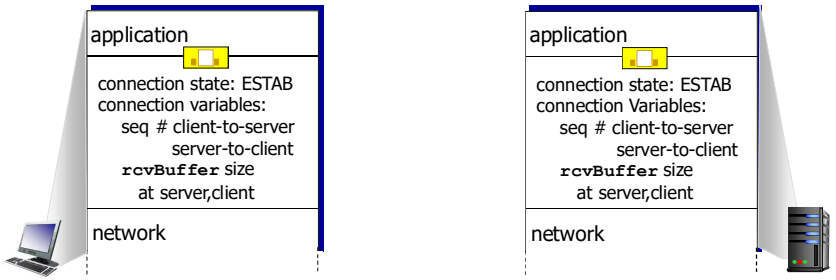
3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理
3.7 TCP 拥塞控制

TCP连接管理

在正式交换数据之前，发送方和接收方握手建立通信关系：

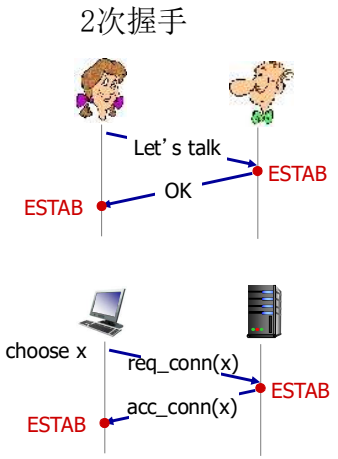
- 同意建立连接（每一方都知道对方愿意建立连接）
- 同意连接参数




```
Socket clientSocket =
  newSocket("hostname","port
   number");
```

```
Socket connectionSocket =
  welcomeSocket.accept();
```


同意建立连接

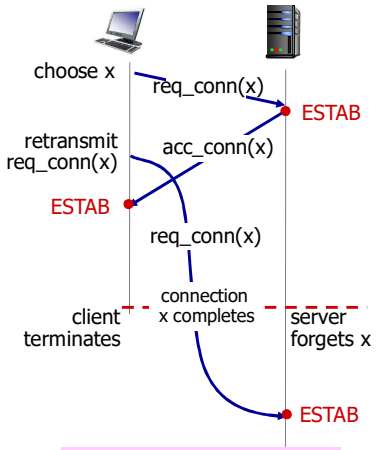


Q:在网络中，2次握手建立连接总是可行吗？

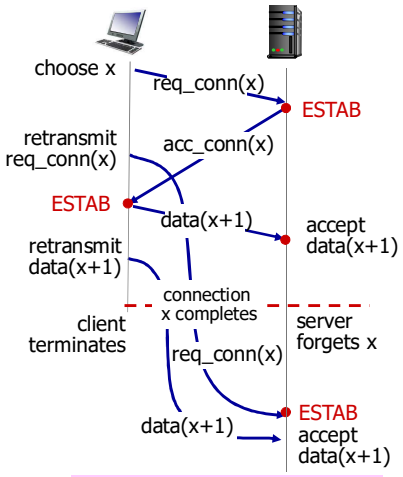
- 变化的延迟（连接请求的段没有丢，但可能超时）
- 由于丢失造成的重传 (e.g. req_conn(x))
- 报文乱序
- 相互看不到对方

同意建立连接

2次握手的失败场景：

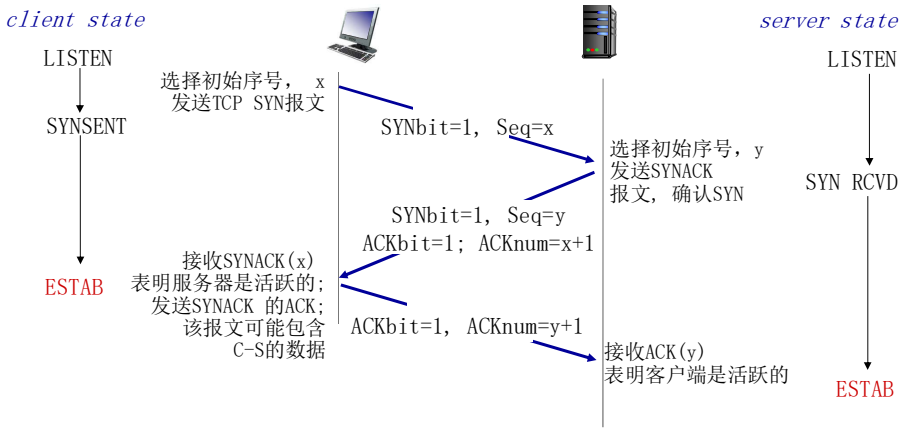


半连接
(只在服务器端维护了连接!)



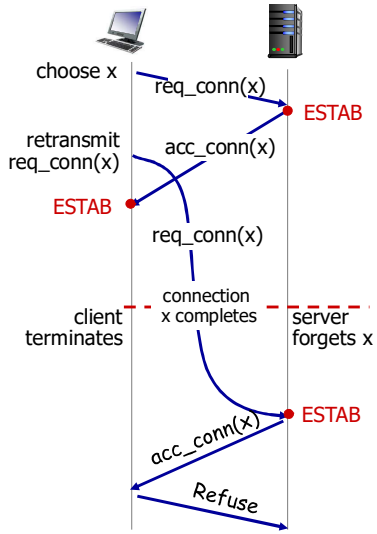
老的数据被当成新的数据接受了

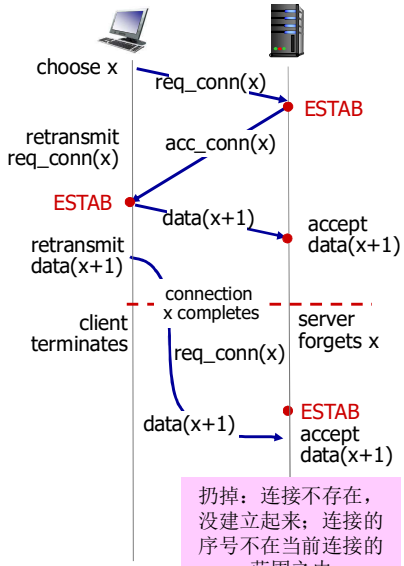
TCP 3次握手



解决方案：变化的初始序号+双方确认对方的序号（3次握手）

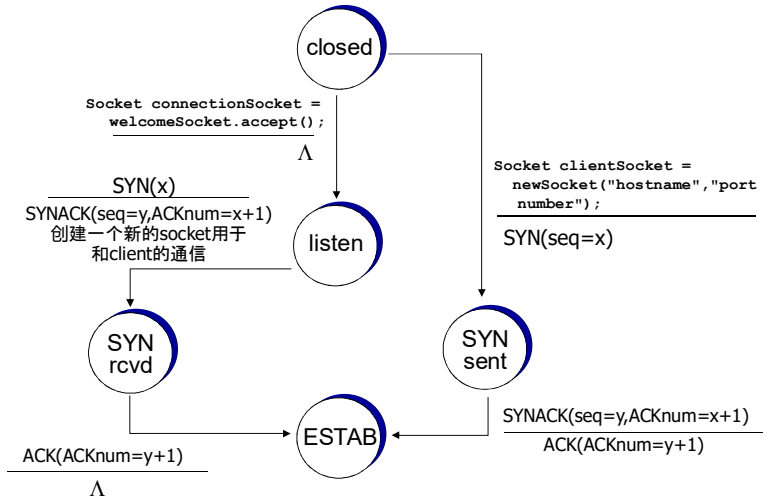
3次握手解决：半连接和接收老数据问题





扔掉：连接不存在，
没建立起来；连接的
序号不在当前连接的
范围之内

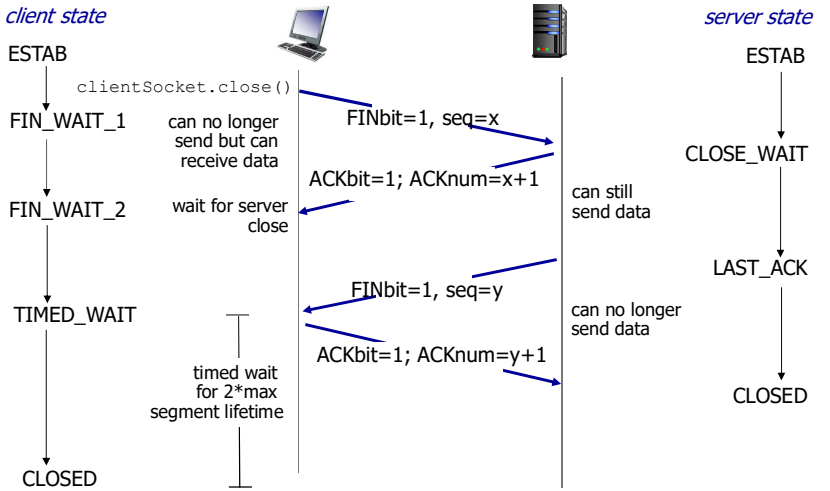
TCP 3次握手：FSM



TCP：关闭连接

- 客户端，服务器分别关闭它自己这一侧的连接
  - 发送FIN bit = 1的TCP段
- 一旦接收到FIN，用ACK回应
  - 接到FIN段，ACK可以和它自己发出的FIN段一起发送
- 可以处理同时的FIN交换

TCP：关闭连接



第3章：提纲

3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

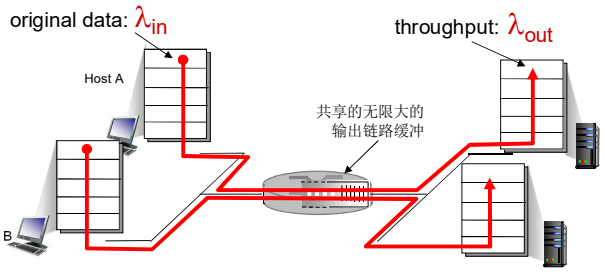
3.6 拥塞控制原理
3.7 TCP 拥塞控制

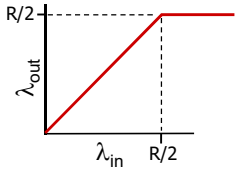
拥塞控制原理

拥塞:
- 非正式的定义: "太多的数据需要网络传输，超过了网络的处理能力"
- 与流量控制不同
- 拥塞的表现:
  - 分组丢失 (路由器缓冲区溢出)
  - 分组经历比较长的延迟(在路由器的队列中排队)
- 网络中前10位的问题!

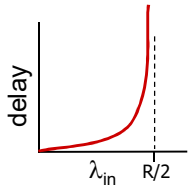
拥塞的原因/代价: 场景1

- 2个发送端，2个接收端
- 一个路由器，具备无限大的缓冲
- 输出链路带宽：R
- 没有重传





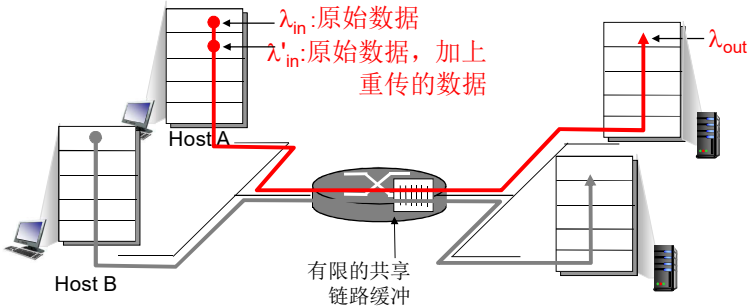
- 每个连接的最大吞吐量：R/2



- 当进入的速率λ_{in}接近链路链路带宽R时，延迟增大

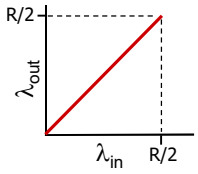
拥塞的原因/代价: 场景2

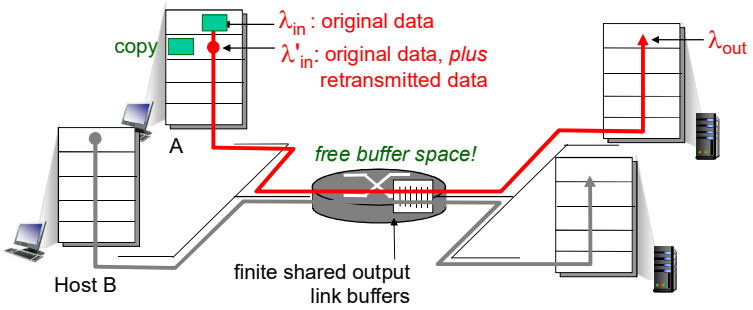
- 一个路由器，有限的缓冲
- 分组丢失时，发送端重传
  - 应用层的输入=应用层输出：$\lambda_{in} = \lambda_{out}$
  - 传输层的输入包括重传：$\lambda'_{in} \geq \lambda_{in}$



拥塞的原因/代价: 场景2

理想化：发送端有完美的信息
发送端知道什么时候路由器的缓冲是可用的
- ✓ 只在缓冲可用时发送
- ✓ 不会丢失:$\lambda'_{in} = \lambda_{in}$

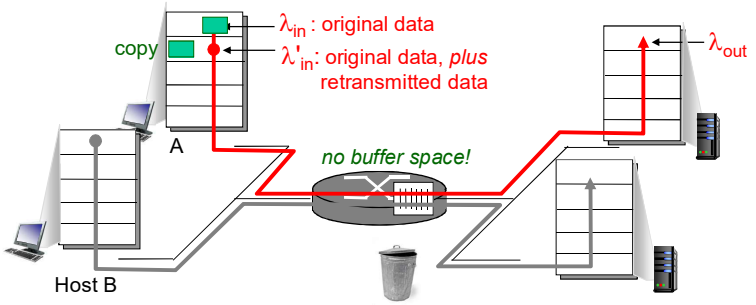




拥塞的原因/代价: 场景2

理想化：掌握丢失信息

分组可以丢失，在路由器由于缓冲器满而被丢弃
- 如果知道分组丢失了，发送方重传分组

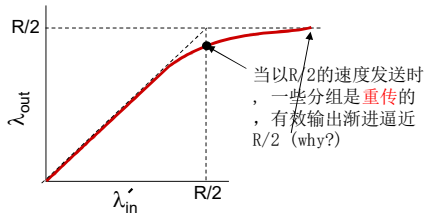


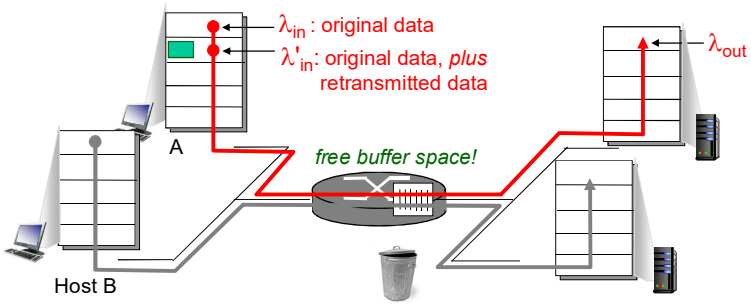
拥塞的原因/代价: 场景2

理想化：掌握丢失信息

分组可以丢失，在路由器由于缓冲器满而被丢弃

- 如果发送端知道分组丢失了，发送方重传分组

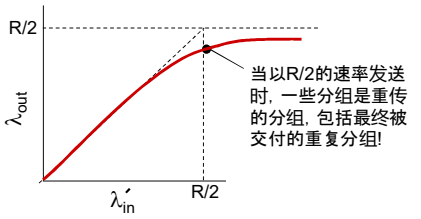


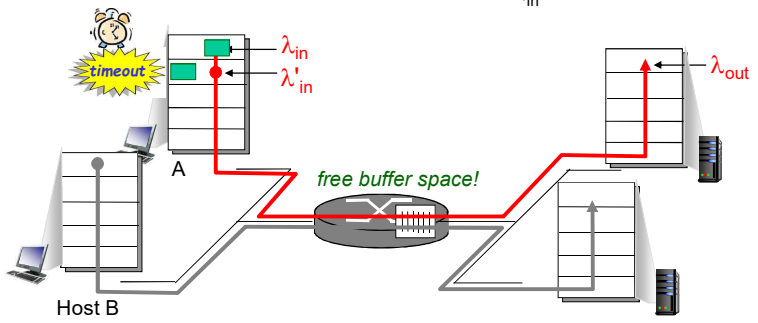


拥塞的原因/代价: 场景2

现实情况：重复

- 分组可能丢失，由于缓冲器满而被丢弃
- 发送端最终超时，发送第2个拷贝，2个分组都被传出

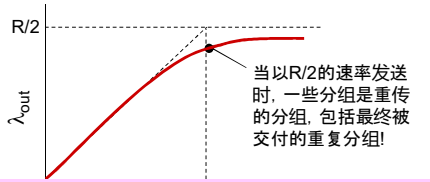




拥塞的原因/代价: 场景2

现实情况：重复

- 分组可能丢失，由于缓冲器满而被丢弃
- 发送端最终超时，发送第2个拷贝，2个分组都传到



输出比输入少原因：1）重传的丢失分组；2）没有必要重传的重复分组

拥塞的“代价”：

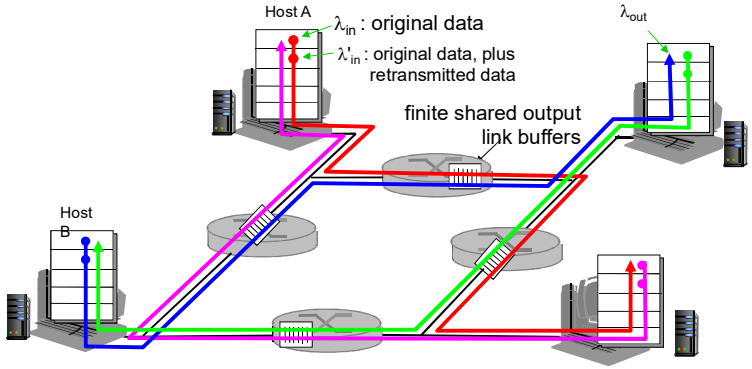
- 为了达到一个有效输出，网络需要做更多的工作（重传）
- 没有必要的重传，链路中包括了多个分组的拷贝
  - 是那些没有丢失，经历的时间比较长（拥塞状态）但是超时的分组
  - 降低了的“goodput”

拥塞的原因/代价: 场景3

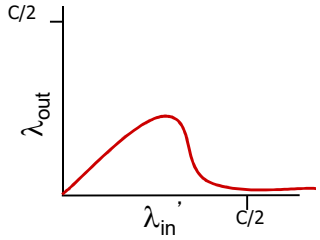
- 4个发送端
- 多重路径
- 超时 / 重传

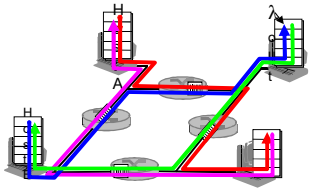
Q: 当 λ_{in} λ'_{in} 增加时，会发生什么?

A: 当红色的λ'_{in}增加时，所有到来的蓝色分组都在最上方的队列中丢弃了，蓝色吞吐->0



拥塞的原因/代价: 场景3





又一个拥塞的代价:

- 当分组丢失时，任何"关于这个分组的上游传输能力"都被浪费了

拥塞控制方法

2种常用的拥塞控制方法:

端到端拥塞控制:

- 没有来自网络的显式反馈
- 端系统根据延迟和丢失事件推断是否有拥塞
- TCP采用的方法

网络辅助的拥塞控制:

- 路由器提供给端系统以反馈信息
  - 单个bit置位，显示有拥塞 (SNA, DECbit, TCP/IP ECN, ATM)
  - 显式提供发送端可以采用的速率

案例学习: ATM ABR 拥塞控制

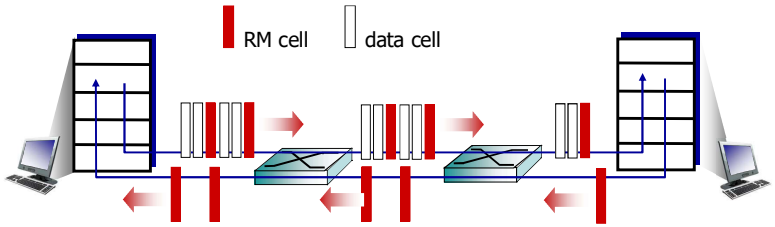
ABR: available bit rate:

- "弹性服务"
- 如果发送端的路径 "轻载"
  - 发送方使用可用带宽
- 如果发送方的路径拥塞了
  - 发送方限制其发送的速度到一个最小保障速率上

RM (资源管理) 信元:

- 由发送端发送,在数据信元中间隔插入
- RM信元中的比特被交换机设置 ("网络辅助")
  - NI bit: no increase in rate (轻微拥塞)速率不要增加了
  - CI bit: congestion indication 拥塞指示
- 发送端发送的RM 信元被接收端返回, 接收端不做任何改变

案例学习: ATM ABR 拥塞控制



- 在RM信元中的2个字节 ER (explicit rate)字段
  - 拥塞的交换机可能会降低信元中ER的值
  - 发送端发送速度因此是最低的可支持速率
- 数据信元中的EFCI bit: 被拥塞的交换机设置成1
  - 如果在管理信元RM前面的数据信元EFCI被设置成了1, 接收端在返回的RM信元中设置CI bit

第3章：提纲

3.1 概述和传输层服务
3.2 多路复用与解复用
3.3 无连接传输：UDP
3.4 可靠数据传输的原理
3.5 面向连接的传输：TCP
- 段结构
- 可靠数据传输
- 流量控制
- 连接管理

3.6 拥塞控制原理
3.7 TCP 拥塞控制

TCP 拥塞控制：机制

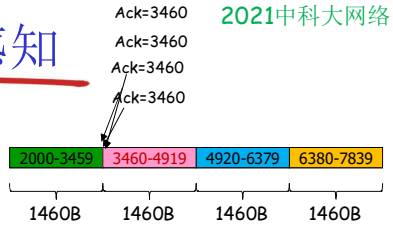
- 端到端的拥塞控制机制
  - 路由器不向主机有关拥塞的反馈信息
    - 路由器的负担较轻
    - 符合网络核心简单的TCP/IP架构原则
  - 端系统根据自身得到的信息，判断是否发生拥塞，从而采取动作

拥塞控制的几个问题

- 如何检测拥塞
  - 轻微拥塞
  - 拥塞
- 控制策略
  - 在拥塞发送时如何动作，降低速率
    - 轻微拥塞，如何降低
    - 拥塞时，如何降低
  - 在拥塞缓解时如何动作，增加速率

TCP 拥塞控制：拥塞感知

发送端如何探测到拥塞?



- 某个段超时了（丢失事件）：拥塞
  - 超时时间到，某个段的确认没有来
  - 原因1：网络拥塞（某个路由器缓冲区没空间了，被丢弃）概率大
  - 原因2：出错被丢弃了（各级错误，没有通过校验，被丢弃）概率小
  - 一旦超时，就认为拥塞了，有一定误判，但是总体控制方向是对的
- 有关某个段的3次重复ACK：轻微拥塞
  - 段的第1个ack，正常，确认绿段，期待红段
  - 段的第2个重复ack，意味着红段的后一段收到了，蓝段乱序到达
  - 段的第2、3、4个ack重复，意味着红段的后第2、3、4个段收到了，橙段乱序到达，同时红段丢失的可能性很大（后面3个段都到了，红段都没到）
  - 网络这时还能够进行一定程度的传输，拥塞但情况要比第一种好

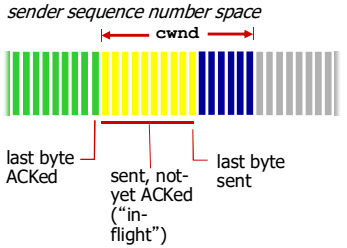
TCP 拥塞控制：速率控制方法

如何控制发送端发送的速率

- 维持一个拥塞窗口的值：CongWin
- 发送端限制已发送但是未确认的数据量（的上限）：

$$\text{LastByteSent} - \text{LastByteAcked} \le \text{CongWin}$$

- 从而粗略地控制发送方的往网络中注入的速率



$$\text{rate} \approx \frac{\text{CongWin}}{\text{RTT}} \text{bytes/sec}$$

TCP 拥塞控制：速率控制方法

如何控制发送端发送的速率

- CongWin是动态的，是感知到的网络拥塞程度的函数
  - 超时或者3个重复ack，CongWin↓
    - 超时时：**CongWin降为1MSS，进入SS阶段然后再倍增到CongWin/2（每个RTT），从而进入CA阶段**
    - **3个重复ack ：CongWin降为CongWin/2，CA阶段**
  - 否则（正常收到Ack，没有发送以上情况）：CongWin跃跃欲试↑
    - **SS阶段：加倍增加(每个RTT)**
    - **CA阶段：线性增加(每个RTT)**

TCP拥塞控制和流量控制的联合动作

联合控制的方法：

- 发送端控制*发送但是未确认*的量同时也不能够超过接收窗口，满足流量控制要求
  - SendWin=min{CongWin, RecvWin}
  - 同时满足 拥塞控制和流量控制要求

TCP 拥塞控制：策略概述

拥塞控制策略：

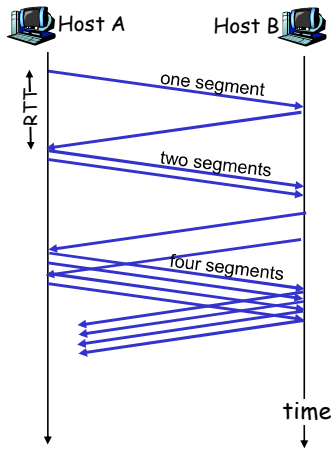
- 慢启动
- AIMD：线性增、乘性减少
- 超时事件后的保守策略

TCP 慢启动

- 连接刚建立，**CongWin = 1 MSS**
  - 如: MSS = 1460bytes & RTT = 200 msec
  - 初始速率 = 58.4kbps
- 可用带宽可能>> MSS/RTT
  - 应该尽快加速，到达希望的速率
- 当连接开始时，指数性增加发送速率，直到发生丢失的事件
  - 启动初值很低
  - 但是速度很快

TCP 慢启动（续）

- 当连接开始时，指数性增加（每个RTT）发送速率直到发生丢失事件
  - 每一个RTT， CongWin加倍
  - 每收到一个ACK时，CongWin加1（why）
  - 慢启动阶段：只要不超时或3个重复ack，一个RTT，CongWin加倍
- 总结: 初始速率很慢，但是加速却是指数性的
  - 指数增加，SS时间很短，长期来看可以忽略



TCP 拥塞控制：AIMD

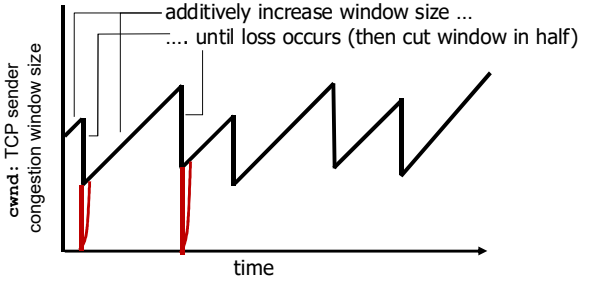
乘性减:

丢失事件后将CongWin降为1，将CongWin/2作为阈值，进入慢启动阶段（倍增直到CongWin/2）

加性增：

当CongWin>阈值时，一个RTT如没有发生丢失事件,将CongWin加1MSS: *探测*

AIMD saw tooth behavior: probing for bandwidth



TCP拥塞控制：AIMD

- 当收到3个重复的*ACKs*:
  - **CongWin** 减半
  - 窗口（缓冲区大小）之后线性增长
- 当超时事件发生时:
  - **CongWin被设置成 1 MSS，进入SS阶段**
  - 之后窗口指数增长
  - 增长到一个阈值（上次发生拥塞的窗口的一半）时，再线性增加

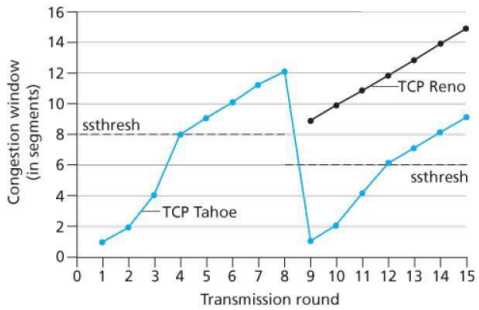
思路

- 3个重复的ACK表示网络还有一定的段传输能力
- 超时之前的3个重复的ACK表示“警报”

改进(续)

Q:什么时候应该将指数性增长变成线性？

A:在超时之前，当**CongWin变成上次发生超时的窗口的一半**



实现:

- 变量：Threshold
- 出现丢失，Threshold设置成 CongWin的1/2

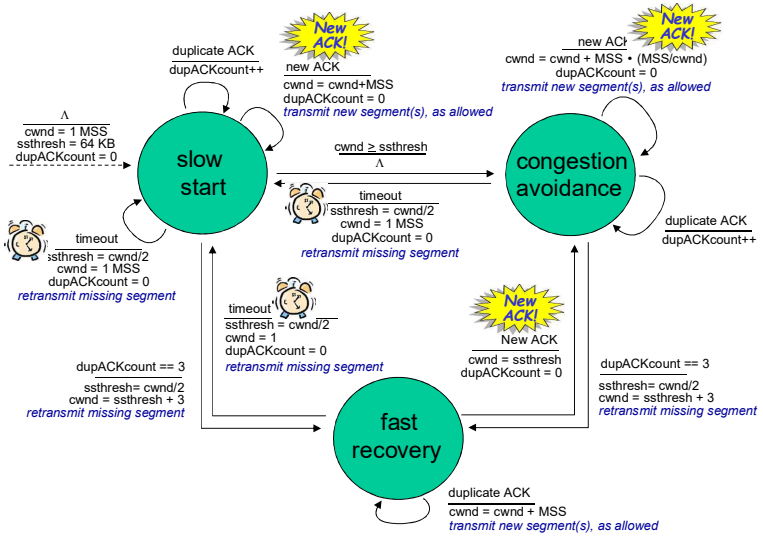
总结: TCP拥塞控制

- 当**CongWin＜Threshold**, 发送端处于慢启动阶段（**slow-start**）, 窗口指数性增长.
- 当**CongWin〉Threshold**, 发送端处于拥塞避免阶段（**congestion-avoidance**）, 窗口线性增长.
- 当收到三个重复的*ACKs* **(triple duplicate ACK), Threshold设置成 CongWin/2, CongWin=Threshold+3.**
- 当超时事件发生时**timeout**, **Threshold=CongWin/2 CongWin=1 MSS**，进入SS阶段

TCP 发送端拥塞控制

事件	状态	TCP 发送端行为	解释
以前没有收到ACK的data被ACKed	慢启动(SS)	CongWin = CongWin + MSS If (CongWin > Threshold) 状态变成 “CA”	每一个RTT CongWin 加倍
以前没有收到ACK的data被ACKed	拥塞避免 (CA)	CongWin = CongWin+MSS * (MSS/CongWin)	加性增加, 每一个RTT对CongWin 加一个 1 MSS
通过收到3个重复的ACK，发现丢失的事件	SS or CA	Threshold = CongWin/2, CongWin = Threshold+3, 状态变成“CA”	快速重传, 实现乘性的减. CongWin 没有变成1 MSS.
超时	SS or CA	Threshold = CongWin/2, CongWin = 1 MSS, 状态变成“SS”	进入slow start
重复的 ACK	SS or CA	对被ACKed 的segment， 增加重复ACK的计数	CongWin and Threshold 不变

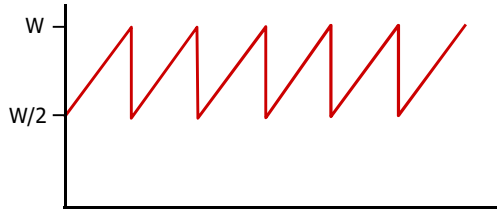
总结： TCP拥塞控制



TCP 吞吐量

- TCP的平均吞吐量是多少，使用窗口window尺寸W和RTT来描述?
  - 忽略慢启动阶段，假设发送端总有数据传输
- W：发生丢失事件时的窗口尺寸（单位：字节）
  - 平均窗口尺寸（#in-flight字节）：3/4W
  - 平均吞吐量：RTT时间吞吐3/4W

$$\text{avg TCP thruput} = \frac{3}{4}\frac{W}{RTT} \text{ bytes/sec}$$



TCP 未来: TCP over “long, fat pipes”

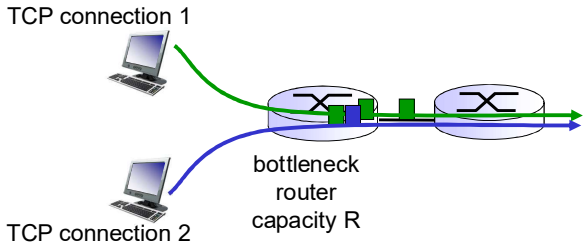
- 例如: 1500字节 / 段, 100ms RTT（往返延迟R=200ms）, 如果需要10 Gbps 吞吐量
- T=0.75W/R -> 0.75W=TR=12.5MB=83333段
- 平均需要窗口大小 0.75W = 83,333 in-flight 在传段
- 吞吐量用丢失率表示:

$$T = \frac{1.22 \cdot MSS}{RTT\sqrt{L}}$$

- ➜ L = $2 \cdot 10^{-10}$ （为了达到10Gbps的吞吐，平均50亿段丢失一个） *非常非常小的丢失率！可能远远低于链路的物理丢失率，达不到的*
- 如果需要网络带宽增加到未来可用的10Gbps，那就需要更新的TCP版本!

TCP 公平性

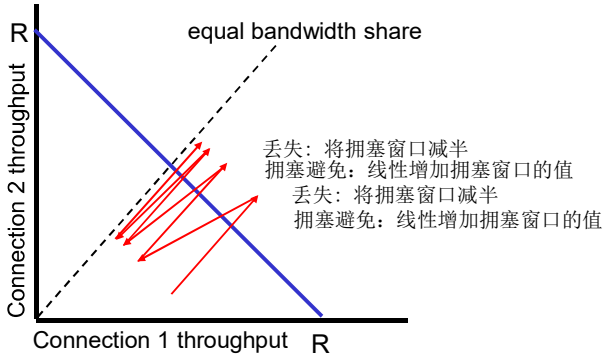
公平性目标: 如果 K个TCP会话分享一个链路带宽为R的瓶颈，每一个会话的有效带宽为 R/K



为什么TCP是公平的?

2个竞争的TCP会话:

- 加性增加，斜率为1, 吞吐量增加
- 乘性减，吞吐量比例减少



公平性 (续)

公平性和 UDP

- 多媒体应用通常不是用 TCP
  - 应用发送的数据速率希望不受拥塞控制的节制
- 使用UDP:
  - 音视频应用泵出数据的速率是恒定的, 忽略数据的丢失
- 研究领域: TCP 友好性

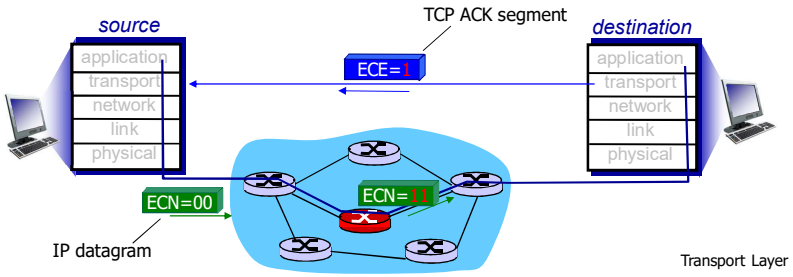
公平性和并行TCP连接

- 2个主机间可以打开多个并行的TCP连接
- Web浏览器
- 例如: 带宽为R的链路支持了9个连接;
  - 如果新的应用要求建1个TCP连接,获得带宽R/10
  - 如果新的应用要求建11个TCP连接,获得带宽R/2

Explicit Congestion Notification (ECN)

网络辅助拥塞控制:

- TOS字段中2个bit被网络路由器标记，用于指示是否发生拥塞
- 拥塞指示被传送到接收主机
- 在接收方-到发送方的ACK中，接收方(在IP数据报中看到了拥塞指示）设置ECE bit，指示发送方发生了拥塞



第三章 总结(1/2)

- 传输层提供的服务
  - 应用进程间的逻辑通信
    - Vs 网络层提供的是主机到主机的通信服务
  - 互联网上传输层协议：UDP TCP
    - 特性
- 多路复用和解复用
  - 端口：传输层的SAP
  - 无连接的多路复用和解复用
  - 面向连接的多路复用和解复用
- 实例1：无连接传输层协议 UDP
  - 多路复用解复用
  - UDP报文格式
  - 检错机制：校验和
- 可靠数据传输原理
  - 问题描述
  - 停止等待协议：
    - Rdt1.0 rdt2.0,2.1 ,2.2 Rdt 3.0
  - 流水线协议
    - GBN
    - SR

第三章 总结（2/2）

- 实例2：面向连接的传输层协议-TCP
  - 概述：TCP特性
  - 报文段格式
    - 序号，超时机制及时间
  - TCP可靠传输机制
  - 重传，快速重传
  - 流量控制
  - 连接管理
    - 三次握手
    - 对称连接释放
- 拥塞控制原理
  - 网络辅助的拥塞控制
  - 端到端的拥塞控制
- TCP的拥塞控制
  - AIMD
  - 慢启动
  - 超时之后的保守策略

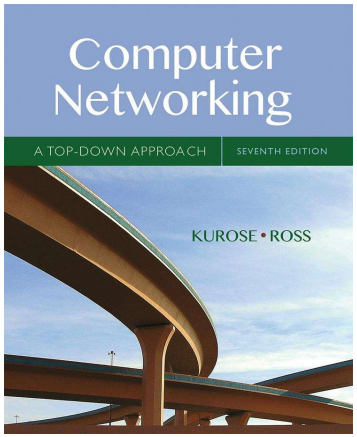
第三章 展望

下二章:

- 离开网络"边缘" (应用层和传输层)
- 深入到网络的"核心"
- 2个关于网络层的章
  - 数据平面
  - 控制平面

第 4 章
网络层：数据平面

中国科学技术大学
自动化系 郑烇
改编自Jim kurose,Keith Ross



Computer Networking: A Top Down Approach
7th edition
Jim Kurose, Keith Ross
Pearson/Addison Wesley
April 2016

第4章 网络层：数据平面

- 4. 1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

第4章 网络层：数据平面

- 4. 1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

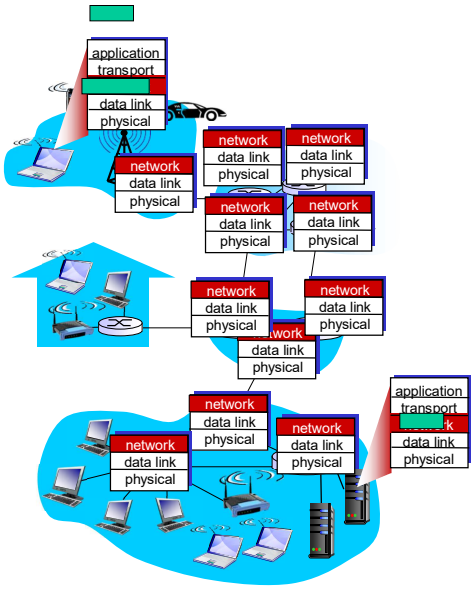
第4章 网络层：数据平面

本章目标：

- 理解网络服务的基本原理，聚焦于其数据平面
  - 网络服务模型
  - 转发和路由
  - 路由器工作原理
  - 通用转发
- 互联网中网络层协议的实例和实现

网络层服务

- 在发送主机和接收主机对之间传送段（segment）
- 在发送端将段封装到数据报中
- 在接收端，将段上交给传输层实体
- 网络层协议存在于每一个主机和路由器
- 路由器检查每一个经过它的IP数据报的头部



网络层的关键功能

网络层功能：

- *转发*: 将分组从路由器的输入接口转发到合适的输出接口
- *路由*: 使用路由算法来决定分组从发送主机到目标接收主机的路径
  - 路由选择算法
  - 路由选择协议

旅行的类比：

- 转发: 通过单个路口的过程
- 路由: 从源到目的的路由路径规划过程

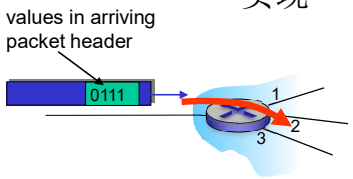
网络层：数据平面、控制平面

数据平面

- 本地，每个路由器功能
- 决定从路由器输入端口到达的分组如何转发到输出端口
- 转发功能：
  - 传统方式：基于目标地址+转发表
  - SDN方式：基于多个字段+流表

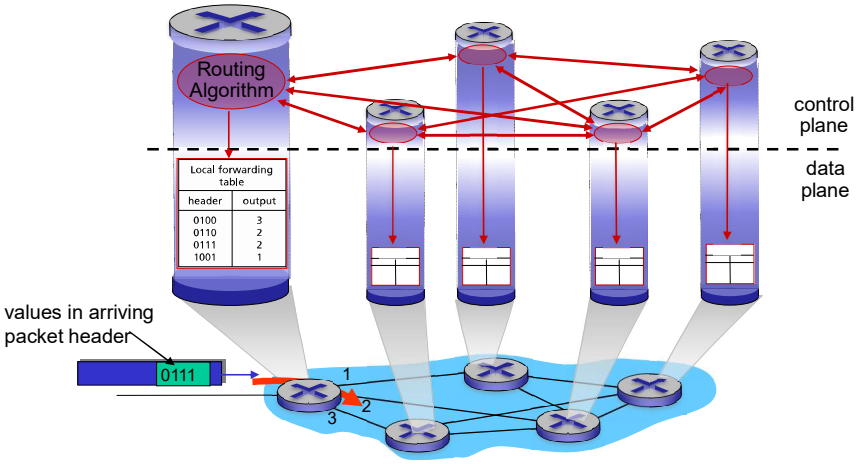
控制平面

- 网络范围内的逻辑
- 决定数据报如何在路由器之间路由，决定数据报从源到目标主机之间的端到端路径
- 2个控制平面方法:
  - 传统的路由算法: 在路由器中被实现
  - *software-defined networking (SDN)*: 在远程的服务器中实现

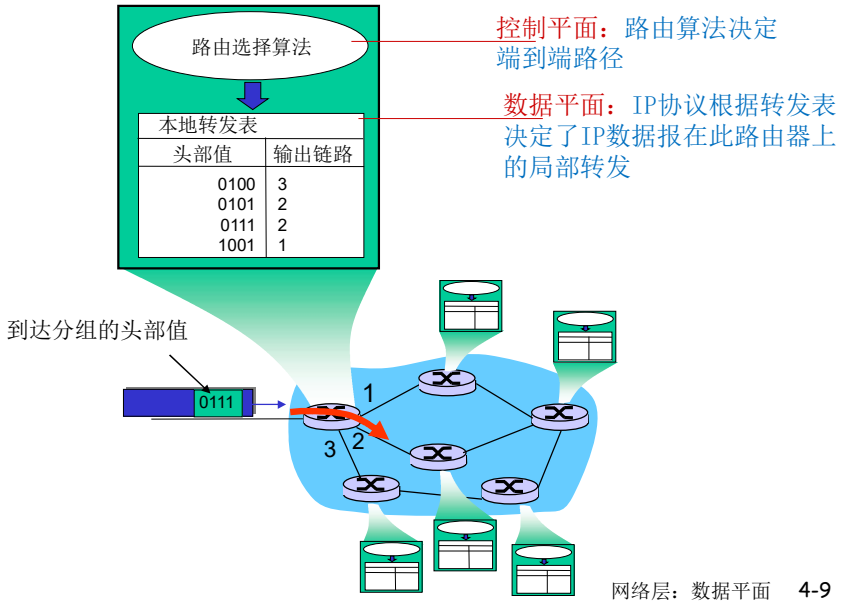


传统方式：每-路由器(Per-router)控制平面

在每一个路由器中的单独路由器算法元件，在控制平面进行交互

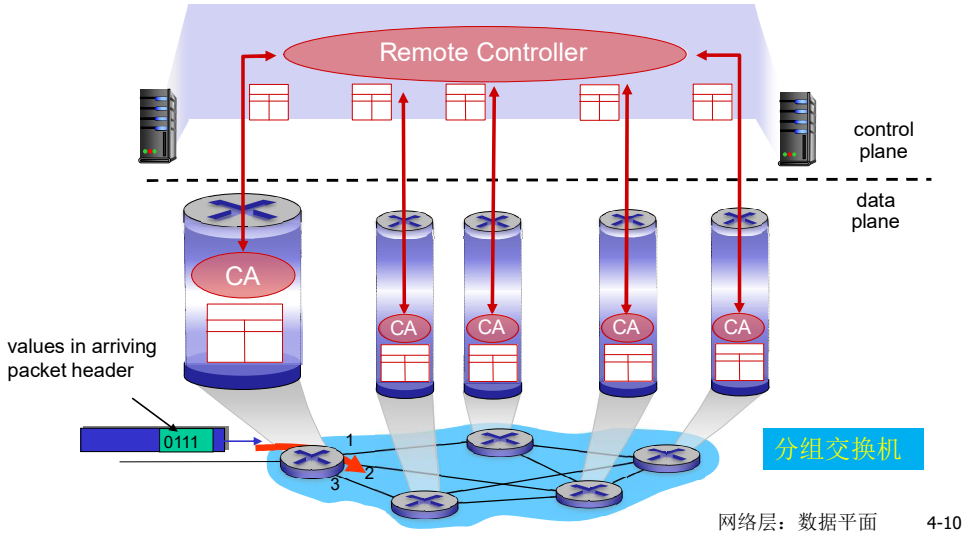


传统方式：路由和转发的相互作用



SDN方式：逻辑集中的控制平面

一个不同的（通常是远程的）控制器与本地控制代理（CAs）交互



网络服务模型

Q: 从发送方主机到接收方主机传输数据报的“通道”，网络提供什么样的服务模型？

对于单个数据报的服务：

- 可靠传送
- 延迟保证，如：少于40ms的延迟

对于数据报流的服务：

- 保序数据报传送
- 保证流的最小带宽
- 分组之间的延迟差

连接建立

- 在某些网络架构中是第三个重要的功能
  - ATM, frame relay, X.25
- 在分组传输之前，在两个主机之间，在通过一些路由器所构成的路径上建立一个网络层连接
  - 涉及到路由器
- 网络层和传输层连接服务区别：
  - 网络层：在2个主机之间，涉及到路径上的一些路由器
  - 传输层：在2个进程之间，很可能只体现在端系统上(TCP连接)

网络层服务模型:

网络架构	服务模型	保证？ 带宽	丢失	保序	延迟	拥塞反馈
Internet	best effort	none	no	no	no	no (inferred via loss)
ATM	CBR 恒定速率	constant rate	yes	yes	yes	no congestion
ATM	VBR 变化速率	guaranteed rate	yes	yes	yes	no congestion
ATM	ABR 可用比特率	guaranteed minimum	no	yes	no	yes
ATM	UBR 不指名比特率	none	no	yes	no	no

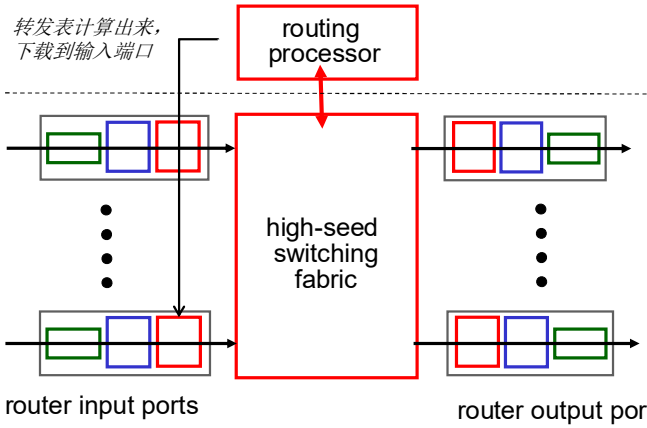
第4章 网络层：数据平面

- 4. 1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

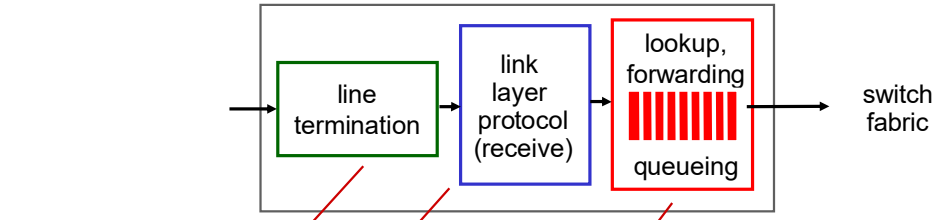
路由器结构概况

高层面(非常简化的)通用路由器体系架构

- 路由：运行路由选择算法 / 协议 (RIP, OSPF, BGP)-生成路由表
- 转发：从输入到输出链路交换数据报-根据路由表进行分组的转发



输入端口功能



分布式交换：

- 根据数据报头部的信息如：目的地址，在输入端口内存中的转发表中查找合适的输出端口（匹配+行动）
- 基于目标的转发：仅仅依赖于IP数据报的目标IP地址（传统方法）
- 通用转发：基于头部字段的任意集合进行转发

基于目标的转发

转发表

Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

Q: 但是如果地址范围如果没有划分的特别规整，会发生什么？

最长前缀匹配

longest prefix matching

当给定目标地址查找转发表时，采用最长地址前缀匹配的目标地址表项

Destination Address Range	Link interface
11001000 00010111 00010*** *********	0
11001000 00010111 00011000 *********	1
11001000 00010111 00011*** *********	2
otherwise	3

examples:

DA: 11001000 00010111 00010110 10100001 which interface?

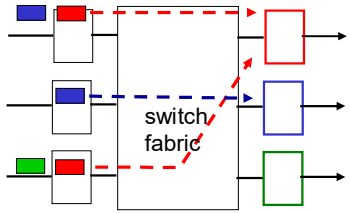
DA: 11001000 00010111 00011000 10101010 which interface?

最长前缀匹配

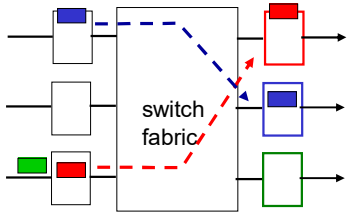
- 我们将会在学习IP地址时，简单讲解为什么要采用最长前缀匹配
- 最长前缀匹配：在路由器中经常采用TCAMs(ternary content addressable memories)硬件来完成
  - 内容可寻址：将地址交给TCAM，它可以在一个时钟周期内检索出地址，不管表空间有多大
  - Cisco Catalyst系列路由器: 在TCAM中可以存储多达约为1百万条路由表项

输入端口缓存

- 当交换机构的速率小于输入端口的汇聚速率时 ➡ 在输入端口可能要排队
  - *排队延迟以及由于输入缓存溢出造成丢失!*
- Head-of-the-Line (HOL) blocking: 排在队头的数据报阻止了队列中其他数据报向前移动



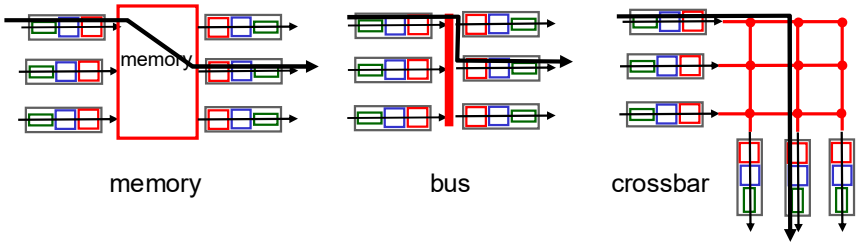
输出端口竞争:
只能有一个红色分组被传递，交换到一个输出端口.
下面红色的分组被阻塞



一个分组时间:
绿色分组经历了头端阻塞

交换结构

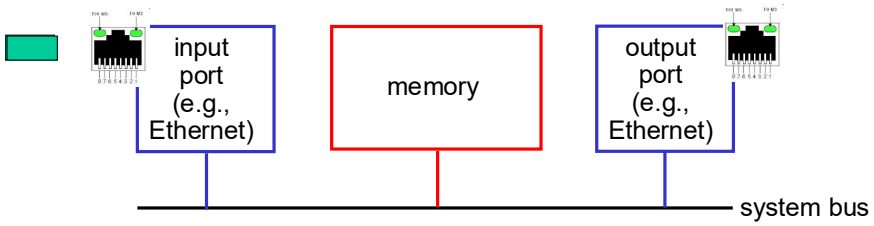
- 将分组从输入缓冲区传输到合适的输出端口
- 交换速率：分组可以按照该速率从输入传输到输出
  - 运行速度经常是输入/输出链路速率的若干倍
  - N 个输入端口：交换机构的交换速度是输入线路速度的N倍比较理想，才不会成为瓶颈
- 3种典型的交换机构



通过内存交换

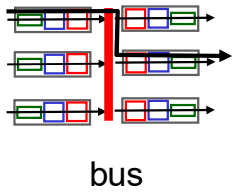
第一代路由器：

- 在CPU直接控制下的交换，采用传统的计算机
- 分组被拷贝到系统内存，CPU从分组的头部提取出目标地址，查找转发表，找到对应的输出端口，拷贝到输出端口
- 转发速率被内存的带宽限制 (数据报通过BUS两遍)
- 一次只能转发一个分组



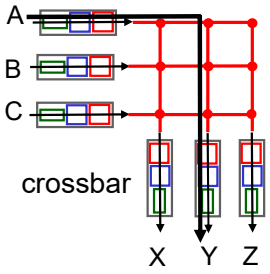
通过总线交换

- 数据报通过共享总线，从输入端口转发到输出端口
- 总线竞争: 交换速度受限于总线带宽
- 1次处理一个分组
- 1 Gbps bus, Cisco 1900； 32 Gbps bus, Cisco 5600；对于接入或企业级路由器，速度足够（但不适合区域或骨干网络）

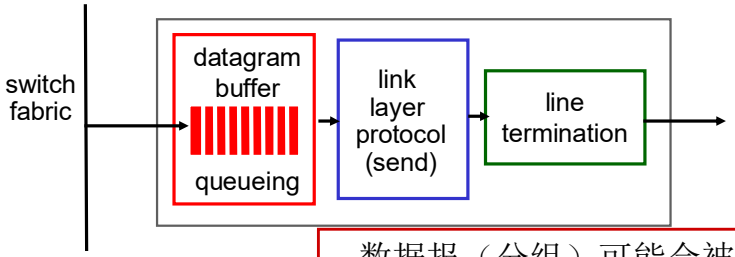


通过互联网络(crossbar等)的交换

- 同时并发转发多个分组，克服总线带宽限制
- Banyan（榕树）网络，crossbar(纵横)和其它的互联网络被开发，将多个处理器连接成多处理器
- 当分组从端口A到达，转给端口Y；控制器短接相应的两个总线
- 高级设计：将数据报分片为固定长度的信元，通过交换网络交换
- Cisco12000：以60Gbps的交换速率通过互联网络



输出端口

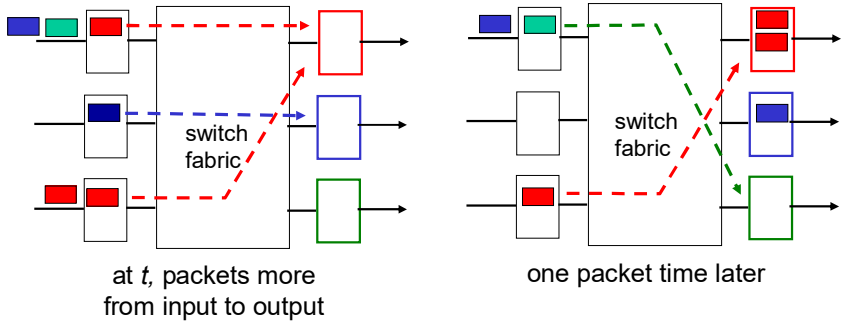


数据报（分组）可能会被丢弃，由于拥塞，缓冲区没有空间

- 当数据报从交换机构的到达速度比传输速率快就需要输出端口缓存
- 由调度规则选择排队的数据报进行传输

优先权调度-谁会获得最优性能，网络中立？

输出端口排队



- 假设交换速率R_{switch}是R_{line}的N倍（N：输入端口的数量）
- 当多个输入端口同时向输出端口发送时，缓冲该分组（当通过交换网络到达的速率超过输出速率则缓存）
- 排队带来延迟，由于输出端口缓存溢出则丢弃数据报！

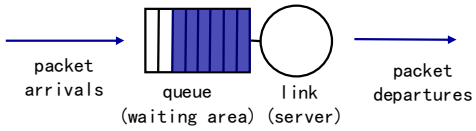
需要多少缓存?

- RFC 3439 拇指规则（经验性规则）：平均缓存大小=典型的RTT（例如：250ms）倍于链路容量C
  - e.g., C = 10 Gpbs link
  - 250ms*10Gbps=2.5 Gbit buffer
- 最近的一些推荐：有*N（非常大）*个流，缓存大小等于

$$\frac{RTT \cdot C}{\sqrt{N}}$$

调度机制

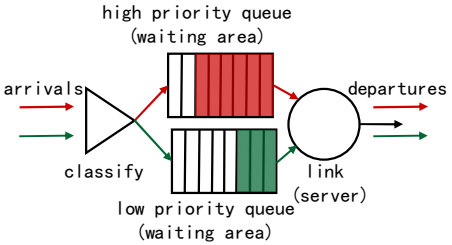
- *调度:* 选择下一个要通过链路传输的分组
- *FIFO (first in first out) scheduling:* 按照分组到来的次序发送
  - 现实例子?
  - *丢弃策略:* 如果分组到达一个满的队列，哪个分组将会被抛弃?
    - *tail drop:* 丢弃刚到达的分组
    - *priority:* 根据优先权丢失/移除分组
    - *random:* 随机地丢弃/移除

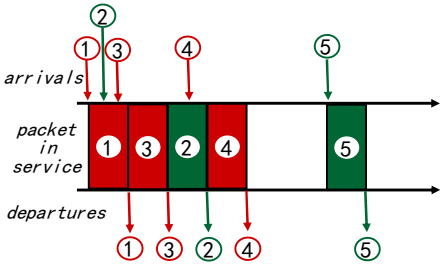


调度策略：优先权

*优先权调度：*发送最高优先权的分组

- 多类，不同类别有不同的优先权
  - 类别可能依赖于标记或者其他的头部字段，e.g. IP source/dest，port numbers，ds，etc.
  - 先传高优先级的队列中的分组，除非没有
  - 高（低）优先权中的分组传输次序：FIFO
  - 现实生活中的例子?

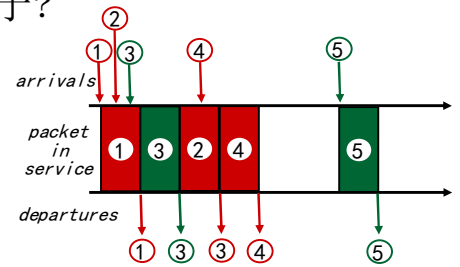




调度策略：其他的

Round Robin (RR) scheduling:

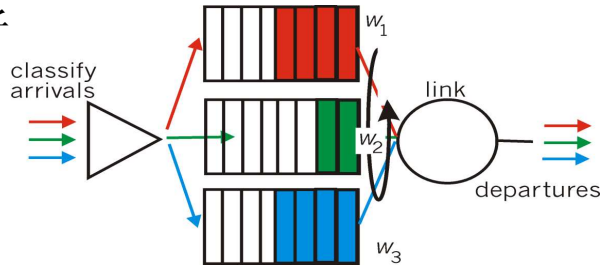
- 多类
- 循环扫描不同类型的队列，发送完一类的一个分组，再发送下一个类的一个分组，循环所有类
- 现实例子?



调度策略：其他的

Weighted Fair Queuing (WFQ):

- 一般化的Round Robin
- 在一段时间内，每个队列得到的服务时间是：$W_i/(XIGMA(W_i))$ *t，和权重成正比
- 每个类在每一个循环中获得不同权重的服务量
- 现实例子

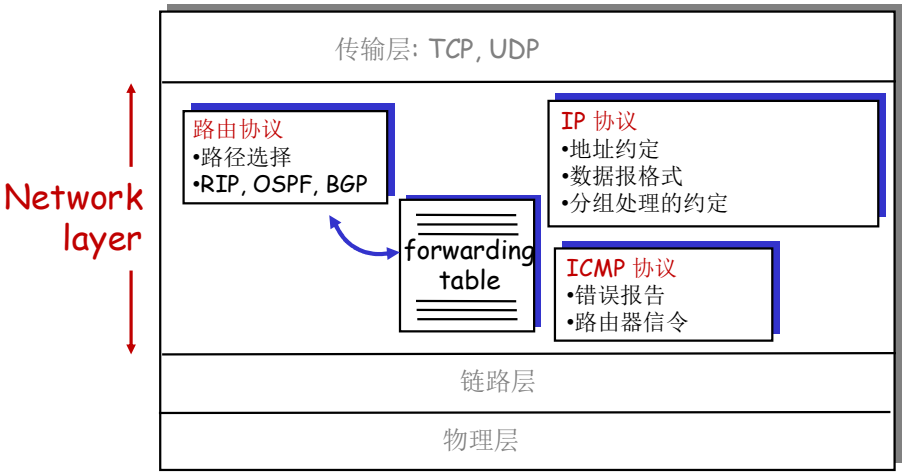


第4章 网络层：数据平面

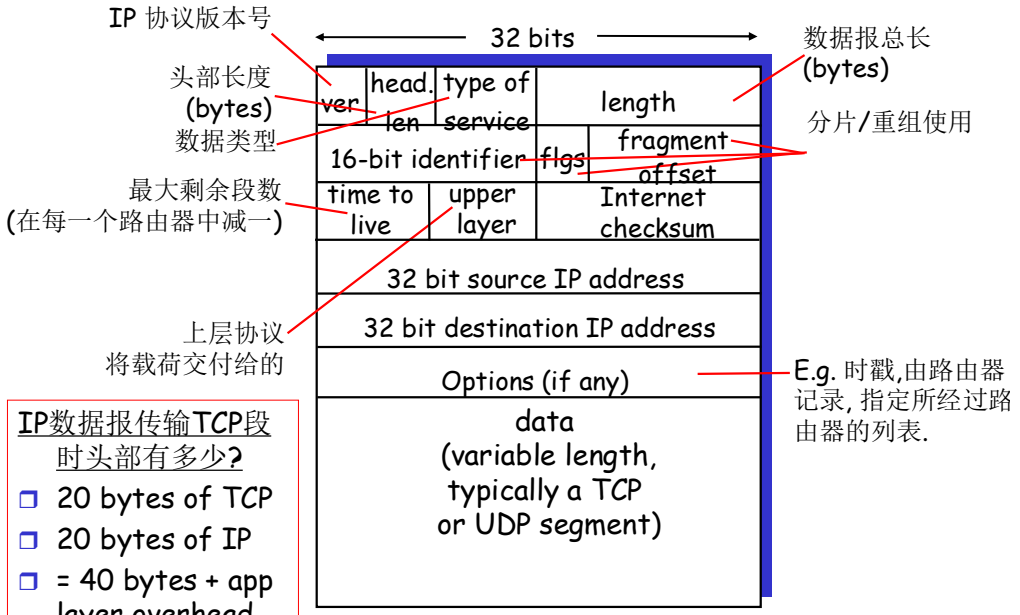
- 4.1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

互联网的网络层

主机,路由器中的网络层功能：



IP 数据报格式

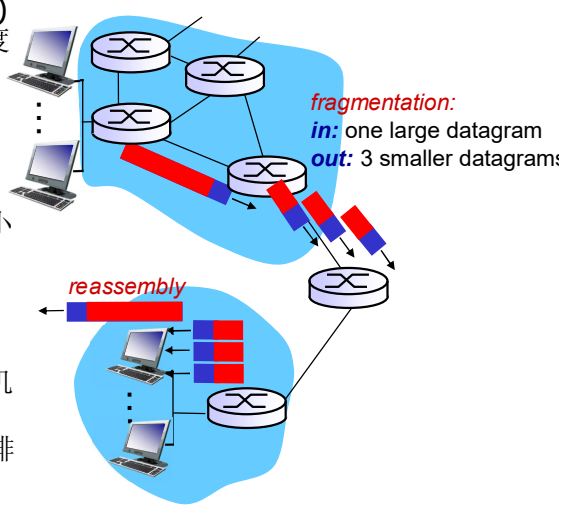


IP数据报传输TCP段时头部有多少?

- 20 bytes of TCP
- 20 bytes of IP
- = 40 bytes + app layer overhead

IP 分片和重组(Fragmentation & Reassembly)

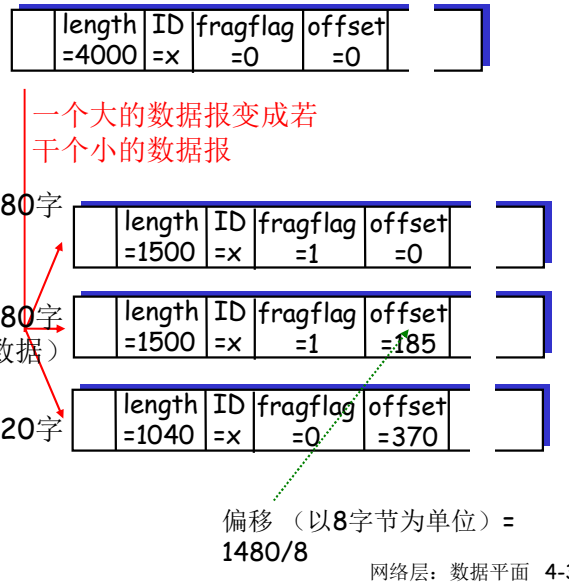
- 网络链路有MTU (最大传输单元) -链路层帧所携带的最大数据长度
  - 不同的链路类型
  - 不同的MTU
- 大的IP数据报在网络上被分片 (“fragmented”)
  - 一个数据报被分割成若干个小的数据报
    - 相同的ID
    - 不同的偏移量
    - 最后一个分片标记为0
  - “重组”只在最终的目标主机进行
  - IP头部的信息被用于标识，排序相关分片



IP 分片和重组

例子

- 4000 字节数据报
  - 20字节头部
  - 3980字节数据
- MTU = 1500 bytes
- 第一片：20字节头部+1480字节数据
  - 偏移量：0
- 第二片：20字节头部+1480字节数据（1480字节应用数据）
  - 偏移量：1480/8=185
- 第三片：20字节头部+1020字节数据（应用数据）
  - 偏移量：2960/8=370

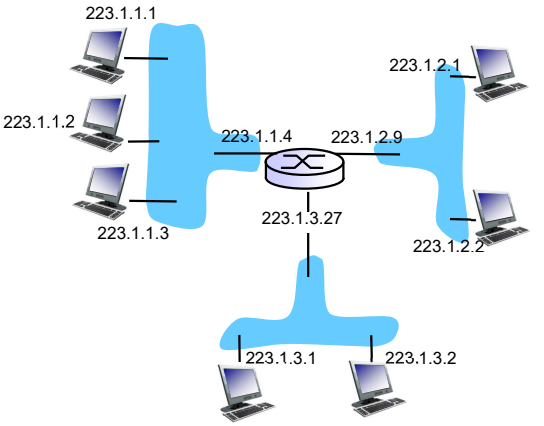


第4章 网络层：数据平面

- 4.1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

IP 编址：引论

- IP 地址：32位标示，对主机或者路由器的接口编址
- *接口*：主机/路由器和物理链路的连接处
  - 路由器通常拥有多个接口
  - 主机也有可能有多个接口
  - IP地址和每一个接口关联
- 一个IP地址和一个接口相关联



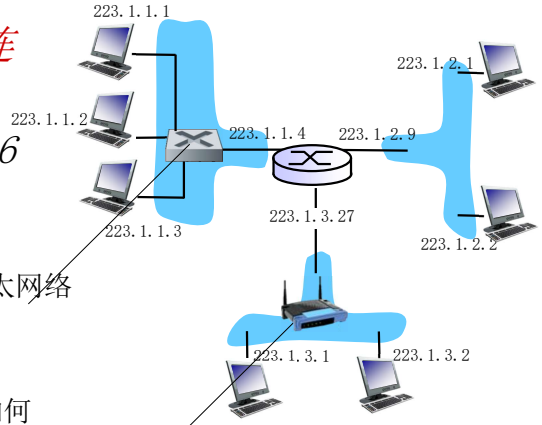
223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

IP 编址：引论

Q：这些接口是如何连接的?

A：我们将会在第5，6章学习



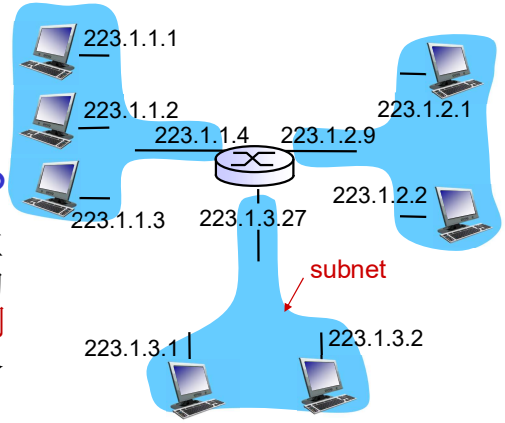
A: 有线以太网网口链接到以太网络交换机连接

*目前：*无需担心一个接口是如何接到另外一个接口（中间没有路由器）

A: 无线WiFi接口被WiFi基站连接

子网(Subnets)

- IP地址：
  - 子网部分(高位bits)
  - 主机部分(地位bits)
- *什么是子网(subnet)？*
  - 一个子网内的节点（主机或者路由器）它们的IP地址的高位部分相同，这些节点构成的网络的一部分叫做子网
  - 无需路由器介入，子网内各主机可以在物理上相互直接到达

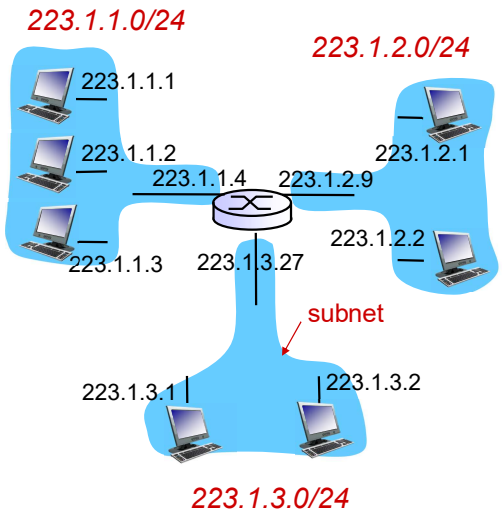


包括3个子网的网络

子网

方法：

- 要判断一个子网，将每一个接口从主机或者路由器上分开，构成了一个个网络的孤岛
- 每一个孤岛（网络）都是一个都可以被称之为subnet.

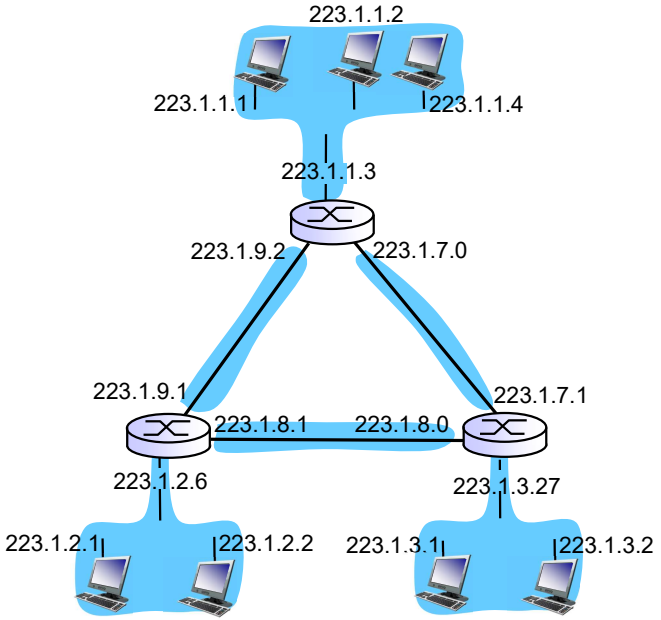


子网掩码：11111111 11111111 11111111 00000000

Subnet mask: /24

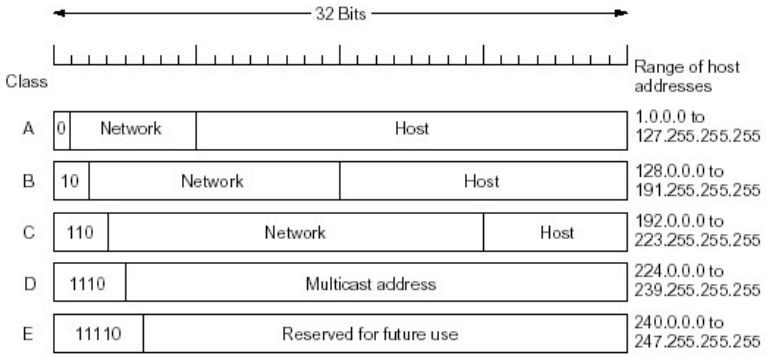
子网

几个?



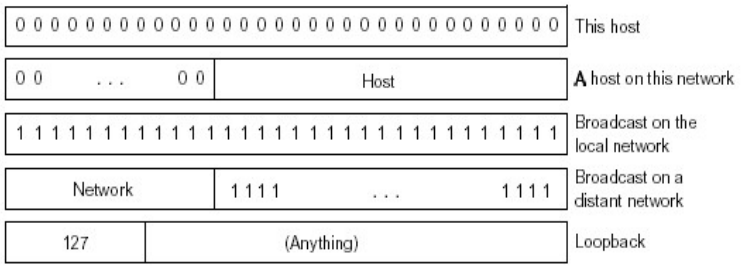
IP 地址分类

- Class A：126 networks，16 million hosts
- Class B：16382networks，64 K hosts
- Class C：2 million networks，254 host
- Class D：multicast
- Class E：reserved for future



特殊IP地址

- 一些约定：
  - 子网部分：全为 0---本网络
  - 主机部分：全为0---本主机
  - 主机部分：全为1--广播地址，这个网络的所有主机
- 特殊IP地址



内网(专用)IP地址

- 专用地址：地址空间的一部份供专用地址使用
- 永远不会被当做公用地址来分配，不会与公用地址重复
  - 只在局部网络中有意义，区分不同的设备
- 路由器不对目标地址是专用地址的分组进行转发
- 专用地址范围
  - Class A 10.0.0.0-10.255.255.255 MASK 255.0.0.0
  - Class B 172.16.0.0-172.31.255.255 MASK 255.255.0.0
  - Class C 192.168.0.0-192.168.255.255 MASK 255.255.255.0

IP 编址: CIDR

CIDR: Classless InterDomain Routing

（无类域间路由）

- 子网部分可以在任意的位置
- 地址格式: a.b.c.d/x, 其中 x 是 地址中子网号的长度

200.23.16.0/23

子网掩码：11111111 11111111 11111110 00000000

子网掩码(subnet mask)

- 32bits , 0 or 1 in each bit
  - 1: bit位置表示子网部分
  - 0:bit位置表示主机部分
- 原始的A、B、C类网络的子网掩码分别是
  - A：255.0.0.0 ：11111111 00000000 0000000 00000000
  - B：255.255.0.0：11111111 11111111 0000000 00000000
  - C：255.255.255.0：11111111 11111111 11111111 00000000
- CIDR下的子网掩码例子：
  - 11111111 11111111 11111100 00000000
- 另外的一种表示子网掩码的表达方式
  - /#
  - 例：/22：表示前面22个bit为子网部分

转发表和转发算法

Destination Subnet Num	Mask	Next hop	Interface
202.38.73.0	255.255.255.192	IPx	Lan1
202.38.64.0	255.255.255.192	IPy	Lan2
.....			
Default	-	IPz	Lan0

- 获得IP数据报的目标地址
- 对于转发表中的每一个表项
  - ✓如 (IP Des addr) & (mask)== destination, 则按照表项对应的接口转发该数据报
  - ✓ 如果都没有找到,则使用默认表项转发数据报

如何获得一个IP地址

Q: 主机如何获得一个IP地址?

- 系统管理员将地址配置在一个文件中
  - Wintel: control-panel->network->configuration->tcp/ip->properties
  - UNIX: /etc/rc.config
- DHCP: Dynamic Host Configuration Protocol: 从服务器中动态获得一个IP地址
  - “plug-and-play”

DHCP: Dynamic Host Configuration Protocol

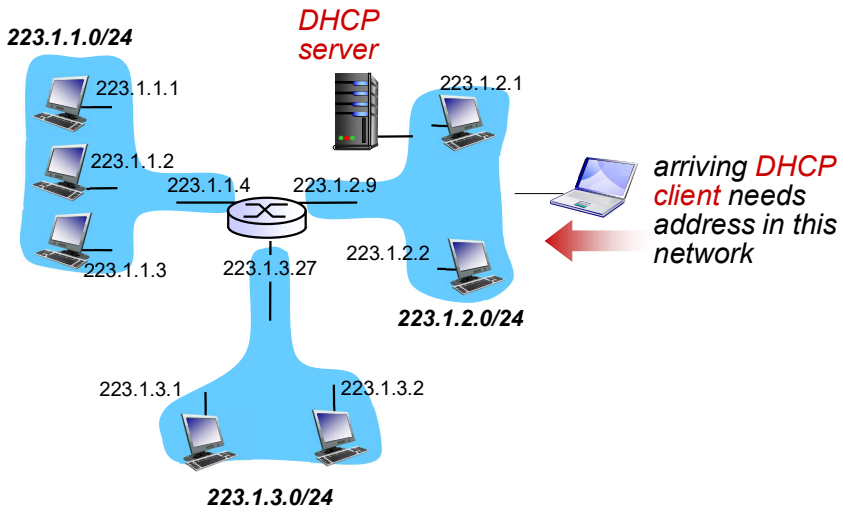
目标: 允许主机在加入网络的时候，动态地从服务器那里获得IP地址：

- 可以更新对主机在用IP地址的租用期-租期快到了
- 重新启动时，允许重新使用以前用过的IP地址
- 支持移动用户加入到该网络（短期在网）

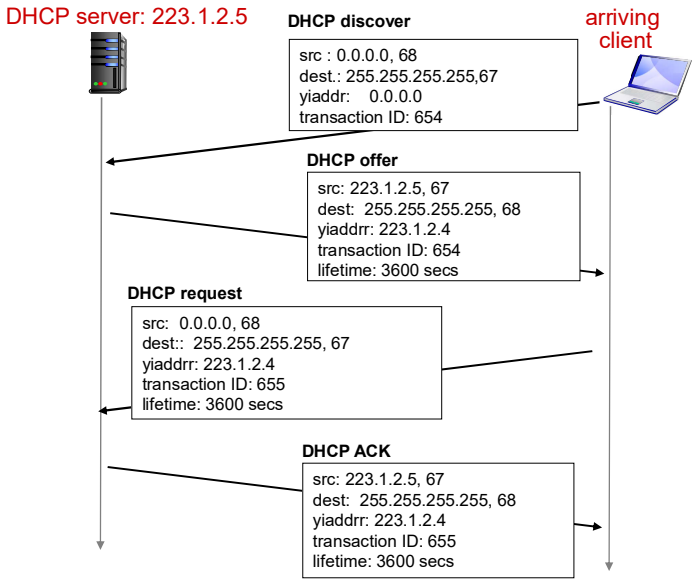
DHCP工作概况:

- 主机广播“DHCP discover”报文[可选]
- DHCP 服务器用 “DHCP offer”提供报文响应[可选]
- 主机请求IP地址：发送 “DHCP request” 报文
- DHCP服务器发送地址： “DHCP ack” 报文

DHCP client-server scenario



DHCP client-server scenario

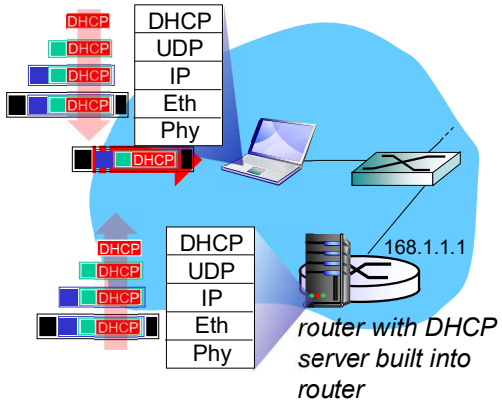


DHCP: 不仅仅是IP addresses

DHCP 返回:

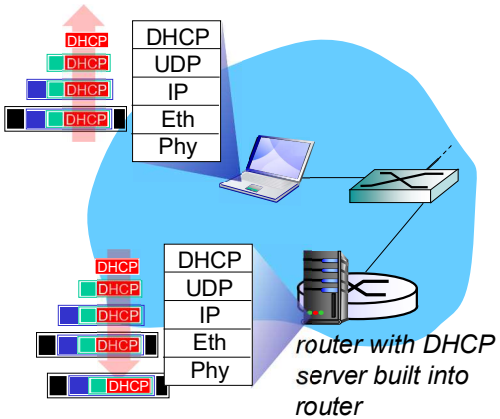
- IP 地址
- 第一跳路由器的IP地址（默认网关）
- DNS服务器的域名和IP地址
- 子网掩码 (指示地址部分的网络号和主机号)

DHCP: 实例



- 联网笔记本需要获取自己的IP地址，第一跳路由器地址和DNS服务器：采用DHCP协议
- DHCP 请求被封装在UDP段中，封装在IP数据报中，封装在以太网的帧中
- 以太网帧在局域网范围内广播（dest: FFFFFFFFFFFF），被运行DHCP服务的路由器收到
- 以太网帧解封装成IP，IP解封装成UDP，解封装成DHCP

DHCP: 实例



- DHCP服务器生成DHCP ACK，包含客户端的IP地址，第一跳路由器的IP地址和DNS域名服务器的IP地址
- DHCP服务器封装的报文所在的帧转发到客户端，在客户端解封装成DHCP报文
- 客户端知道它自己的IP地址，DNS服务器的名字和IP地址，第一跳路由器的IP地址

DHCP: Wireshark 输出(home LAN)

request

```
Message type: Boot Request (1)
Hardware type: Ethernet
Hardware address length: 6
Hops: 0
Transaction ID: 0x6b3a11b7
Seconds elapsed: 0
Bootp flags: 0x0000 (Unicast)
Client IP address: 0.0.0.0 (0.0.0.0)
Your (client) IP address: 0.0.0.0 (0.0.0.0)
Next server IP address: 0.0.0.0 (0.0.0.0)
Relay agent IP address: 0.0.0.0 (0.0.0.0)
Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)
Server host name not given
Boot file name not given
Magic cookie: (OK)
Option: (t=53,l=1) DHCP Message Type = DHCP Request
Option: (61) Client identifier
    Length: 7; Value: 010016D323688A;
    Hardware type: Ethernet
    Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)
Option: (t=50,l=4) Requested IP Address = 192.168.1.101
Option: (t=12,l=5) Host Name = "nomad"
Option: (55) Parameter Request List
    Length: 11; Value: 010F03062C2E2F1F21F92B
    1 = Subnet Mask; 15 = Domain Name
    3 = Router; 6 = Domain Name Server
    44 = NetBIOS over TCP/IP Name Server
    .....
```

reply

```
Message type: Boot Reply (2)
Hardware type: Ethernet
Hardware address length: 6
Hops: 0
Transaction ID: 0x6b3a11b7
Seconds elapsed: 0
Bootp flags: 0x0000 (Unicast)
Client IP address: 192.168.1.101 (192.168.1.101)
Your (client) IP address: 0.0.0.0 (0.0.0.0)
Next server IP address: 192.168.1.1 (192.168.1.1)
Relay agent IP address: 0.0.0.0 (0.0.0.0)
Client MAC address: Wistron_23:68:8a (00:16:d3:23:68:8a)
Server host name not given
Boot file name not given
Magic cookie: (OK)
Option: (t=53,l=1) DHCP Message Type = DHCP ACK
Option: (t=54,l=4) Server Identifier = 192.168.1.1
Option: (t=1,l=4) Subnet Mask = 255.255.255.0
Option: (t=3,l=4) Router = 192.168.1.1
Option: (6) Domain Name Server
    Length: 12; Value: 445747E2445749F244574092;
    IP Address: 68.87.71.226;
    IP Address: 68.87.73.242;
    IP Address: 68.87.64.146
Option: (t=15,l=20) Domain Name = "hsd1.ma.comcast.net."
```

如何获得一个IP地址

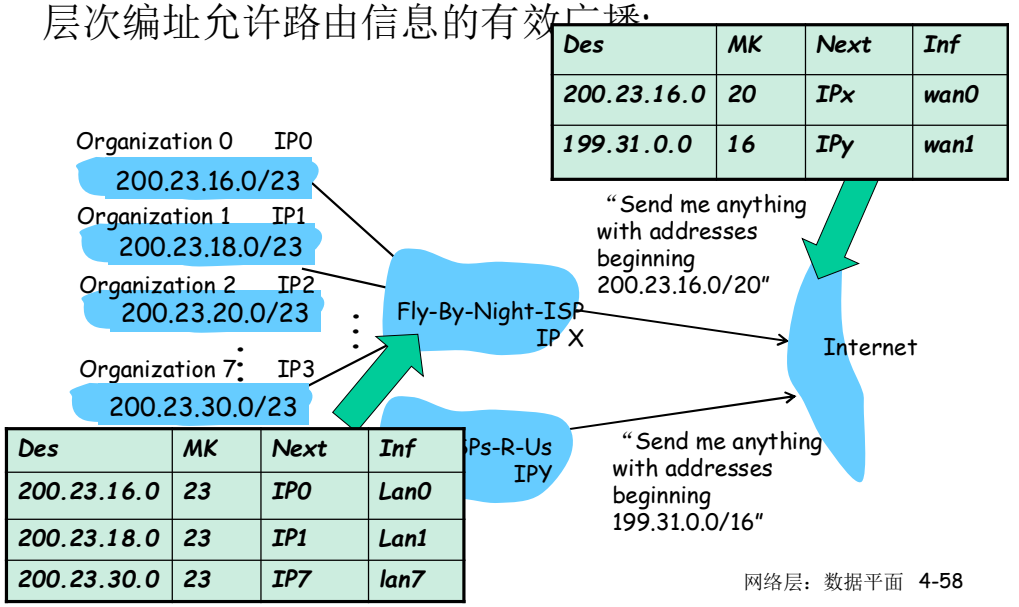
Q: 如何获得一个网络的子网部分?

A: 从ISP获得地址块中分配一个小地址块

ISP's block	11001000 00010111 00010000	00000000	200.23.16.0/20
Organization 0	11001000 00010111 00010000	00000000	200.23.16.0/23
Organization 1	11001000 00010111 00010010	00000000	200.23.18.0/23
Organization 2	11001000 00010111 00010100	00000000	200.23.20.0/23
...	.....	....	....
Organization 7	11001000 00010111 00011110	00000000	200.23.30.0/23

层次编址: 路由聚集（route aggregation）

层次编址允许路由信息的有效广播:



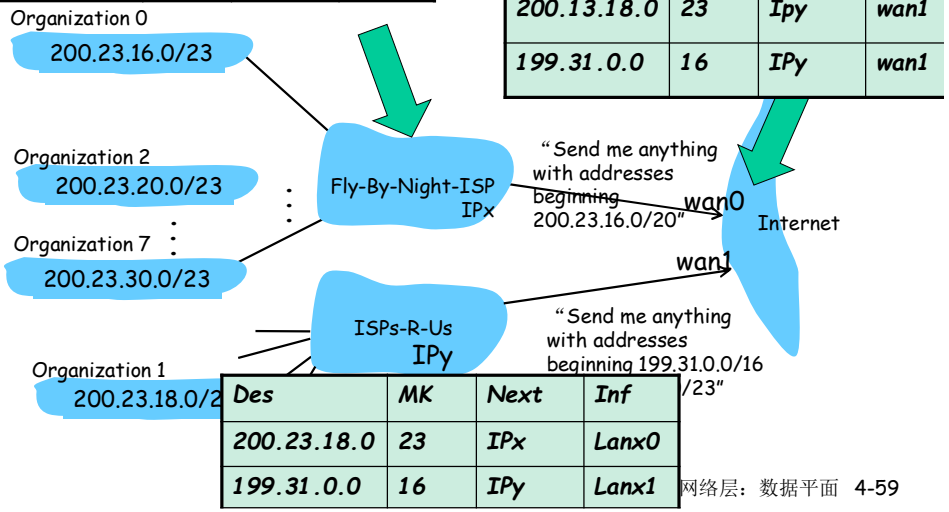
Des	MK	Next	Inf
200.23.16.0	20	IPx	wan0
199.31.0.0	16	IPy	wan1

Des	MK	Next	Inf
200.23.16.0	23	IP0	Lan0
200.23.18.0	23	IP1	Lan1
200.23.30.0	23	IP7	lan7

层次编址: 特殊路由信息(more specific routes)

Des	MK	Next	Inf
200.23.16.0	23	IP0	Lan0
200.23.20.0	23	IP2	Lan1
200.23.30.0	23	IP7	lan7

Des	MK	Next	Inf
200.23.16.0	20	IPx	wan0
200.13.18.0	23	Ipy	wan1
199.31.0.0	16	IPy	wan1



Des	MK	Next	Inf
200.23.18.0	23	IPx	Lanx0
199.31.0.0	16	IPy	Lanx1

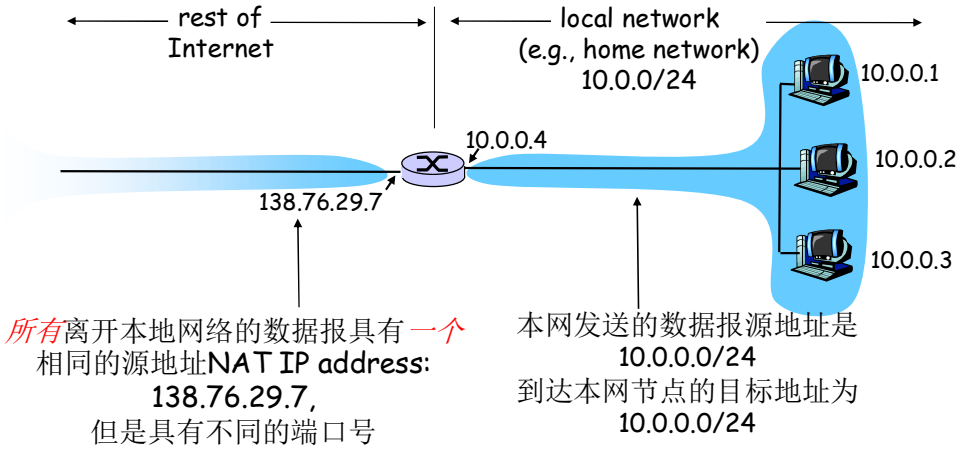
IP 编址: 如何获得一块地址

Q: 一个ISP如何获得一个地址块?

A: ICANN: Internet Corporation for Assigned Names and Numbers

- 分配地址
- 管理DNS
- 分配域名，解决冲突

NAT: Network Address Translation



NAT: Network Address Translation

- 动机：本地网络只有一个有效IP地址：
  - 不需要从ISP分配一块地址，可用一个IP地址用于所有的（局域网）设备--省钱
  - 可以在局域网改变设备的地址情况下而无须通知外界
  - 可以改变ISP（地址变化）而不需要改变内部的设备地址
  - 局域网内部的设备没有明确的地址，对外是不可见的--安全

NAT: Network Address Translation

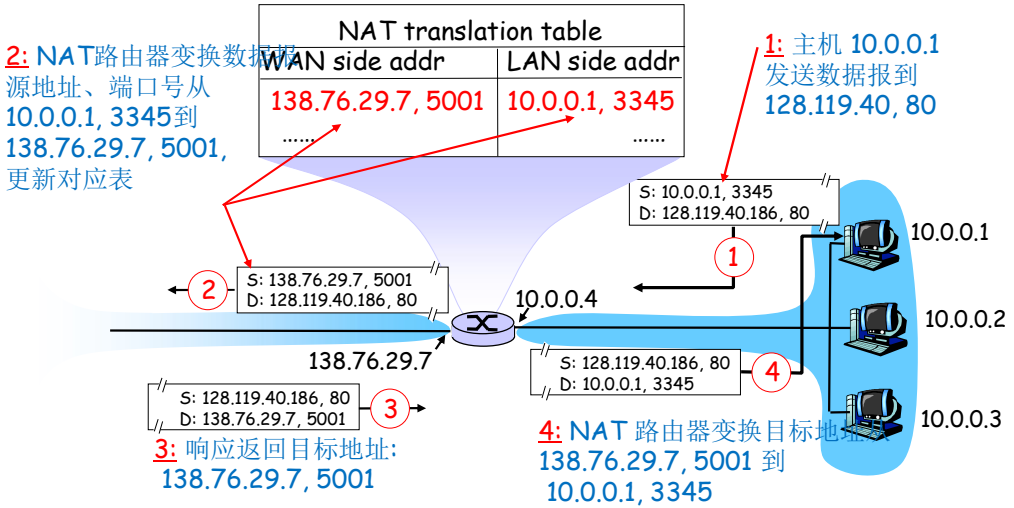
实现：NAT 路由器必须：

- **外出数据包**：替换源地址和端口号为NAT IP地址和新的端口号，目标IP和端口不变
  ...远端的C/S将会用NAP IP地址，新端口号作为目标地址

- **记住**每个转换替换对（在NAT转换表中）
  .. 源IP，端口 vs NAP IP，新端口

- **进入数据包**：替换目标IP地址和端口号，采用存储在NAT表中的mapping表项，用（源IP，端口）

NAT: Network Address Translation

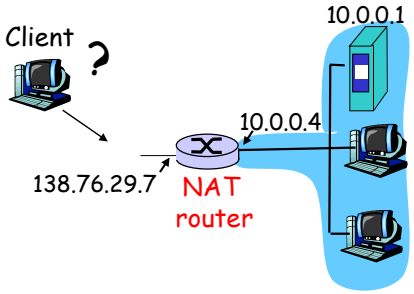


NAT: Network Address Translation

- 16-bit端口字段:
  - 6万多个同时连接，一个局域网!
- 对NAT是有争议的:
  - 路由器只应该对第3层做信息处理，而这里对端口号（4层）作了处理
  - 违反了end-to-end 原则
    - 端到端原则：复杂性放到网络边缘
      - 无需借助中转和变换，就可以直接传送到目标主机
    - NAT可能要被一些应用设计者考虑, eg, P2P applications
    - 外网的机器无法主动连接到内网的机器上
  - 地址短缺问题可以被IPv6 解决
  - NAT穿越： 如果客户端需要连接在NAT后面的服务器，如何操作

NAT 穿越问题（略）

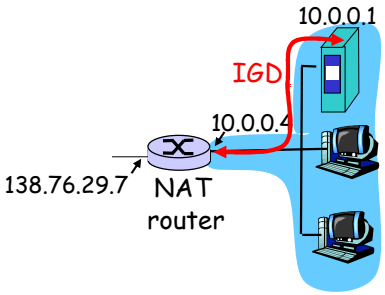
- 客户端需要连接地址为10.0.0.1的服务器
  - 服务器地址10.0.0.1 LAN本地地址 (客户端不能够使用其作为目标地址)
  - 整网只有一个外部可见地址: 138.76.29.7
- 方案1: 静态配置NAT：转发进来的对服务器特定端口连接请求
  - e.g., (123.76.29.7, port 2500) 总是转发到10.0.0.1 port 25000



NAT 穿越问题（略）

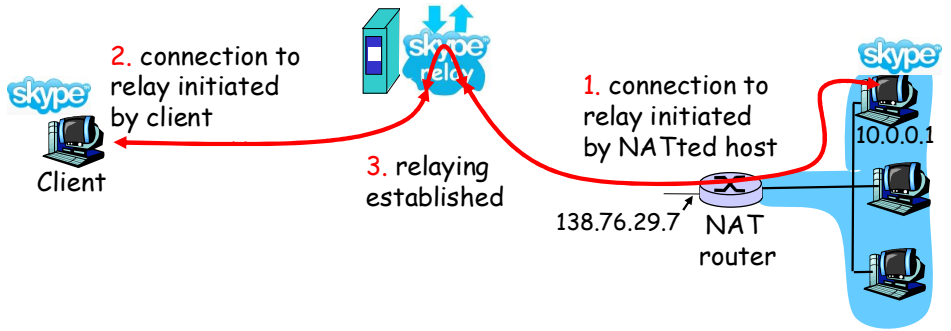
- 方案2: Universal Plug and Play (UPnP) Internet Gateway Device (IGD) 协议. 允许NATted主机可以:
  - 获知网络的公共 IP地址 (138.76.29.7)
  - 列举存在的端口映射
  - 增/删端口映射 (在租用时间内)

  i.e., 自动化静态NAT端口映射配置



NAT 穿越问题（略）

- 方案 3: 中继 (used in Skype)
  - NAT后面的服务器建立和中继的连接
  - 外部的客户端链接到中继
  - 中继在2个连接之间桥接



第4章 网络层：数据平面

- 4. 1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

IPv6：动机

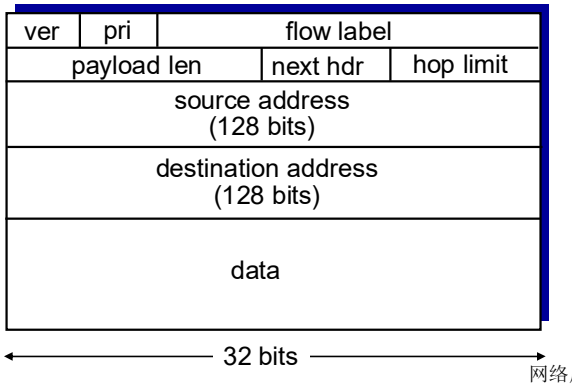
- 初始动机：32-bit地址空间将会被很快用完
- 另外的动机：
  - 头部格式改变帮助加速处理和转发
    - TTL-1
    - 头部checksum
    - 分片
  - 头部格式改变帮助QoS

IPv6 数据报格式：

- 固定的40 字节头部
- 数据报传输过程中，不允许分片

IPv6 头部 (Cont)

Priority: 标示流中数据报的优先级
Flow Label: 标示数据报在一个“flow.”
(“flow”的概念没有被严格的定义)
Next header: 标示上层协议

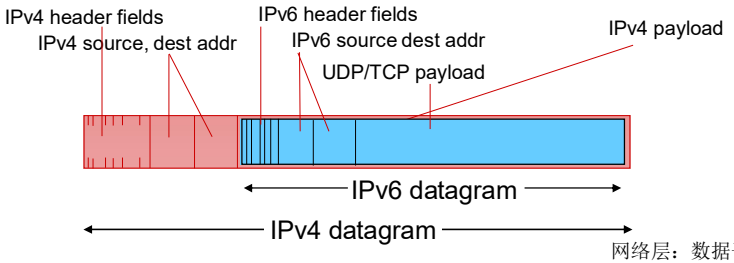


和IPv4的其它变化

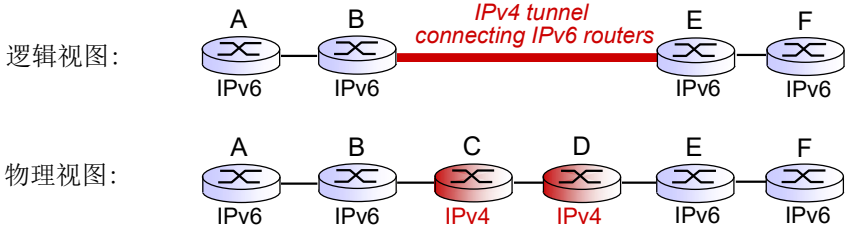
- *Checksum:* *被移除掉，降低在每一段中的处理速度*
- *Options:* 允许，但是在头部之外，被 “Next Header” 字段标示
- *ICMPv6:* ICMP的新版本
  - 附加了报文类型，e.g. “Packet Too Big”
  - 多播组管理功能

从IPv4到IPv6的平移

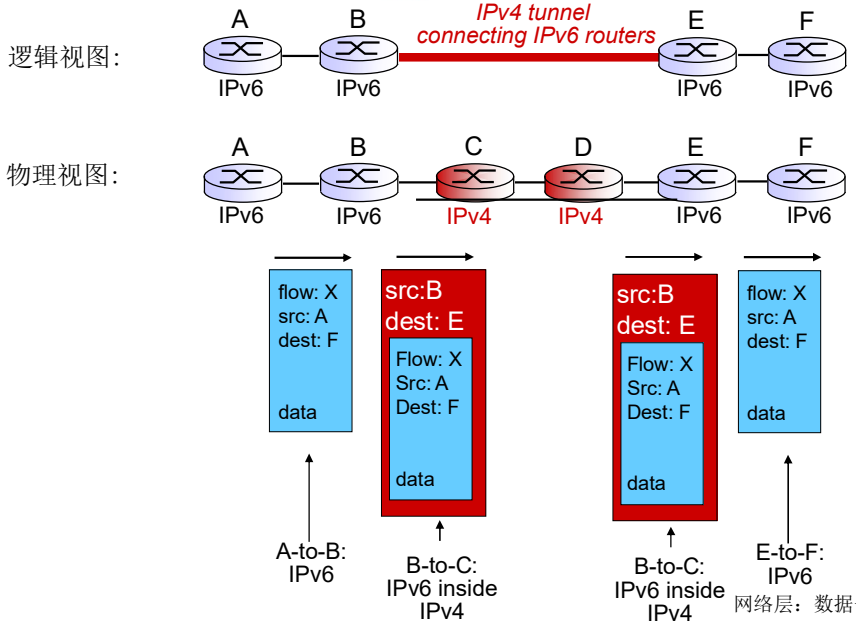
- 不是所有的路由器都能够同时升级的
  - 没有一个标记日 “flag days”
  - 在IPv4和IPv6路由器混合时，网络如何运转?
- *隧道:* 在IPv4路由器之间传输的IPv4数据报中携带IPv6数据报



隧道(Tunneling)



隧道(Tunneling)



IPv6：应用

- Google: 8% 的客户通过IPv6访问谷歌服务
- NIST: 全美国1/3的政府域支持IPv6
- *估计还需要很长时间进行部署*
  - 20年以上!
  - 看看过去20年来应用层面的变化: WWW, Facebook, streaming media, Skype, …
  - *为什么?*

第4章 网络层：数据平面

- 4.1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

网络层功能为例的数据平面和控制平面

网络层功能：

- *转发：* 对于从某个端口到来的分组转发到合适的输出端口
- *路由：* 决定分组从源端到目标端的路径
  - *路由算法*

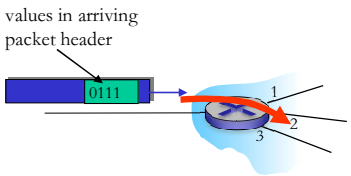
类比：旅行

- *转发：* 一个多岔路口的进入和转出过程
- *路由：* 规划从源到目标的旅行路径

网络层：数据平面和控制平面

数据平面

- 本地的、每个路由器的功能
- 决定某个从某个端口进入的分组从从哪个端口输出
- 转发功能

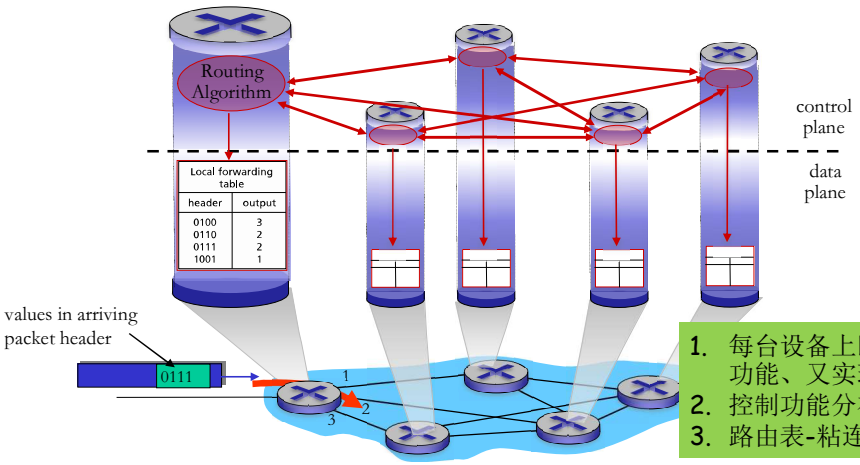


控制平面

- 网络范围的逻辑
- 决定分组端到端穿行于各个路由器的路径

每个路由器(Per Route)的控制平面

每个路由器上都有实现路由算法元件（它们之间需要相互交互）- 形成传统IP实现方式的控制平面



1. 每台设备上既实现控制功能、又实现数据平面
2. 控制功能分布式实现
3. 路由表-粘连

数量众多、功能各异的中间盒

- 路由器的网络层功能：
  - IP转发：对于到来的分组按照路由表决定如何转发，数据平面
  - 路由：决定路径，计算**路由表**；处在控制平面
- 还有其他种类繁多网络设备（中间盒）：
  - 交换机；防火墙；NAT；IDS；负载均衡设备
  - 未来：不断增加的需求和相应的网络设备
  - 需要不同的设备去实现不同的网络功能
    - 每台设备集成了控制平面和数据平面的功能
    - 控制平面分布式地实现了各种控制平面功能
    - 升级和部署网络设备非常困难

网络设备控制平面的实现方式特点

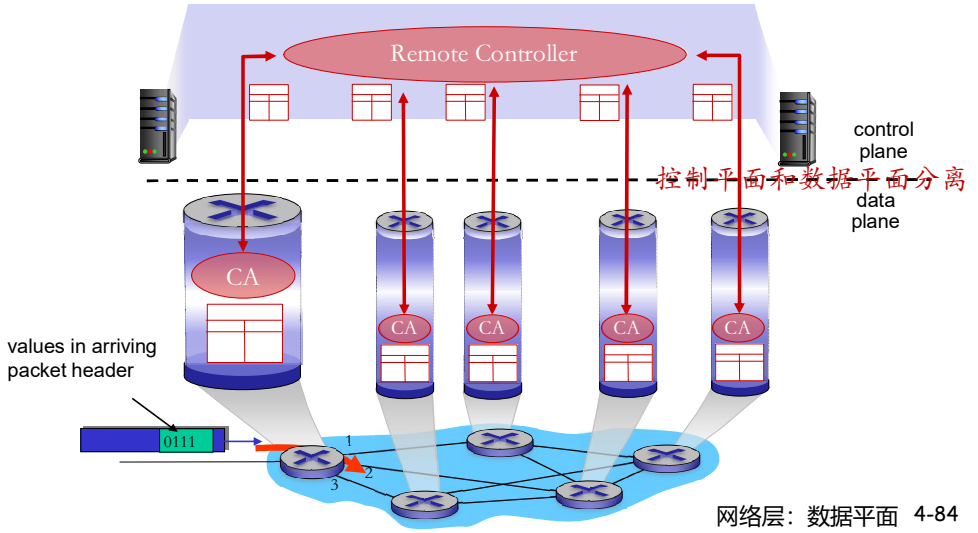
- 互联网网络设备：传统方式都是通过分布式，每台设备的方法来实现数据平面和控制平面功能
  - *垂直集成：每台*路由器或其他网络设备，包括：
    - 1）硬件、在私有的操作系统；
    - 2）互联网标准协议(IP, RIP, IS-IS, OSPF, BGP)的私有实现
    - 从上到下都由一个厂商提供（代价大、被设备商“绑架”“）
  - 每个设备都实现了数据平面和控制平面的事情
    - 控制平面的功能是分布式实现的
  - 设备基本上只能（分布式升级困难）按照固定方式工作，控制逻辑固化。不同的网络功能需要不同的“middleboxes”：防火墙、负载均衡设备、NAT boxes, ..
- （数据+控制平面）集成>（控制逻辑）分布->固化
  - 代价大；升级困难；管理困难等

传统方式实现网络功能的问题

- 问题：
  - 垂直集成>昂贵、不便于创新的生态
  - 分布式、固化设备功能==网络设备种类繁多
    - 无法改变路由等工作逻辑，无法实现流量工程等高级特性
    - 配置错误影响全网运行；升级和维护会涉及到全网设备：**管理困难**
    - 要增加新的网络功能，需要设计、实现以及部署新的特定设备，设备种类繁多
- ~2005: 开始重新思考网络控制平面的处理方式
  - 集中：远程的控制器集中实现控制逻辑
  - 远程：数据平面和控制平面的分离

SDN：逻辑上集中的控制平面

一个不同的（通常是远程）控制器和CA交互，控制器决定分组转发的逻辑（可编程），CA所在设备执行逻辑。



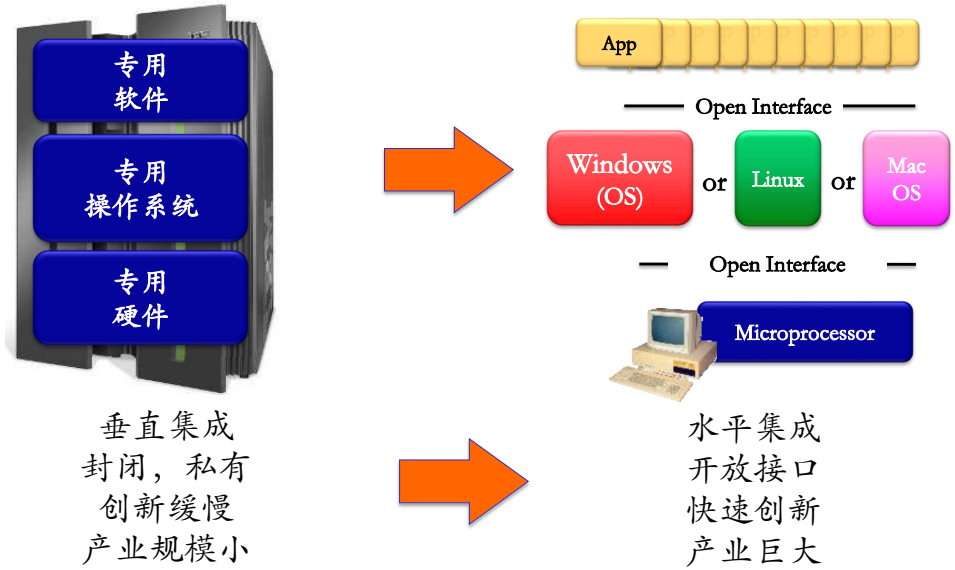
SDN的主要思路

- 网络设备数据平面和控制平面分离
- 数据平面-分组交换机
  - 将路由器、交换机和目前大多数网络设备的功能进一步**抽象**成：按照流表（由控制平面设置的控制逻辑）进行PDU（帧、分组）的动作（包括转发、丢弃、拷贝、泛洪、阻塞）
  - **统一化**设备功能：SDN交换机（分组交换机），执行控制逻辑
- 控制平面-控制器+网络应用
  - 分离、集中
  - 计算和下发控制逻辑：流表

SDN控制平面和数据平面分离的优势

- 水平集成控制平面的开放实现（而非私有实现），创造出好的产业生态，促进发展
  - 分组交换机、控制器和各种控制逻辑网络应用app可由不同厂商生产，专业化，引入竞争形成良好生态
- **集中**式实现控制逻辑，网络管理容易：
  - 集中式控制器了解网络状况，编程简单，传统方式困难
  - 避免路由器的误配置
- 基于流表的**匹配+行动**的工作方式允许“可编程的”分组交换机
  - 实现流量工程等高级特性
  - 在此框架下实现各种新型（未来）的网络设备

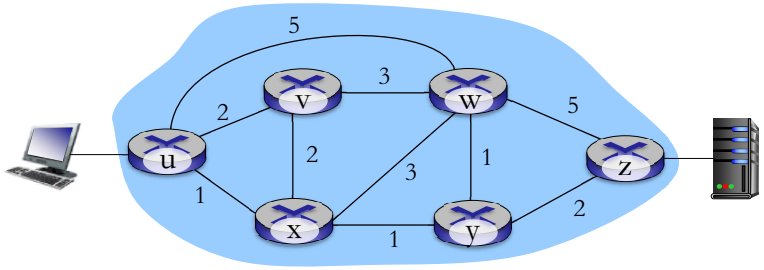
类比：主框架到PC的演变



垂直集成
封闭，私有
创新缓慢
产业规模小

水平集成
开放接口
快速创新
产业巨大

流量工程：传统路由比较困难

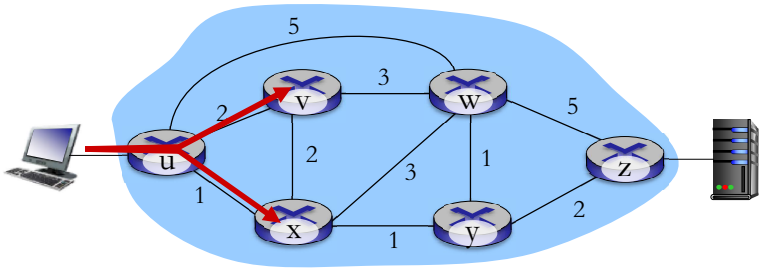


Q: 网管如果需要u到z的流量走uvwz,x到z的流量走xwyz，怎么办?

A: 需要定义链路的代价，流量路由算法以此运算（IP路由面向目标，无法操作）(或者需要新的路由算法)!

链路权重只是控制旋钮，错!

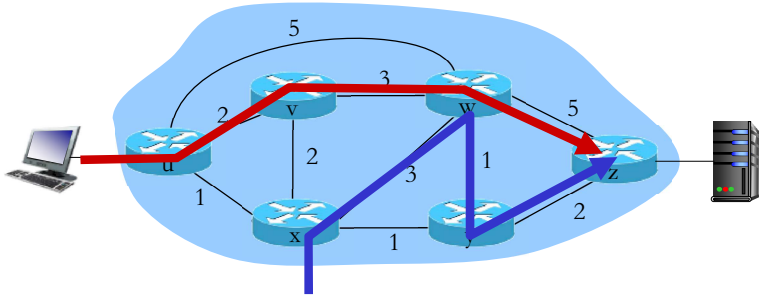
流量工程：困难



Q: 如果网管需要将u到z的流量分成2路：uvwz 和uxyz（负载均衡），怎么办？（IP路由面向目标）

A: 无法完成(在原有体系下只有使用新的路由选择算法，而在全网部署新的路由算法是个大的事情)

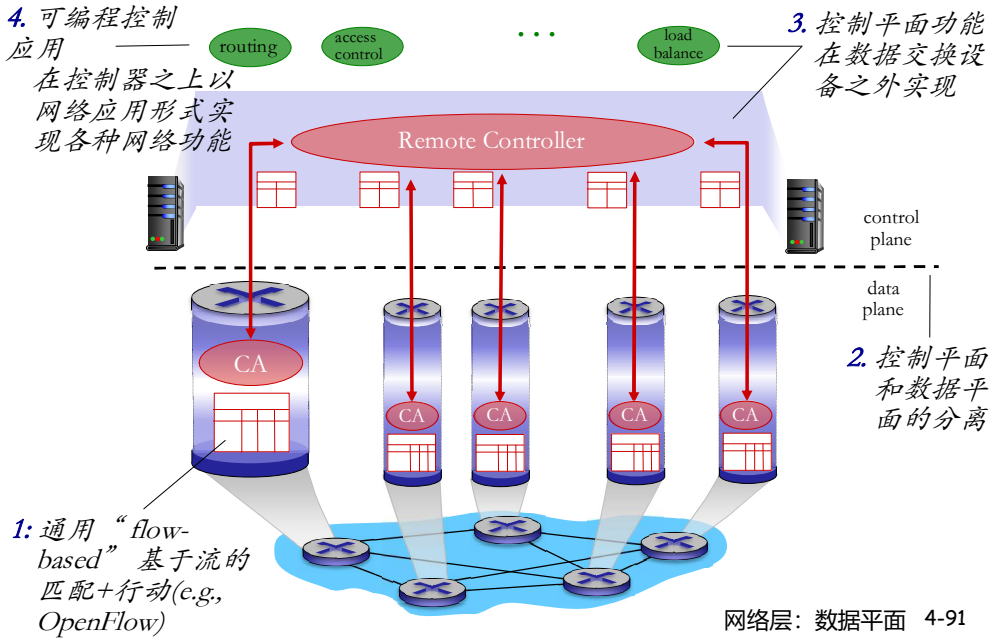
流量工程：困难



Q: 如果需要w对蓝色的和红色的流量采用不同的路由，怎么办？

A: 无法操作 (基于目标的转发，采用LS, DV 路由)

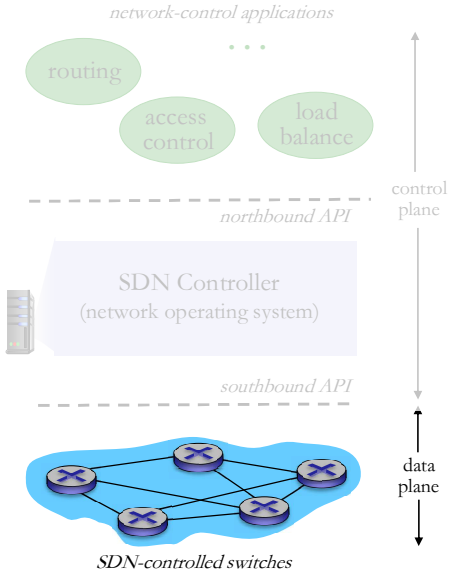
SDN特点



SDN 架构: 数据平面交换机

数据平面交换机

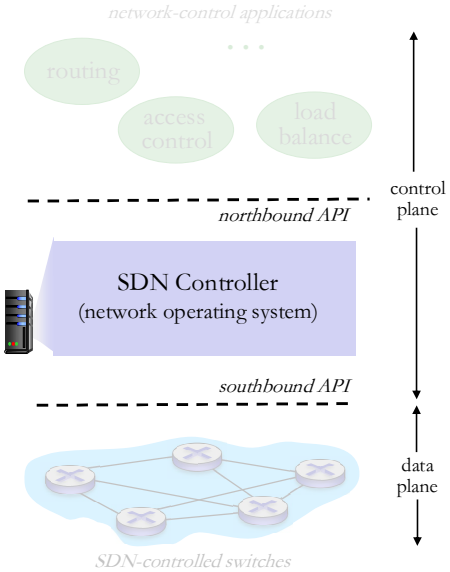
- 快速，简单，商业化交换设备采用硬件实现通用转发功能
- 流表被控制器计算和安装
- 基于南向API（例如OpenFlow），SDN控制器访问基于流的交换机
  - 定义了哪些可以被控制哪些不能
- 也定义了和控制器的协议 (e.g., OpenFlow)



SDN 架构: SDN控制器

SDN 控制器(网络OS):

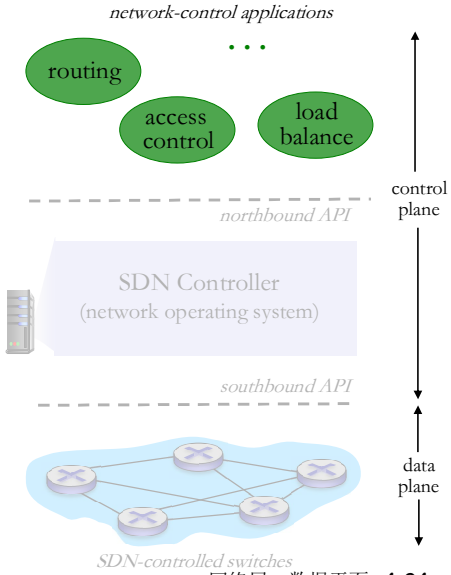
- 维护网络状态信息
- 通过上面的北向API和网络控制应用交互
- 通过下面的南向API和网络交换机交互
- 逻辑上集中，但是在实现上通常由于性能、可扩展性、容错性以及鲁棒性采用分布式方法



SDN 架构: 控制应用

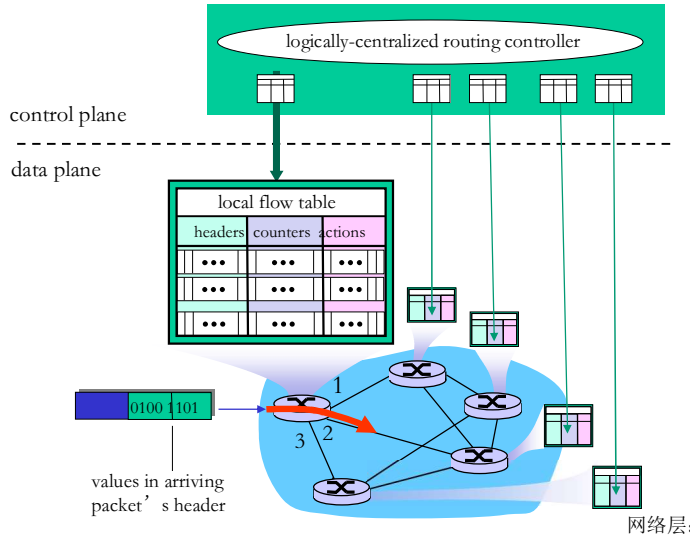
网络控制应用:

- 控制的大脑： 采用下层提供的服务（SDN控制器提供的API)，实现网络功能
  - 路由器 交换机
  - 接入控制 防火墙
  - 负载均衡
  - 其他功能
- *非绑定:* 可以被第三方提供，与控制器厂商以通常上不同，与分组交换机厂商也可以不同



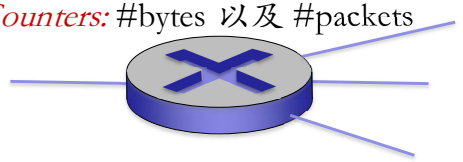
通用转发和SDN

每个路由器包含一个流表（被逻辑上集中的控制器计算和分发）



OpenFlow 数据平面抽象

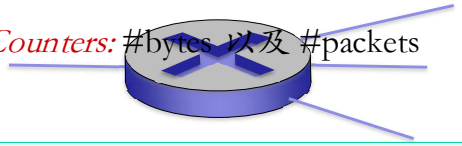
- *流:* 由分组（帧）头部字段所定义
- 通用转发: 简单的分组处理规则
  - *模式:* 将分组头部字段和流表进行匹配
  - *行动:* 对于匹配上的分组，可以是丢弃、转发、修改、将匹配的分组发送给控制器
  - *优先权Priority*: 几个模式匹配了，优先采用哪个，消除歧义
  - *计数器Counters:* #bytes 以及 #packets



路由器中的流表定义了路由器的匹配+行动规则（流表由控制器计算并下发）

OpenFlow 数据平面抽象

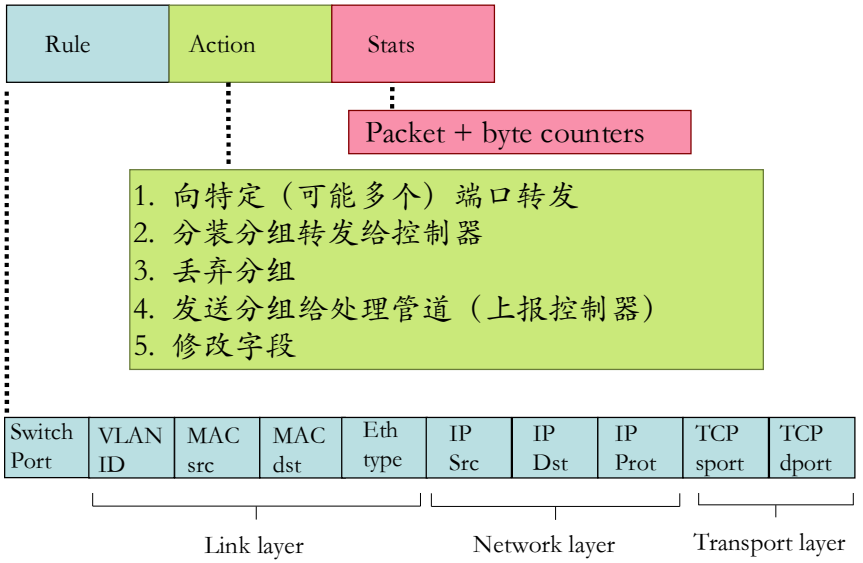
- *流*: 由头部字段所定义
- 通用转发: 简单的分组处理规则
  - *模式Pattern*: 将分组头部字段和流表进行匹配
  - *行动*：对于匹配上的分组，可以是*丢弃、转发、修改、将匹配的分组发送给控制器*
  - *优先权Priority*: 几个模式匹配了，优先采用哪个，消除歧义
  - *计数器Counters*: #bytes 以及 #packets



* : wildcard

1. src=1.2.*.*, dest=3.4.5.* → drop
2. src = *.*.*.*, dest=3.4.*.* → forward(2)
3. src=10.1.2.3, dest=*.*.*.* → send to controller

OpenFlow: 流表的表项结构



例子

基于目标的转发

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	51.6.0.8	*	*	*	port6

IP 数据报目标地址是51.6.0.8 应该被通过端口6转发

防火墙:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Forward
*	*	*	*	*	*	*	*	*	22	drop

不转发（阻塞）所有具有目标TCP端口号是22的分组

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Forward
*	*	*	*	*	128.119.1.1	*	*	*	*	drop

所有由128.119.1.1发送的分组都应该被阻塞

例子

基于层2目标的转发:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	22:A7:23:11:E1:02	*	*	*	*	*	*	*	*	port3

所有层2源MAC地址是 22:A7:23:11:E1:02都应该被向端口3转发

OpenFlow 抽象

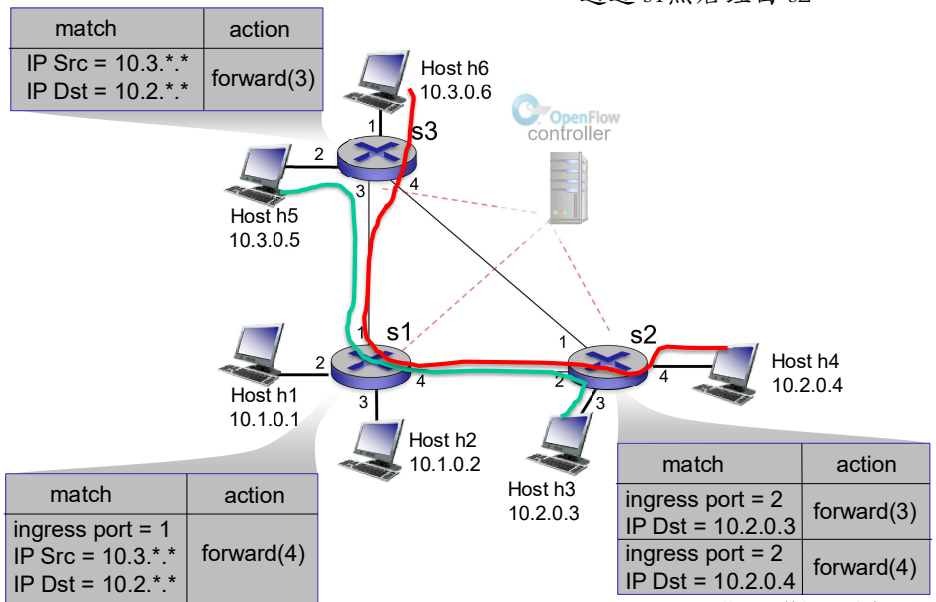
- *match+action:* 统一化各种网络设备提供的功能

- 路由器
  - *match:* 最长前缀匹配
  - *action:* 通过一条链路转发
- 交换机
  - *match:* 目标MAC地址
  - *action:* 转发或者泛
- 防火墙
  - *match:* IP地址和TCP/UDP端口号
  - *action:* 允许或者禁止
- NAT
  - *match:* IP地址和端口号
  - *action:* 重写地址和

目前几乎所有的网络设备都可以在这个 匹配+行动模式 框架进行描述，具体化为各种网络设备包括未来的网络设备

OpenFlow 例子

例子: 来自H5和H6的数据报应该被发向H3或者H4通过 s1然后经由 s2



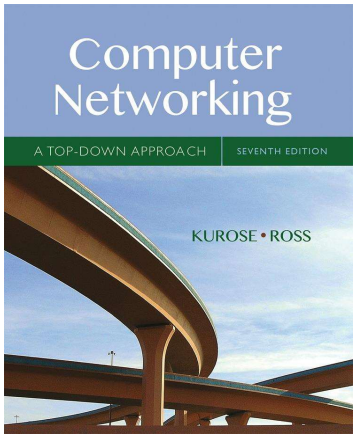
第4章 网络层：数据平面

- 4. 1 导论
  - 数据平面
  - 控制平面
- 4.2 路由器组成
- 4.3 IP: Internet Protocol
  - 数据报格式
  - 分片
  - IPv4地址
  - NAT：网络地址转换
  - IPv6
- 4.4 通用转发和SDN
  - SDN架构
  - 匹配
  - 行动
  - OpenFLow有关“匹配+行动”的运行实例

问题: 转发表(基于目标的转发)和流表（通用转发）是如何计算出来的？

答案: 通过控制平面(下一章)

第 5 章
网络层：控制平面



Computer Networking: A Top Down Approach
7th edition
Jim Kurose, Keith Ross
Pearson/Addison Wesley
April 2016

中国科学技术大学
自动化系 郑烇
改编自Jim kurose,Keith Ross

第5章：网络层控制平面

本章目标： 理解网络层控制平面的工作原理

- 传统路由选择算法
- SDN 控制器
- ICMP:Internet Control Message Protocol
- 网络管理（略）

以及它们在互联网上的实例和实现：

- OSPF, BGP, OpenFlow, ODL 和ONOS控制器, ICMP, SNMP

第5章：提纲

5.1 导论
5.2 路由选择算法
- link state
- distance vector

5.3 因特网中自治系统内部的路由选择
- RIP
- OSPF

5.4 ISP之间的路由选择: BGP
5.5 SDN控制平面
5.6 ICMP: 因特网控制报文协议
5.7 网络管理和SNMP(略)

网络层功能

回顾：2个网络层功能：

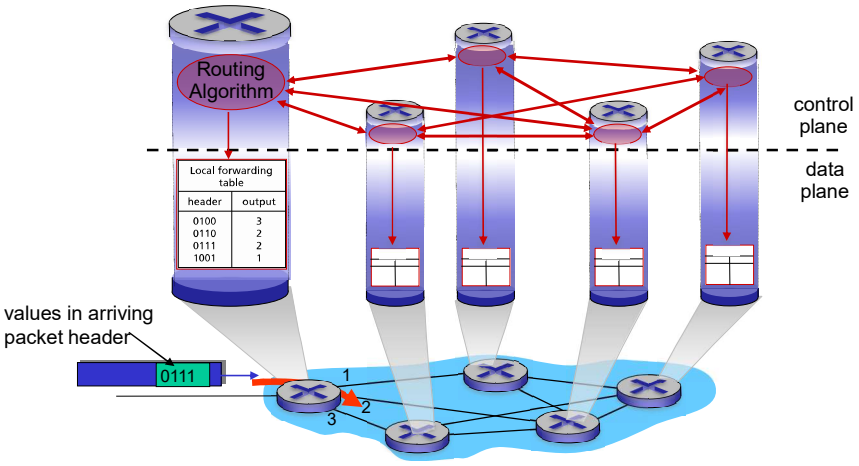
- *转发:* 将分组从路由器的一个输入端口移到合适的输出端口 — 数据平面
- *路由:* 确定分组从源到目标的路径 — 控制平面

2种构建网络控制平面功能的方法：

- 每个路由器控制功能实现（传统）
- 逻辑上集中的控制功能实现(software defined networking)

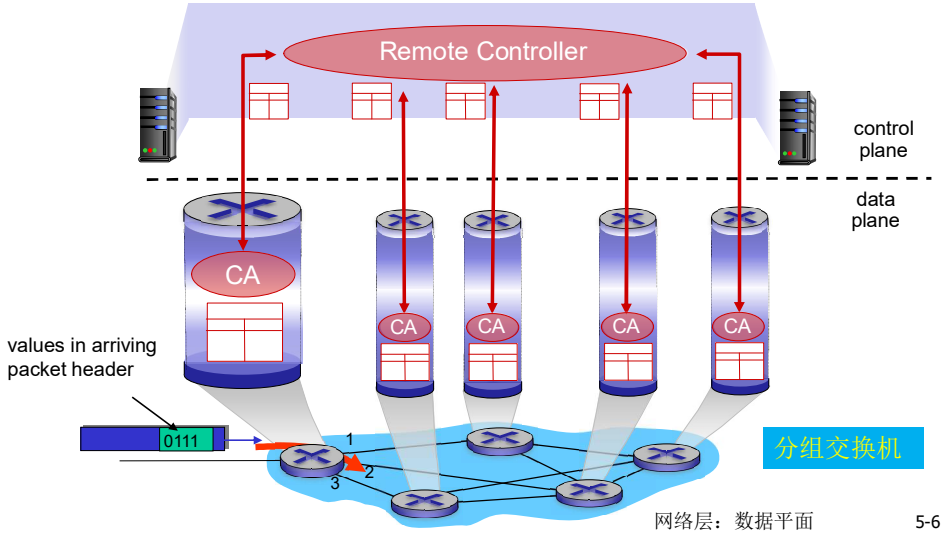
传统方式：每-路由器(Per-router)控制平面

在每一个路由器中的单独路由器算法元件，在控制平面进行交互



SDN方式：逻辑上集中的控制平面

一个不同的（通常是远程的）控制器与本地控制代理（CAs）交互



第5章：提纲

5.1 导论

5.2 路由选择算法

- link state
- distance vector

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择：BGP

5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

5.7 网络管理和SNMP(略)

路由(route)的概念

- 路由:按照某种指标(传输延迟,所经过的站点数目等)找到一条从源节点到目标节点的较好路径
  - 较好路径: 按照某种指标较小的路径
  - 指标:站数, 延迟,费用,队列长度等, 或者是一些单纯指标的加权平均
  - 采用什么样的指标,表示网络使用者希望网络在什么方面表现突出,什么指标网络使用者比较重视

- 以网络为单位进行路由（路由信息通告+路由计算）
  - 网络为单位进行路由，路由信息传输、计算和匹配的代价低
  - 前提条件是：一个网络所有节点地址前缀相同，且物理上聚集
  - 路由就是：计算网络 到其他网络如何走的问题

路由(route)的概念

- 网络到网络的路由= 路由器-路由器之间路由
  - 网络对应的路由器到其他网络对应的路由器的路由
  - 在一个网络中：路由器-主机之间的通信，链路层解决
  - 到了这个路由器就是到了这个网络

- 路由选择算法(routing algorithm):网络层软件的一部分,完成路由功能

网络的图抽象

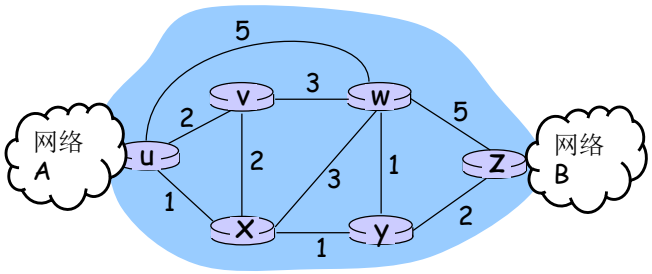
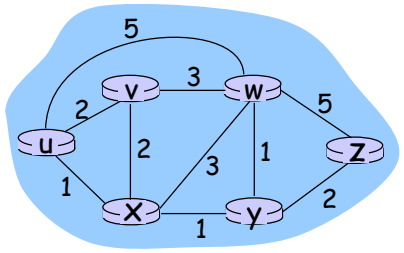


图: G = (N,E)

N = 路由器集合 = { u, v, w, x, y, z }

E = 链路集合 ={ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) }
边有代价

图抽象：边和路径的代价



• c(x,x') = 链路的代价 (x,x')

- e.g., c(w,z) = 5

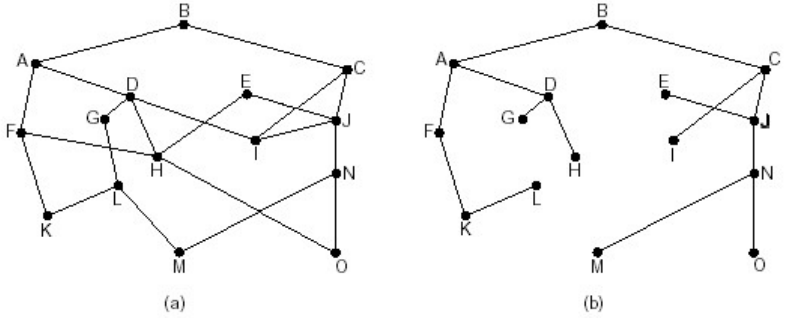
•代价可能总为 1
•或是 链路带宽的倒数
•或是 拥塞情况的倒数

Cost of path $(x_1, x_2, x_3, \ldots, x_p) = c(x_1,x_2) + c(x_2,x_3) + \ldots + c(x_{p-1},x_p)$

路由的输入：拓扑、边的代价、源节点
输出的输出：源节点的汇集树

最优化原则(optimality principle)

- 汇集树(sink tree)
  - 此节点到所有其它节点的最优路径形成的树
  - 路由选择算法就是为所有路由器找到并使用汇集树



路由的原则

- 路由选择算法的原则
  - 正确性(correctness):算法必须是正确的和完整的,使分组一站一站接力，正确发向目标站；完整：目标所有的站地址，在路由表中都能找到相应的表项；没有处理不了的目标站地址；
  - 简单性(simplicity):算法在计算机上应简单：最优但复杂的算法，时间上延迟很大，不实用，不应为了获取路由信息增加很多的通信量；
  - 健壮性(robustness):算法应能适应通信量和网络拓扑的变化：通信量变化，网络拓扑的变化算法能很快适应；不向很拥挤的链路发数据，不向断了的链路发送数据；

路由的原则（续）

- 路由选择算法的原则
  - 稳定性(stability)：产生的路由不应该摇摆
  - 公平性(fairness)：对每一个站点都公平
  - 最优性(optimality)：某一个指标的最优，时间上，费用上，等指标，或综合指标；实际上，获取最优的结果代价较高，可以是次优的

路由算法分类

全局或者局部路由信息?

全局:
- 所有的路由器拥有完整的拓扑和边的代价的信息
- “link state” 算法

分布式:
- 路由器只知道与它有物理连接关系的邻居路由器，和到相应邻居路由器的代价值
- 叠代地与邻居交换路由信息、计算路由信息
- “distance vector” 算法

静态或者动态的?

静态:
- 路由随时间变化缓慢

动态:
- 路由变化很快
  - 周期性更新
  - 根据链路代价的变化而变化

非自适应算法(non-adaptive algorithm)：不能适应网络拓扑和通信量的变化,路由表是事先计算好的

自适应路由选择(adaptive algorithm)：能适应网络拓扑和通信量的变化

第5章：提纲

5.1 导论

5.2 路由选择算法
- link state
- distance vector

5.3 因特网中自治系统内部的路由选择
- RIP
- OSPF

5.4 ISP之间的路由选择：BGP

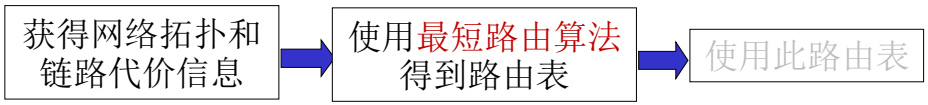
5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

5.7 网络管理和SNMP(略)

LS路由的工作过程

- 配置LS路由选择算法的路由工作过程
  - 各点通过各种渠道获得整个网络拓扑，网络中所有链路代价等信息（这部分和算法没关系，属于协议和实现）
  - 使用LS路由算法,计算本站点到其它站点的最优路径(汇集树),得到路由表
  - 按照此路由表转发分组(datagram方式)
    - 严格意义上说不是路由的一个步骤
    - 分发到输入端口的网络层



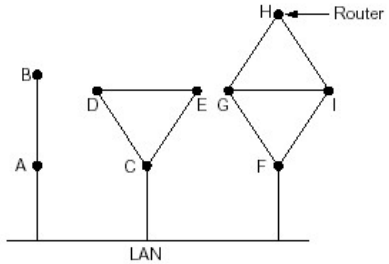
链路状态路由选择(link state routing)

- LS路由的基本工作过程
  1. 发现相邻节点,获知对方网络地址
  2. 测量到相邻节点的代价(延迟,开销)
  3. 组装一个LS分组,描述它到相邻节点的代价情况
  4. 将分组通过扩散的方法发到所有其它路由器

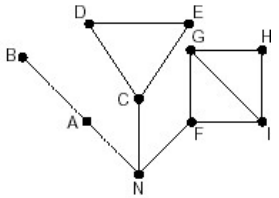
  以上4步让每个路由器获得拓扑和边代价
  5. 通过Dijkstra算法找出最短路径（这才是路由算法）
     1. 每个节点独立算出来到其他节点（路由器=网络）的最短路径
     2. 迭代算法：第k步能够知道本节点到k个其他节点的最短路径

链路状态路由选择(link state routing)

1. 发现相邻节点,获知对方网络地址
   - 一个路由器上电之后,向所有线路发送HELLO分组
   - 其它路由器收到HELLO分组,回送应答,在应答分组中,告知自己的名字(全局唯一)
   - 在LAN中,通过广播HELLO分组,获得其它路由器的信息,可以认为引入一个人工节点



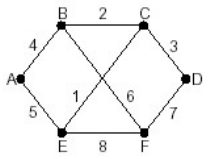
(a)



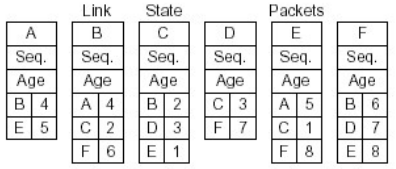
(b)

链路状态路由选择(link state routing)

2. 测量到相邻节点的代价(延迟,开销)
   - 实测法,发送一个分组要求对方立即响应
   - 回送一个ECHO分组
   - 通过测量时间可以估算出延迟情况
3. 组装一个分组,描述相邻节点的情况
   - 发送者名称
   - 序号,年龄
   - 列表: 给出它相邻节点,和它到相邻节点的延迟



(a)



(b)

链路状态路由选择(link state routing)

4. 将分组通过扩散的方法发到所有其它路由器
   - 顺序号:用于控制无穷的扩散,每个路由器都记录(源路由器,顺序号),发现重复的或老的就不扩散
     - 具体问题1: 循环使用问题
     - 具体问题2: 路由器崩溃之后序号从0开始
     - 具体问题3:序号出现错误
   - 解决问题的办法:年龄字段(age)
     - 生成一个分组时,年龄字段不为0
     - 每个一个时间段,AGE字段减1
     - AGE字段为0的分组将被抛弃

链路状态路由选择(link state routing)

4. 将分组通过扩散的方法发到所有其它路由器（续）
   - 关于扩散分组的数据结构
   - ✓ Source :从哪个节点收到LS分组
   - ✓ Seq,Age:序号,年龄
   - ✓ Send flags:发送标记,必须向指定的哪些相邻站点转发LS分组
   - ✓ ACK flags:本站点必须向哪些相邻站点发送应答
   - ✓ DATA:来自source站点的LS分组
   - ✓ 节点B的数据结构

			Send flags			ACK flags			
Source	Seq.	Age	A	C	F	A	C	F	Data
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

链路状态路由选择(link state routing)

5. 通过Dijkstra算法找出最短路径
   - 路由器获得各站点LS分组和整个网络的拓扑
   - 通过Dijkstra算法计算出到其它各路由器的最短路径(汇集树)
   - 将计算结果安装到路由表中

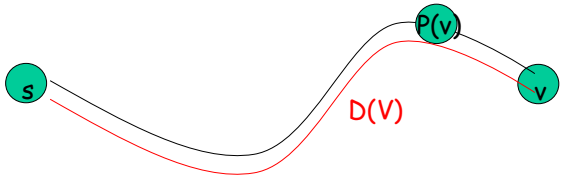
这才是路由算法

- LS的应用情况
  - OSPF协议是一种LS协议,被用于Internet上
  - IS-IS(intermediate system- intermediate system): 被用于Internet主干中, Netware

链路状态路由选择(link state routing)

- 符号标记:
  - c(i,j): 从节点i 到j链路代价(初始状态下非相邻节点之间的链路代价为∞)
  - D(v): 从源节点到节点V的当前路径代价(节点的代价)
  - p(v): 从源到节点V的路径前序节点
  - N': 当前已经知道最优路径的的节点集合(永久节点的集合)

要计算S到其他所有节点的路由

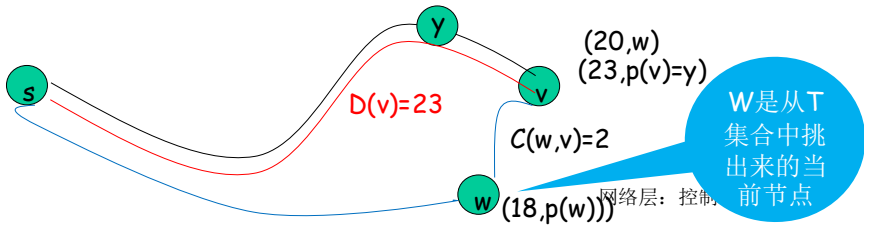


链路状态路由选择(link state routing)

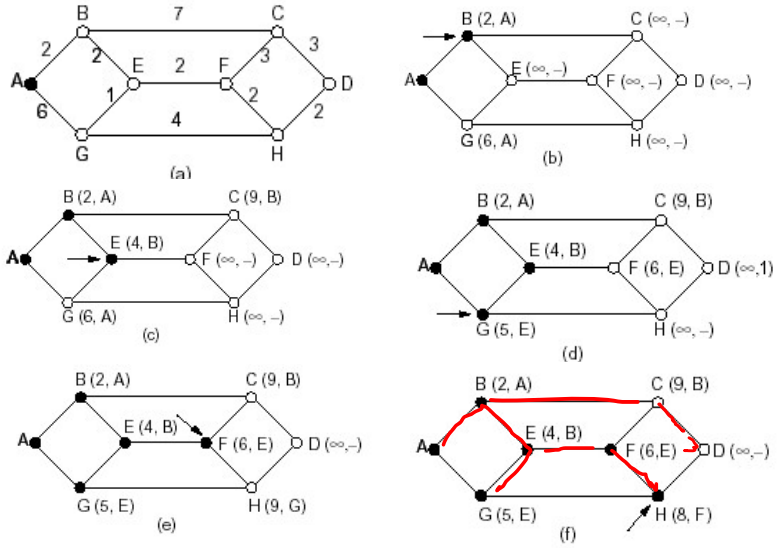
- LS路由选择算法的工作原理
  - **节点标记：**每一个节点使用(D(v),p(v)) 如：(3,B)标记
    - D(v)从源节点由已知最优路径到达本节点的**距离**
    - P(v)**前序节点**来标注
  - 2类节点
    - 临时节点(tentative node) :还没有找到从源节点到此节点的最优路径的节点
    - 永久节点(permanent node) N':已经找到了从源节点到此节点的最优路径的节点

链路状态路由选择(link state routing)

- 初始化
  - 除了源节点外,所有节点都为临时节点
  - 节点代价除了与源节点代价相邻的节点外,都为∞
- 从所有临时节点中找到一个节点代价最小的临时节点,将之变成永久节点(当前节点)W
- 对此节点的所有在临时节点集合中的邻节点(V)
  - 如 D(v)>D(w) + c(w,v), 则重新标注此点, (D(W)+C(W,V), W)
  - 否则，不重新标注
- 开始一个新的循环



例子:

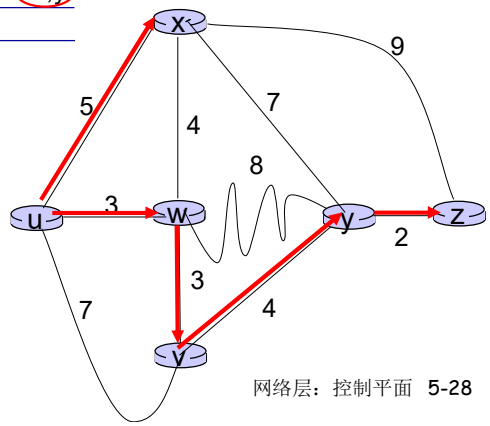


Dijkstra算法的例子

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

$$e.g.,\ D(v) = \min(D(v), D(w) + c(w,v))$$
$$= \min\{7, 3+3\} = 6$$

destination	link
v	(u,v)
x	(u,x)
y	(u,w)
w	(u,w)
z	(u,w)



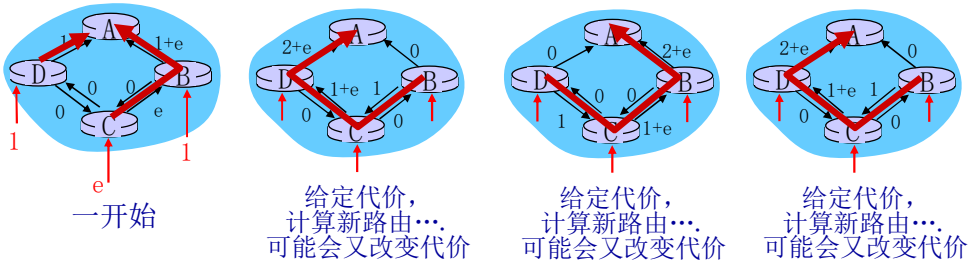
Dijkstra算法的讨论

算法复杂度： n节点

- 每一次迭代：需要检查所有不在永久集合N中节点
- n(n+1)/2 次比较：$O(n^2)$
- 有很有效的实现：O(nlogn)

可能的震荡：

- e.g.,链路代价=链路承载的流量：



第5章：提纲

5.1 导论
5.2 路由选择算法
- link state
- distance vector

5.3 因特网中自治系统内部的路由选择
- RIP
- OSPF

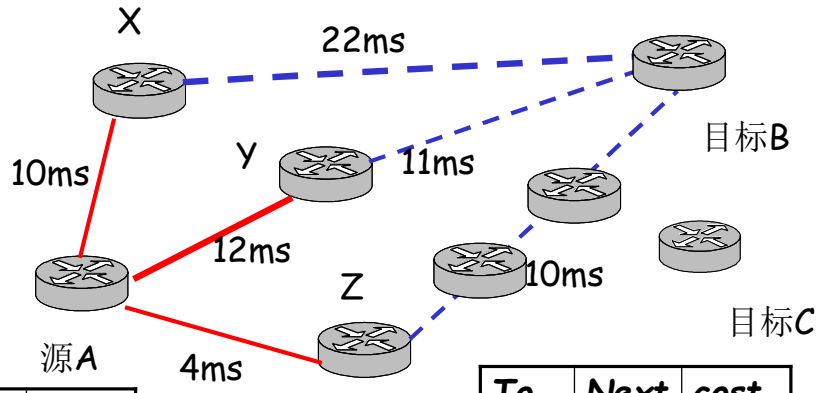
5.4 ISP之间的路由选择：BGP
5.5 SDN控制平面
5.6 ICMP：因特网控制报文协议
5.7 网络管理和SNMP(略)

距离矢量路由选择(distance vector routing)

- 动态路由算法之一
- DV算法历史及应用情况
  - 1957 Bellman, 1962 Ford Fulkerson
  - 用于ARPANET, Internet(RIP)
    DECnet , Novell, AppleTalk
- 距离矢量路由选择的基本思想
  - 各路由器维护一张路由表,结构如图(其它代价)
  - 各路由器与相邻路由器交换路由表(待续)
  - 根据获得的路由信息,更新路由表(待续)

To 目标	Next 下个	cost
A	Z	14
.....		.....

距离矢量路由选择(distance vector routing)



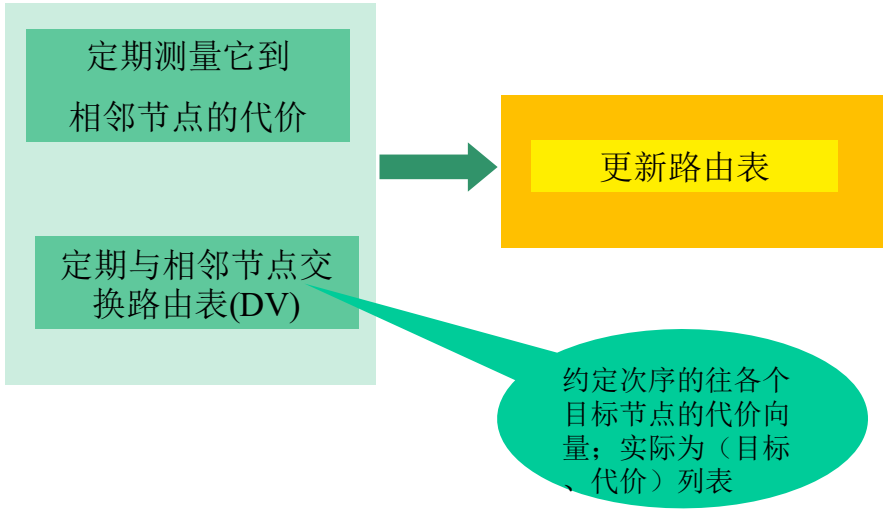
To	Next	cost
B	Z	14
.....		.....

To	Next	cost
B	Q	10
.....		.....

距离矢量路由选择(distance vector routing)

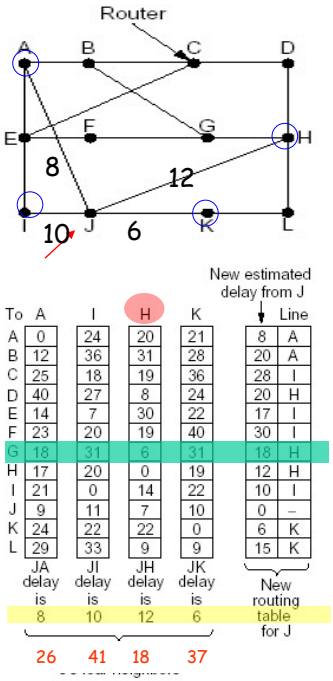
- 代价及相邻节点间代价的获得
  - ✓ 跳数(hops), 延迟(delay),队列长度
  - ✓ 相邻节点间代价的获得：通过实测
- 路由信息的更新
  - ✓ 根据实测 得到本节点A到相邻站点的代价（如:延迟）
  - ✓ 根据各相邻站点声称它们到目标站点B的代价
  - ✓ 计算出本站点A经过各相邻站点到目标站点B的代价
  - ✓ **找到一个最小的代价，和相应的下一个节点Z，到达节点B经过此节点Z，并且代价为A-Z-B的代价**
  - ✓ 其它所有的目标节点一个计算法

距离矢量路由选择(distance vector routing)



距离矢量路由：例子1

- 以当前节点J为例,相邻节点A,I,H,K
- J测得到A,I,H,K的延迟为8ms,10ms,12ms,6ms
- 通过交换DV, 从A,I,H,K获得到它们到G的延迟为18ms,31ms,6ms,31ms
- 因此从J经过A,I,H,K到G的延迟为26ms,41ms,**18ms**, 37ms
- 将到G的路由表项更新为18ms, 下一跳为：H
- 其它目标一样，除了本节点J



距离矢量算法

Bellman-Ford 方程(动态规划)

设

$d_x(y)$:= 从x到y的最小路径代价

那么

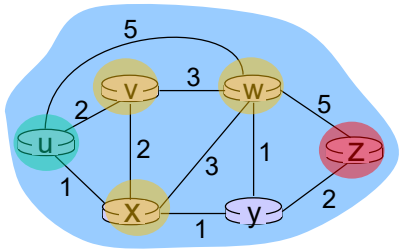
$$d_x(y) = \min_v \{c(x,v) + d_v(y)\}$$

从邻居v到目标y的代价

x到邻居v的代价

取所有x的邻居取最小的v

Bellman-Ford例子



明显的，$d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

由B-F 方程得到：

$$d_u(z) = \min \{ c(u,v) + d_v(z), \; c(u,x) + d_x(z), \; c(u,w) + d_w(z) \}$$
$$= \min \{2 + 5, \; 1 + 3, \; 5 + 3\} = 4$$

那个能够达到目标z最小代价的节点x，就在到目标节点的下一条路径上，在转发表中使用

距离矢量算法

- $D_x(y)$ = 节点x到y代价最小值的估计
  - x 节点维护距离矢量$D_x = [D_x(y): y \in N]$
- 节点x:
  - 知道到所有邻居v的代价：c(x,v)
  - 收到并维护一个它邻居的距离矢量集
  - 对于每个邻居，x 维护 $D_v = [D_v(y): y \in N]$

距离矢量算法

核心思路:

- 每个节点都将自己的距离矢量估计值传送给邻居，定时或者DV有变化时，让对方去算
- 当x从邻居收到DV时，自己运算，更新它自己的距离矢量
  - 采用B-F equation:

$D_x(y) \leftarrow \min_v\{c(x,v) + D_v(y)\}$ 对于每个节点 $y \in N$

X往y的代价　x到邻居v代价　v声称到y的代价

- $D_x(y)$估计值最终收敛于实际的最小代价值$d_x(y)$
  - 分布式、迭代算法

距离矢量算法

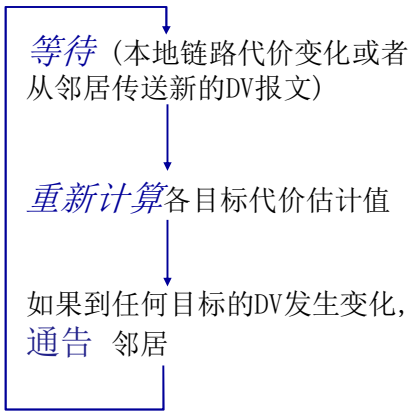
异步式, 迭代: 每次本地迭代被以下事件触发:

- 本地链路代价变化了
- 从邻居来了DV的更新消息

分布式:

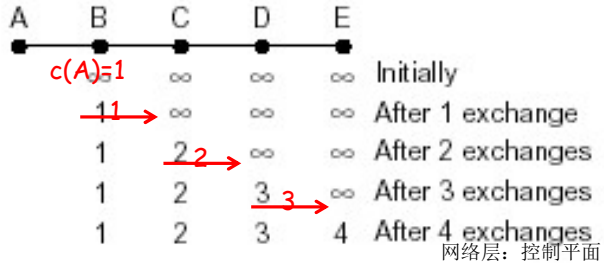
- 每个节点只是在自己的DV改变之后向邻居通告
  - 然后邻居们在有必要的时候通知他们的邻居

每个节点:



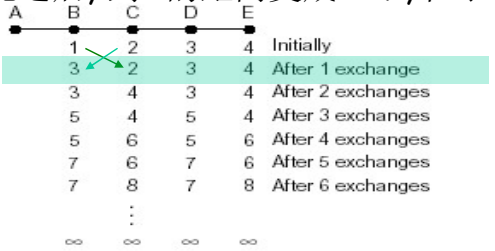
距离矢量路由选择(distance vector routing)

- DV的无穷计算问题
  - DV的特点
    - 好消息传的快 坏消息传的慢
  - 好消息的传播以每一个交换周期前进一个路由器的速度进行
    - 好消息:某个路由器接入或有更短的路径
    - 举例



距离矢量路由选择(distance vector routing)

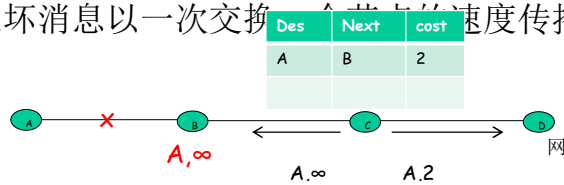
- DV的无穷计算问题
  - 坏消息的传播速度非常慢(无穷计算问题)
  - 例子:
    - 第一次交换之后，B从C处获得信息,C可以到达A(C-A,要经过B本身),但是路径是2,因此B变成3,从C处走
    - 第二次交换,C从B处获得消息，B可以到达A,路径为3,因此,C到A从B走,代价为3
    - 无限此之后，到A的距离变成INF,不可达

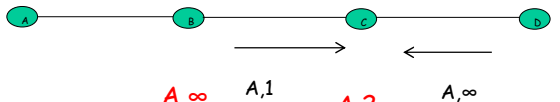


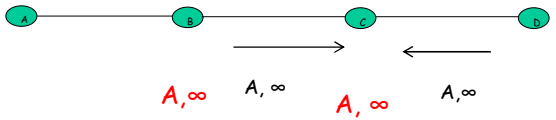
=Min(left,right)+1

距离矢量路由选择(distance vector routing)

- 水平分裂(split horizon)算法
  - 一种对无穷计算问题的解决办法
    - C知道要经过B才能到达A，所以C向B报告它到A的距离为INF；C 告诉D它到A的真实距离
    - D告诉E,它到A的距离,但D告诉C它通向A的距离为INF
    - 第一次交换: B通过测试发现到A的路径为INF,而C也告诉B到A的距离为INF,因此,B到A的距离为INF
    - 第二次交换: C从B和D那里获知,到A的距离为INF,因此将它到A的距离为INF
    - .....坏消息以一次交换一个节点的速度传播

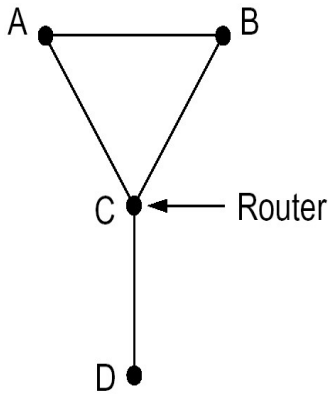






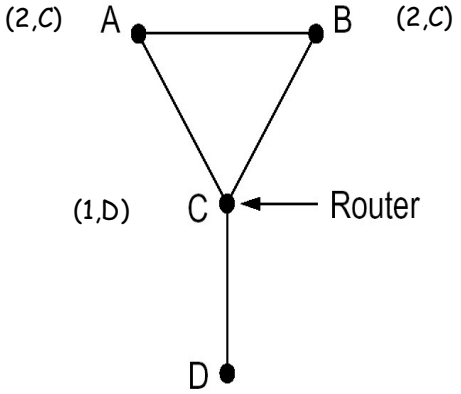
距离矢量路由选择

- 水平分裂(split horizon)算法
  - 水平分裂的问题:在某些拓扑形式下会失败（存在环路）
  - 例子:
    - ✓ A,B到D的距离为2, C到D的距离为1
    - ✓ 如果C-D路径失败
    - ✓ C获知到D为INF,从A,B获知到D的距离为INF,因此C认为D不可达
    - ✓ A从C获知D的距离为INF,但从B处获知它到D的距离为2,因此A到B的距离为3,从B走
    - ✓ B也有类似的问题
    - ✓ 经过无限次之后,A和B都知道到D的距离为INF



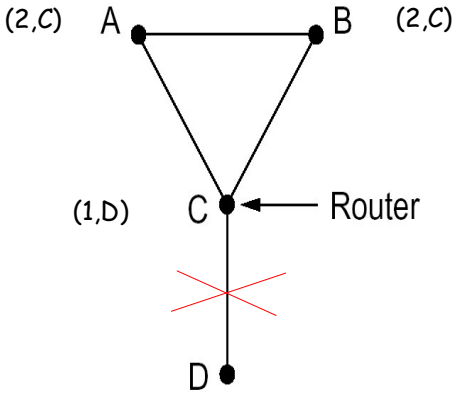
距离矢量路由选择

- 水平分裂(split horizon)算法



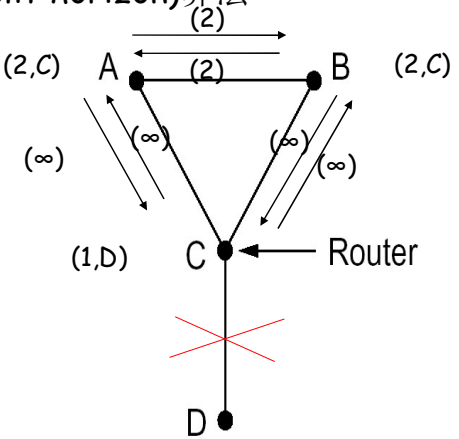
距离矢量路由选择

- 水平分裂(split horizon)算法



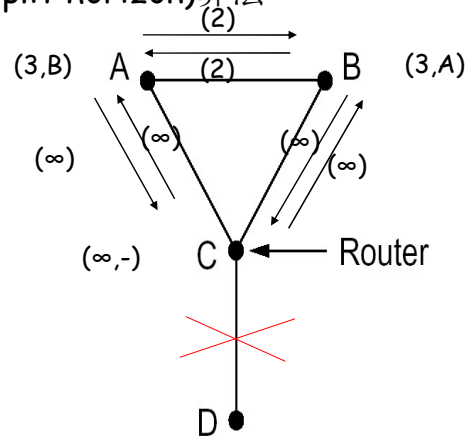
距离矢量路由选择

- 水平分裂(split horizon)算法



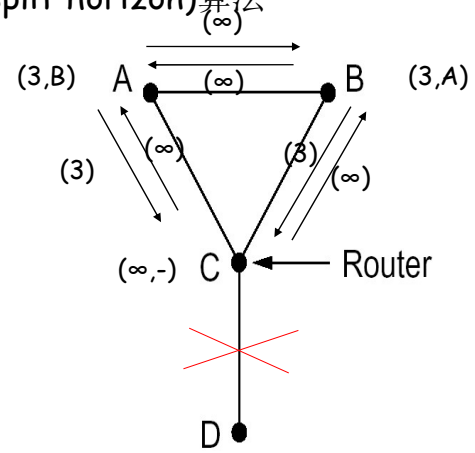
距离矢量路由选择

- 水平分裂(split horizon)算法



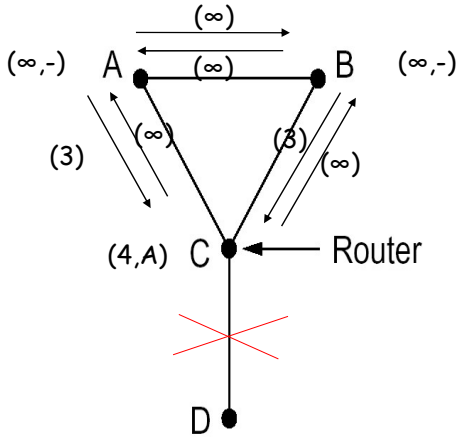
距离矢量路由选择

- 水平分裂(split horizon)算法



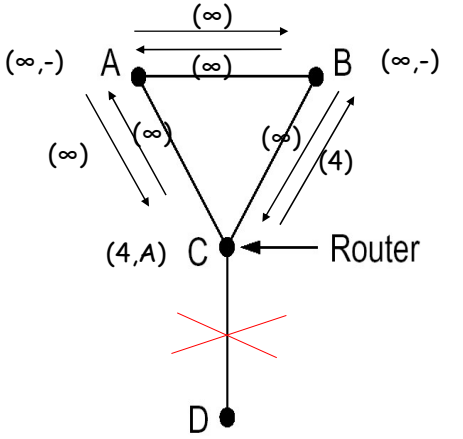
距离矢量路由选择

- 水平分裂(split horizon)算法



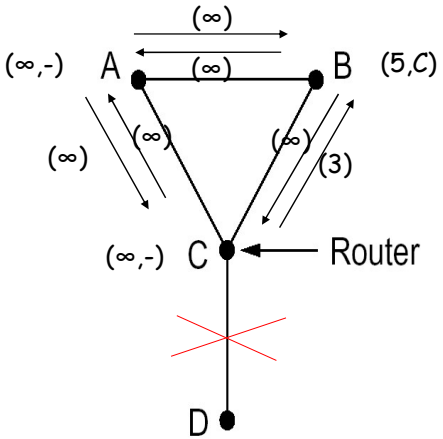
距离矢量路由选择

- 水平分裂(split horizon)算法



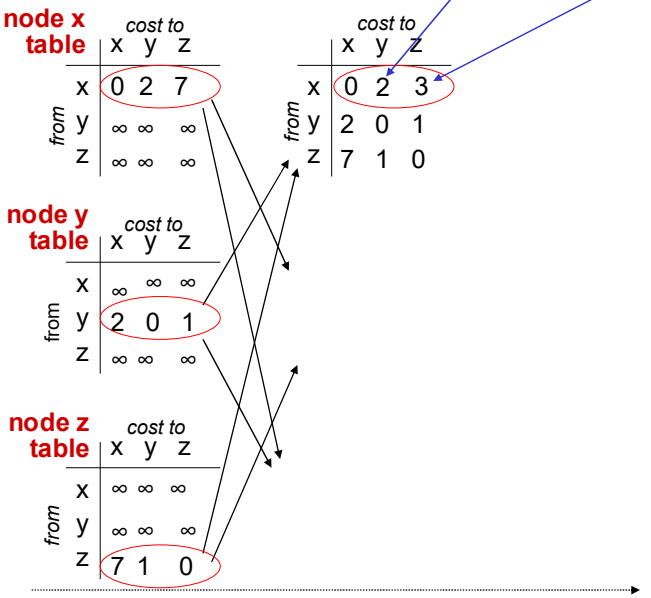
距离矢量路由选择

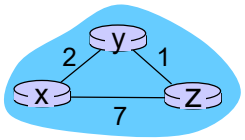
- 水平分裂(split horizon)算法



$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$
$= \min\{2+0 , 7+1\} = 2$

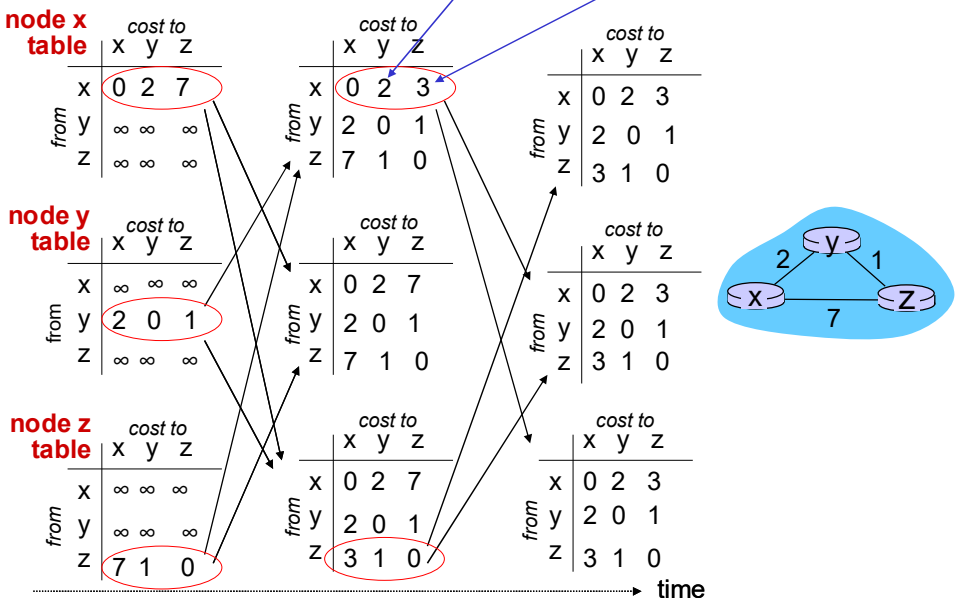
$D_x(z) = \min\{c(x,y) +$
$D_y(z), c(x,z) + D_z(z)\}$
$= \min\{2+1 , 7+0\} = 3$





$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$
$= \min\{2+0 , 7+1\} = 2$

$D_x(z) = \min\{c(x,y) +$
$D_y(z), c(x,z) + D_z(z)\}$
$= \min\{2+1 , 7+0\} = 3$



LS 和 DV 算法的比较

消息复杂度（DV胜出）

- LS: 有 n 节点, E 条链路,发送报文O(nE)个
  - 局部的路由信息；全局传播
- DV: 只和邻居交换信息
  - 全局的路由信息，局部传播

收敛时间（LS胜出）

- LS: $O(n^2)$ 算法
  - 有可能震荡
- DV: 收敛较慢
  - 可能存在路由环路
  - count-to-infinity 问题

健壮性: 路由器故障会发生什么（LS胜出）

LS:

- 节点会通告不正确的链路代价
- 每个节点只计算自己的路由表
- 错误信息影响较小，局部，路由较健壮

DV:

- DV 节点可能通告对全网所有节点的不正确路径代价
  - 距离矢量
- 每一个节点的路由表可能被其它节点使用
  - 错误可以扩散到全网

2种路由选择算法都有其优缺点，而且在互联网上都有应用

第5章：提纲

5.1 导论

5.2 路由选择算法

- link state
- distance vector

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择：BGP

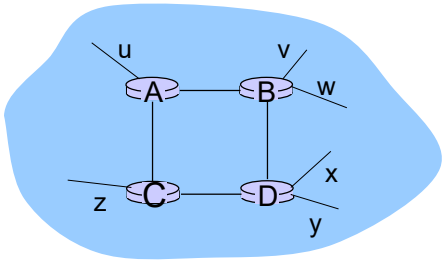
5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

5.7 网络管理和SNMP(略)

RIP (Routing Information Protocol)

- 在 1982年发布的BSD-UNIX 中实现
- Distance vector 算法
  - 距离矢量:每条链路cost=1，# of hops (max = 15 hops) 跳数
  - DV每隔30秒和邻居交换DV，通告
  - 每个通告包括：最多25个目标子网



从路由器A到目标子网：

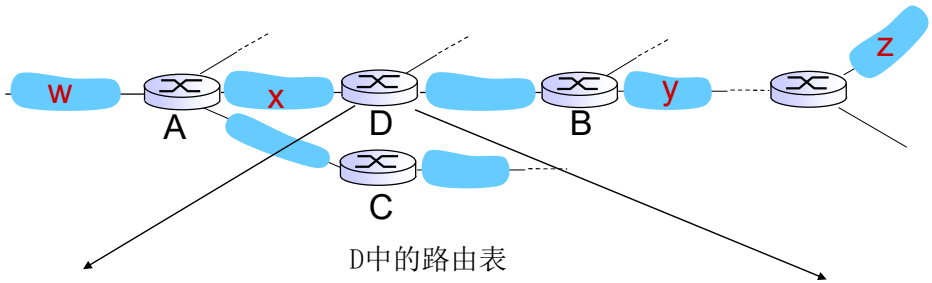
subnet	hops
u	1
v	2
w	2
x	3
y	3
z	2

RIP 通告（advertisements）

- DV: 在邻居之间每30秒交换通告报文
  - 定期，而且在改变路由的时候发送通告报文
  - 在对方的请求下可以发送通告报文
- 每一个通告: 至多AS内部的25个目标网络的DV
  - 目标网络+跳数

一次公告最多25个子网
最大跳数为16

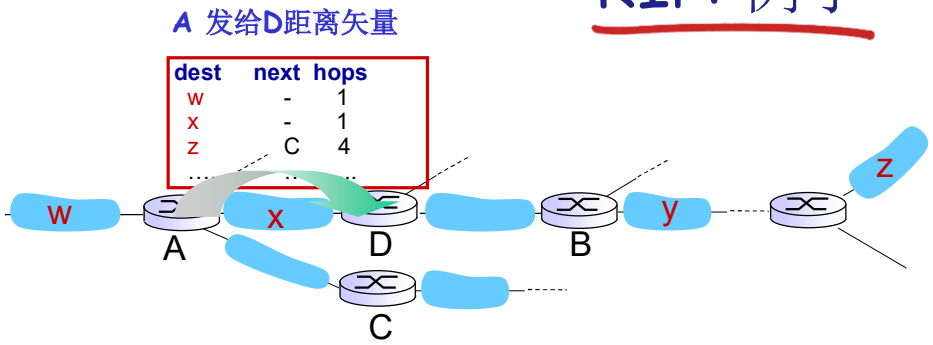
RIP: 例子



D中的路由表

destination subnet	next router	# hops to dest
w	A	2
y	B	2
z	B	7
x	--	1
....	....	....

RIP: 例子



routing table in router D

destination subnet	next router	# hops to dest
w	A	2
y	B	2
z	B → A	7 → 5
x	--	1
....	....	....

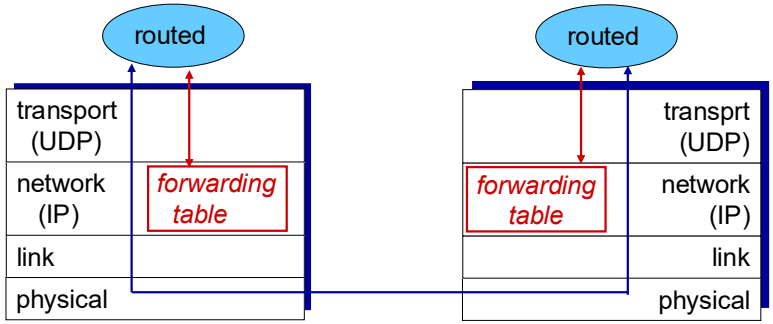
RIP: 链路失效和恢复

如果180秒没有收到通告信息-->邻居或者链路失效

- 发现经过这个邻居的路由已失效
- 新的通告报文会传递给邻居
- 邻居因此发出新的通告 (如果路由变化的话)
- 链路失效快速(?)地在整网中传输
- 使用毒性逆转（poison reverse）阻止ping-pong回路 (不可达的距离：跳数无限 = 16 段)

RIP 进程处理

- RIP 以应用进程的方式实现：route-d (daemon)
- 通告报文通过UDP报文传送，周期性重复
- 网络层的协议使用了传输层的服务，以应用层实体的方式实现



第5章：提纲

5.1 导论

5.2 路由选择算法

- link state
- distance vector

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择: BGP

5.5 SDN控制平面

5.6 ICMP: 因特网控制报文协议

5.7 网络管理和SNMP(略)

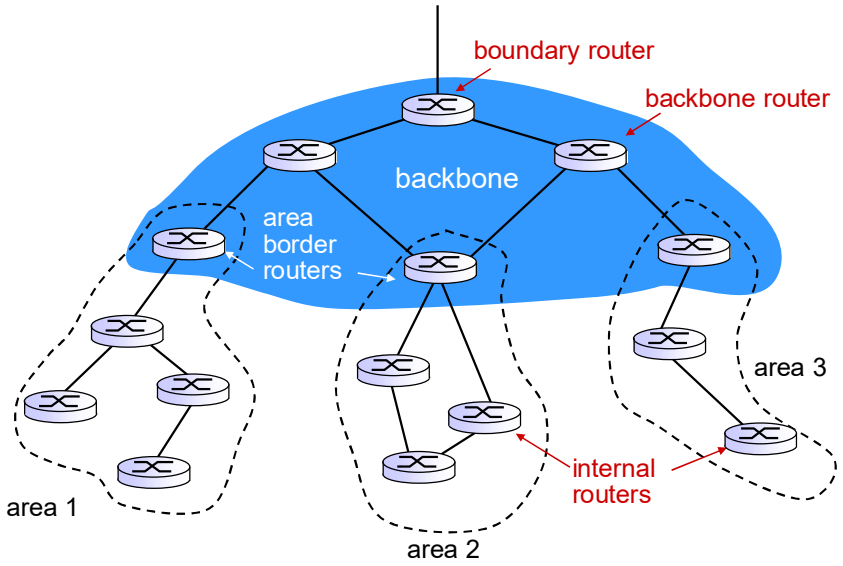
OSPF (Open Shortest Path First)

- “open”：标准可公开获得
- 使用LS算法
  - LS 分组在网络中（一个AS内部）分发
  - 全局网络拓扑、代价在每一个节点中都保持
  - 路由计算采用Dijkstra算法
- OSPF通告信息中携带：每一个邻居路由器一个表项
- 通告信息会传遍AS全部（通过泛洪）
  - 在IP数据报上直接传送OSPF报文 (而不是通过UDP和TCP)
- IS-IS路由协议：几乎和OSPF一样

OSPF “高级” 特性(在RIP中的没有的)

- 安全: 所有的OSPF报文都是经过认证的 (防止恶意的攻击)
- 允许有多个代价相同的路径存在 (在RIP协议中只有一个)
- 对于每一个链路，对于不同的TOS有多重代价矩阵
  - 例如：卫星链路代价对于尽力而为的服务代价设置比较低，对实时服务代价设置的比较高
  - 支持按照不同的代价计算最优路径，如：按照时间和延迟分别计算最优路径
- 对单播和多播的集成支持:
  - Multicast OSPF (MOSPF) 使用相同的拓扑数据库，就像在OSPF中一样
- 在大型网络中支持层次性OSPF

层次化的OSPF路由



层次性的OSPF路由

- 2个级别的层次性: 本地, 骨干
  - 链路状态通告仅仅在本地区域Area范围内进行
  - 每一个节点拥有本地区域的拓扑信息；
    - 关于其他区域，知道去它的方向，通过区域边界路由器（最短路径）
- **区域边界路由器:** “汇总（聚集）”到自己区域内网络的距离, 向其它区域边界路由器通告.
- **骨干路由器:** 仅仅在骨干区域内，运行OSPF路由
- **边界路由器:** 连接其它的AS's.

第5章：提纲

5.1 导论

5.2 路由选择算法

- link state
- distance vector

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择：BGP

5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

5.7 网络管理和SNMP(略)

层次路由

- 一个平面的路由
  - 一个网络中的所有路由器的地位一样
  - 通过LS, DV，或者其他路由算法，所有路由器都要知道其他所有路由器（子网）如何走
  - 所有路由器在一个平面
- 平面路由的问题
  - 规模巨大的网络中，路由信息的存储、传输和计算代价巨大
    - DV: 距离矢量很大，且不能够收敛
    - LS：几百万个节点的LS分组的泛洪传输，存储以及最短路径算法的计算
  - 管理问题：
    - 不同的网络所有者希望按照自己的方式管理网络
    - 希望对外隐藏自己网络的细节
    - 当然，还希望和其它网络互联

层次路由

- 层次路由：将互联网分成一个个AS(路由器区域)
  - 某个区域内的路由器集合，自治系统“autonomous systems” (AS)
  - 一个AS用AS Number（ASN)唯一标示
  - 一个ISP可能包括1个或者多个AS
- 路由变成了：2个层次路由
  - **AS内部路由**：在同一个AS内路由器运行相同的路由协议
    - “intra-AS” routing protocol：内部网关协议
    - 不同的AS可能运行着不同的内部网关协议
    - 能够解决规模和管理问题
    - 如：RIP,OSPF,IGRP
    - 网关路由器：AS边缘路由器，可以连接到其他AS
  - AS间运行**AS间路由协议**
    - “inter-AS” routing protocol：外部网关协议
    - 解决AS之间的路由问题，完成AS之间的互联互通

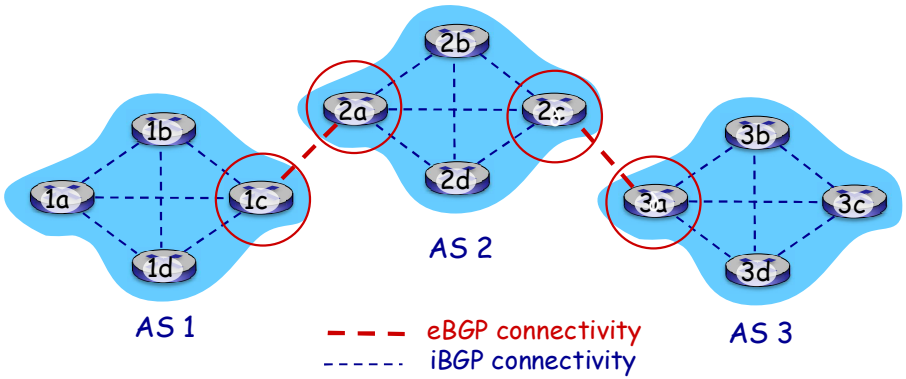
层次路由的优点

- 解决了规模问题
  - 内部网关协议解决：AS内部数量有限的路由器相互到达的问题, AS内部规模可控
    - 如AS节点太多，可分割AS，使得AS内部的节点数量有限
  - AS之间的路由的规模问题
    - 增加一个AS，对于AS之间的路由从总体上来说，只是增加了一个节点=子网（每个AS可以用一个点来表示）
    - 对于其他AS来说只是增加了一个表项，就是这个新增的AS如何走的问题
    - 扩展性强：规模增大，性能不会减得太多
- 解决了管理问题
  - 各个AS可以运行不同的内部网关协议
  - 可以使自己网络的细节不向外透露

互联网AS间路由：BGP

- BGP (Border Gateway Protocol): 自治区域间路由协议"事实上的"标准
  - "将互联网各个AS粘在一起的胶水"
- BGP 提供给每个AS以以下方法：
  - eBGP: 从相邻的ASes那里获得子网可达信息
  - iBGP: 将获得的子网可达信息传遍到AS内部的所有路由器
  - 根据子网可达信息和策略来决定到达子网的"好"路径
- 允许子网向互联网其他网络通告"我在这里"
- 基于距离矢量算法（路径矢量）
  - 不仅仅是距离矢量，还包括到达各个目标网络的详细路径（AS序号的列表）能够避免简单DV算法的路由环路问题

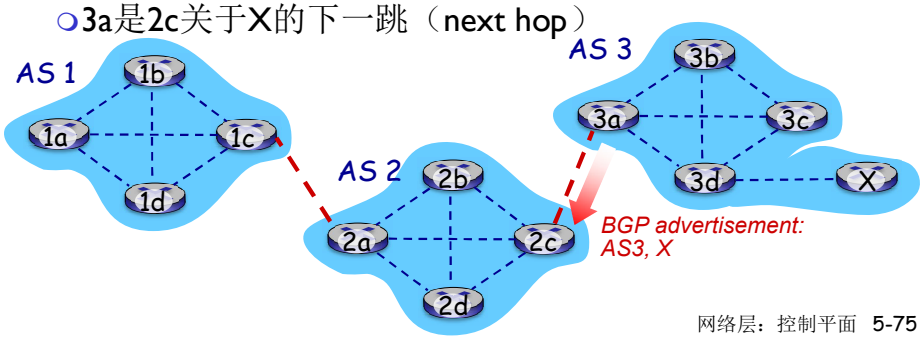
eBGP, iBGP 连接



网关路由器同时运行eBGP和iBGP协议

BGP基础

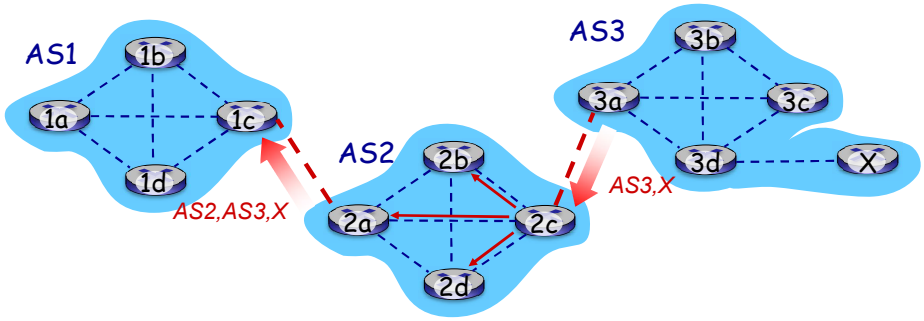
- BGP 会话: 2个BGP路由器("peers")在一个半永久的TCP连接上交换BGP报文:
  - 通告向不同目标子网前缀的"路径"（BGP是一个"路径矢量"协议）
- 当AS3网关路由器3a向AS2的网关路由器2c通告路径： AS3,X
  - 3a参与AS内路由运算，知道本AS所有子网X信息
  - 语义上：AS3向AS2承诺，它可以向子网X转发数据报
  - 3a是2c关于X的下一跳（next hop）



路径的属性& BGP 路由

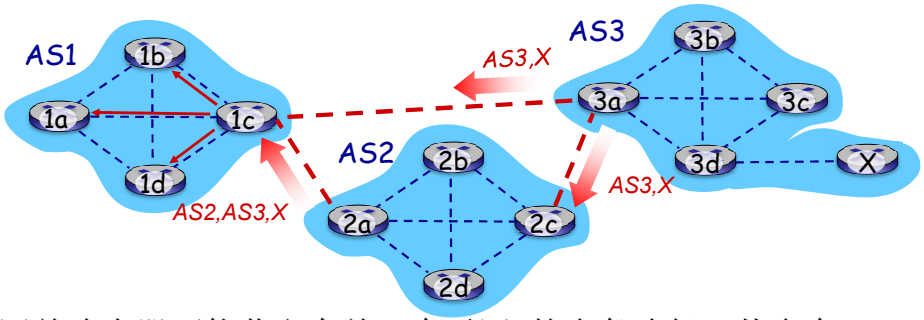
- 当通告一个子网前缀时，通告包括 BGP 属性
  - prefix + attributes = "route"
- 2个重要的属性:
  - AS-PATH: 前缀的通告所经过的AS列表: AS 67 AS 17
    - 检测环路；多路径选择
    - 在向其它AS转发时，需要将自己的AS号加在路径上
  - NEXT-HOP: 从当前AS到下一跳AS有多个链路，在NETX-HOP属性中，告诉对方通过那个 I 转发.
  - 其它属性：路由偏好指标，如何被插入的属性
- 基于策略的路由：
  - 当一个网关路由器接收到了一个路由通告，使用输入策略来接受或过滤（accept/decline.）
    - 过滤原因例1：不想经过某个AS，转发某些前缀的分组
    - 过滤原因例2：已经有了一条往某前缀的偏好路径
  - 策略也决定了是否向它别的邻居通告收到的这个路由信息

BGP 路径通告



- 路由器AS2.2c从AS3.3a接收到的AS3,X路由通告 (通过 eBGP)
- 基于AS2的输入策略，AS2.2c决定接收AS3,X的通告，而且通过iBGP）向AS2的所有路由器进行通告
- 基于AS2的策略，AS2路由器2a通过eBGP向AS1.1c路由器通告AS2,AS3,X 路由信息
  - 路径上加上了 AS2自己作为AS序列的一跳

BGP 路径通告



网关路由器可能获取有关一个子网X的多条路径，从多个eBGP会话上：

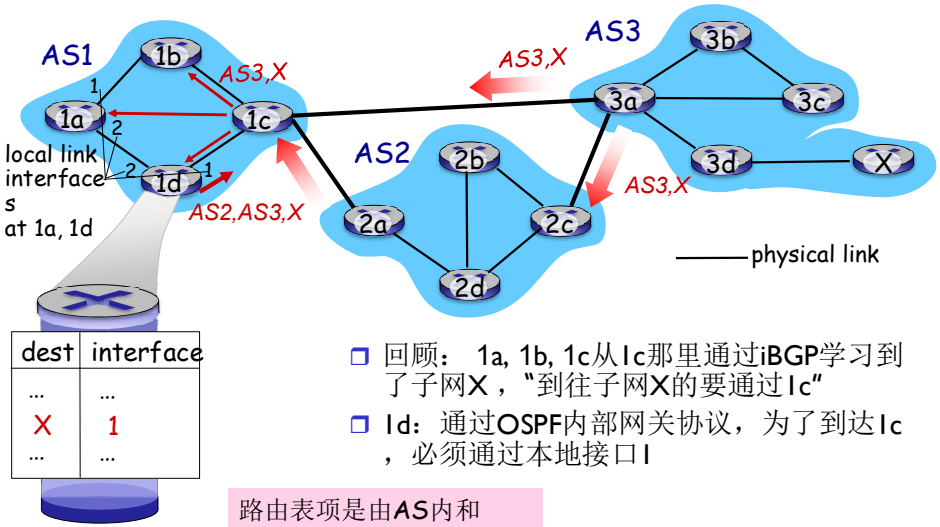
- AS1 网关路由器1c从2a学习到路径：*AS2, AS3, X*
- AS1网关路由器1c从3a处学习到路径AS3, X
- 基于策略，AS1路由器1c选择了路径：AS3, X，而且通过iBGP告诉所有AS1内部的路由器

BGP报文

- 使用TCP协议交换BGP报文.
- BGP 报文:
  - OPEN: 打开TCP连接，认证发送方
  - UPDATE: 通告新路径 (或者撤销原路径)
  - KEEPALIVE：在没有更新时保持连接，也用于对 OPEN 请求确认
  - NOTIFICATION: 报告以前消息的错误，也用来关闭连接

BGP, OSPF, 转发表表项

Q:路由器是如何设置到这些远程子网前缀的转发表表项的？

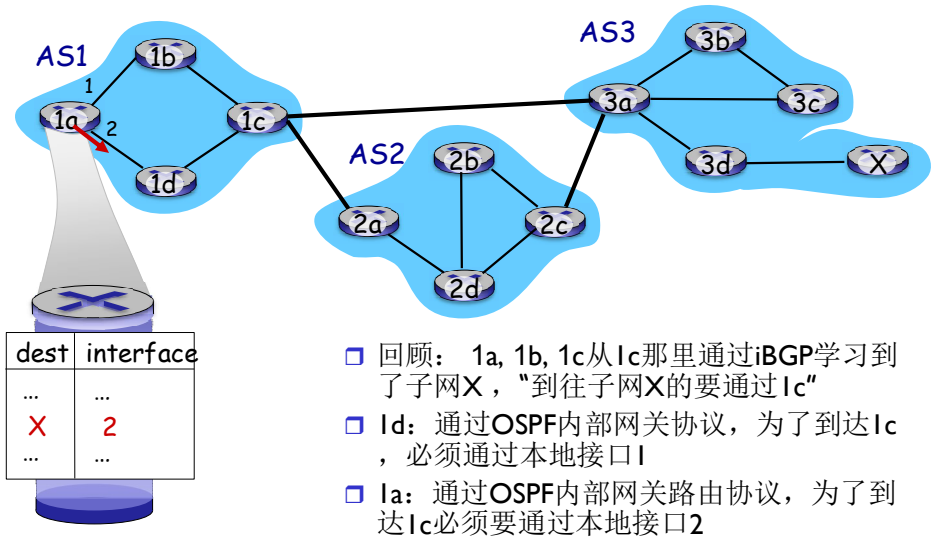


- 回顾： 1a, 1b, 1c从1c那里通过iBGP学习到了子网X，“到往子网X的要通过1c”
- 1d：通过OSPF内部网关协议，为了到达1c，必须通过本地接口1

路由表项是由AS内和AS间的路由共同决定的

BGP, OSPF, 转发表表项

Q:路由器是如何设置到这些远程子网前缀的转发表表项的？

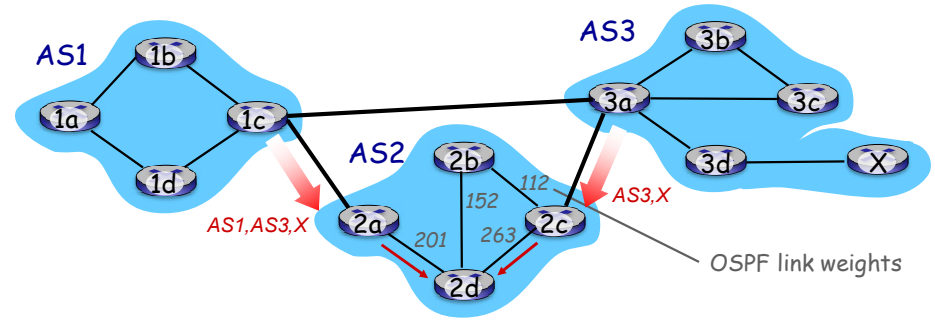


- 回顾： 1a, 1b, 1c从1c那里通过iBGP学习到了子网X，“到往子网X的要通过1c”
- 1d：通过OSPF内部网关协议，为了到达1c，必须通过本地接口1
- 1a：通过OSPF内部网关路由协议，为了到达1c必须要通过本地接口2

BGP 路径选择

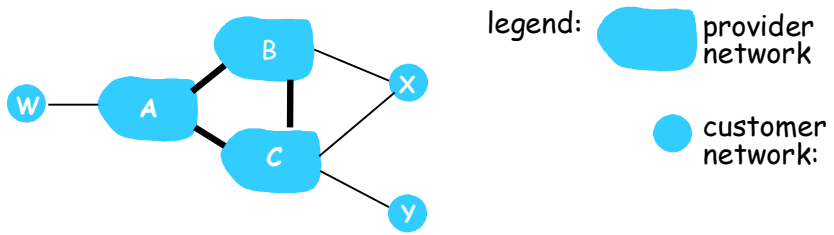
- 路由器可能获得一个网络前缀的多个路径，路由器必须进行路径的选择，路由选择可以基于：
  1. 本地偏好值属性: 偏好策略决定
  2. 最短AS-PATH ： AS的跳数
  3. 最近的NEXT-HOP路由器:热土豆路由
  4. 附加的判据：使用BGP标示
- 一个前缀对应着多种路径，采用消除规则直到留下一条路径

热土豆路由



- 2d通过iBGP获知，它可以通过2a或者2c到达X
- 热土豆策略：选择具备最小内部区域代价的网关作为往X的出口（如：2d选择2a，即使往X可能有比较多的AS跳数）：不要操心域间的代价！

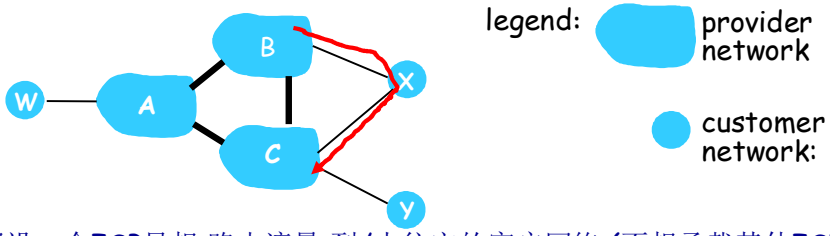
BGP: 通过路径通告执行策略



假设一个ISP只想 路由流量 到/去往它的客户网络 (不想承载其他ISPs之间的流量，即不通告：不是去往我的客户，也不是来自我的客户)

- A 向B和C通告路径Aw
- B选择不向C通告BAw:
  - B从CBAw的路由上无法获得收益，因为C,A,w都不是B的客户
  - C从而无法获知 CBAw路径的存在：每个ISP感知到的网络和真实不一致
- C可能会通过 CAw (而不是使用B)最终路由到w

BGP: 通过路径通告执行策略



假设一个ISP只想 路由流量 到/去往它的客户网络 (不想承载其他ISPs之间的流量，即不通告：不是去往我的客户，也不是来自我的客户)

- A,B,C 是 提供商网络：
- X,W,Y 是桩网络（stub networks）或者叫端网络
- X 是双重接入的，多宿桩网络，接入了2个网络
- 策略强制让X:
  - X不想路由 从B通过X到C的 分组
  - 因而X就不通告给B，它实际上可以路由到C

为什么内部网关协议和外部网关协议如此不同?

策略:

- Inter-AS: 管理员需要控制通信路径，谁在使用它的网络进行数据传输；
- Intra-AS: 一个管理者，所以无需策略;
  - AS内部的各子网的主机尽可能地利用资源进行快速路由

规模:

- AS间路由必须考虑规模问题，以便支持全网的数据转发
- AS内部路由规模不是一个大的问题
  - 如果AS 太大，可将此AS分成小的AS；规模可控
  - AS之间只不过多了一个点而已
  - 或者AS内部路由支持层次性，层次性路由节约了表空间，降低了更新的数据流量

性能:

- Intra-AS: 关注性能
- Inter-AS: 策略可能比性能更重要

第5章：提纲

5.1 导论

5.2 路由选择算法

- link state
- distance vector

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择: BGP

5.5 SDN控制平面

5.6 ICMP: 因特网控制报文协议

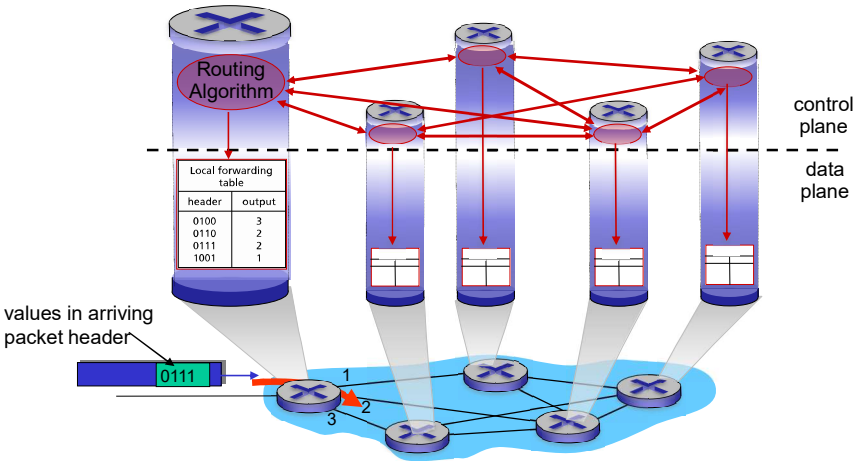
5.7 网络管理和SNMP(略)

Software defined networking (SDN)

- 互联网络网络层：历史上都是通过分布式、每个路由器的实现
  - *单个路由器包含了：交换设备硬件、私有路由器OS（如：思科IOS）和其上运行的互联网标准协议*(IP, RIP, IS-IS, OSPF, BGP)*的私有实现*
  - 需要不同的中间盒来实现不同网络层功能：防火墙，负载均衡设备和NAT…
- ~2005: 点燃重新思考互联网控制平面的兴趣

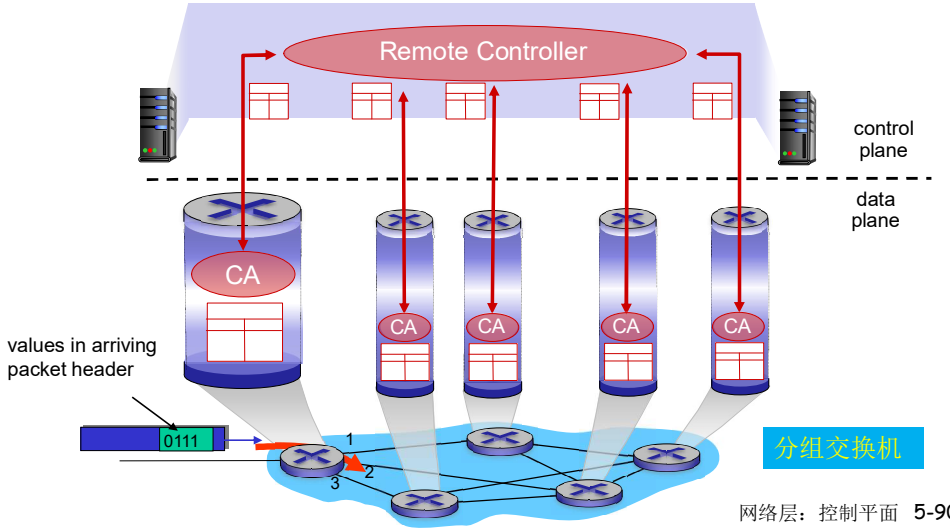
传统方式：每-路由器(Per-router)控制平面

在每一个路由器中的单独路由器算法元件，在控制平面进行交互



SDN方式：逻辑上集中的控制平面

一个不同的（通常是远程的）控制器与本地控制代理（CAs）交互



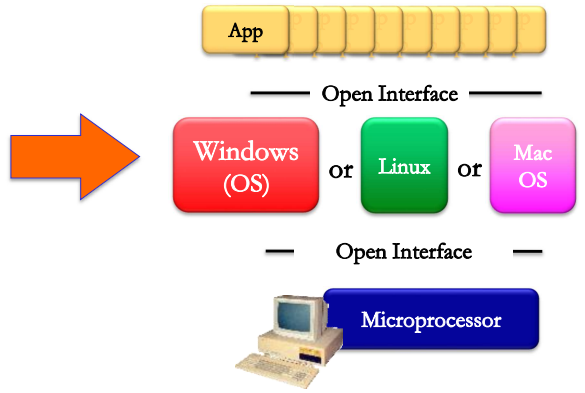
Software defined networking (SDN)

为什么需要一个逻辑上集中的控制平面?

- 网络**管理更加容易**：避免路由器的错误配置，对于通信流的弹性更好
- 基于流表的转发（回顾一下OpenFlow API），允许“**可编程**”的路由器
  - 集中式“编程”更加容易：集中计算流表然后分发
  - 传统方式分布式“编程”困难：在每个单独的路由器上分别运行分布式的算法，得到转发表（部署和升级代价低）
    - 而且要求各分布式计算出的转发表都得基本正确
- 控制平面的开放实现（非私有）
  - 新的竞争生态

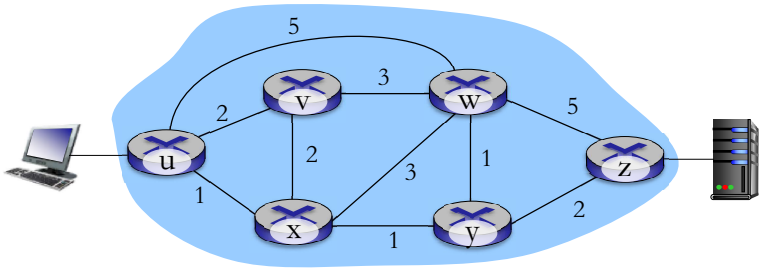
类比: 主框架到PC的演变

垂直集成
封闭，私有
创新缓慢
产业规模小



水平集成
开放接口
快速创新
产业巨大

流量工程：传统路由比较困难

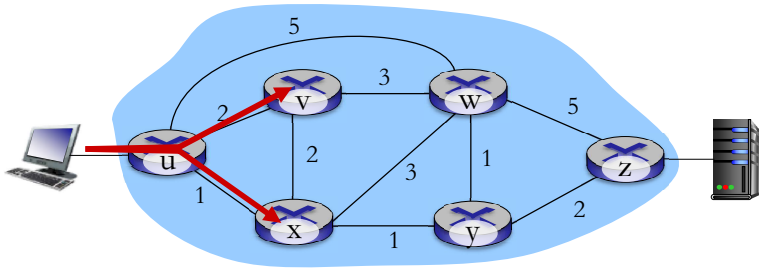


Q: 网管如果需要u到z的流量走uvwz,x到z的流量走xwyz，怎么办?

A: 需要定义链路的代价，流量路由算法以此运算（IP路由面向目标，无法操作）(或者需要新的路由算法)!

只是链路权重控制旋钮，错!
不仅仅链路代价是控制旋钮，控制平面的策略也是

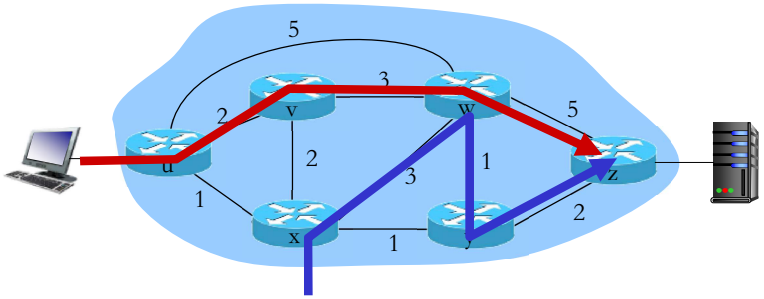
流量工程：困难



Q: 如果网管需要将u到z的流量分成2路：uvwz 和uxyz (负载均衡)，怎么办?（IP路由面向目标）

A: 无法完成(在原有体系下只有使用新的路由选择算法，而在全网部署新的路由算法是个大的事情)

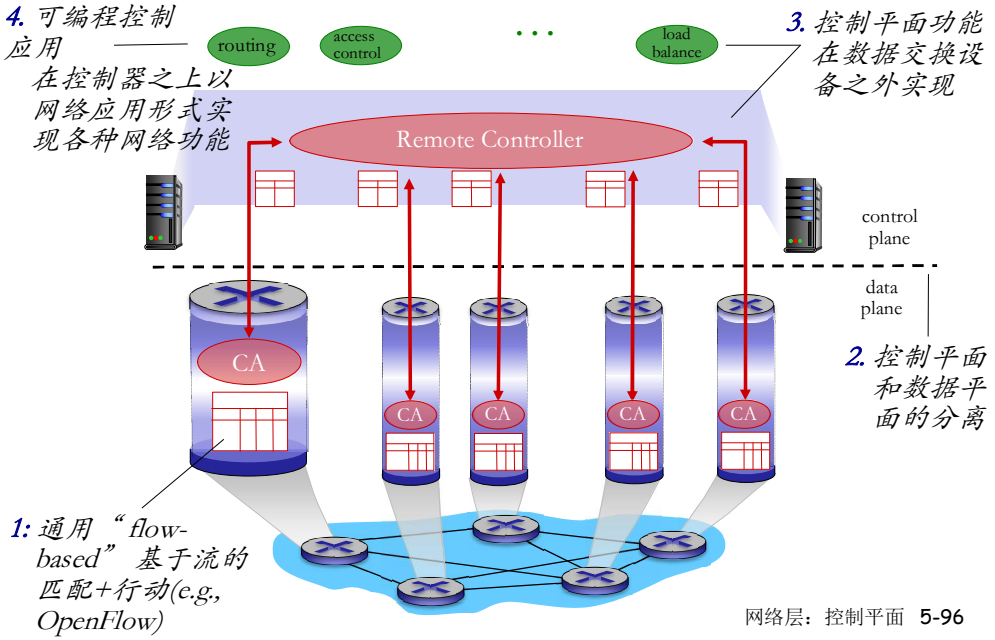
流量工程：困难



Q: 如果需要w对蓝色的和红色的流量采用不同的路由，怎么办?

A: 无法操作 (基于目标的转发，采用LS, DV 路由)

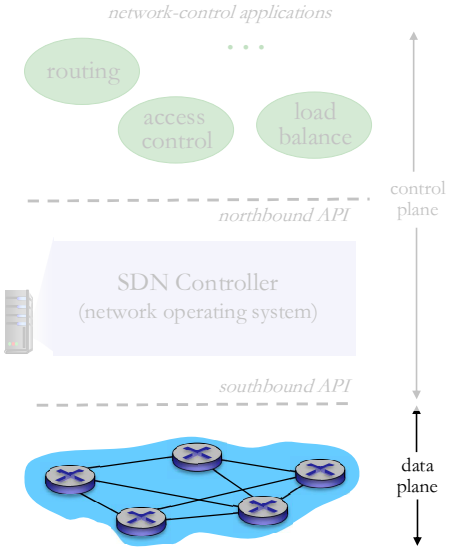
SDN特点



SDN 架构: 数据平面交换机

数据平面交换机

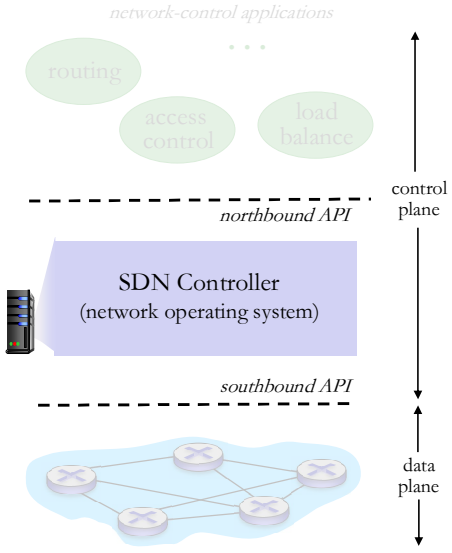
- 快速，简单，商业化交换设备采用硬件实现**通用**转发功能
- 流表被控制器计算和安装
- 基于南向API（例如OpenFlow），SDN控制器访问基于流的交换机
  - 定义了哪些可以被控制哪些不能
- 也定义了和控制器的协议 (e.g., OpenFlow)



SDN 架构: SDN控制器

SDN 控制器(网络OS):

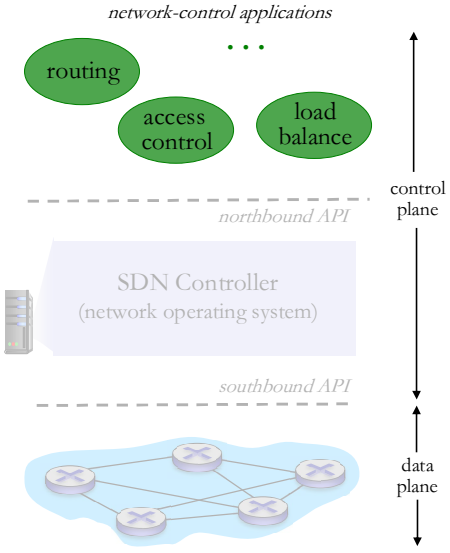
- 维护网络状态信息
- 通过上面的北向API和网络控制应用交互
- 通过下面的南向API和网络交换机交互
- 逻辑上集中，但是在实现上通常由于性能、可扩展性、容错性以及鲁棒性采用分布式方法实现



SDN 架构: 控制应用

网络控制应用:

- 控制的大脑： 采用下层提供的服务（SDN控制器提供的API)，实现网络功能
  - 路由器 交换机
  - 接入控制 防火墙
  - 负载均衡
  - 其他功能
- *非绑定:* 可以被第三方提供，与控制器厂商以通常上不同，与分组交换机厂商也可以不同

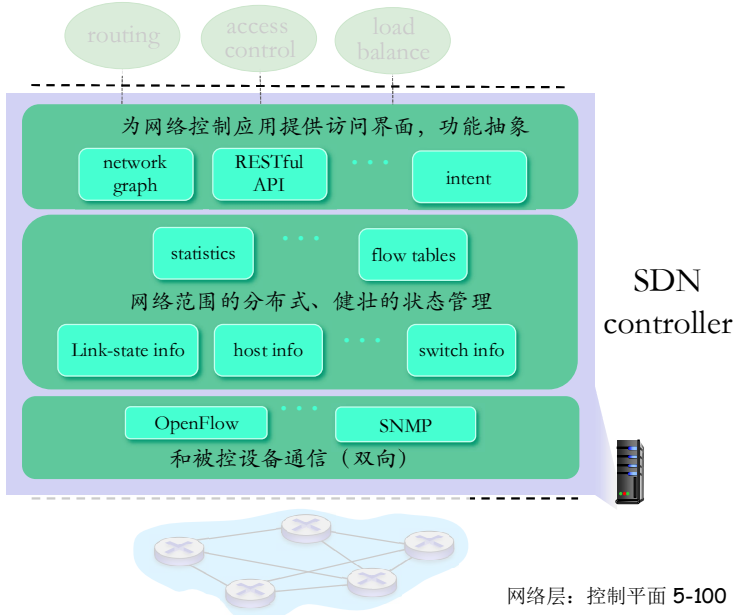


SDN控制器里的元件

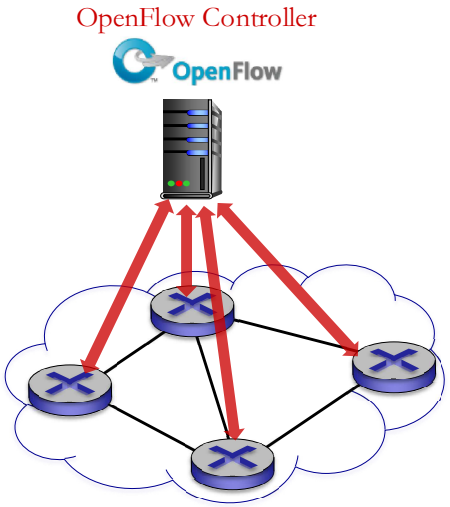
网络控制应用的界面层 抽象API

网络范围的状态管理层: 网络链路、交互设备和服务的状态: *分布式数据库*

通信层: SDN控制器和SDN交换机之间进行通信



OpenFlow 协议

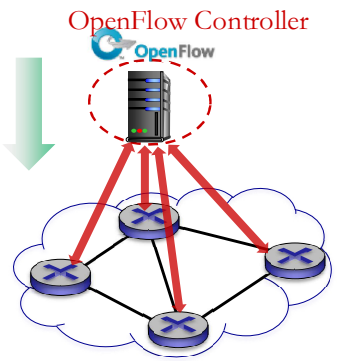


- 控制器和SDN交换机交互的协议
- 采用TCP 来交换报文
  - 加密可选
- 3种OpenFlow报文类型
  - 控制器>交换机
  - 异步（交换机>控制器）
  - 对称 (misc)

OpenFlow: 控制器-交换机报文

一些关键的控制器到交换机的报文

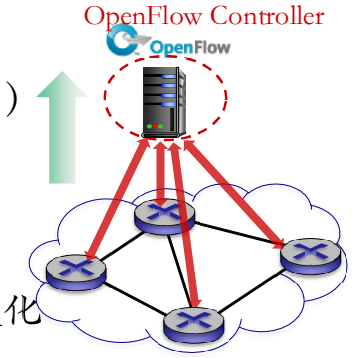
- *特性：* 控制器查询交换机特性，交换机应答
- *配置：* 交换机查询/设置交换机的配置参数
- *修改状态：* 增加删除修改OpenFlow表中的流表
- *packet-out:* 控制器可以将分组通过特定的端口发出



OpenFlow: 交换机到控制器的报文

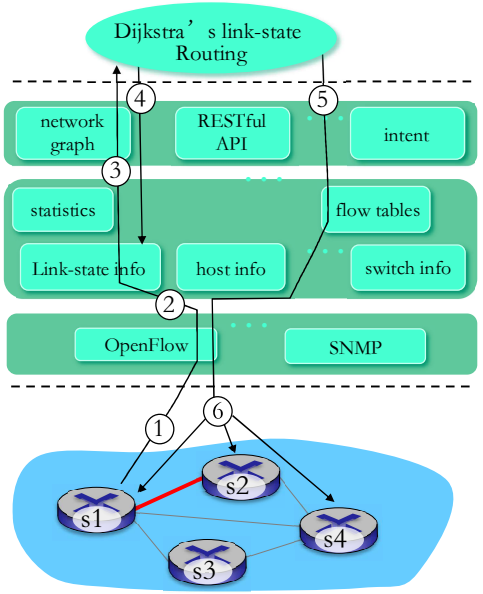
一些关键的交换机到控制器的报文

- *分组进入:* 将分组（和它的控制）传给控制器，见来自控制器的packet-out报文
- *流移除:* 在交换机上删除流表项
- *端口状态:* 通告控制器端口的变化



幸运的是，网络管理员不需要直接通过创建/发送流表来编程交换机，而是采用在控制器上的app自动运算和配置

SDN: 控制/数据平面交互的例子



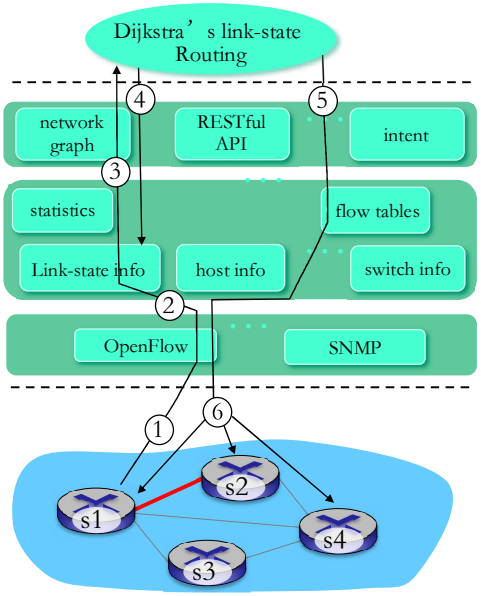
① S1, 经历了链路失效，采用OpenFlow报文通告控制器:
端口状态报文

② SDN 控制器接收OpenFlow报文，更新链路状态信息

③ Dijkstra路由算法应用被调用（前面注册过这个状态变化消息）

④ Dijkstra路由算法访问控制器中的网络拓扑信息，链路状态信息计算新路由

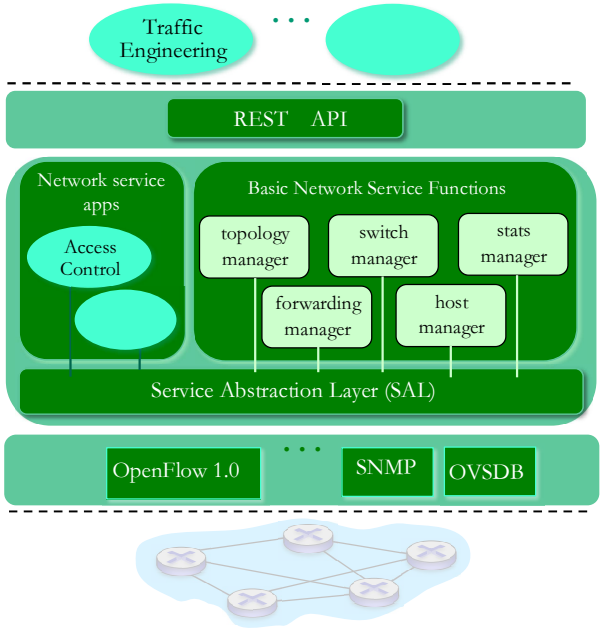
SDN: 控制/数据平面交互的例子



⑤ 链路状态路由app和SDN控制器中流表计算元件交互，计算出新的所需流表

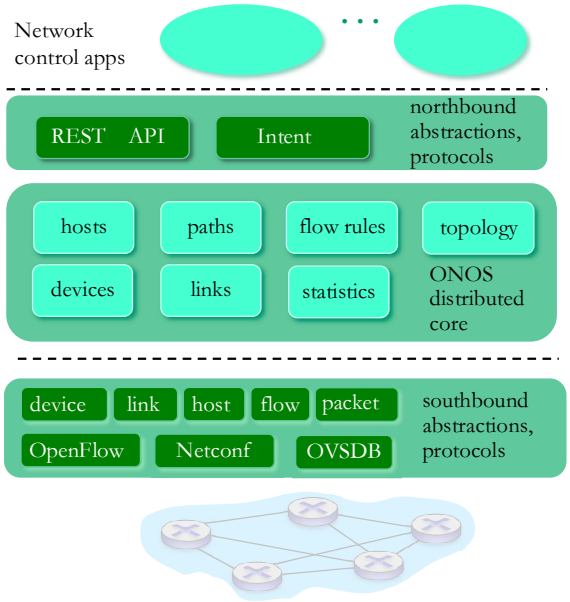
⑥ 控制器采用OpenFlow在交换机上安装新的需要更新的流表

OpenDaylight (ODL) 控制器



- ODL Lithium 控制器
- 网络应用可以在SDN控制内或者外面
- 服务抽象层SAL：和内部以及外部的应用以及服务进行交互

ONOS 控制器



- 控制应用和控制器分离（应用app在控制器外部）
- 意图框架：服务的高级规范：描述什么而不是如何
- 相当多的重点聚焦在分布式核心上，以提高服务的可靠性，性能的可扩展性

SDN: 面临的挑战

- 强化控制平面：可信、可靠、性能可扩展性、安全的分布式系统
  - 对于失效的鲁棒性：利用为控制平面可靠分布式系统的强大理论
  - 可信任，安全：从开始就进行铸造
- 网络、协议满足特殊任务的需求
  - e.g., 实时性，超高可靠性、超高安全性
- 互联网络范围内的扩展性
  - 而不是仅仅在一个AS的内部部署，全网部署

第5章：提纲

5.1 导论

5.2 路由选择算法

- link state
- distance vector

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择：BGP

5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

5.7 网络管理和SNMP(略)

ICMP: Internet Control Message Protocol

- 由主机、路由器、网关用于传达网络层控制信息
  - 错误报告：主机不可到达、网络、端口、协议
  - Echo 请求和回复（ping）
- ICMP处在网络层，但是是在IP协议的上面
  - ICMP消息由IP数据报承载
- ICMP 报文:
  - 类型
  - 编码
  - 加上IP数据报的头8B
    - 第一个导致该ICMP报文的IP数据报

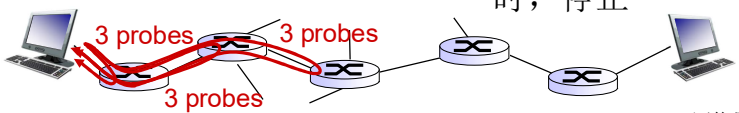
Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

Traceroute and ICMP

- 源主机发送一系列UDP段给目标主机
  - 第一个：TTL =1
  - 第二个： TTL=2, etc.
  - 一个不可达的端口号
- 当nth数据报到达nth路由器
  - 路由器抛弃数据报
  - 然后发送一个给源的ICMP报文 (type 11, code 0)
  - 报文包括了路由器的名字和IP地址
- 当 ICMP报文到达，源端计算RTT
- 对于一个nTraceroute做3次

停止的判据

- UDP 段最终到达目标主机
- 目标返回给源主机 ICMP “端口不可达”报文 (type 3, code 3)
- 当源主机获得这个报文时，停止



第5章：提纲

5.1 导论

5.2 路由选择算法

- link state
- distance vector

5.3 因特网中自治系统内部的路由选择

- RIP
- OSPF

5.4 ISP之间的路由选择：BGP

5.5 SDN控制平面

5.6 ICMP：因特网控制报文协议

5.7 网络管理和SNMP(略)

什么是网络管理?

- 自治系统（autonomous systems, aka "network"): 1000多个相互的软件和硬件部件
- 其他复杂系统也需要被监视和控制:
  - 喷气飞机
  - 核电站
  - 其他例子?

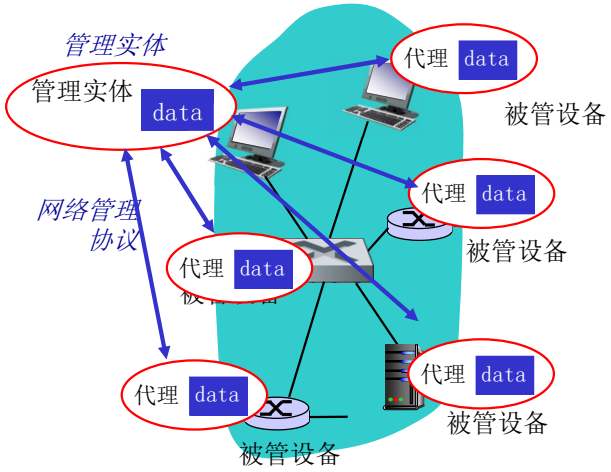
"网络管理"包括了硬件、软件和人类元素的设置，综合和协调，以便监测，测试，轮询，配置，分析，评价和控制网络和网元资源，用合理的成本满足实时性，运行能和服务质量的要求；

网络管理的5大功能

- 性能管理:
  - 性能(利用率、吞吐量)量化、测量、报告、分析和控制不同网络部件的性能
  - 涉及到的部件：单独部件（网卡，协议实体），端到端的路径
- 故障管理：记录、检测和响应故障；
  - 性能管理为长期监测设备性能
  - 故障管理：突然发生的强度大的性能降低，强调对故障的响应
- 配置管理：跟踪设备的配置，管理设备配置信息
- 账户管理：定义、记录和控制用户和设备访问网络资源
  - 限额使用、给予使用的收费，以及分配资源访问权限
- 安全管理：定义安全策略，控制对网络资源的使用

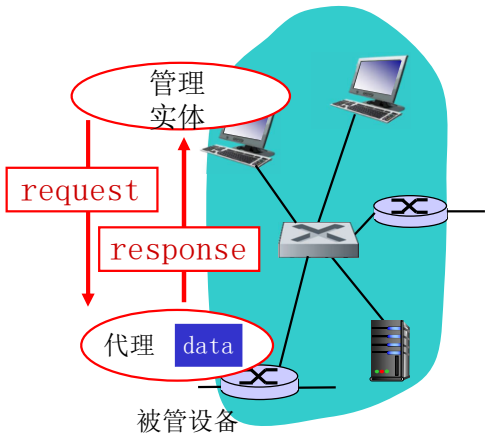
网络管理架构

定义：

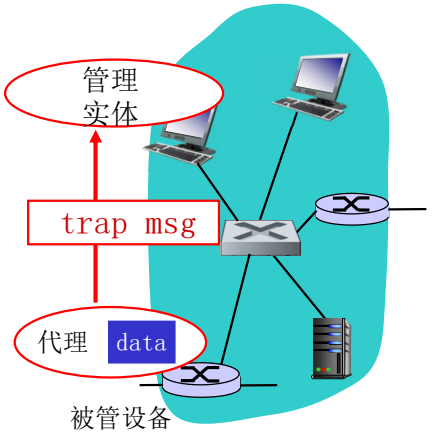


被管设备
包含若干
被管对象
它们的数据被收集在
Management Information Base (MIB)

SNMP协议



请求/响应模式

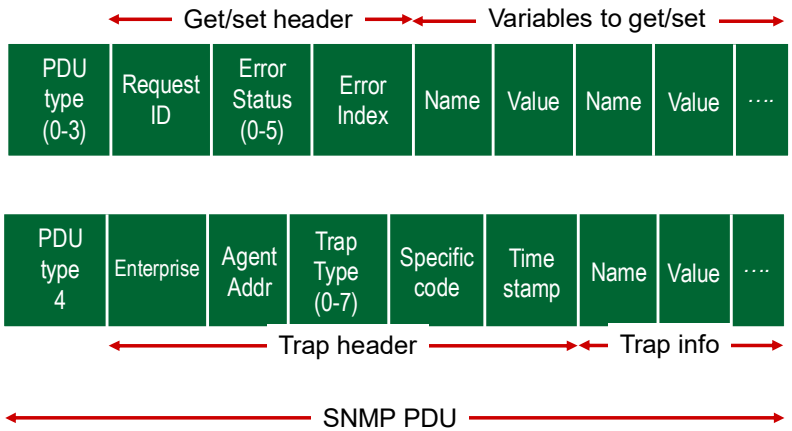


陷阱模式

SNMP 协议：报文类型

报文类型	功能
GetRequest GetNextRequest GetBulkRequest	管理实体-代理：“给我数据” (instance, next in list, block)
InformRequest	实体-实体：给你MIB值
SetRequest	实体-代理：set MIB value
Response	代理-实体：值，对请求的响应
Trap	代理-实体：异常事件的报告

SNMP 协议：报文类型



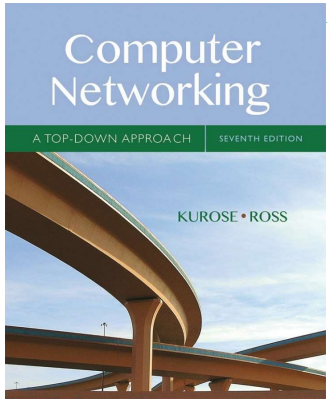
第五章：总结

- 网络层控制平面的方法
  - 每个路由器控制（传统方法）
  - 逻辑上集中的控制 (software defined networking)
- 传统路由选择算法
  - 在互联网上的实现：RIP , OSPF, BGP
- SDN控制器
  - 实际中的实现：ODL, ONOS
- Internet Control Message Protocol
- 网络管理和SNMP协议

下一站：链路层

第6章
链路层和局域网

中国科学技术大学
自动化系 郑烇
改编自Jim kurose, Keith Ross



Computer Networking: A Top Down Approach

7th edition
Jim Kurose, Keith Ross
Pearson/Addison Wesley
April 2016

第6章：链路层和局域网

导引：

- 网络层解决了一个网络如何到达另外一个网络的路由问题
- 在一个网络内部如何由一个节点（主机或者路由器）到达另外一个相邻节点
  - 链路层的点到点传输层功能

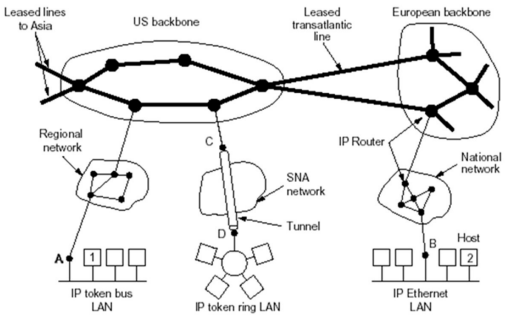
第6章：链路层和局域网

目标：

- 理解数据链路层服务的原理：
  - 检错和纠错
  - 共享广播信道：多点接入（多路访问）
  - 链路层寻址
  - LAN:以太网、WLAN、VLANs
  - 可靠数据传输，流控制：*解决!*
- 实例和各种链路层技术的实现

网络节点的连接方式

Q：一个子网中的若干节点是如何连接到一起的



- 点到点连接
- 多点连接：
  - 共享型介质
  - 通过网络交换机

数据链路层和局域网

- WAN:网络形式采用点到点链路
  - 带宽大、距离远（延迟大）>带宽延迟积大
  - 如果采用多点连接方式
    - 竞争方式：一旦冲突代价大
    - 令牌等协调方式：在其中协调节点的发送代价大
- 点到点链路的链路层服务实现非常简单，封装和解封装
- LAN一般采用多点连接方式
  - 连接节点非常方便
  - 接到共享型介质上（或网络交换机），就可以连接所有其他节点
- 多点连接方式网络的链路层功能实现相当复杂
  - 多点接入：协调各节点对共享性介质的访问和使用
  - 竞争方式：冲突之后的协调；
  - 令牌方式：令牌产生，占有和释放等

内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

内容提纲

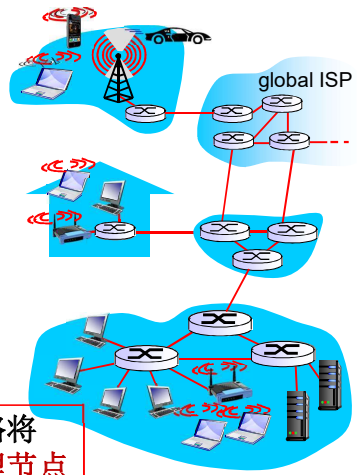
6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

链路层：导论

一些术语:

- 主机和路由器是节点（网桥和交换机也是）：**nodes**
- 沿着通信路径,连接个相邻节点通信信道的是链路：**links**
  - 有线链路
  - 无线链路
  - 局域网，共享性链路
- 第二层协议数据单元帧**frame**，封装数据报



数据链路层负责从一个节点通过链路将（帧中的）数据报发送到**相邻的物理节点**（一个子网内部的2节点）

链路层：上下文

- 数据报（分组）在不同的链路上以不同的链路协议传送：
  - 第一跳链路：以太网
  - 中间链路：帧中继链路
  - 最后一跳802.11：
- 不同的链路协议提供不同的服务
  - e.g.,比如在链路层上提供（或没有）可靠数据传送

传输类比

- 从Princeton到Lausanne
  - 轿车: Princeton to JFK
  - 飞机: JFK to Geneva
  - 火车: Geneva to Lausanne
- 旅行者=数据报datagram
- 交通段=通信链路 communication link
- 交通模式=链路层协议：数据链路层和局域网 protocol
- 票务代理=路由算法 routing algorithm

链路层服务

- 成帧，链路接入：
  - 将数据报封装在帧中，加上帧头、帧尾部
  - 如果采用的是共享性介质，信道接入获得信道访问权
  - 在帧头部使用"MAC"（物理）地址来标示源和目的
    - 不同于IP地址
- 在（一个网络内）相邻两个节点完成可靠数据传递
  - 已经学过了（第三章）
  - 在低出错率的链路上（光纤和双绞线电缆）很少使用
  - 在无线链路经常使用：出错率高
    - Q: 为什么在链路层和传输层都实现了可靠性

一般化的链路层服务，不是所有的链路层都提供这些服务
一个特定的链路层只是提供其中一部分的服务

链路层服务（续）

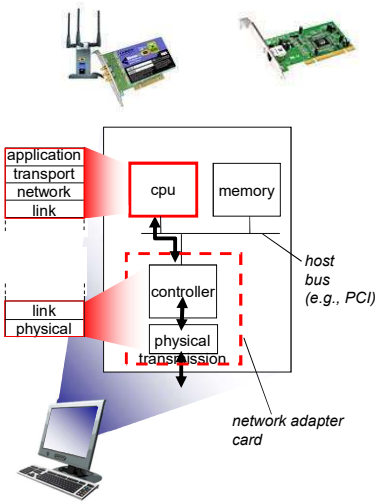
- 在相邻节点间（一个子网内）进行可靠的转发
  - 我们已经学习过（见第三章）！
  - 在低差错链路上很少使用 (光纤,一些双绞线)
    - 出错率低，没有必要在每一个帧中做差错控制的工作，协议复杂
      - 发送端对每一帧进行差错控制编码，根据反馈做相应的动作
      - 接收端进行差错控制解码，反馈给发送端（ACK，NAK）
    - 在本层放弃可靠控制的工作，在网络层或者是传输层做可靠控制的工作，或者根本就不做可靠控制的工作
  - 在高差错链路上需要进行可靠的数据传送
    - 高差错链路：无线链路；
    - Q：为什么要在采用无线链路的网络上，链路层做可靠数据传输工作；还要在传输层做端到端的可靠性工作？
    - 原因：出错率高，如果在链路层不做差错控制工作，漏出去的错误比较高；到了上层如果需要可靠控制的数据传输代价会很大
      - 如不做local recovery 工作，总体代价大

链路层服务（续）

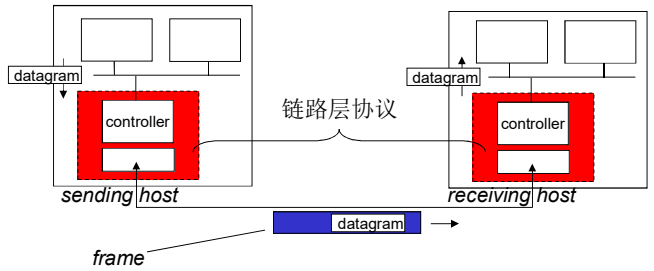
- 流量控制：
  - 使得相邻的发送和接收方节点的速度匹配
- 错误检测：
  - 差错由信号衰减和噪声引起
  - 接收方检测出的错误：
    - 通知发送端进行重传或丢弃帧
- 差错纠正：
  - 接收端检查和纠正bit错误，不通过重传来纠正错误
- 半双工和全双工：
  - 半双工：链路可以双向传输，但一次只有一个方向

链路层在哪里实现？

- 在每一个主机上
  - 也在每个路由器上
  - 交换机的每个端口上
- 链路层功能在"适配器"上实现 (aka *network interface card* NIC) 或者在一个芯片组上
  - 以太网卡，802.11 网卡；以太网芯片组
  - 实现链路层和相应的物理层功能
- 接到主机的系统总线上
- 硬件、软件和固件的综合体



适配器通信



- 发送方：
  - 在帧中封装数据报
  - 加上差错控制编码，实现RDT和流量控制功能等
- 接收方
  - 检查有无出错，执行rdt和流量控制功能等
  - 解封装数据报，将至交给上层

适配器是半自治的
实现了链路和物理层功能

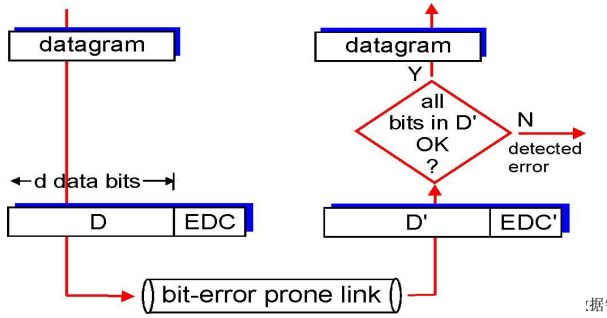
内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

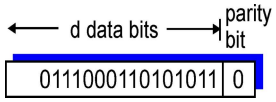
错误检测

EDC=差错检测和纠正位（冗余位）
D =数据由差错检测保护，可以包含头部字段
错误检测不是100%可靠的!
- 协议会漏检一些错误，但是很少
- 更长的EDC字段可以得到更好的检测和纠正效果

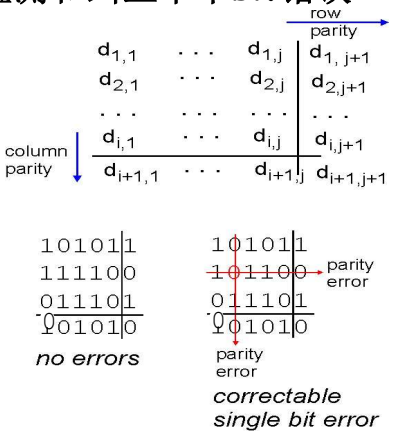


奇偶校验

单bit奇偶校验: **检测单个bit级错误**



2维奇偶校验: **检测和纠正单个bit错误**



Internet校验和

目标: 检测在传输报文段时的错误（如位翻转），（注：仅仅用在传输层）

发送方:

- 将报文段看成16-bit整数
- 报文段的校验和: 和 (1'的补码和)
- 发送方将checksum的值放在‘UDP校验和’字段

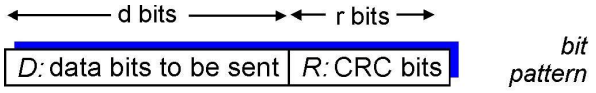
接收方:

- 计算接收到的报文段的校验和
- 检查是否与携带校验和字段值一致:
  - 不一致：检出错误
  - 一致：没有检出错误，但可能还是有错误

有更简单的检查方法
全部加起来看是不是全1

检验和：CRC（循环冗余校验）

- 强大的差错检测码
- 将数据比特 D, 看成是二进制的数据
- 生成多项式*G*：双方协商r+1位模式（r次方）
  - 生成和检查所使用的位模式
- 目标:选择r位 CRC附加位R，使得
  - <D,R> 正好被 G整除 (modulo 2)
  - 接收方知道 G, 将 <D,R>除以 G. 如果非0余数: 检查出错误!
  - 能检出所有少于r+1位的突发错误
- 实际中广泛使用（以太网、802.11 WiFi、ATM）



$D * 2^r$ XOR R　　*mathematical formula*

CRC 例子

需要:

$D \cdot 2^r$ XOR R = nG

等价于:

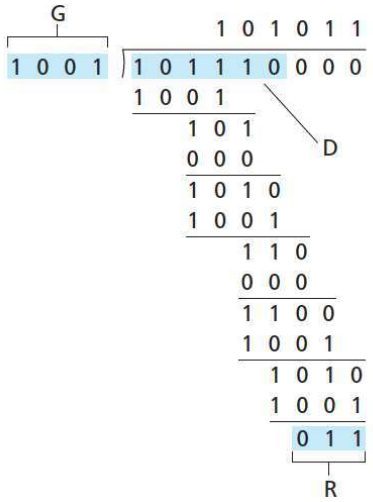
$D \cdot 2^r$ = nG XOR R

等价于:

两边同除G

得到余数 R=..

$$R = \text{remainder}\left[\frac{D \cdot 2^r}{G}\right]$$



CRC性能分析

- 突发错误和突发长度
- CRC检错性能描述
  - 能够检查出所有的1bit错误
  - 能够检查出所有的双bits的错误
  - 能够检查出所有长度 =r或者<r 位的错误
  - 出现长度为 r+1的突发错误，检查不出的概率是 $\frac{1}{2^{r-1}}$
  - 出现长度大于r+1的突发错误，检查不出的概率 $\frac{1}{2^{r}}$

内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

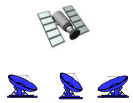
多路访问链路和协议

两种类型的链路（一个子网内部链路连接形式）：

- 点对点
  - 拨号访问的PPP
  - 以太网交换机和主机之间的点对点链路
- 广播 (共享线路或媒体)
  - 传统以太网
  - HFC上行链路
  - 802.11无线局域网

shared wire (e.g., cabled Ethernet)

shared RF (e.g., 802.11 WiFi)



shared RF (satellite)

humans at a cocktail party (shared air, acoustical)

多路访问协议

- 单个共享的广播型链路
- 2个或更多站点同时传送: 冲突（collision）
  - 多个节点在同一个时刻发送，则会收到2个或多个信号叠加

多路访问协议（介质访问控制协议：MAC）

- 分布式算法-决定节点如何使用共享信道，即：决定节点什么时候可以发送？
- 关于共享控制的通信必须用借助信道本身传输！
  - 没有带外的信道，各节点使用其协调信道使用
  - 用于传输控制信息

理想的多路访问协议

给定：Rbps的广播信道

必要条件：

1. 当一个节点要发送时，可以R速率发送.
2. 当M个节点要发送，每个可以以R/M的平均速率发送
3. 完全分布的:
   - 没有特殊节点协调发送
   - 没有时钟和时隙的同步
4. 简单

MAC（媒体访问控制）协议：分类

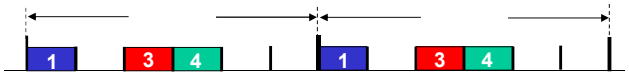
3大类:

- 信道划分
  - 把信道划分成小片（时间、频率、编码）
  - 分配片给每个节点专用
- 随机访问
  - 信道不划分，允许冲突
  - 冲突后恢复
- 依次轮流
  - 节点依次轮流
  - 但是有很多数据传输的节点可以获得较长的信道使用权

a.信道划分MAC协议：TDMA

TDMA:time division multiple access

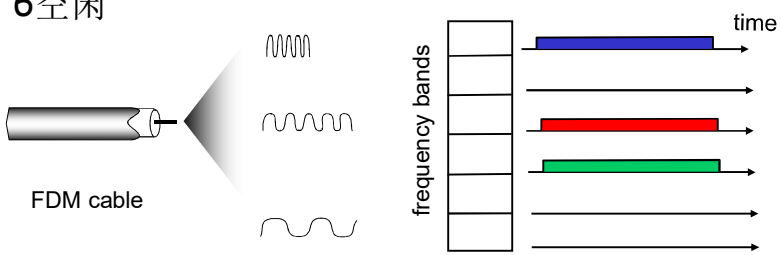
- 轮流使用信道，信道的时间分为周期
- 每个站点使用每周期中固定的时隙(长度=帧传输时间)传输帧
- 如果站点无帧传输，时隙空闲-》浪费
- 如：6站LAN，1、3、4有数据报，时隙2、5、6空闲



a.信道划分MAC协议：FDMA

FDMA: frequency division multiple access

- 信道的有效频率范围被分成一个个小的频段
- 每个站点被分配一个固定的频段
- 分配给站点的频段如果没有被使用，则空闲
- 例如：6站LAN，1、3、4有数据报，频段2、5、6空闲



a.码分多路访问（CDMA）

- CDMA (code division multiple access) :
  - 所有站点在整个频段上同时进行传输, 采用编码原理加以区分
  - 完全无冲突
  - 假定:信号同步很好,线性叠加
- 比方
  - TDM:不同的人在不同的时刻讲话
  - FDM:不同的组在不同的小房间里通信
  - CDMA:不同的人使用不同的语言讲话

b.随机存取协议

- 当节点有帧要发送时
  - 以**信道带宽的全部 R bps**发送
  - 没有节点间的预先协调
- 两个或更多节点同时传输，会发生➜冲突 “collision”
- 随机存取协议规定:
  - 如何检测冲突
  - 如何从冲突中恢复（如：通过稍后的重传）
- 随机MAC协议:
  - 时隙ALOHA
  - ALOHA
  - CSMA, CSMA/CD, CSMA/CA

b.1 时隙ALOHA

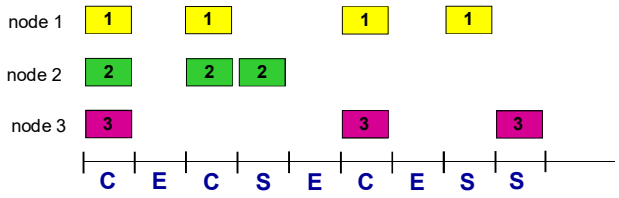
假设

- 所有帧是等长的
- 时间被划分成相等的**时隙**，每个时隙可发送一帧
- 节点只在时隙开始时发送帧
- 节点在时钟上是同步的
- 如果两个或多个节点在一个时隙传输，所有的站点都能检测到冲突

运行

- 当节点获取新的帧，在下一个时隙传输
- 传输时没有检测到冲突，成功
  - 节点能够在下一时隙发送新帧
- 检测时如果检测到冲突，失败
  - 节点在每一个随后的时隙以概率p重传帧直到成功

b.1时隙ALOHA



优点

- 节点可以以信道带宽全速连续传输
- 高度分布：仅需要节点之间在时隙上的同步
- 简单

缺点

- 存在冲突，浪费时隙
- 即使有帧要发送，仍然有可能存在空闲的时隙
- 节点检测冲突的时间<帧传输的时间
  - 必须传完
- 需要时钟上同步

时隙ALOHA的效率(Efficiency)

效率：当有很多节点，每个节点有很多帧要发送时，x%的时隙是成功传输帧的时隙

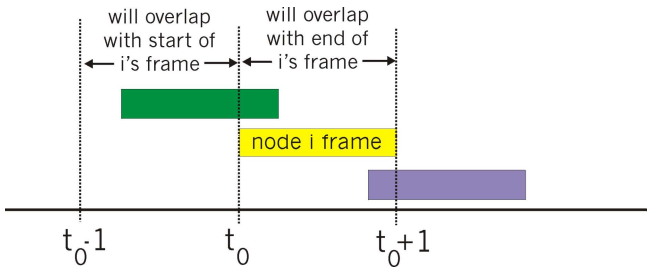
- 假设N个节点，每个节点都有很多帧要发送，在每个时隙中的传输概率是p
- 一个节点成功传输概率是$p(1-p)^{N-1}$
- 任何一个节点的成功概率是= $Np(1-p)^{N-1}$

- N个节点的最大效率：求出使$f(P)=Np(1-p)^{N-1}$最大的p*
- 代入P*得到最大$f(p^*)=Np^*(1-p^*)^{N-1}$
- N为无穷大时的极限为1/e=0.37

最好情况：信道利用率37%

b.2 纯ALOHA(非时隙)

- 无时隙ALOHA：简单、无须节点间在时间上同步
- 当有帧需要传输：马上传输
- 冲突的概率增加：
  - 帧在t_0发送，和其它在$[t_0-1, t_0+1]$区间内开始发送的帧冲突
  - 和当前帧冲突的区间（其他帧在此区间开始传输）增大了一倍



纯ALOHA的效率

P(指定节点成功) = P(节点传输) ·

P(其它节点在$[t_0-1, t_0]$不传) ·

P(其它节点在$[t_0, t_0+1$不传]

$= p \cdot (1-p)^{N-1} \cdot (1-p)^{N-1} = p \cdot (1-p)^{2(N-1)}$

选择最佳的p、N趋向无穷大…

$= 1/(2e) = 17.5\%$

效率比时隙ALOHA更差了!

b.3 CSMA(载波侦听多路访问)

Aloha: 如何提高ALOHA的效率
发之前不管有无其他节点在传输

CSMA: 在传输前先侦听信道:
- 如果侦听到信道空闲，传送整个帧
- 如果侦听到信道忙，推迟传送

- 人类类比：不要打断别人正在进行的说话!

CSMA冲突

冲突仍然可能发生:

由传播延迟造成：两个节点可能侦听不到正在进行的传输

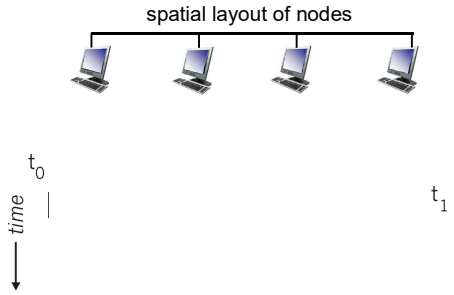
冲突:

整个冲突帧的传输时间都被浪费了，是无效的传输(红黄区域)

注意：

传播延迟（距离）决定了冲突的概率

节点依据本地的信道使用情况来判断全部信道的使用情况



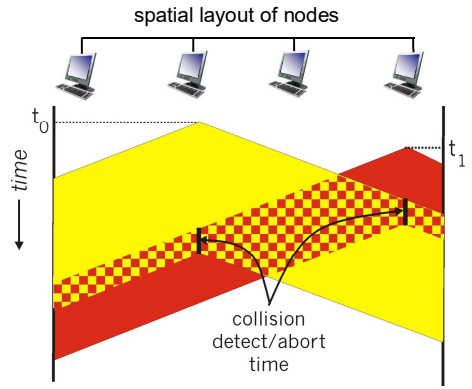
b.4 CSMA/CD(冲突检测)

CSMA/CD:

- 载波侦听CSMA：和在CSMA中一样发送前侦听信道
- 没有传完一个帧就可以在短时间内检测到冲突
- 冲突发生时则传输终止，减少对信道的浪费

- 冲突检测CD技术，有线局域网中容易实现：
  - 检测信号强度，比较传输与接收到的信号是否相同
  - 通过周期的过零点检测
- 人类类比：礼貌的对话人

CSMA/CD（冲突检测）



对信道的浪费减少了

以太网CSMA/CD算法

1. 适配器获取数据报，创建帧
2. 发送前：侦听信道CS
   1)闲：开始传送帧
   2)忙：一直等到闲再发送
3. 发送过程中，冲突检测CD
   1)没有冲突:成功
   2)检测到冲突:放弃,之后尝试重发
4. 发送方适配器检测到冲突，除放弃外，还发送一个Jam信号，所有听到冲突的适配器也是如此

   强化冲突：让所有站点都知道冲突
5. 如果放弃，适配器进入指数退避状态

   在第m次失败后，适配器随机选择一个{0，1，2，，2^{m-1}}中K，等待K*512位时，然后转到步骤2

exponential backoff

二进制指数退避算法

以太网CSMA/CD算法（续）

指数退避:

- 目标：适配器试图适应当前负载，在一个变化的碰撞窗口中随机选择时间点尝试重发
  - 高负载：重传窗口时间大，减少冲突，但等待时间长
  - 低负载：使得各站点等待时间少，但冲突概率大

- 首次碰撞：在{0，1}选择K；延迟K*512位时
- 第2次碰撞：在{0，1，2，3}选择K
- 第10次碰撞：在{0，1，2，3，……，1023}选择K

CSMA/CD效率

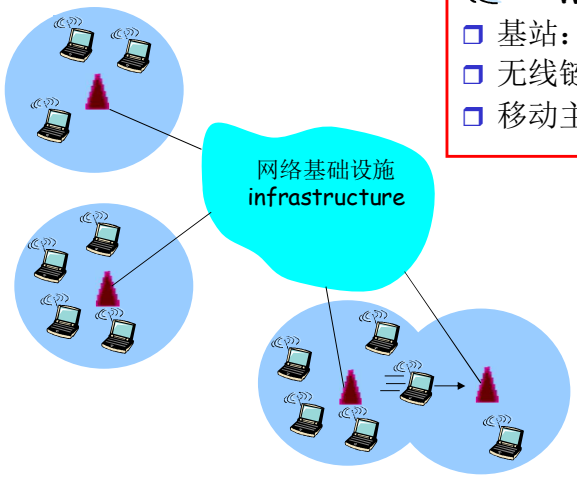
- T_{prop} = LAN上2个节点的最大传播延迟
- t_{trans} = 传输最大帧的时间

$$efficiency = \frac{1}{1 + 5t_{prop}/t_{trans}}$$

- 效率变为1
  - 当t_{prop} 变成0时
  - 当t_{trans} 变成无穷大时
- 比ALOHA更好的性能，而且简单，廉价，分布式！

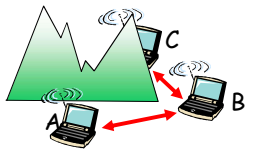
6.5 无线局域网 CSMA/CA

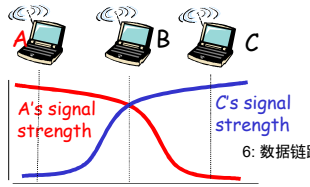
WLAN构成
- 基站：AP
- 无线链路
- 移动主机节点



无线局域网中的 MAC：CSMA/CA

- 冲突：2+ 站点（AP或者站点）在同一个时刻发送
- 802.11: CSMA - 发送前侦听信道
  - 不会和其它节点正在进行的传输发生冲突
- 802.11: 没有冲突检测!
  - 无法检测冲突：自身信号远远大于其他节点信号
  - 即使能CD：冲突!=成功
  - 目标: *avoid collisions:* CSMA/C(ollision)A(voidance)
    - 无法CD，一旦发送一股脑全部发送完毕，不CD
    - 为了避免无CD带来的信道利用率低的问题，事前进行冲突避免





无线局域网: CSMA/CA

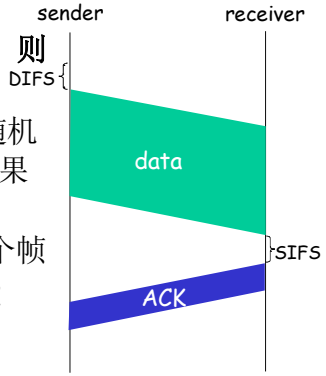
发送方

1 如果站点侦测到信道空闲持续**DIFS长，则传输整个帧** (no CD)

2 如果侦测到信道忙碌，那么 选择一个随机回退值，并在信道空闲时递减该值；如果信道忙碌，回退值不会变化

到数到0时（只生在信道闲时）发送整个帧

如果没有收到ACK, 增加回退值，重复2



802.11 接收方

- 如果帧正确，则在SIFS后发送ACK

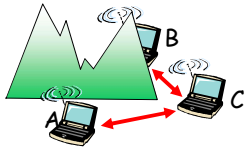
(无线链路特性，需要每帧确认；例如：由于隐藏终端问题，在接收端可能形成干扰,接收方没有正确地收到。链路层可靠机制)

IEEE 802.11 MAC 协议: CSMA/CA

- 在count down时，侦听到了信道空闲为什么不发送，而要等到0时在发送
  - 2个站点有数据帧需要发送，第三个节点正在发送
  - LAN CD：让2者听完第三个节点发完，立即发送
    - ✓冲突：放弃当前的发送，避免了信道的浪费于无用冲突帧的发送
    - ✓代价不昂贵
  - WLAN : CA
    - ✓无法CD，一旦发送就必须发完，如冲突信道浪费严重，代价高昂
    - ✓思想：尽量事先避免冲突，而不是在发生冲突时放弃然后重发
    - ✓听到发送的站点，分别选择随机值，回退到0发送
      - 不同的随机值，一个站点会胜利
      - 失败站点会冻结计数器，当胜利节点发完再发

IEEE 802.11 MAC 协议: CSMA/CA

- 无法完全避免冲突
  - 两个站点相互隐藏：
    - A,B 相互隐藏，C在传输
    - A,B选择了随机回退值
    - 一个节点如A胜利了，发送
    - 而B节点收不到，顺利count down到0 发送
    - A,B的发送在C附近形成了干扰



  - 选择了非常靠近的随机回退值：
    - A,B选择的值非常近
    - A到0后发送
    - 但是这个信号还没到达B时
    - B也到0了，发送
    - 冲突

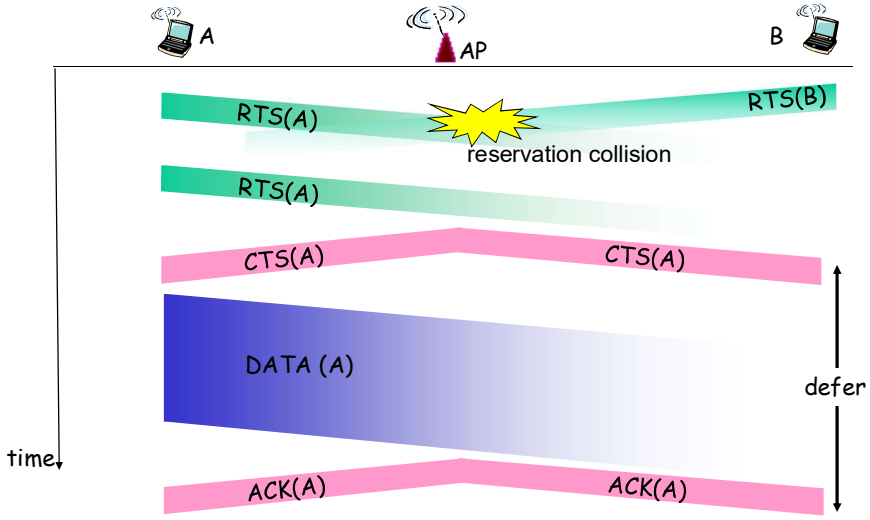
冲突避免(续)

思想: 允许发送方“预约”信道，而不是随机访问该信道: 避免长数据帧的冲突（可选项）

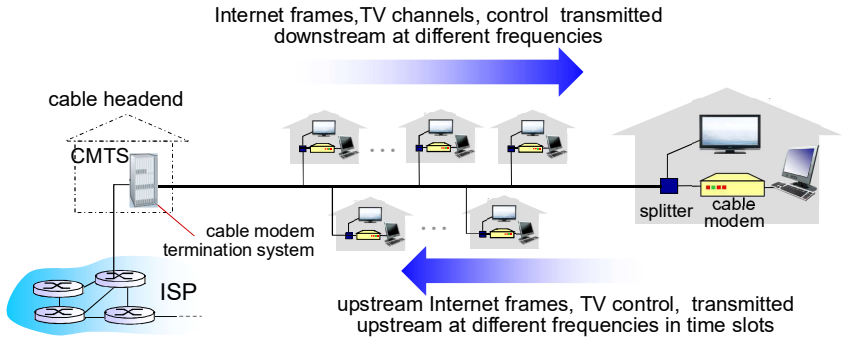
- 发送方首先使用CSMA向BS发送一个小的RTS分组
  - RTS可能会冲突（但是由于比较短，浪费信道较少）
- BS广播 clear-to-send CTS，作为RTS的响应
- CTS能够被所有涉及到的节点听到
  - 发送方发送数据帧
  - 其它节点抑制发送

采用小的预约分组，可以完全避免数据帧的冲突

冲突避免：RTS-CTS 交换

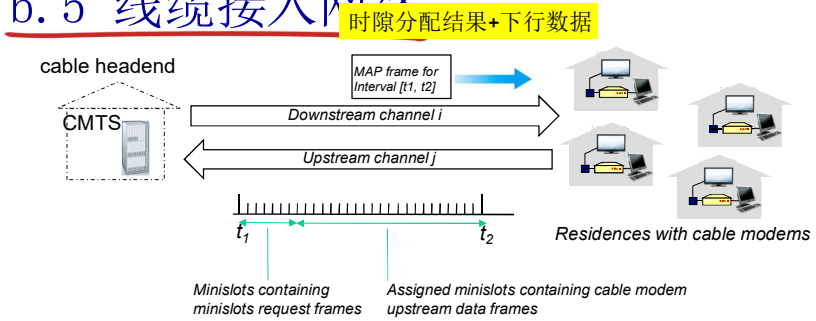


b. 5 线缆接入网络



- 多个40Mbps 下行(广播)信道,FDM
  - 下行：通过FDM分成若干信道，互联网、数字电视等
  - 互联网信道：只有1个CMTS在其上传输
- 多个30 Mbps上行的信道,FDM
  - 多路访问：所有用户使用；接着TDM分成微时隙
  - 部分时隙：分配；部分时隙：竞争；

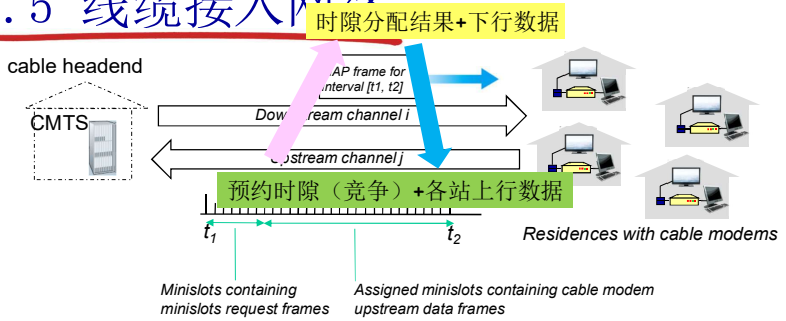
b. 5 线缆接入网络



DOCSIS: data over cable service interface spec

- 采用FDM进行信道的划分：若干上行、下行信道
- 下行信道:
  - 在下行MAP帧中：CMTS告诉各节点微时隙分配方案，分配给各站点的上行**微时隙**
  - 另外：头端传输下行数据（给各个用户）

b. 5 线缆接入网络



DOCSIS: TDM上行信道

- 采用TDM的方式将上行信道分成若干**微时隙**：MAP指定
- 站点采用分配给它的**微时隙**上行数据传输：分配
- 在特殊的上行**微时隙**中，各站点请求上行**微时隙**：竞争
  - 各站点对于该时隙的使用是随机访问的
  - 一旦碰撞（请求不成功，结果是：在下行的MAP中没有为它分配，则二进制退避）选择时隙上传输

c.轮流(Taking Turns)MAC协议

信道划分MAC协议:

- 共享信道在高负载时是有效和公平的
- 在低负载时效率低下
  - 只能等到自己的时隙开始发送或者利用1/N的信道频率发送
  - 当只有一个节点有帧传时，也只能够得到1/N个带宽分配

随机访问MAC协议

- 在低负载时效率高：单个节点可以完全利用信道全部带宽
- 高负载时：冲突开销较大，效率极低，时间很多浪费在冲突中

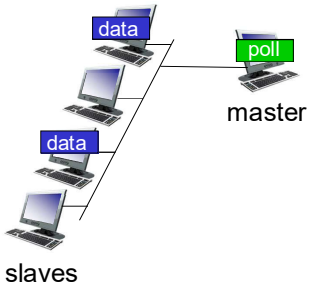
轮流协议

有2者的优点!

轮流MAC协议

论询:

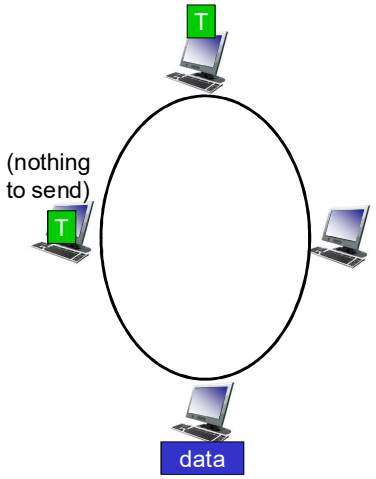
- 主节点邀请从节点依次传送
- 从节点一般比较“dumb”
- 缺点:
  - 轮询开销：轮询本身消耗信道带宽
  - 等待时间：每个节点需等到主节点轮询后开始传输，即使只有一个节点，也需要等到轮询一周后才能够发送
  - 单点故障：主节点失效时造成整个系统无法工作



轮流MAC协议

令牌传递:

- 控制令牌(token)循环从一个节点到下一个节点传递
- 令牌报文：特殊的帧
- 缺点:
  - 令牌开销：本身消耗带宽
  - 延迟：只有等到抓住令牌，才可传输
  - 单点故障 (token):
    - 令牌丢失系统级故障，整个系统无法传输
    - 复杂机制重新生成令牌



MAC 协议总结

- 多点接入问题：对于一个共享型介质，各个节点如何协调对它的访问和使用?
  - 信道划分：按时间、频率或者编码
    - TDMA、FDMA、CDMA
  - 随机访问 (动态)
    - ALOHA, S-ALOHA, CSMA, CSMA/CD
    - 载波侦听: 在有些介质上很容易 (wire：有线介质), 但在有些介质上比较困难 (wireless：无线)
    - CSMA/CD ： 802.3 Ethernet网中使用
    - CSMA/CA ： 802.11WLAN中使用
  - 依次轮流协议
    - 集中：由一个中心节点轮询；分布：通过令牌控制
    - 蓝牙、FDDI、令牌环

内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

MAC 地址和ARP

- 32bitIP地址：
  - 网络层地址
  - 前n-1跳：用于使数据报到达目的IP子网
  - 最后一跳：到达子网中的目标节点
- LAN（MAC/物理/以太网）地址：
  - 用于使帧从一个网卡传递到与其物理连接的另一个网卡(在同一个物理网络中)
  - 48bit MAC地址固化在适配器的ROM，有时也可以通过软件设定
  - 理论上全球任何2个网卡的MAC地址都不相同
  - e.g.: 1A-2F-BB-76-09-AD

16进制表示（每一位代表4个bits）

网络地址和mac地址分离

1. IP地址和MAC地址的作用不同
   a) IP地址是分层的
      - 一个子网所有站点网络号一致，路由聚集，减少路由表
        - 需要一个网络中的站点地址网络号一致，如果捆绑需要定制网卡非常麻烦
      - 希望网络层地址是配置的；IP地址完成网络到网络的交付

   b) mac地址是一个平面的
      - 网卡在生产时不知道被用于哪个网络，因此给网卡一个唯一的标示，用于区分一个网络内部不同的网卡即可
      - 可以完成一个物理网络内部的节点到节点的数据交付

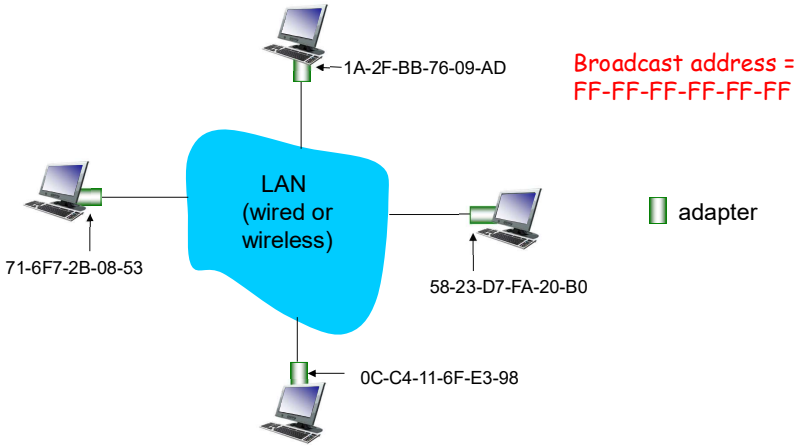
网络地址和mac地址分离

1. 分离好处
   a) 网卡坏了，ip不变，可以捆绑到另外一个网卡的mac上
   b) 物理网络还可以除IP之外支持其他网络层协议，链路协议为任意 上层网络协议， 如IPX等

2. 捆绑的问题
   a) 如果仅仅使用IP地址，不用mac地址，那么它仅支持IP协议
   b) 每次上电都要重新写入网卡 IP地址；
   c) 另外一个选择就是不使用任何地址；不用MAC地址，则每到来一个帧都要上传到IP层次，由它判断是不是需要接受，干扰一次

LAN 地址和ARP

局域网上每个适配器都有一个唯一的LAN地址

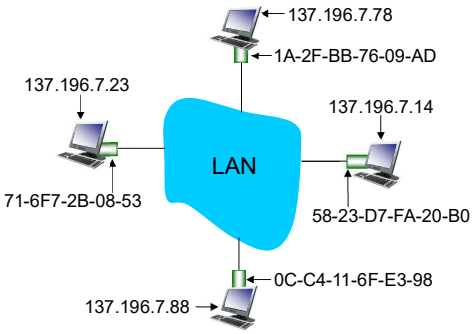


LAN 地址和ARP（续）

- MAC地址由IEEE管理和分配
- 制造商购入MAC地址空间（保证唯一性）
- 类比:
  - (a)MAC地址：社会安全号
  - (b)IP地址：通讯地址
- MAC平面地址 ➜ 支持移动
  - 可以将网卡到接到其它网络
- IP地址有层次-不能移动
  - 依赖于节点连接的IP子网，与子网的网络号相同（有与其相连的子网相同的网络前缀）

ARP: Address Resolution Protocol

问题:已知B的IP地址，如何确定B的MAC地址?



- 在LAN上的每个IP节点都有一个ARP表
- ARP表：包括一些LAN节点IP/MAC地址的映射

  < IP address; MAC address; TTL>
  - TTL时间是指地址映射失效的时间
  - 典型是20min

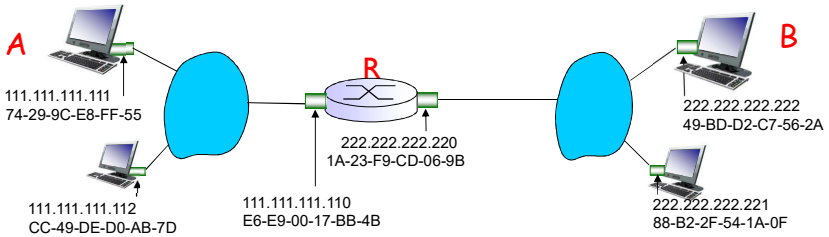
ARP协议：在同一个LAN (网络)

- A要发送帧给B(B的IP地址已知)， 但B的MAC地址不在A的ARP表中
- A广播包含B的IP地址的ARP查询包
  - Dest MAC address = FF-FF-FF-FF-FF-FF
  - LAN上的所有节点都会收到该查询包
- B接收到ARP包，回复A自己的MAC地址
  - 帧发送给A
  - 用A的MAC地址（单播）
- A在自己的ARP表中，缓存IP-to-MAC地址映射关系，直到信息超时
  - 软状态: 靠定期刷新维持的系统状态
  - 定期刷新周期之间维护的状态信息可能和原有系统不一致
- ARP是即插即用的
  - 节点自己创建ARP的表项
  - 无需网络管理员的干预

路由到其他LAN

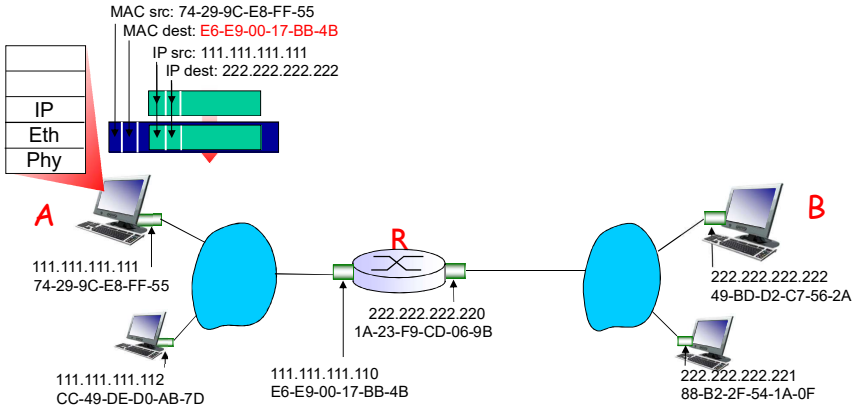
Walkthrough :发送数据报：由A通过R到B，假设A知道B的IP地址

- 在R上有两个ARP表，分别对应两个LAN
- 在源主机的路由表中，发现到目标主机的下一跳时111.111.111.110
- 在源主机的ARP表中，发现其MAC地址是E6-E9-00-17-BB-4B, etc



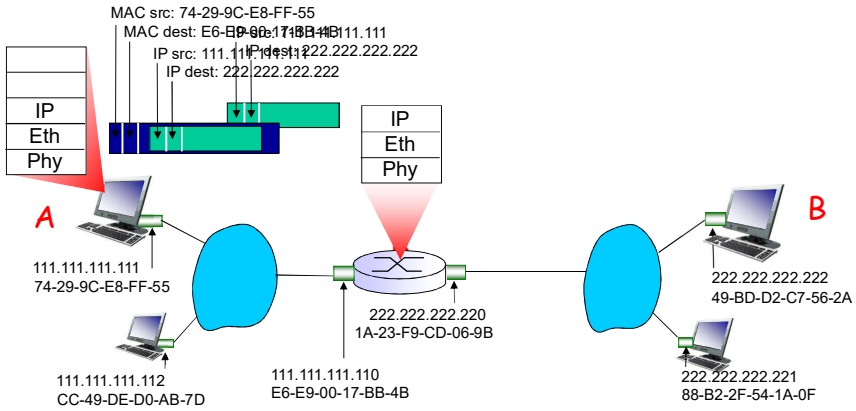
编址：路由到其他LAN

- A创建数据报，源IP地址：A；目标IP地址：B
- A创建一个链路层的帧，目标MAC地址是R，该帧包含A到B的IP数据报



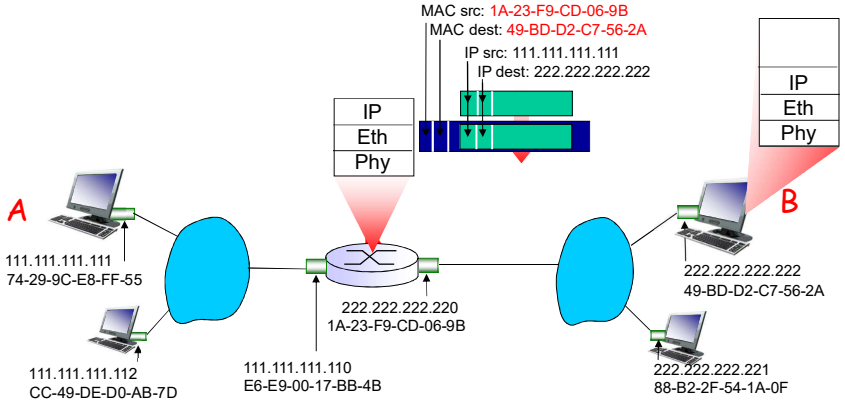
编址：路由到其他LAN

- 帧从A发送到R
- 帧被R接收到，从中提取出IP分组，交给上层IP协议实体



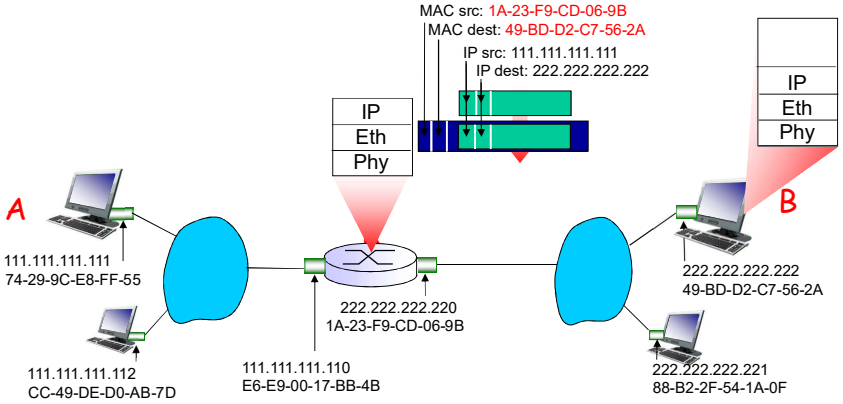
编址：路由到其他LAN

- R转发数据报，数据报源IP地址为A，目标IP地址为B
- R创建一个链路层的帧，目标MAC地址为B，帧中包含 A到B的IP 数据报



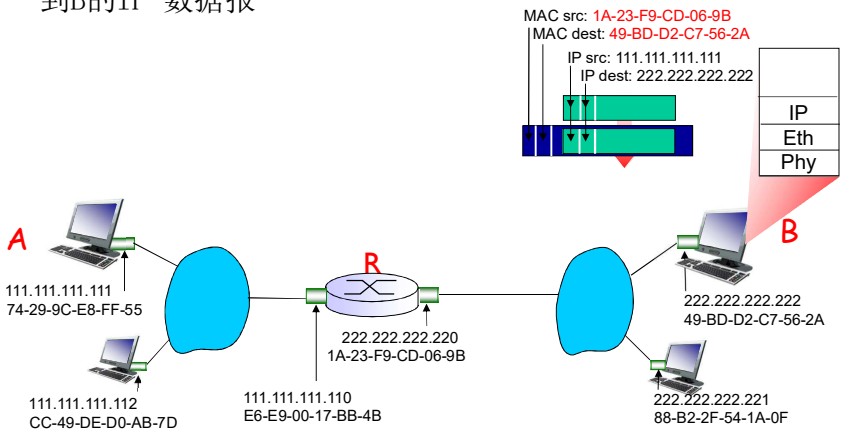
编址：路由到其他LAN

- R 转发数据报，源IP地址为A，目标IP地址为B
- R创建一个链路层的帧，目标MAC地址为B，帧中包含 A到B的IP 数据报



编址：路由到其他LAN

- R 转发数据报，源IP地址为A，目标IP地址为B
- R创建一个链路层的帧，目标MAC地址为B，帧中包含 A到B的IP 数据报



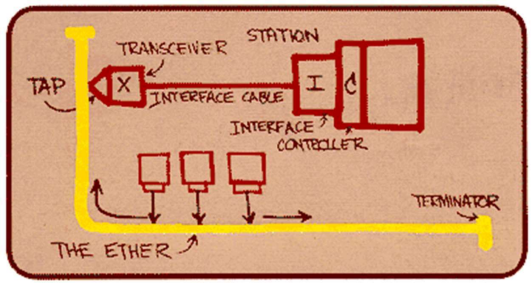
内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

以太网

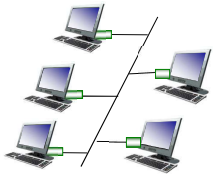
- 目前最主流的LAN技术：98%占有率
- 廉价：30元RMB 100Mbps!
- 最早广泛应用的LAN技术
- 比令牌网和ATM网络简单、廉价
- 带宽不断提升：10M, 100M, 1G, 10G



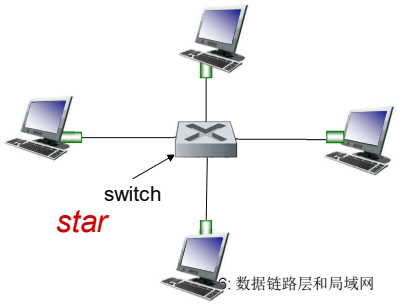
Metcalfe's Ethernet sketch

以太网：物理拓扑

- 总线：在上个世纪90年代中期很流行
  - 所有节点在一个碰撞域内，一次只允许一个节点发送
  - 可靠性差，如果介质破损，截面形成信号的反射，发送节点误认为是冲突，总是冲突
- 星型：目前最主流
  - 连接选择: hub 或者 switch
  - 现在一般是交换机在中心
  - 每个节点以及相连的交换机端口使用（独立的）以太网协议（不会和其他节点的发送产生碰撞）



bus: coaxial cable



以太帧结构

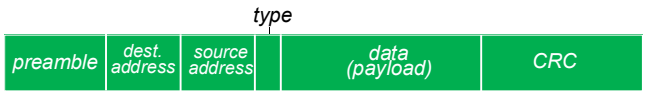
发送方适配器在以太网帧中封装IP数据报，或其他网络层协议数据单元

前导码:

- 7B 10101010 + 1B 10101011
- 用来同步接收方和发送方的时钟速率
  - 使得接收方将自己的时钟调到发送端的时钟
  - 从而可以按照发送端的时钟来接收所发送的帧

以太帧结构（续）

- 地址：6字节源MAC地址，目标MAC地址
  - 如：帧目标地址=本站MAC地址，或是广播地址，接收，递交帧中的数据到网络层
  - 否则，适配器忽略该帧
- 类型：指出高层协(大多情况下是IP，但也支持其它网络层协议Novell IPX和AppleTalk)
- CRC：在接收方校验
  - 如果没有通过校验，丢弃错误帧

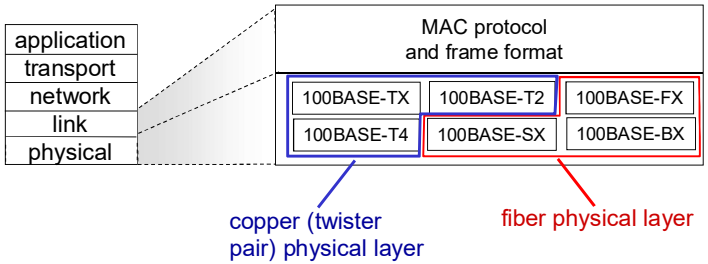


以太网：无连接、不可靠的服务

- 无连接：帧传输前，发送方和接收方之间没有握手
- 不可靠：接收方适配器不发送ACKs或NAKs给发送方
  - 递交给网络层的数据报流可能有gap
  - 如上层使用像传输层TCP协议这样的rdt，gap会被补上(源主机，TCP实体)
  - 否则，应用层就会看到gap
- 以太网的MAC协议：采用二进制退避的CSMA/CD介质访问控制形式

802.3 以太网标准：链路和物理层

- *很多*不同的以太网标准
  - 相同的MAC协议（介质访问控制）和帧结构
  - 不同的速率：2 Mbps、10 Mbps 、100 Mbps 、 1Gbps 、 10G bps
  - 不同的物理层标准
  - 不同的物理层媒介：光纤，同轴电缆和双绞线



以太网使用CSMA/CD

- 没有时隙
- NIC如果侦听到其它NIC在发送就不发送：载波侦听 carrier sense
- 发送时，适配器当侦听到其它适配器在发送就放弃对当前帧的发送，冲突检测 collision detection
- 冲突后尝试重传，重传前适配器等待一个随机时间，随机访问random access

以太网CSMA/CD算法

1. 适配器获取数据报，创建帧
2. 发送前：侦听信道CS
   1)闲：开始传送帧
   2)忙：一直等到闲再发送
3. 发送过程中，冲突检测CD
   1)没有冲突:成功
   2)检测到冲突:放弃,之后尝试重发
4. 发送方适配器检测到冲突，除放弃外，还发送一个Jam信号，所有听到冲突的适配器也是如此
   强化冲突：让所有站点都知道冲突
5. 如果放弃，适配器进入指数退避状态
   在第m次失败后，适配器随机选择一个{0，1，2，，2^{m-1}}中K，等待K*512位时，然后转到步骤2

exponential backoff

二进制指数退避算法

以太网CSMA/CD算法（续）

Jam Signal: 使其它发送方明确知道发生碰撞，48bits

Bit time: 10Mbps以太网 1/10M=0.1us

对于K= 1023，大约等50ms：最坏情况

1023*512*0.1us=50ms

See/interact with Java applet on AWL Web site: highly recommended !

指数退避:

- 目标：适配器试图适应当前负载，在一个变化的碰撞窗口中随机选择时间点尝试重发
  - 高负载：重传窗口时间大，减少冲突，但等待时间长
  - 低负载：使得各站点等待时间少，但冲突概率大
- 首次碰撞：在{0，1}选择K；延迟K*512位时
- 第2次碰撞：在{0，1，2，3}选择K
- 第10次碰撞：在{0，1，2，3，……，1023}选择K

CSMA/CD的效率

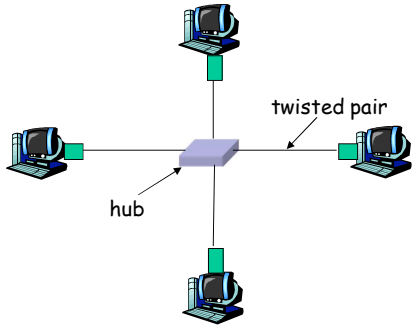
- t_{prop} = LAN上最远2个节点的传播时间
- t_{trans} =最大帧的传输时间

$$efficiency = \frac{1}{1 + 5t_{prop} / t_{trans}}$$

- 效率在 t_{prop}为0时接近1: 不会有冲突
- 当t_{trans} 发送接近无穷大时接近1
  - 一个站点一旦抓住信道，则永远发送
- 比ALOHOA要好：简单、廉价、分布式控制

10BaseT and 100BaseT

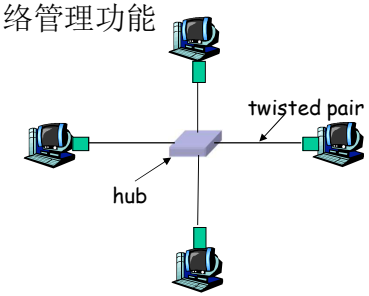
- 100 Mbps 速率 也被称之为 "fast ethernet"
- T代表双绞线
- 节点连接到HUB上: "star topology"物理上星型
  - 逻辑上总线型，盒中总线
- 节点和HUB间的最大距离是100 m



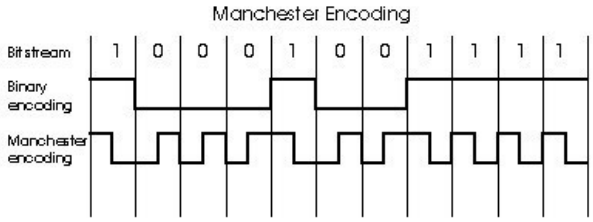
Hubs

Hubs 本质上是物理层的中继器:

- 从一个端口收，转发到所有其他端口
- 速率一致
- 没有帧的缓存
- 在hub端口上没有CSMA/CD机制:适配器检测冲突
- 提供网络管理功能



Manchester 编码



- 在 10BaseT中使用
- 每一个bit的位时中间有一个信号跳变
- 允许在接收方和发送方节点之间进行时钟同步
  - 节点间不需要集中的和全局的时钟
- 10Mbps，使用20M带宽，效率50%
- Hey, this is physical-layer stuff!

100BaseT中的4b5b编码

- 0000->11110
- 0001->01001
- 0010->10100
- 0011->10101
- 0100->01010
- 0101->01011
- 0110->01110
- 0111->01111
- 1000->10010
- 1001->10011
- 1010->10110
- 1011->10111
- 1100->11010
- 1101->11011
- 1110->11100
- 1111->11101

千兆以太网

- 采用标准的以太帧格式
- 允许点对点链路和共享广播信道
- 物理编码：8b10b编码
- 在共享模式，继续使用CSMA/CD MAC技术，节点间需要较短距离以提高利用率
- 交换模式：全双工千兆可用于点对点链路
  - 站点使用专用信道，基本不会冲突，效率高
  - 除非发往同一个目标站点
- 10 Gbps now !

内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

IEEE 802.11 Wireless LAN

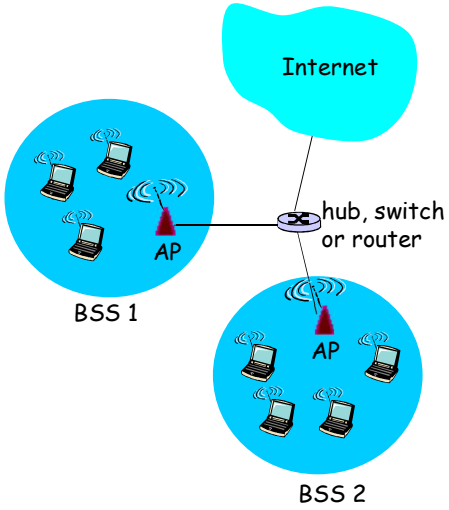
- 802.11b
  - 使用无需许可的2.4-5 GHz 频谱
    - ✓ 无绳电话和微波炉
  - 最高11 Mbps
  - 在物理层采用直接序列扩频 direct sequence spread spectrum (DSSS)
    - ✓所有的主机采用同样的序列码

不同：速率，物理层
相同：MAC, 帧格式

- 802.11a
  - 更高频率5-6 GHz
  - 最高54 Mbps
  - 距离相对短，受多路径影响大
- 802.11g
  - 频率2.4-5 GHz
  - 最大54 Mbps
  - 与802.11b向后兼容
- 802.11n: 多天线MIMO
  - 频率2.4-5 GHz
  - 最高200 Mbps

- 所有的802.11标准都是用CSMA/CA进行多路访问
- 所有的802.11标准都有基站模式和自组织网络模式

802.11 LAN 体系结构

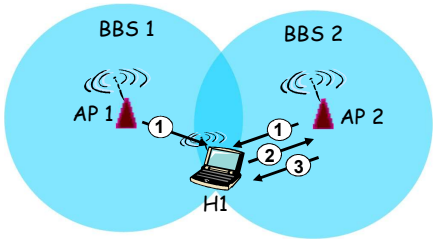


- 无线主机与基站通信
  - 基站base station = 接入点access point (AP)
- 基础设施模式下的基本服务集Basic Service Set (BSS) (aka"cell") 包括以下构件:
  - 无线主机
  - 接入点(AP): 基站
  - 自组织模式下：只有无线主机

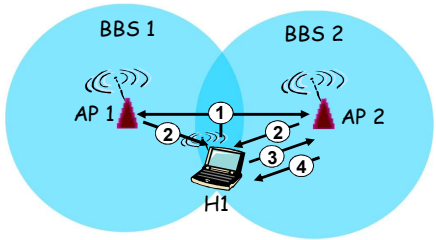
802.11: 信道与关联

- 802.11b: 2.4GHz-2.485GHz 频谱被分为11个相互不同的但是部分重叠的频段
  - AP管理员为AP选择一个频率
  - 可能的干扰: 邻居AP可能选择同样一个信道!
- 主机: 必须在通信之前和AP建立*associate*
  - 扫描所有的信道，侦听包含AP SSID和MAC地址的信标帧
    - ✓ 主动扫描：主机发送探测，接受AP的响应
    - ✓ 被动扫描
  - 选择希望关联的AP
  - 可能需要执行鉴别（认证）[Chapter 8]
    - ✓ 基于MAC、用户名口令
    - ✓ 通过AP的中继，使用RADIUS鉴别服务器进行身份鉴别
  - 将会执行DHCP获得IP地址和AP所在的子网前缀

802.11: 被动/主动扫描

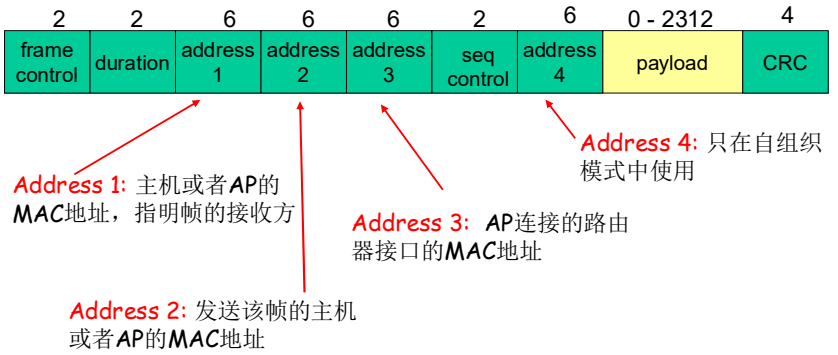


被动扫描:
(1) AP发送信标帧
(2) 关联请求帧的发送：H1向拟关联的AP
(3) 关联响应帧的发送: AP向H1

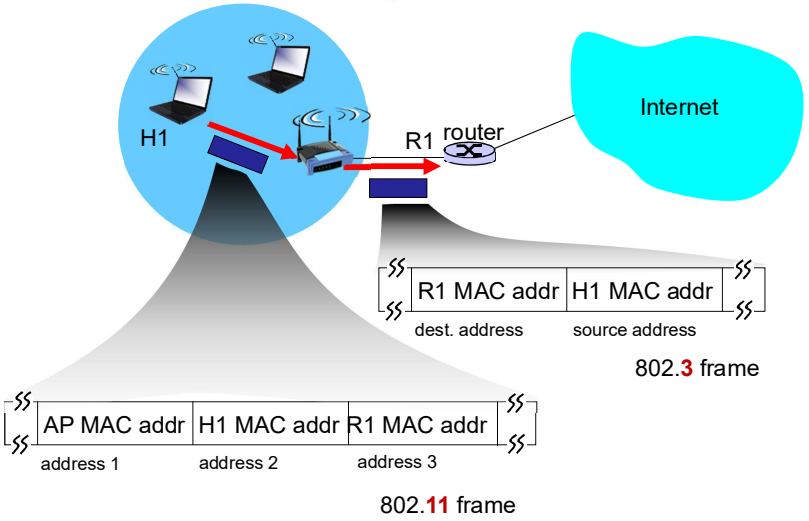


主动扫描:
(1) H1广播探测请求帧
(2) 自AP发送探测响应
(3) H1向选择的AP发送关联请求帧
(4) 选择的AP向H1发送关联的响应帧

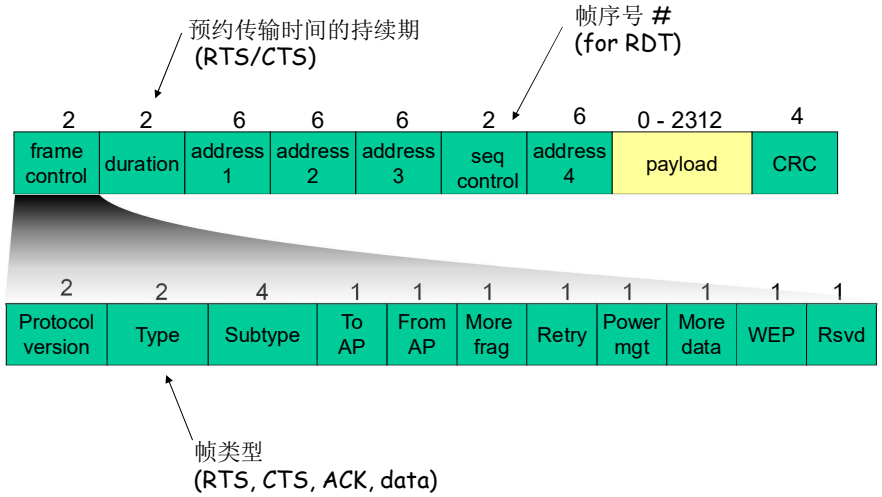
802.11 帧：地址



802.11 帧: 地址



802.11 帧: 续



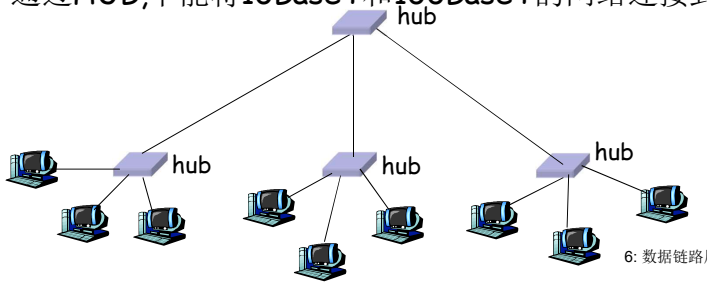
内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

Hub：集线器

- 网段(LAN segments)：可以允许一个站点发送的网络范围
  - 在一个碰撞域，同时只允许一个站点在发送
  - 如果有2个节点同时发送，则会碰撞
  - 通常拥有相同的前缀，比IP子网更详细的前缀
- 所有以hub连到一起的站点处在一个网段，处在一个碰撞域
  - 骨干hub将所有网段连到了一起
- 通过hub可扩展节点之间的最大距离
- 通过HUB,不能将10BaseT和100BaseT的网络连接到一起

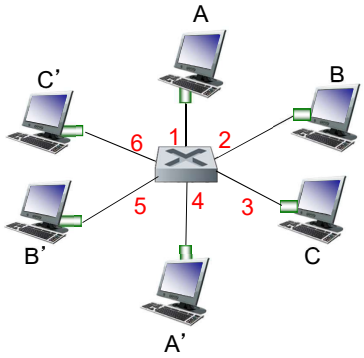


交换机

- 链路层设备：扮演主动角色（端口执行以太网协议）
  - 对帧进行存储和转发
  - 对于到来的帧，检查帧头，根据目标MAC地址进行选择性转发
  - 当帧需要向某个（些）网段进行转发，需要使用CSMA/CD进行接入控制
  - 通常一个交换机端口一个独立网段
- 透明：主机对交换机的存在可以不关心
  - 通过交换机相联的各节点好像这些站点是直接相联的一样
  - 有MAC地址；无IP地址
- 即插即用，自学习：
  - 交换机无需配置

交换机：多路同时传输

- 主机有一个专用和直接到交换机的连接
- 交换机缓存到来的帧
- 对每个帧进入的链路使用以太网协议，没有碰撞；全双工
  - 每条链路都是一个独立的碰撞域
  - MAC协议在其中的作用弱化了
- *交换：* A-to-A' 和 B-to-B' 可以同时传输，没有碰撞



switch with six interfaces (1,2,3,4,5,6)

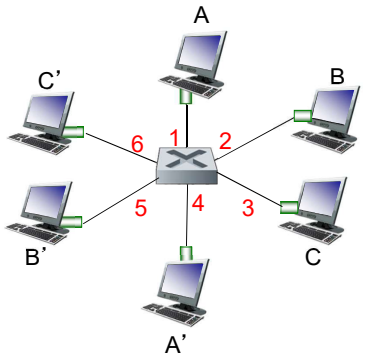
交换机转发表

Q: 交换机如何知道通过接口1到达A，通过接口5到达 B’？

- *A:* 每个交换机都有一个交换表 switch table，每个表项：
  - (主机的MAC地址,到达该MAC经过的接口，时戳)
  - 比较像路由表!

Q: 每个表项是如何创建的？如何维护的？

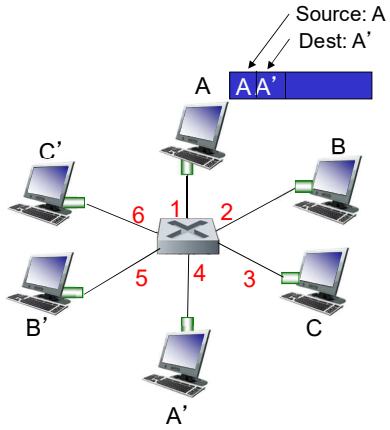
- 有点像路由协议?



switch with six interfaces (1,2,3,4,5,6)

交换机：自学习

- 交换机通过学习得到哪些主机（mac地址）可以通过哪些端口到达
  - 当接收到帧，交换机学习到发送站点所在的端口（网段）
  - 记录发送方MAC地址/进入端口映射关系，在交换表中



MAC addr	interface	TTL
A	*1*	*60*

Switch table (initially empty)

交换机：过滤 / 转发

当交换机收到一个帧：

1.记录进入链路，发送主机的MAC地址

2.使用目标MAC地址对交换表进行索引

3.if entry found for destination
then{
if dest on segment from which frame arrived
then drop the frame
else forward the frame on interface indicated
}
else flood

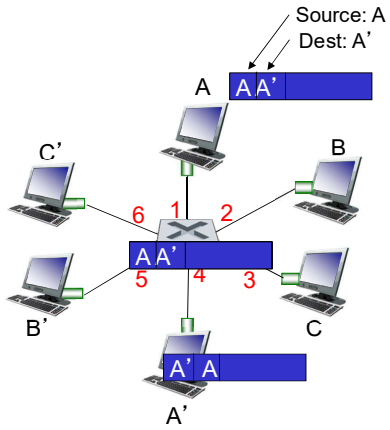
过滤

转发

泛洪：除了帧到达的网段，向所有网络接口

自学习，转发的例子

- 帧的目标： A', 不知道其位置在哪：*泛洪*
- 知道目标A对应的链路：选择性发送到那个端口

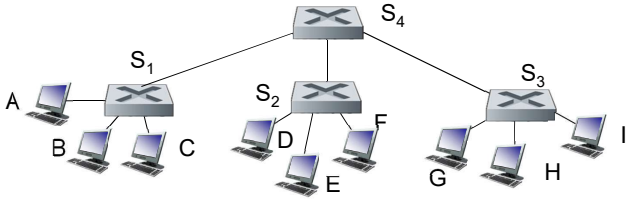


MAC addr	interface	TTL
A	1	60
A '	4	60

switch table (initially empty)

交换机级联

- 交换机可被级联到一起

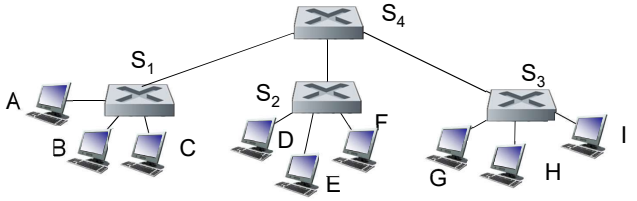


Q: A to G的发送 - 交换机S_1 如何知道经过从S_4 和S_3最终达到F?

- A: 自学习！（和在一个交换机联接所有站点一样!）

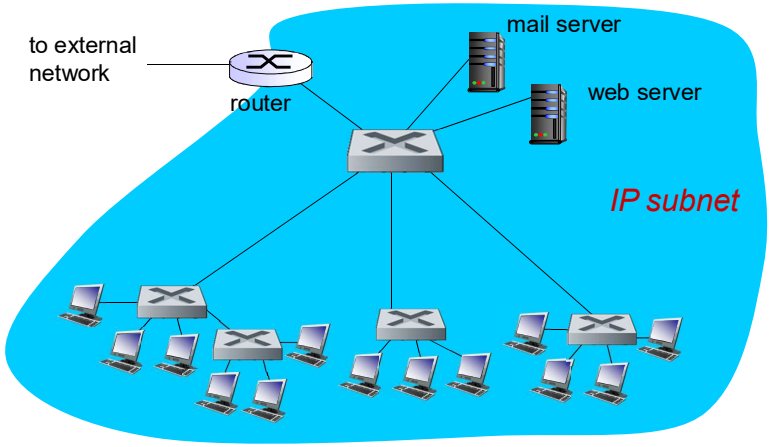
多交换机自学习的例子

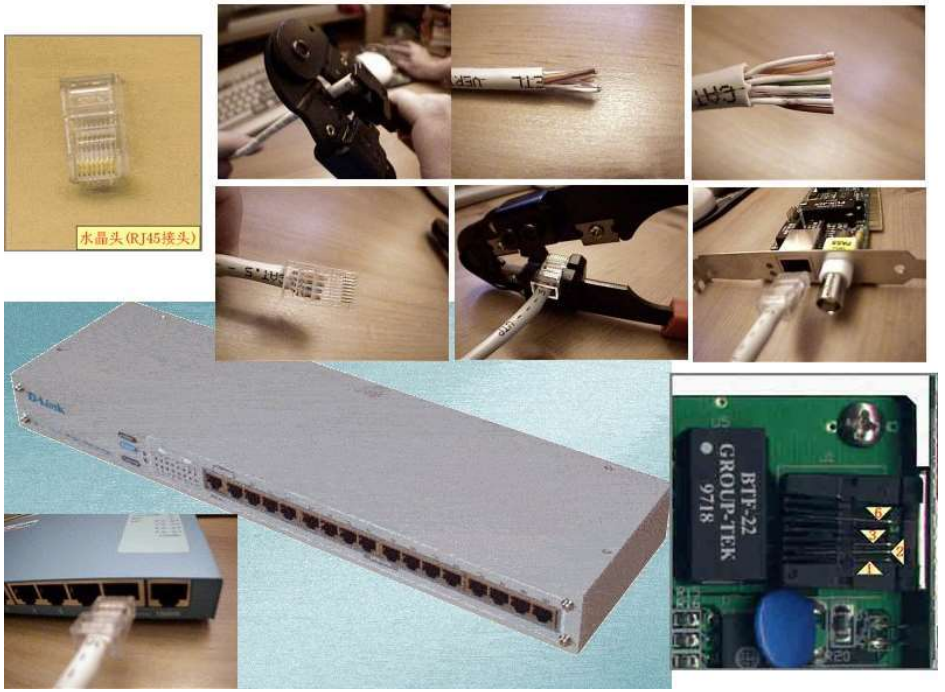
假设C向I发送帧，I给C应答

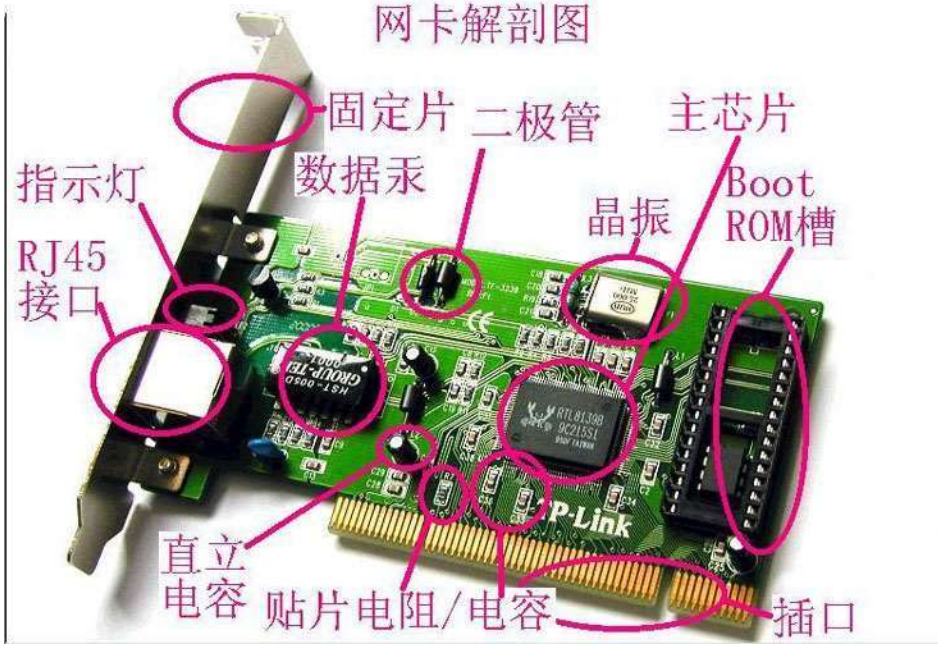


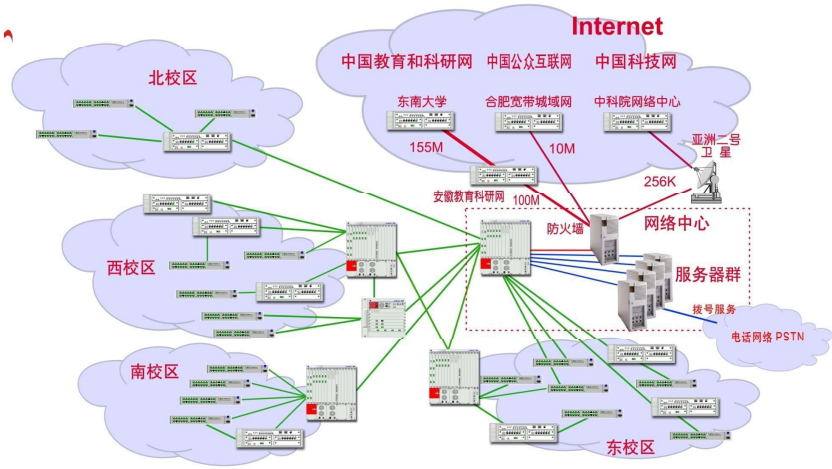
- Q: 显示交换表和帧在S_1, S_2, S_3, S_4的转发

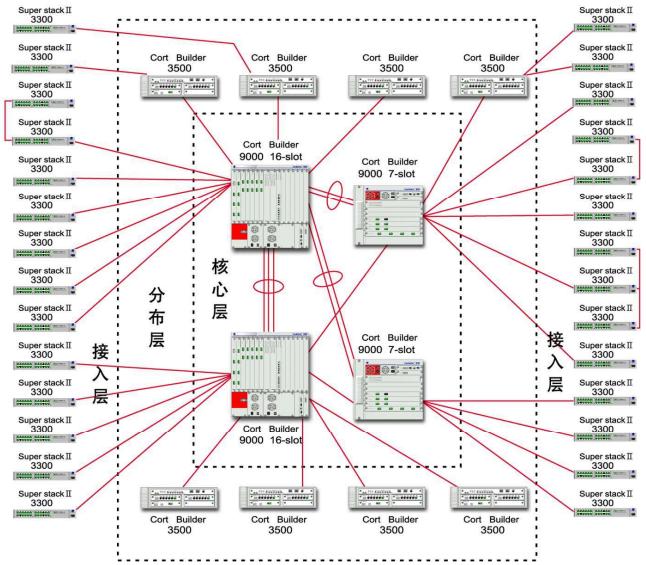
机构网络例子


to external
network
router
mail server
web server
IP subnet


水晶头(RJ45接头)
BTF-22
GROUP-TEK
9718

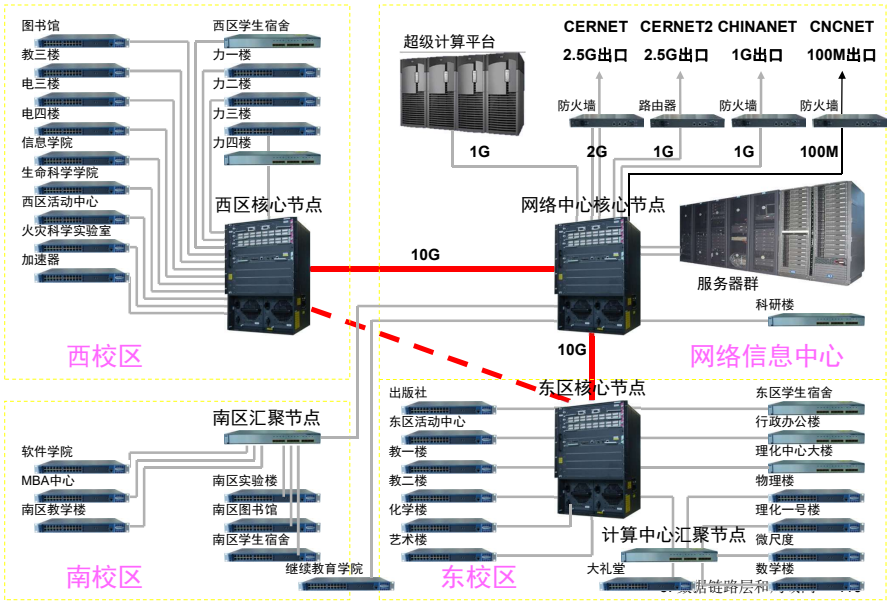

网卡解剖图
固定片
二极管
主芯片
指示灯
数据汞
晶振
Boot
ROM槽
RJ45
接口
直立
电容
贴片电阻/电容
插口
P-Link


中国科大校园网络拓扑结构示意图
Internet
中国教育和科研网
中国公众互联网
中国科技网
北校区
东南大学
合肥宽带城域网
中科院网络中心
155M
10M
256K
亚洲二号
卫星
安徽教育科研网
100M
防火墙
网络中心
服务器群
西校区
拨号服务
电话网络 PSTN
南校区
东校区

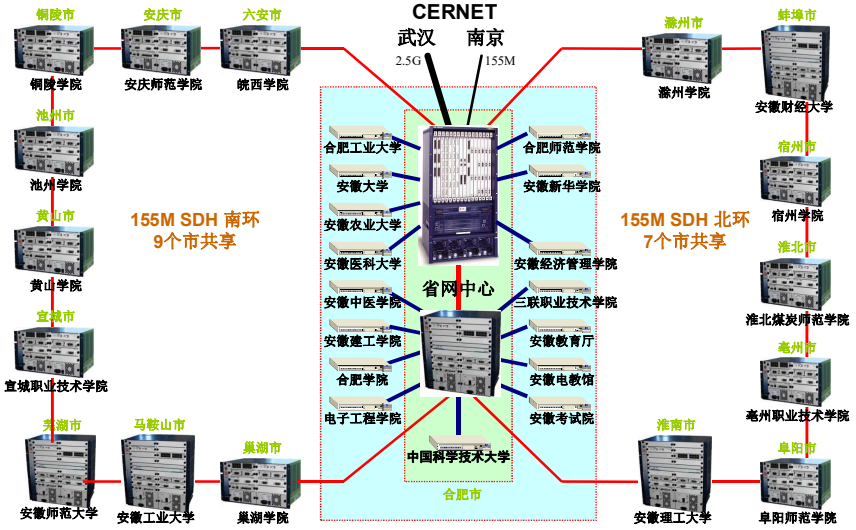


中国科大校园网主干改造方案物理结构图

中国科学技术大学校园网络主干示意图

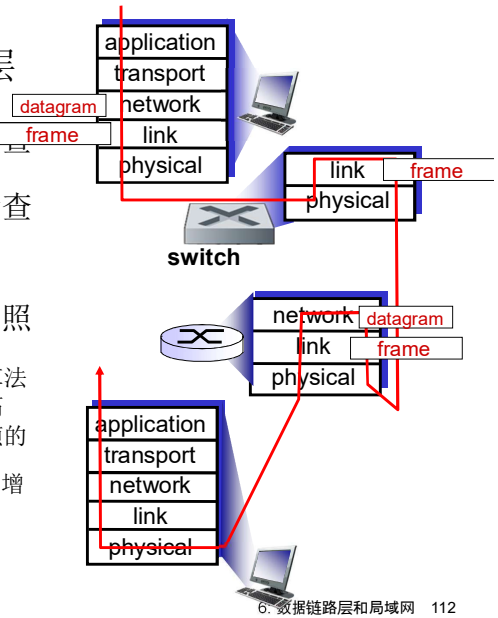


安徽省教育科研网络主干示意图



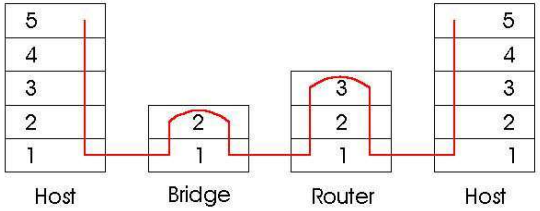
交换机 vs. 路由器

- 都是存储转发设备，但层次不同
  - 交换机：链路层设备（检查链路层头部）
  - 路由器：网络层设备（检查网络层的头部）
- 都有转发表：
  - 交换机：维护交换表，按照MAC地址转发
    - 执行过滤、自学习和生成树算法
    - 即插即用；二层设备，速率高
    - 执行生成树算法，限制广播帧的转发
    - ARP表项随着站点数量增多而增多
  - 路由器：续



交换机 vs. 路由器

- 路由器维护路由表，执行路由算法
  - 路由算法能够避免环路，无需执行生成树算法，可以以各种拓扑构建网络
  - 对广播分组做限制
  - 不是即插即用的，配置网络地址（子网前缀）
  - 三层设备，速率低

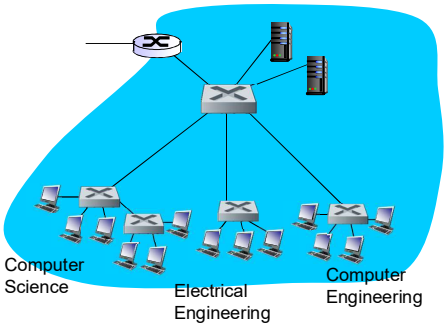


内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- 802.11WLAN
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

VLANs：动机



考虑场景：
- CS用户搬到EE大楼办公室，但是希望连接到CS的交换机?
- 接到多个交换机上
  - 麻烦和浪费：96端口/10个有用
- 如果都接到一个交换机上，在一个广播域
  - 所有的层2广播流量(ARP，DHCP，不知道MAC地址对应端口的帧)都必须穿过整个LAN
  - 安全性/私密性的问题

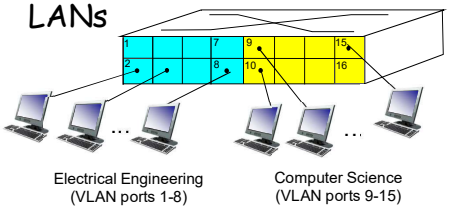
VLANs

Virtual Local Area Network

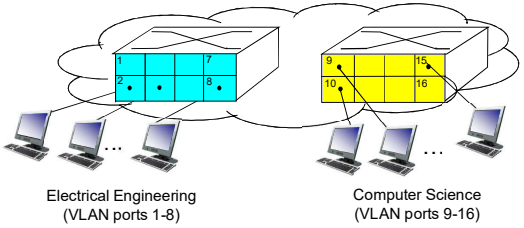
带有VLAN功能的交换机（们）可以被配置成：一个物理LAN基础设施，虚拟成多个LANs

物理上一个交换机
虚拟成多个局域网

基于端口的VLAN: 交换机端口成组(通过交换机管理软件)，以至于单个的交换机可以分成若干虚拟LANs

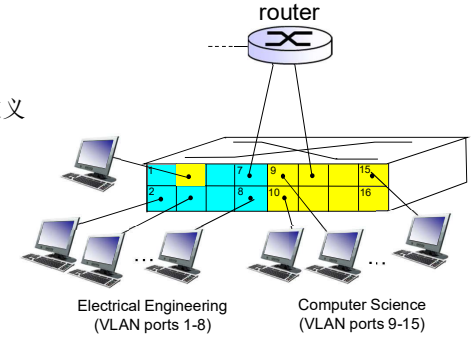


… 就像多个虚拟的交换机

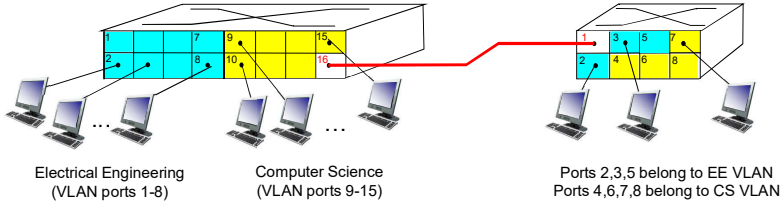


基于端口的VLAN

- *流量隔离：* 从/到1-8端口的流量只会涉及到1-8
  - 也可以基于MAC地址进行VLAN定义
- 动态成员：成员可以在VLANs之间动态分配
- 在VLANs间转发：通过路由器进行转发（就像他们通过各自的交换机相联一样）
  - 实际操作中，设备生产商可以提供：交换机和路由器的单一设备

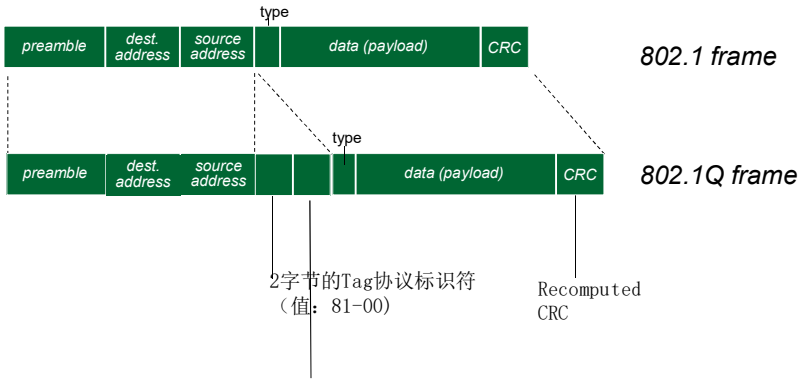


VLANs 互联多个交换机



- 如果有多个交换机，希望它们相连并且共享VLANs信息
- 方法1：各交换机每个VLAN一个端口和另外交换机相应VLAN端口相连->扩展性问题
- *trunk port干线端口:* 多个交换机共享定义的VLAN，在它们之间传输帧
  - 帧在不同交换机上的一个VLAN上转发，不能够再使用vanilla 802.1帧 (必须要携带VLAN ID信息)
  - 802.1q协议增加/移除附加的头部字段，用于在trunk端口上进行帧的转发

802.1Q VLAN 帧格式



内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

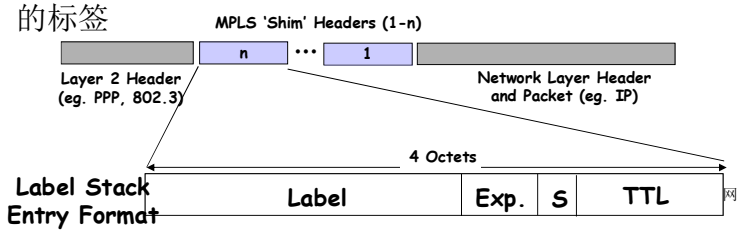
MPLS概述

- 从IP网络来看，将一组支持MPLS的网络虚拟成链路的技术
- 纯IP网络是按照IP地址对分组进行转发的
  - 前缀匹配，转发的方法固定
  - 无法控制IP分组的路径，无法支持流连工程
  - 也无法对一个IP分组流进行资源分配，性能无法保证
- MPLS网络按照标签label进行分组的转发
  - 类似于VC
  - 有基于标签的转发表
  - 基于虚电路表，IP vs 线路交换

in label	out label	dest	out interface
10	6	A	1
12	9	D	0

MPLS概述

- 标签交换的过程：
  - 入口路由器:LER对进入的分组按照EFC的定义打上标签
  - 在MPLS网络中（虚拟成了链路）对分组按照标签进行交换
  - 到了出口路由器，再将标签摘除
  - 支持MPLS的路由器组构成的网络，从IP网络的角度来看虚拟成了链路
- 标签封装：一些列标准定义了在ATM,FR和以太网中如何封装，利用原有网络中的机制VCI，或者定义新的标签



MPLS概述

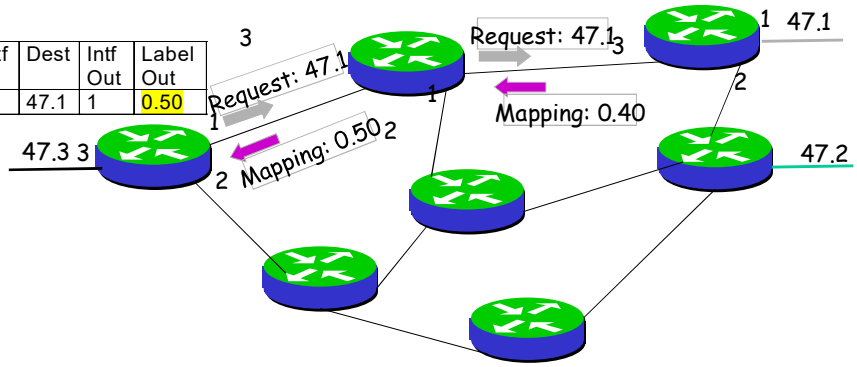
- 建立基于标签的转发表-信令协议：支持逐跳和显式路由：路由信息传播，路由计算(基于Qos，基于策略的)，标签分发
  - LDP CR-LDP
  - RSVP扩展 BGP扩展
- MPLS优点
  - 路由弹性：基于Qos，基于策略的
  - 充分利用已有的硬件ATM，快速转发
  - 支持流连工程，VPN
  - 支持带宽等资源的分配
  - .....

标签分发

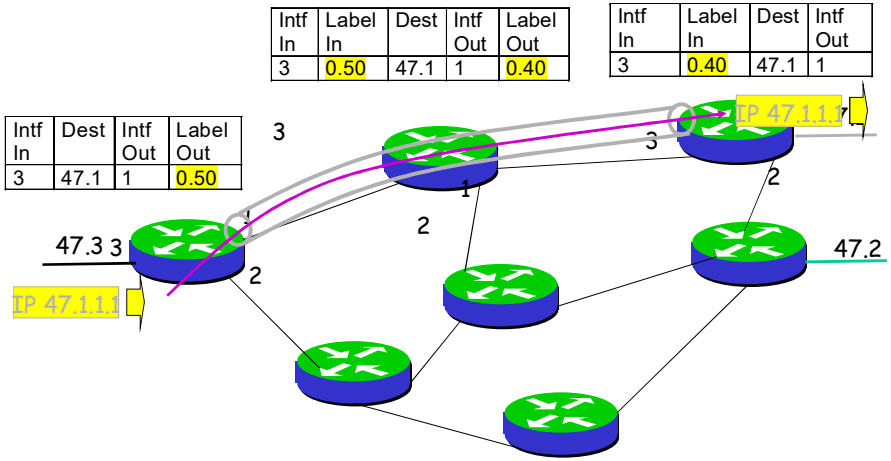
Intf In	Label In	Dest	Intf Out	Label Out
3	0.50	47.1	1	0.40

Intf In	Label In	Dest	Intf Out
3	0.40	47.1	1

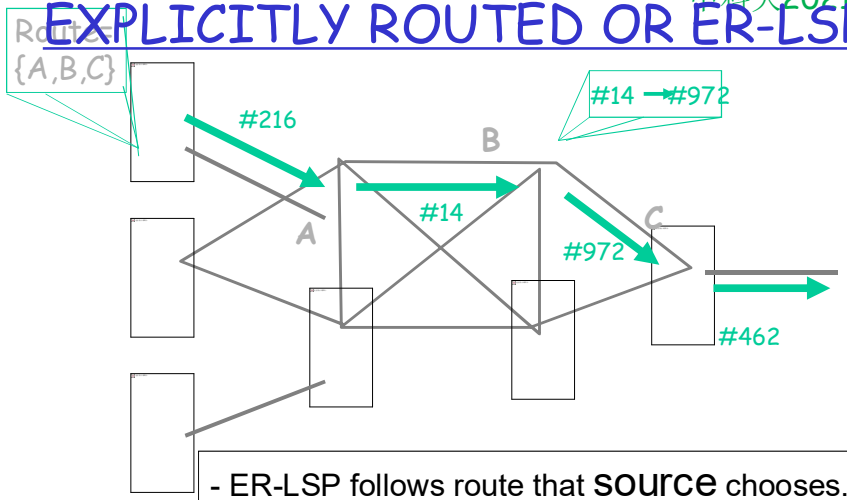
Intf In	Dest	Intf Out	Label Out
3	47.1	1	0.50



Label Switched Path (LSP)

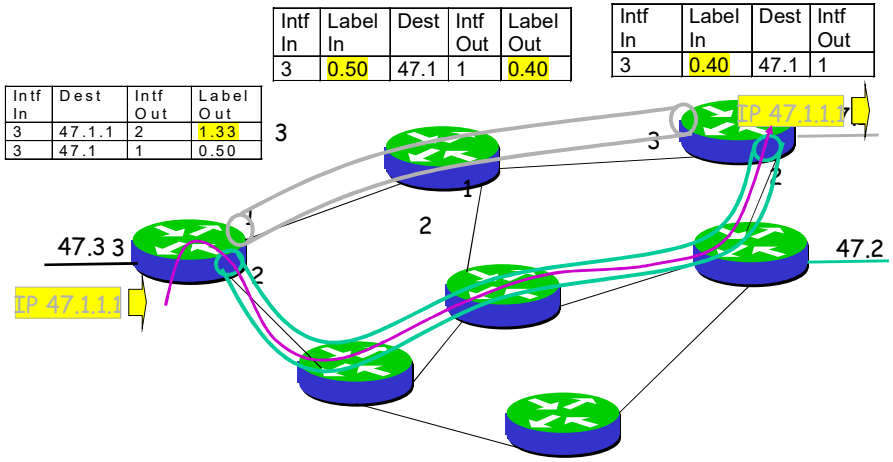


EXPLICITLY ROUTED OR ER-LSP



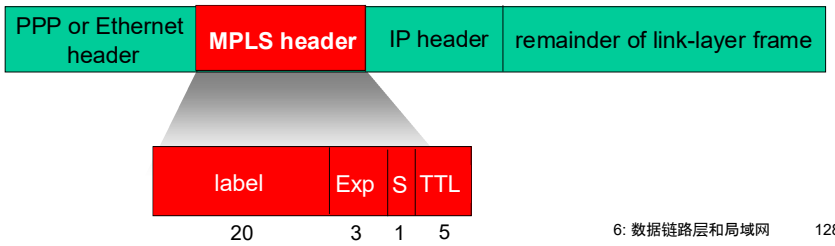
- ER-LSP follows route that **source** chooses. In other words, the control message to establish the LSP (label request) is ***source routed***.

EXPLICITLY ROUTED LSP ER-LSP



Multiprotocol label switching (MPLS)

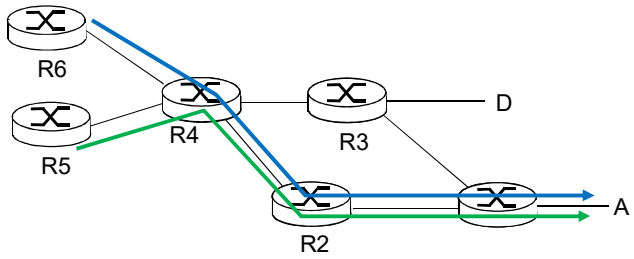
- 初始目的：使用固定长度的标签label进行高速率IP转发（而不是使用IP address,采用最长前缀匹配）
  - 一开始采用固定长度ID进行查表（而不是采用前缀匹配）
  - 借鉴了虚电路的思想（VC）
  - 但是IP数据报仍然保留IP地址！
  - 在帧和其封装的分组之间加入一个垫层，标签交换使能的路由器使用垫层信息进行分组转发，不解析分组目标地址



具有MPLS能力的路由器

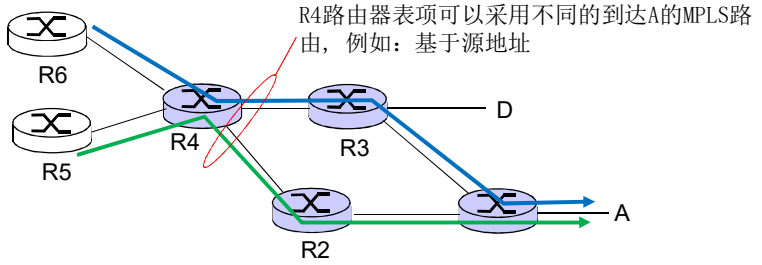
- a.k.a. 标签交换路由器
- 基于标签的值进行分组的转发 (而非检查IP地址)
  - MPLS转发表和IP转发表相互独立
- *弹性:* MPLS转发决策可以和IP不同
  - 采用源地址和目标地址来路由到达同一个目标的流，不同路径（支持流量工程）
  - 如果链路失效，能够快速重新路由: 预先计算好的备份链路 (对于VoIP有效)

MPLS vs IP 路径



- *IP 路由：到达目标的路径仅仅取决于目标地址*

MPLS vs IP 路径

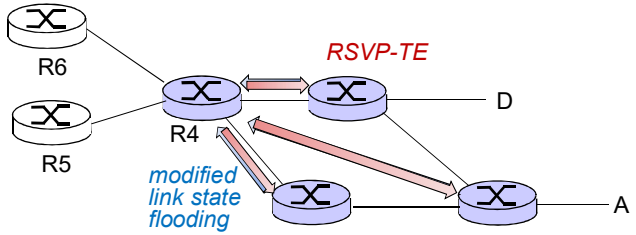


- *IP路由:到达目标的路径仅仅取决于目标地址*

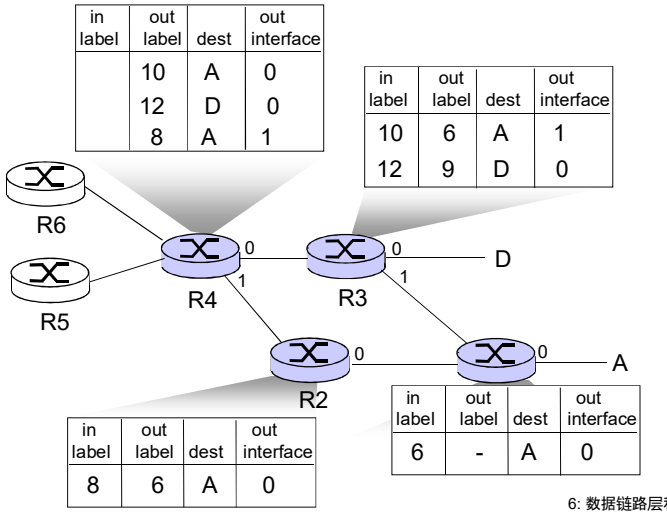
- *MPLS路由：到达目标的路由，可以基于源和目标地址*
  - *快速重新路由：*在链路失效时，采用预先计算好的路径

MPLS 信令

- 修改OSPF, IS-IS链路状态泛洪协议来携带MPLS路由信息
  - *e.g.*, 链路带宽，链路带宽的倒数?
- MPLS使能的路由器采用RSVP-TE信令协议在下游路由器上来建立MPLS转发表



MPLS 转发表



内容提纲

6.1 引论和服务
6.2 差错检测和纠正
6.3 多点访问协议
6.4 LANs
- addressing, ARP
- Ethernet
- switches
- VLANS

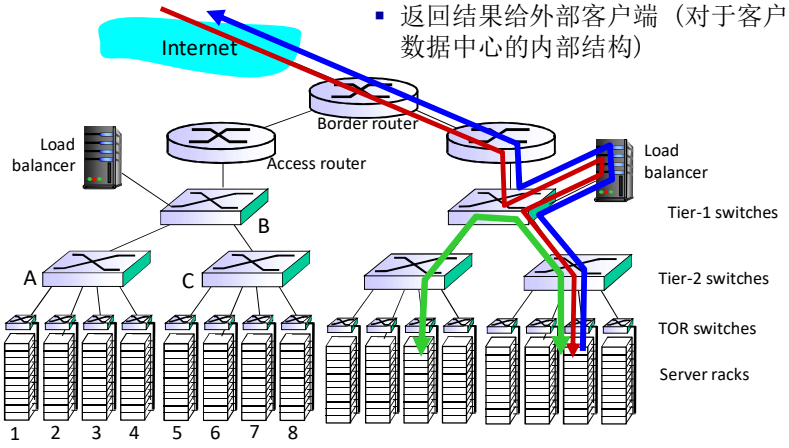
6.5 链路虚拟化：MPLS
6.6 数据中心网络
6.7 a day in the life of web request

数据中心网络

- 数万-数十万台主机构成DC网络，密集耦合、距离临近：
  - 电子商务(e.g. Amazon)
  - 内容服务器(e.g., YouTube, Akamai, Apple, Microsoft)
  - 搜索引擎，数据挖掘 (e.g., Google)
- 挑战：
  - 多种应用，每一种都服务海量的客户端
  - 管理/负载均衡，避免处理、网络和数据的瓶颈

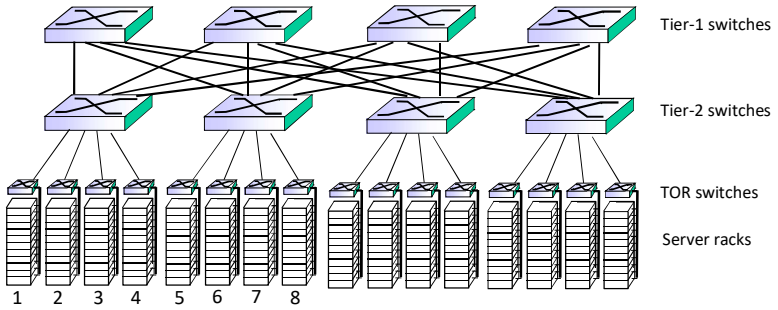
数据中心网络

负载均衡器：应用层路由
- 接受外部的客户端请求
- 将请求导入到数据中心内部
- 返回结果给外部客户端（对于客户端隐藏数据中心的内部结构）



数据中心网络

- 在交换机之间，机器阵列之间有丰富的互连措施：
  - 在阵列之间增加吞吐（多个可能的路由路径）
  - 通过冗余度增加可靠性



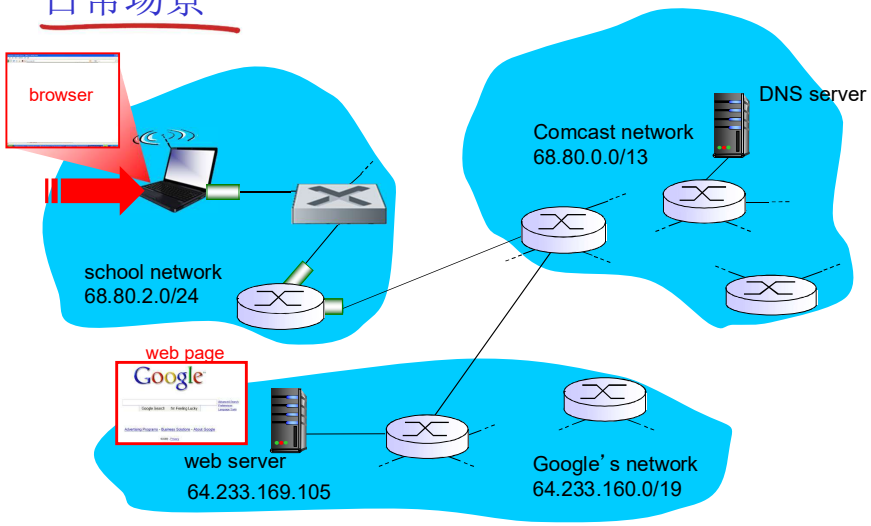
内容提纲

- 6.1 引论和服务
- 6.2 差错检测和纠正
- 6.3 多点访问协议
- 6.4 LANs
  - 编址，ARP
  - 以太网
  - 交换机
  - VLANS
- 6.5 链路虚拟化：MPLS
- 6.6 数据中心网络
- 6.7 a day in the life of web request

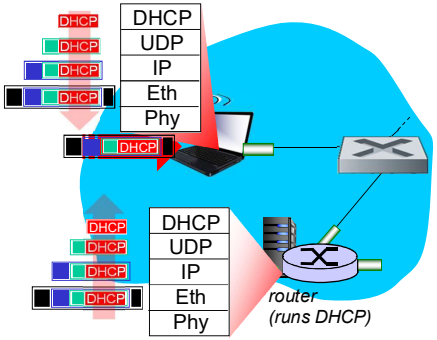
回顾：页面请求的历程

- Top-down的协议栈旅程结束了！
  - 应用层、运输层、网络层和链路层
- 以一个web页面请求的例子：综述！
  - *目标：* 标示、回顾和理解涉及到的协议（所有层次），以一个看似简单的场景：请求www页面
  - *场景：* 学生在校园启动一台笔记本电脑：请求和接受www.google.com

日常场景

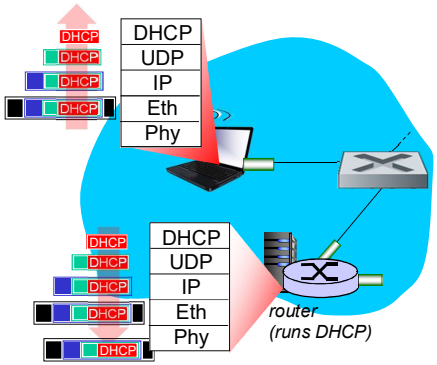


日常场景：... 连接到互联网



- 笔记本需要一个IP地址，第一跳路由器的IP地址，DNS的地址：采用*DHCP*
- DHCP 请求被封装在UDP中，封装在IP，封装在 802.3 以太网帧中
- 以太网的帧在LAN上广播(dest: FFFFFFFFFFFF)，被运行中的DHCP服务器接收到
- 以太网帧中解封装IP分组，解封装UDP，解封装DHCP

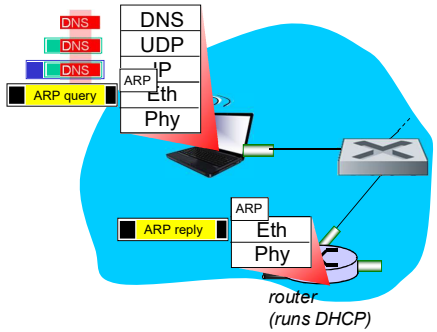
日常场景：... 连接到互联网



- DHCP 服务器生成*DHCP ACK* 包括客户端IP地址，第一跳路由器IP地址和DNS名字服务器地址
- 在DHCP服务器封装，帧通过LAN转发（交换机学习）在客户端段解封装
- 客户端接收DHCP ACK应答

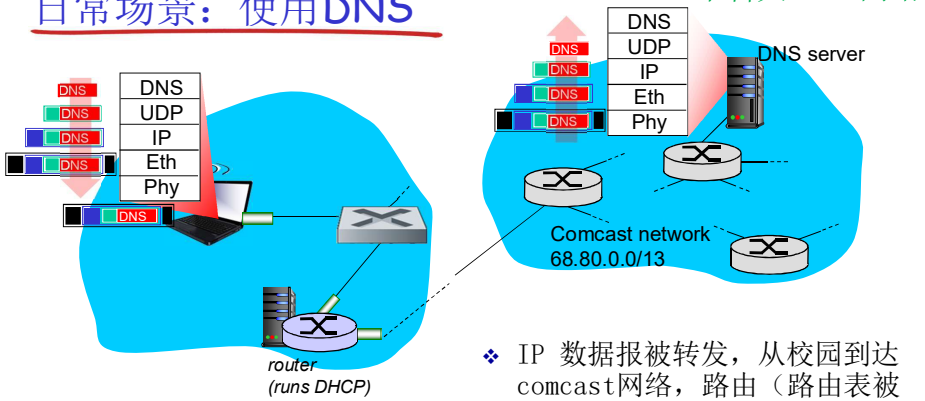
客户端有了IP地址，知道了DNS域名服务器的名字和IP地址
第一跳路由器的IP地址

日常场景..... ARP (DNS之前，HTTP之前)



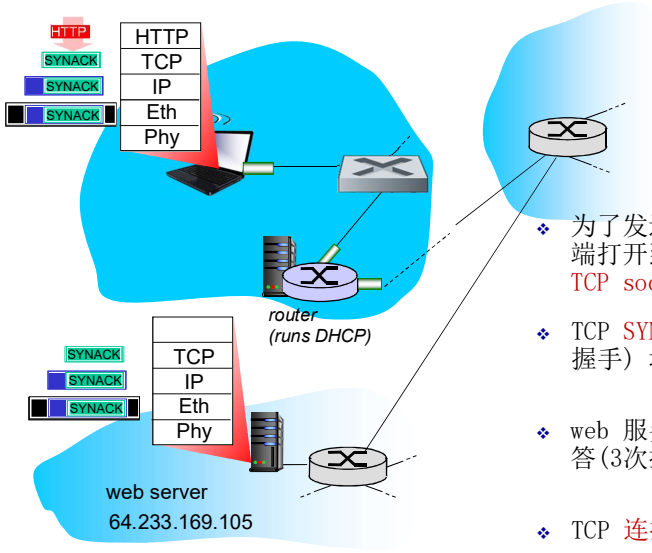
- 在发送*HTTP* request请求之前，需要知道www.google.com的IP地址：*DNS*
- DNS查询被创建，封装在UDP段中，封装在IP数据报中，封装在以太网的帧中. 将帧传递给路由器，但是需要知道路由器的接口：MAC地址：ARP
- ARP查询广播，被路由器接收，路由器用ARP应答，给出其IP地址某个端口的MAC地址
- 客户端现在知道第一跳路由器MAC地址，所以可以发送DNS查询帧了

日常场景：使用DNS



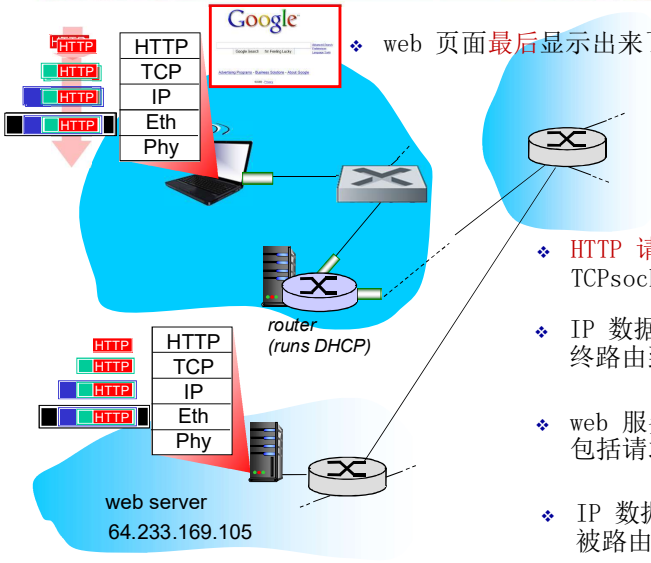
- 包含了DNS查询的IP数据报通过LAN交换机转发，从客户端到第一跳路由器
- IP 数据报被转发，从校园到达comcast网络，路由（路由表被RIP，OSPF，IS-IS 和/或BGP协议创建）到DNS服务器
- 被DNS服务器解封装
- DNS服务器回复给客户端：www.google.com的IP地址

日常场景：...TCP连接携带HTTP报文



- 为了发送HTTP请求，客户端打开到达web服务器的TCP socket
- TCP SYN 段(3次握手的第1次握手) 域间路由到web服务器
- web 服务器用TCP SYNACK 应答(3次握手的第2次握手)
- TCP 连接建立了!

日常场景：..... HTTP请求和应答



- web 页面最后显示出来了
- HTTP 请求发送到TCPsocket中
- IP 数据报包含HTTP请求，最终路由到 www.google.com
- web 服务器用HTTP应答回应(包括请求的页面)
- IP 数据报包含HTTP应答最后被路由到客户端

第6章：总结

- 数据链路层服务背后的原理：
  - 检错、纠错
  - 共享广播式信道：多路访问
  - 链路编址
- 各种链路层技术的实例和实现
  - Ethernet
  - 交换式LANS, VLANs
  - 虚拟成链路层的网络：MPLS
- 综合：一个web页面请求的日常场景

第6章：让我们休息一下

- 自上而下的协议栈的旅行结束了 (除了物理层)
- 坚实地理解了网络的原理和实践
- ..... 可以在这里停一下 ... ... 但是仍然还有很多有趣的话题!
  - 无线网络
  - 多媒体
  - 网络安全